

life.augmented

FreeRTOS on STM32 CMSIS_OS API v2.0

T.O.M.A.S – Technically Oriented Microcontroller Application Services

Introduction and agenda



life.augmented

Purpose of this session

The main goals of this sessions are answers to the following questions:

- What it is an operating system ?
- What are its main components ?
- What are the main advantages and disadvantages of operating system ?
- How to start application development based on OS ?
- When we would need it within embedded MCU based application ?
- What are typical mistakes done during OS based application development ?

How we will do it ?

Complete session will be based on CMSIS_OS v.2 layer which will use FreeRTOS operating system as a base.

As a hardware platform, STM32L476GRT6 has been selected (CortexM4 based STM32 MCU)

As a development tool STM32CubeIDE with STM32L4 Cube library has been used

Session organization

- Session is a combination of the theoretical parts and hands on sessions to illustrate topics
- Within each videos (usually at the end) there are links to other modules, related to the topics just discussed
- Under each video, within its description there are links to:
  - Landing page which contains complete set of information about the session (including link to materials, prerequisites)
  - Link to the ST community where you can post your questions related to the session
  - Link to anonymous survey, where you can pass your opinion about the session

Materials available for this session

- All materials related to this session has been stored on dedicated repository The following list of materials are available for this session:
  - Complete set of slides in pdf format (**./Slides** folder)
  - Solution of each lab part (**./Labs_solutions** folder)
    - Complete set of examples are present within All_FreeRTOS_v2_labs.zip
    - Single examples are additionally stored in dedicated folders in .zip format
- To use solutions, you can import them to your STM32CubeIDE workspace using:

  File->Import->General->Import Existing Projects into Workspace->Select archive file

Agenda 1/3

1. Preparation to the session
2. Operating system – what it is ?
3. FreeRTOS – basic features
4. CMSIS_OS API vs FreeRTOS API
5. FreeRTOS in STM32CubeMX / CubeIDE
6. FreeRTOS configuration
7. FreeRTOS memory management
8. FreeRTOS scheduler

life.augmented

Agenda 2/3

9 FreeRTOS and hardware IRQs

10 FreeRTOS startup step by step

11 Context switching in details

12 Tasks

13 Queues

14 Semaphores

15 Event Flags

16 Thread Flags

Agenda 3/3

17 Resources management

18 Mutexes

19 Software timers

20 Advanced topics (hooks)

21 Low Power support (tickless mode)

22 Troubleshooting and typical mistakes

23 Summary

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Preparation to lab sessions



life.augmented

Prerequisites

- Hardware requirements:
  - PC with preinstalled software (software requirements)
  - NUCLEO-L476RG board
  - mini USB cable
  - [optionally] X-NUCLEO-LPM01A
  - [optionally] micro USB cable
- Software requirements:
  - STM32CubeIDE* (v1.2.1 or more recent one) with installed
    - STM32L4 Cube library v1.15 or more recent one
  - [optionally] STM32CubeMonitorPower



*) you can use STM32CubeMX and selected toolchain supported by this tool instead

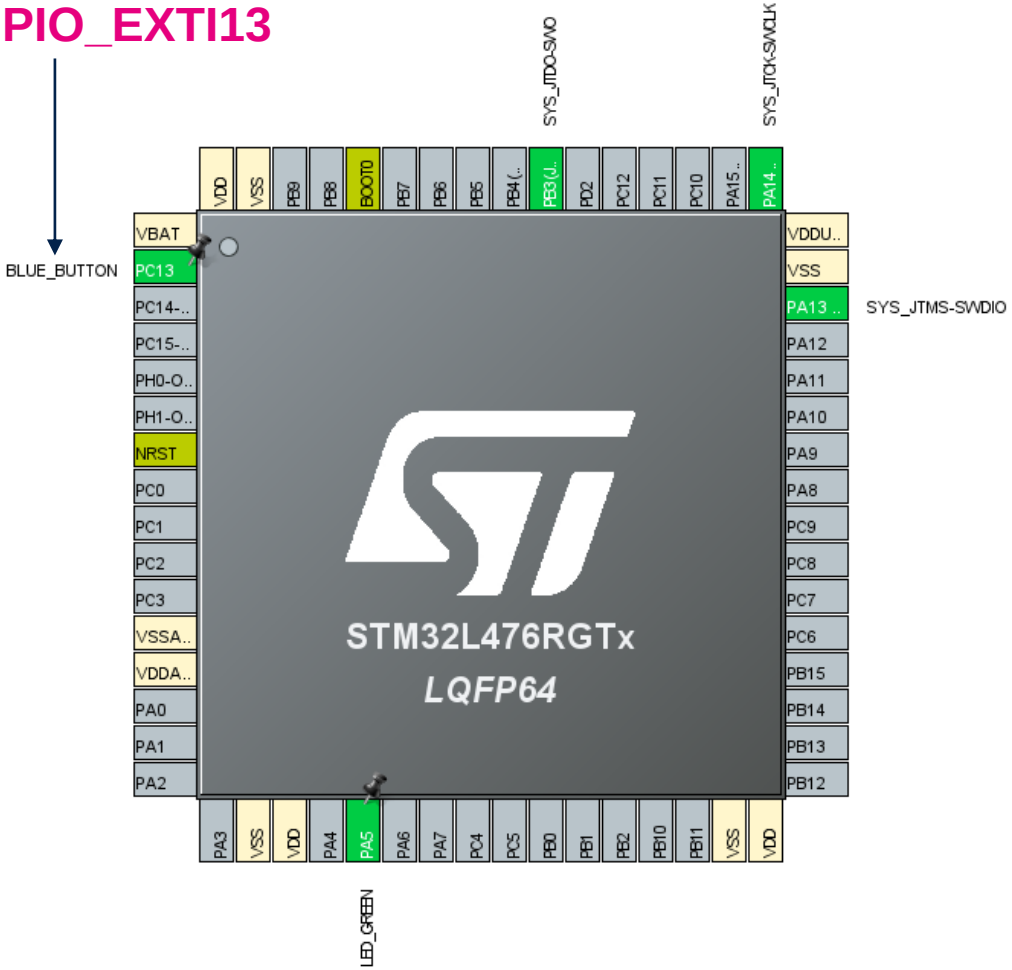
life.augmented

Lab preparation

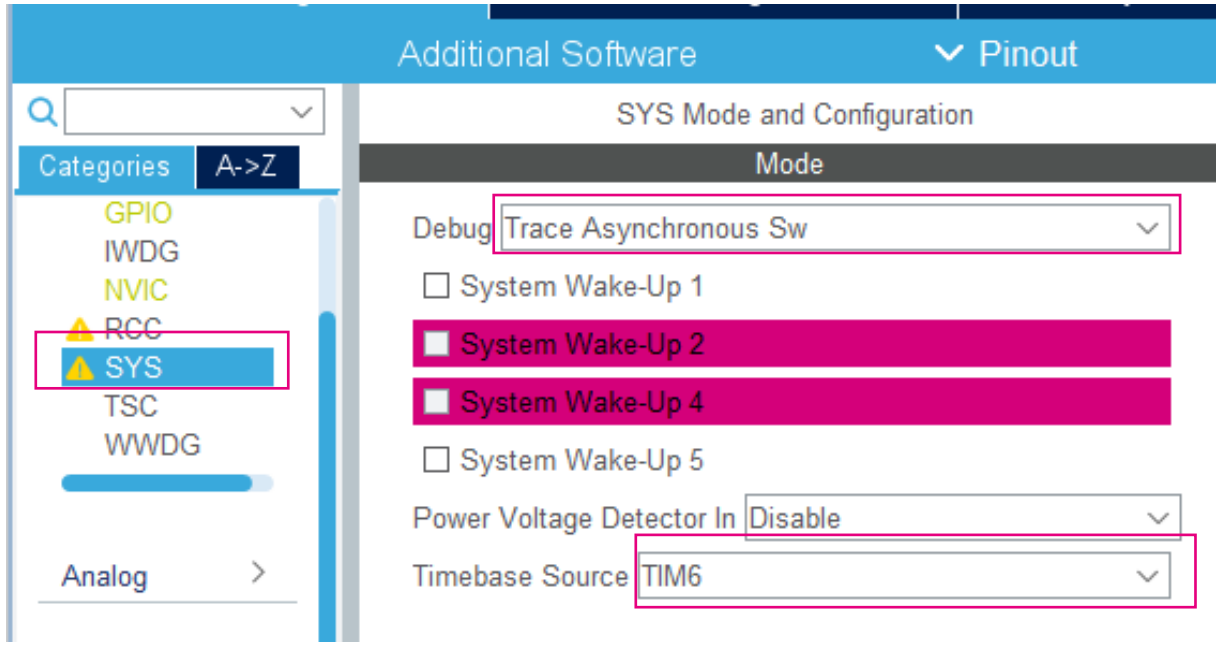
- Using STM32CubeIDE we will create a simple project for STM32L476RGTx MCU
- We will configure there:
  - GPIOA, pin5 as GPIO output, Push-Pull with label LED_GREEN
  - GPIOC, pin13 as EXTI13, with label BLUE_BUTTON with interrupt enabled (NVIC section)
  - SWD as debug interface (with Asynchronous Trace)
  - TimeBase timer changed from SysTick to TIM6
  - Middlewares -> FREERTOS as Interface CMSIS_V2
  - [Optionally] USART2 on PA2, PA3 pins (asynchronous mode, 115200bps, 8b data, 1 stop bit, no parity, no HW control, no IRQs, no DMA usage) -> to redirect prinft, as USART2 is connected to STLink and can be used via Virtual COM port
- We will keep default settings of the clock configuration (4MHz, MSI based)
- We will generate code using HAL libraries model and open it in STM32CubeIDE

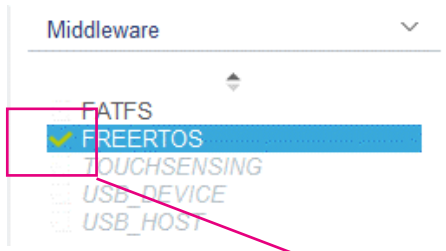
Lab preparation
STM32CubeIDE configuration – peripherals selection

GPIO_EXTI13

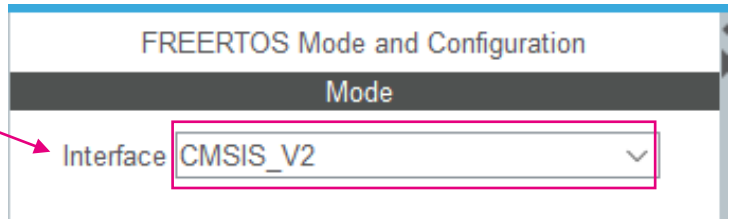


GPIO_Output





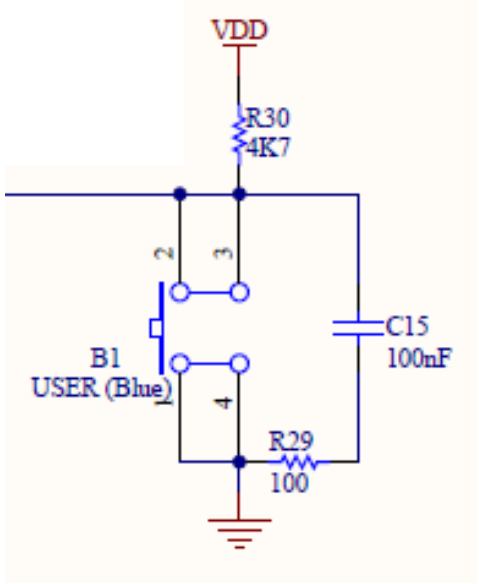
Time base for HAL library as SysTick will be used by FreeRTOS

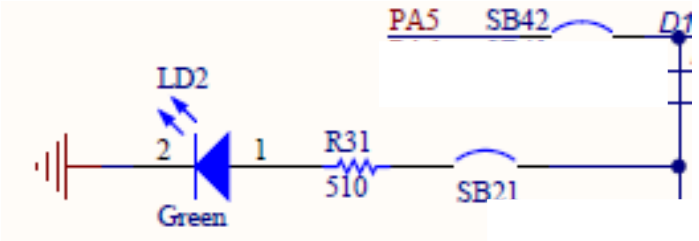


life.augmented

GPIO pins configuration

PC13





PA5

Configuration

Group By Peripherals

GPIO | SYS | NVIC

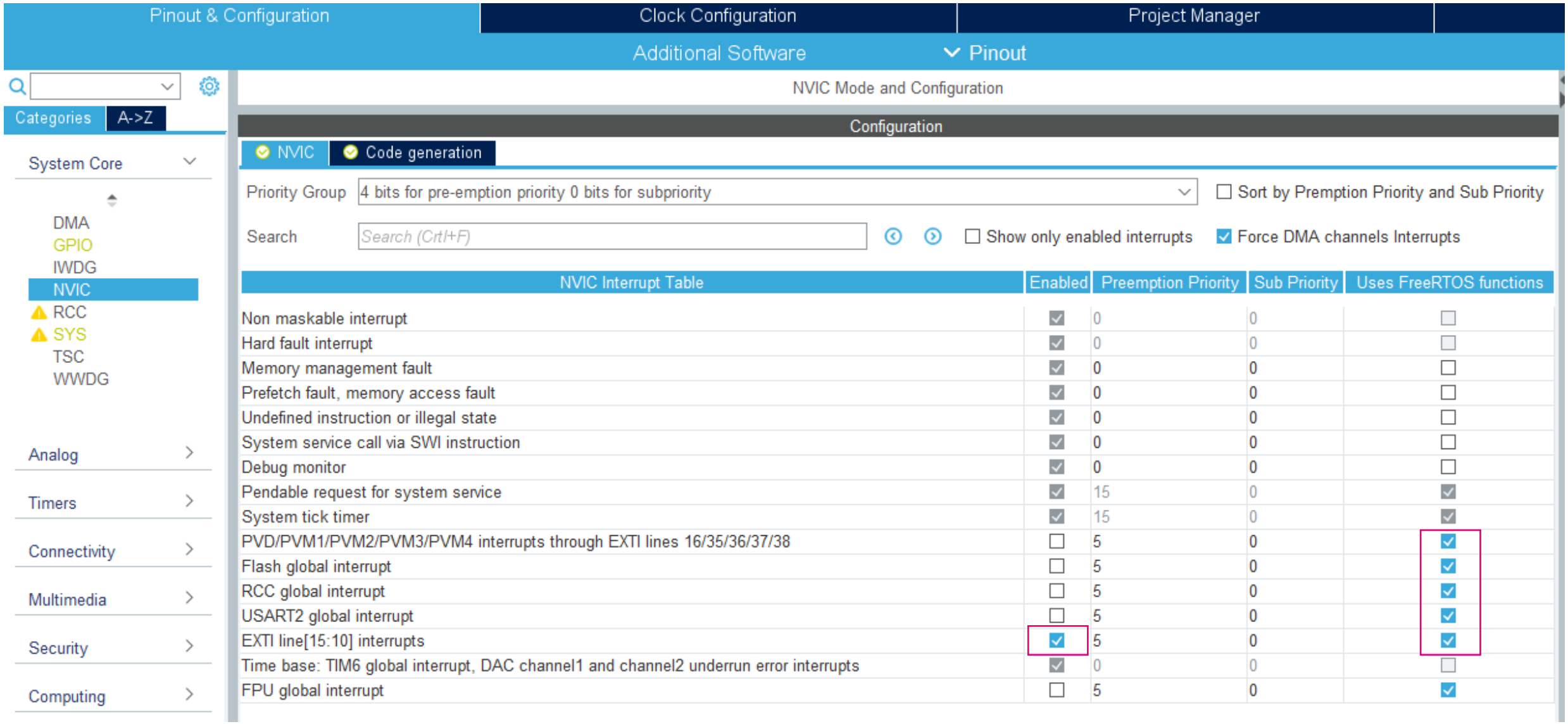
Search Signals

Search (Crtl+F)

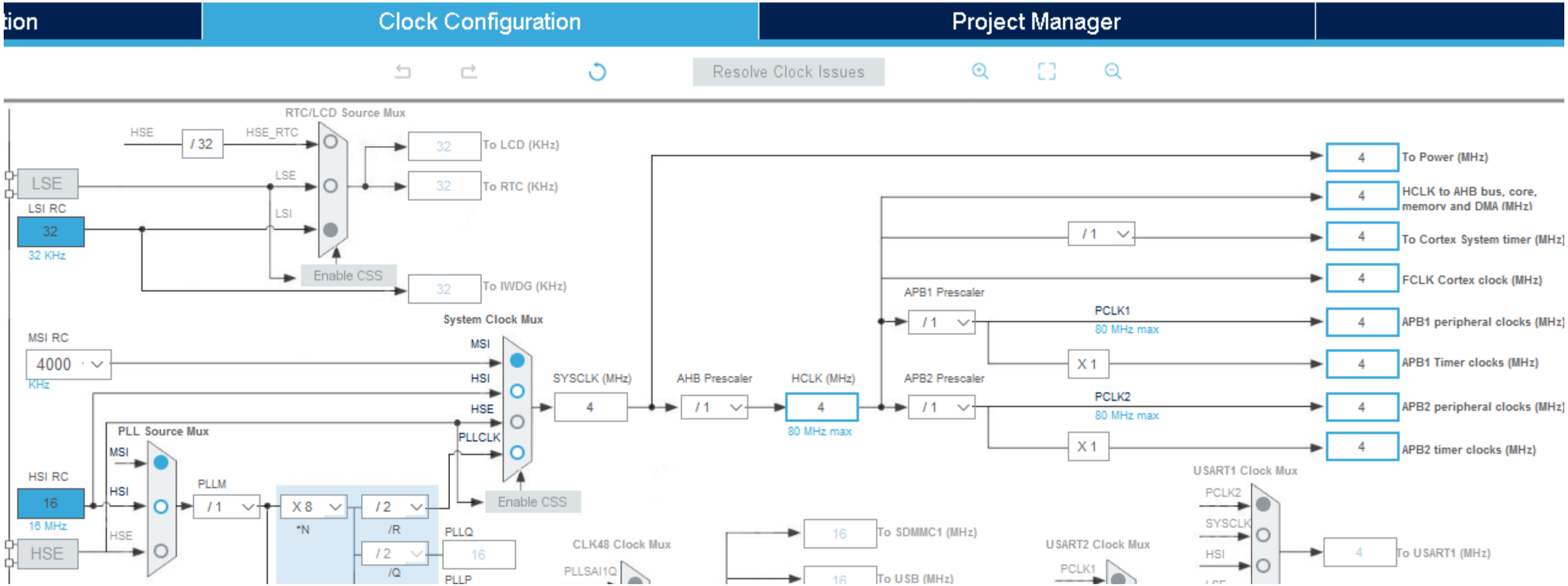
Pin Na...	Signal on Pin	GPIO output ...	GPIO mode	GPIO Pull-up/Pull-down	Maximu...	Fast ...	User Label
PA5	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	n/a	LED_GREEN
PC13	n/a	n/a	External Interrupt Mode with Rising edge trigger detection	No pull-up and no pull-down	n/a	n/a	BLUE_BUTTON

life.augmented

Lab preparation
STM32CubeIDE configuration - NVIC

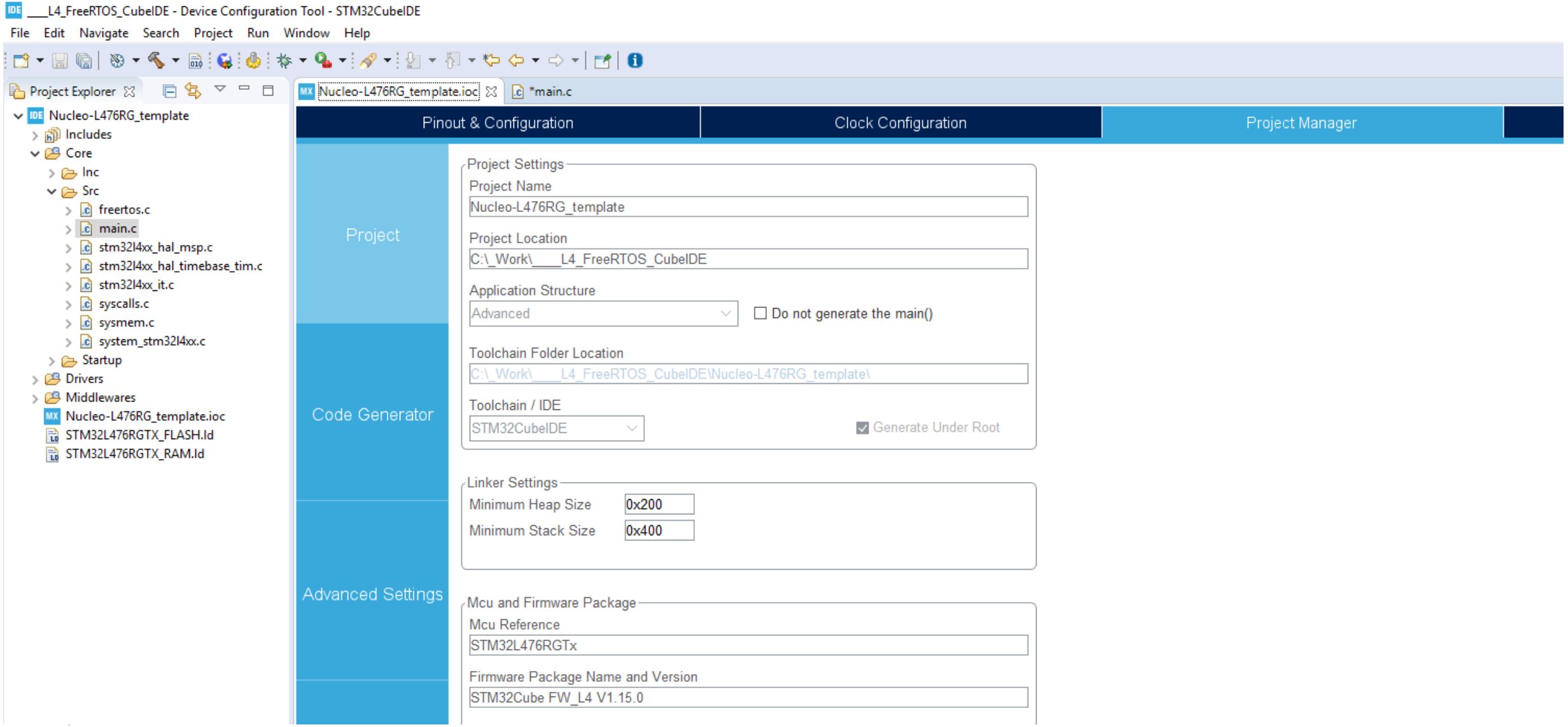


Lab preparation
STM32CubeIDE configuration – clock configuration - DEFAULT



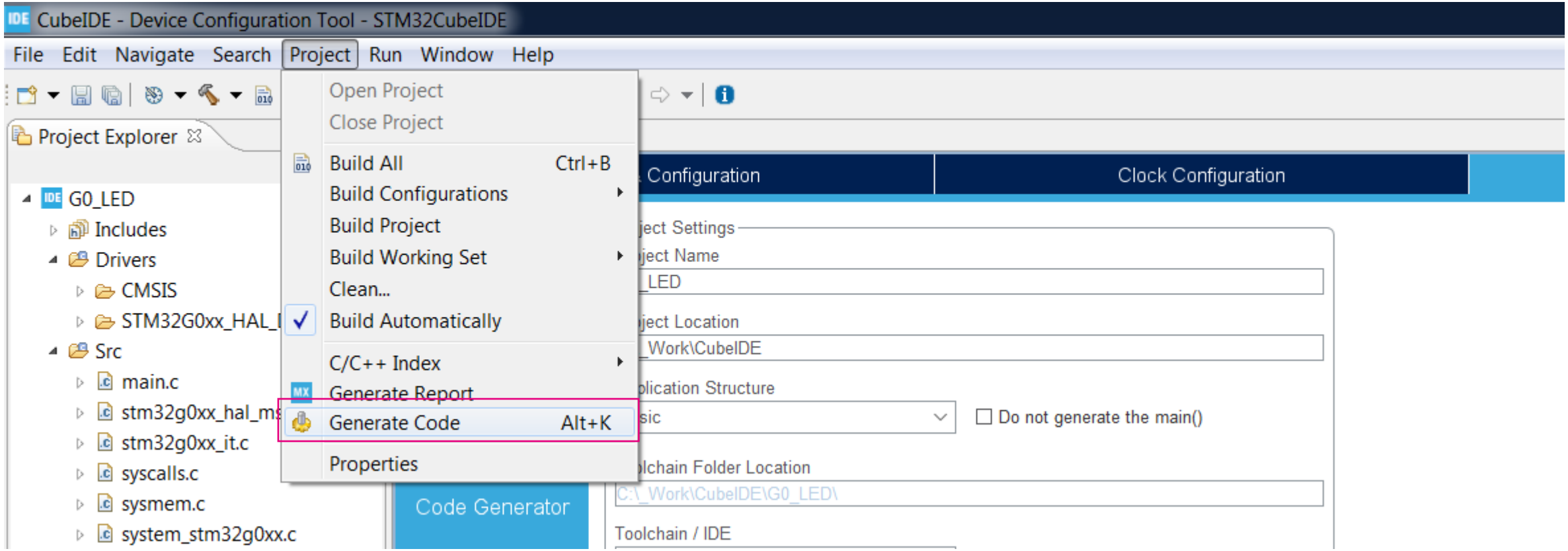
life.augmented

Basic project settings – no change



Code generation

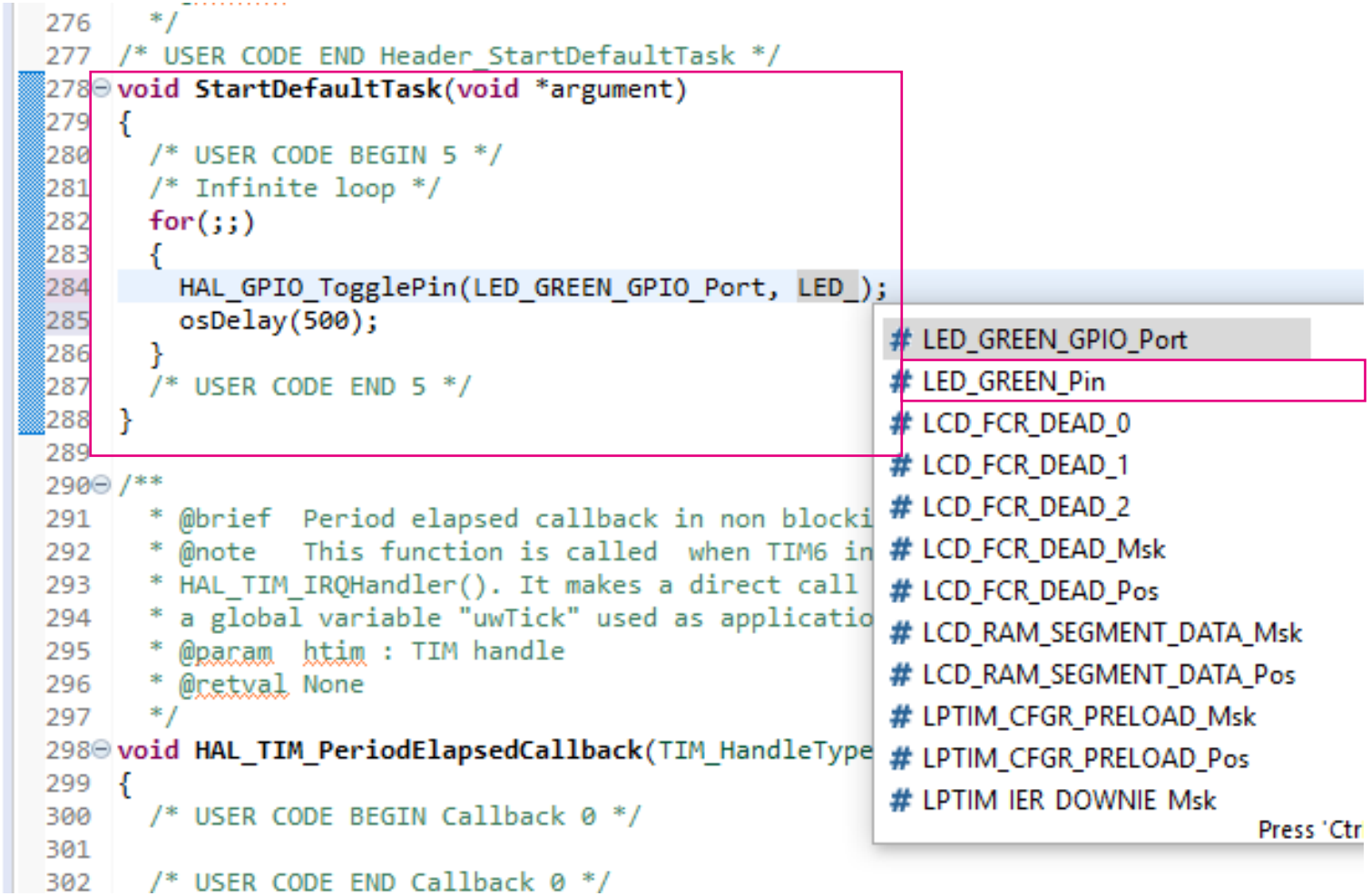
- To generate the code select Project -> Generate Code, press Alt+K or just save the changes (Ctrl+S)



life.augmented

Adding code

- We can use Ctrl+Space help to find proper function or argument
- Please use “USER CODE” sections only
- In our example we need to look for StartDefaultTask() function body and add there GREEN LED blinking (each 500ms)



life.augmented

Final code (main.c file)

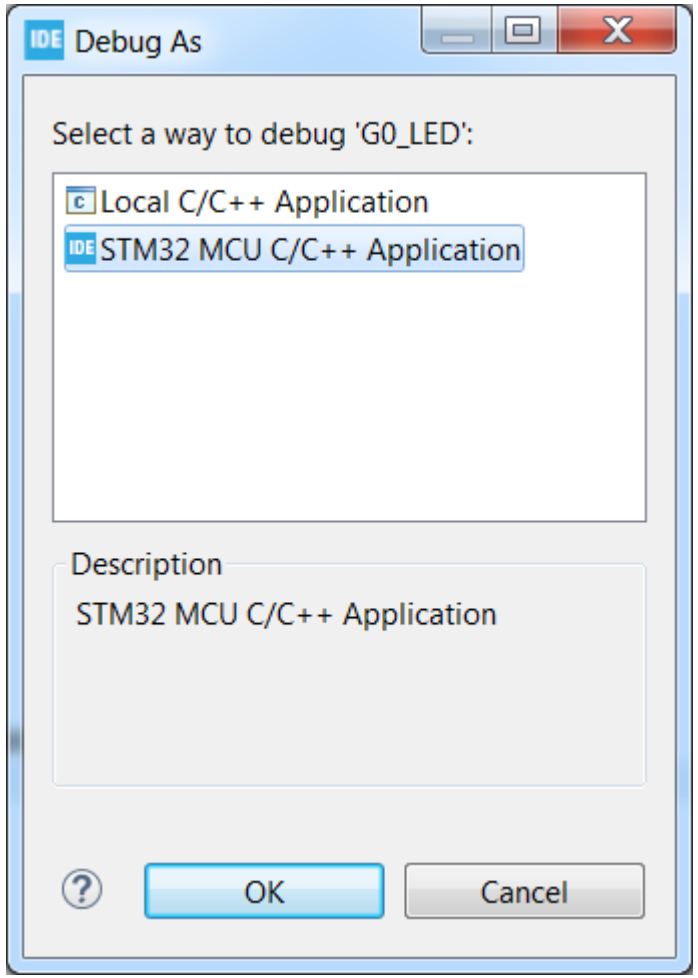
- We need to toggle LED_GREEN pin with 500ms delay in between

```
277 /* USER CODE END Header_StartDefaultTask */
278 void StartDefaultTask(void *argument)
279 {
280   /* USER CODE BEGIN 5 */
281   /* Infinite loop */
282   for(;;)
283   {
284     HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
285     osDelay(500);
286   }
287   /* USER CODE END 5 */
288 }
```

Compile and run debug session

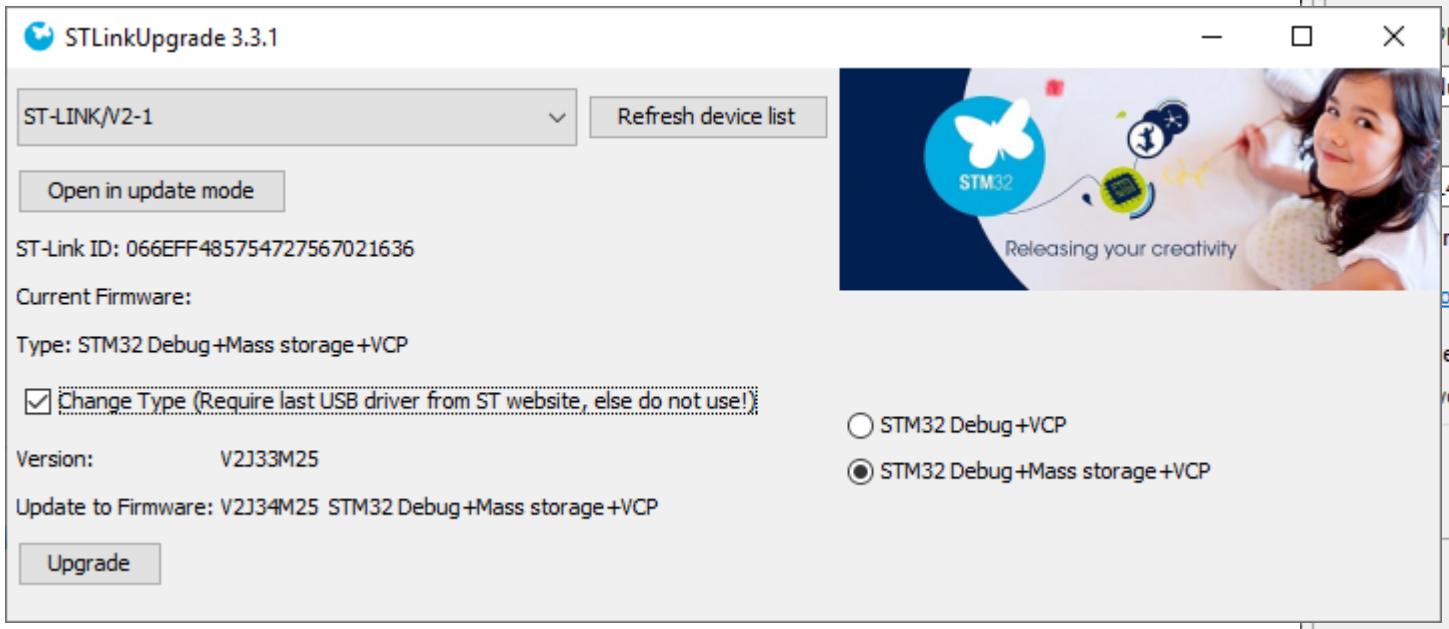
- We can compile the code with  or build with automatic start of debug session using  button

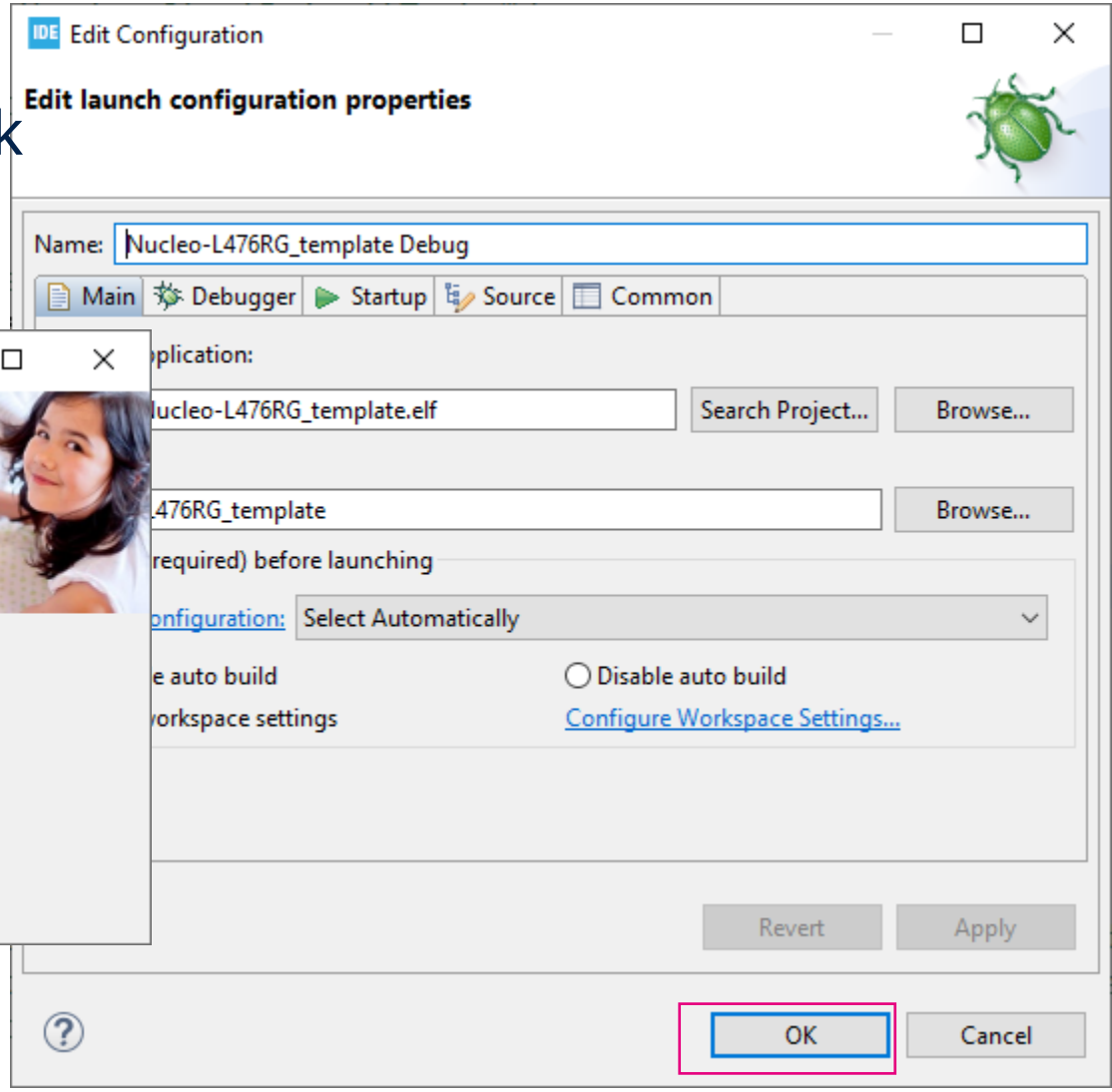
- Starting debug session we will be asked how we would like to debug the code



Debug settings

- When running debug (after successful compilation) there is no need to configure anything for debug
- Just connect target board to STLink and press ok

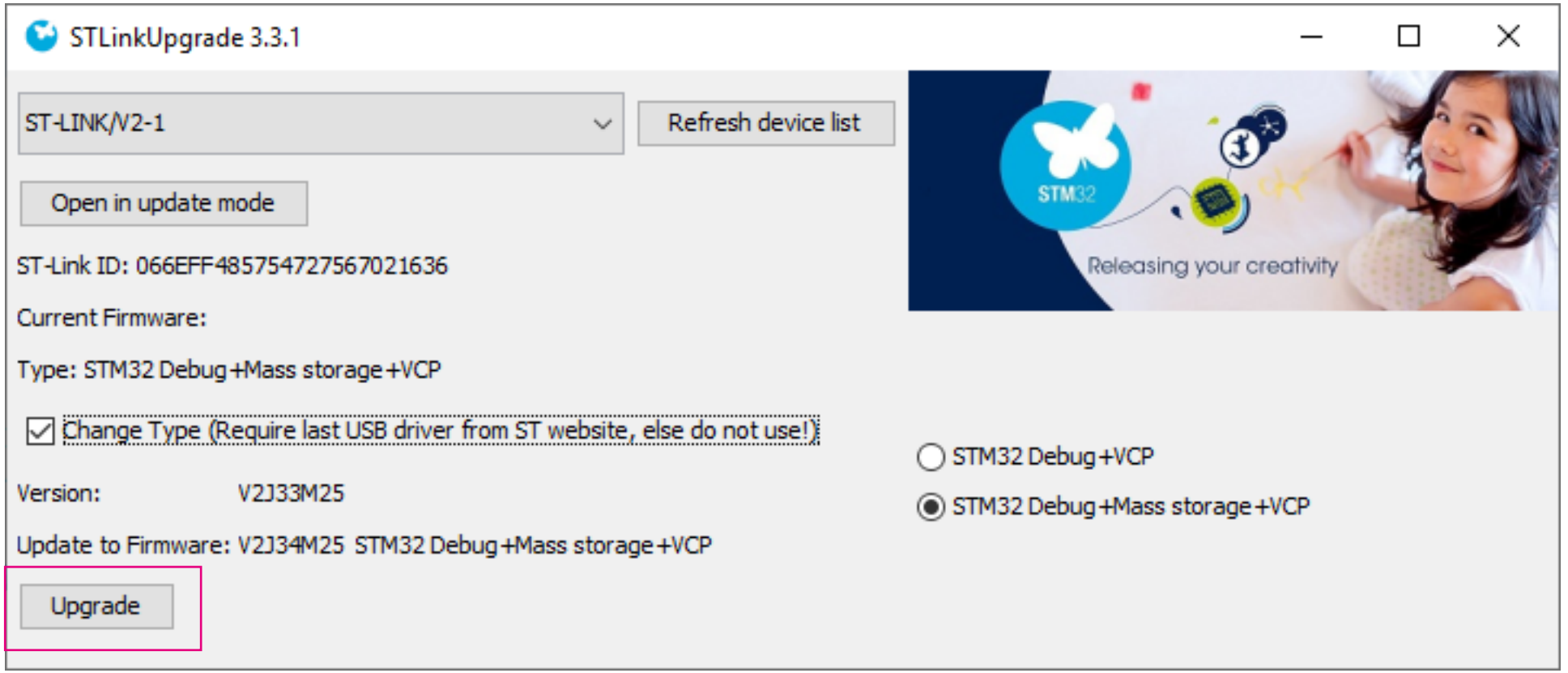




life.augmented

STLink upgrade

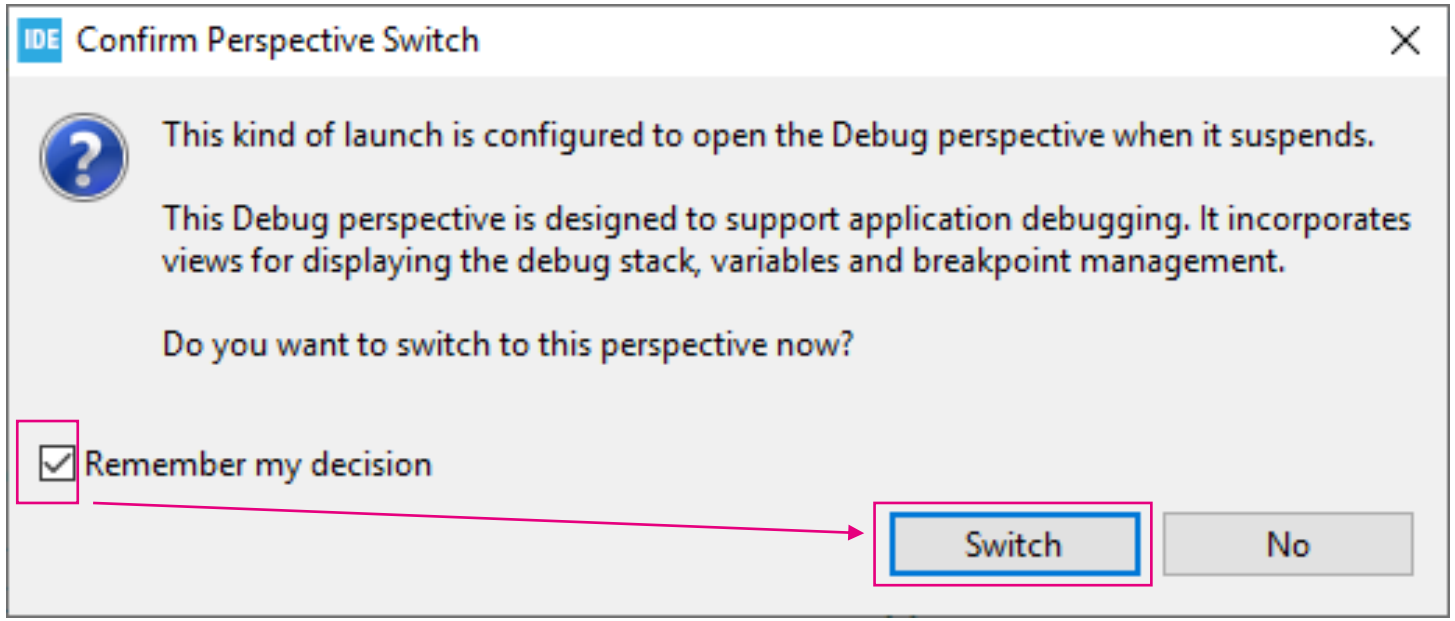
- It may happened that there will be necessity to upgrade your STLink software.
- In such a case agree on this and select settings like below, then press Upgrade
- Then close the window and start debug (bug icon) again 



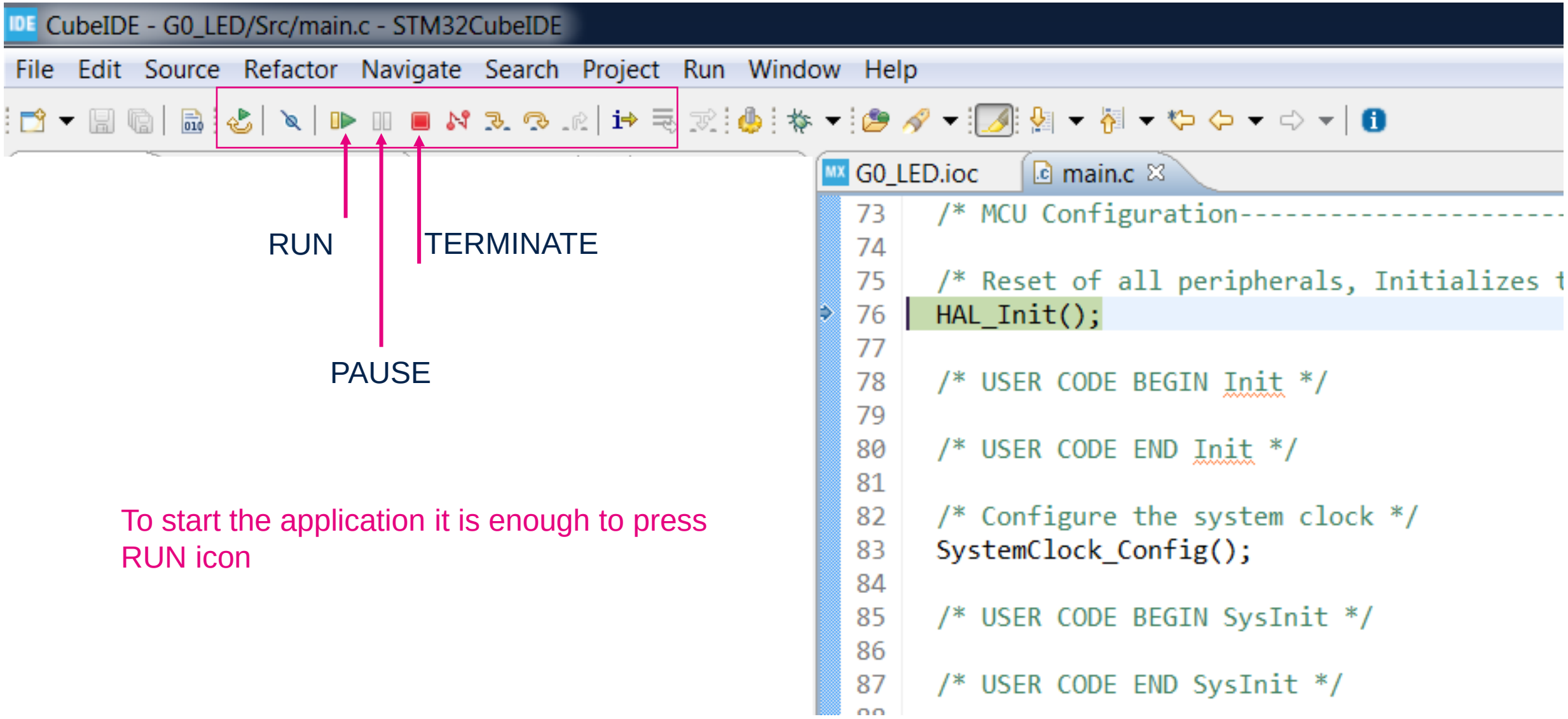
life.augmented

Debug perspective (first run)

- At first run of the debug session there will be a warning message displayed concerning different perspective (debug one) view. Just press „Switch”.



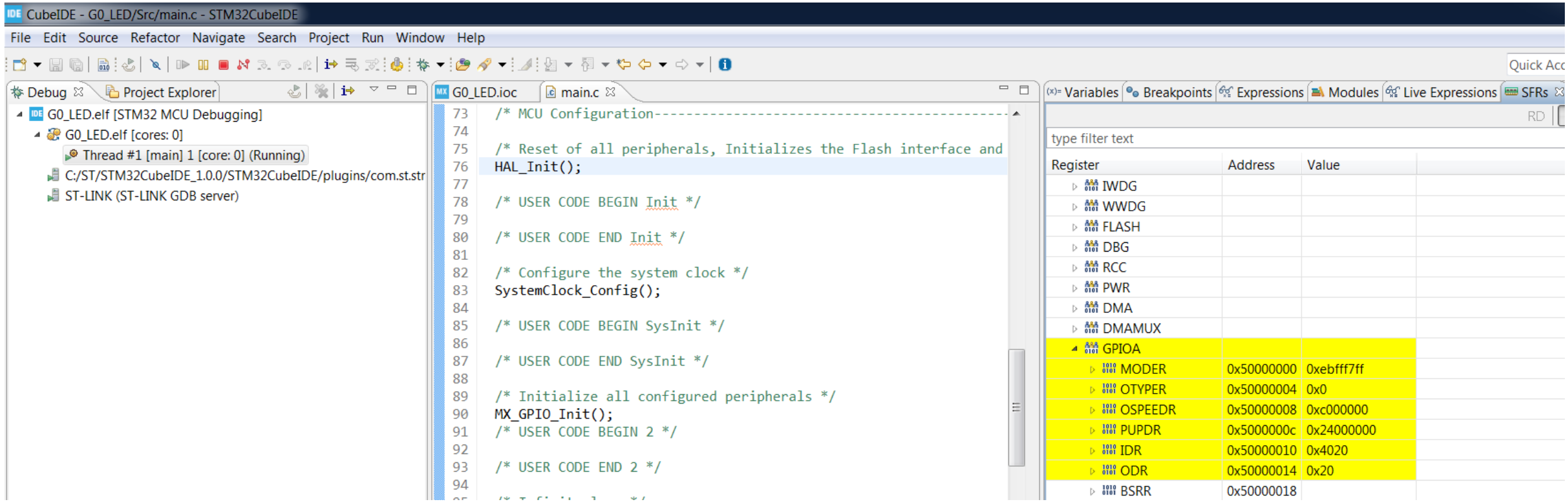
Debug perspective (toolbar)



To start the application it is enough to press RUN icon

Debug perspective – registers view

- Within debug perspective it is possible to monitor code flow, variables and registers content (on application pause, breakpoint stop or live)



Project / debug explorer

Source code

Variables, breakpoints, registers view

life.augmented

… Let’s check it

- After all code processing, we can build the project, start debug session and run the application
- As an effect Green LED should toggle each 0.5 second
- Change of LED state is done in the task called „StartDefaultTask” which is called by OS each 500ms

Task_action() code - theory

- Within most of the hands-on sessions we would need to perform some action within the task
- In all of those places there is Task_action() function call
- You can implement your own Task_action():
  - The easiest way is to use printf() inside, but as this function can be really heavy in terms of RAM usage and execution time, it may cause some unwanted side effects
  - The best is to use existing LEDs or simply use live variables which are supported within STM32CubeIDE. Additionally we may send simple text messages over USART or SWO.

Task_action() code - examples

- Using LED toggling

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
}
```

- Using SWO and Serial Wire Viewer (SWV) to send simple text

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  ITM_SendChar(message);
  ITM_SendChar('\n');
}
```

- Using USART2 (connected to STLink and visible as VCP) to send simple text

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  uint8_t data[]={message,'\n', '\r'};
  HAL_UART_Transmit(&huart2, data, 3, 2);
}
```

life.augmented

Task_action() code – our choice for the session

- Using LED toggling

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
}
```

- Using SWO and Serial Wire Viewer (SWV) to send simple text

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  ITM_SendChar(message);
  ITM_SendChar('\n');
}
```

- Using USART2 (connected to STLink and visible as VCP) to send simple text

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  uint8_t data[]={message,'\n', '\r'};
  HAL_UART_Transmit(&huart2, data, 3, 2);
}
```

life.augmented

Usage example of Task_action() code

- Let’s add a declaration of the function

```
/* USER CODE BEGIN PFP */
void Task_action(char message);
```

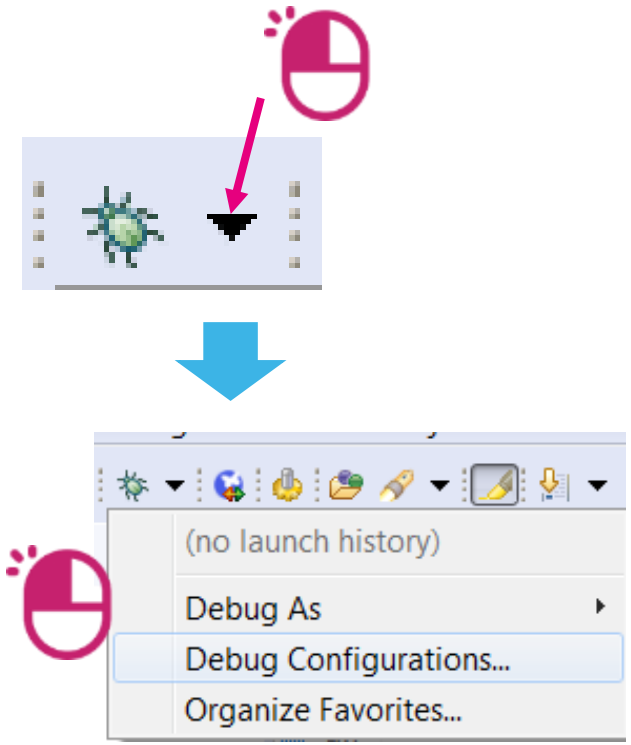
- Let’s modify StartDefaultTask() function to use Task_action() insted of LED blinking

```
void StartDefaultTask(void *argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
        Task_action('1');
        osDelay(500);
  }
  /* USER CODE END 5 */
}
```

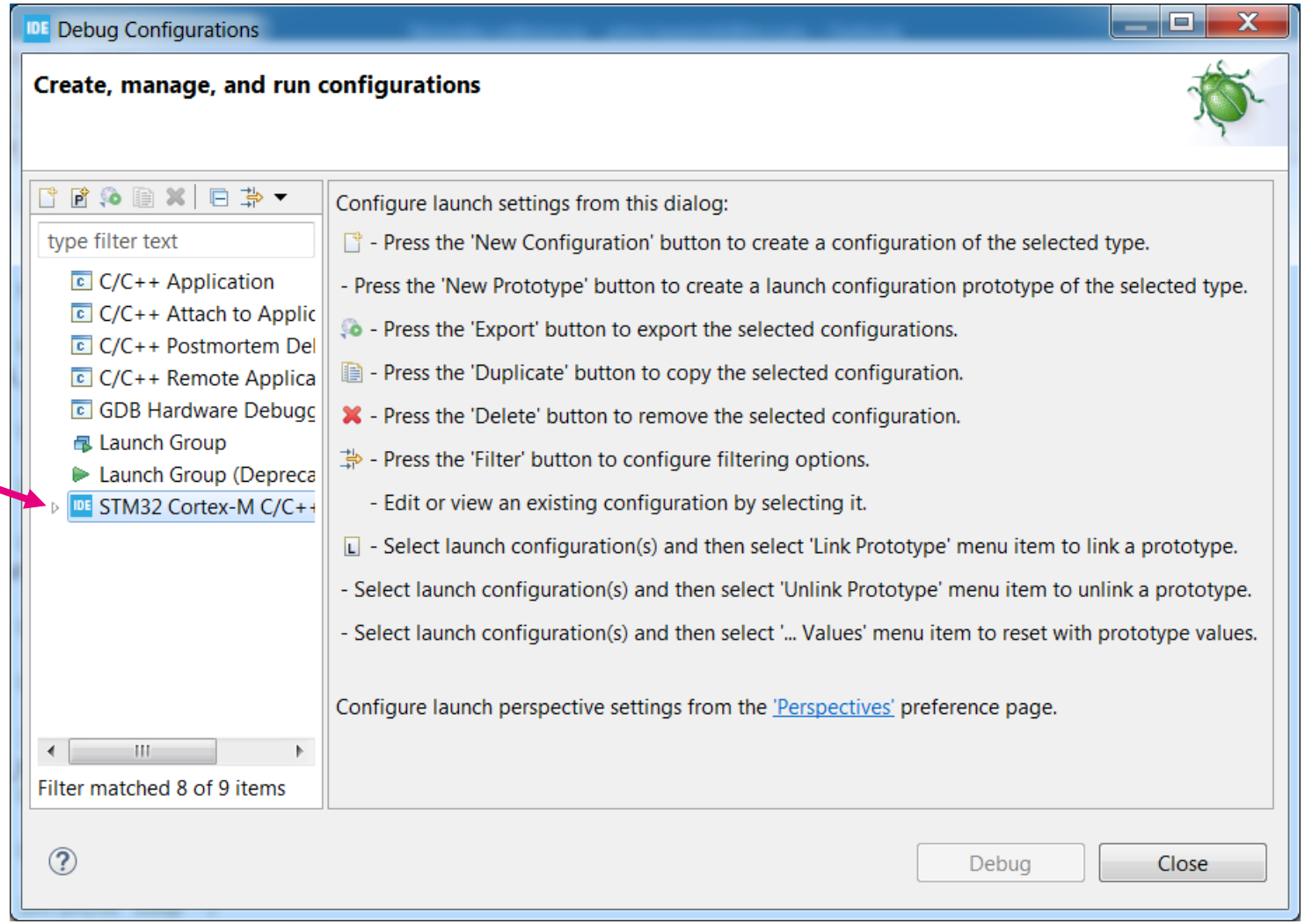
life.augmented

Configuring debug session work with SWO/SWV 1/2

- After the successful compilation we need to configure debug session to use Serial Wire Output



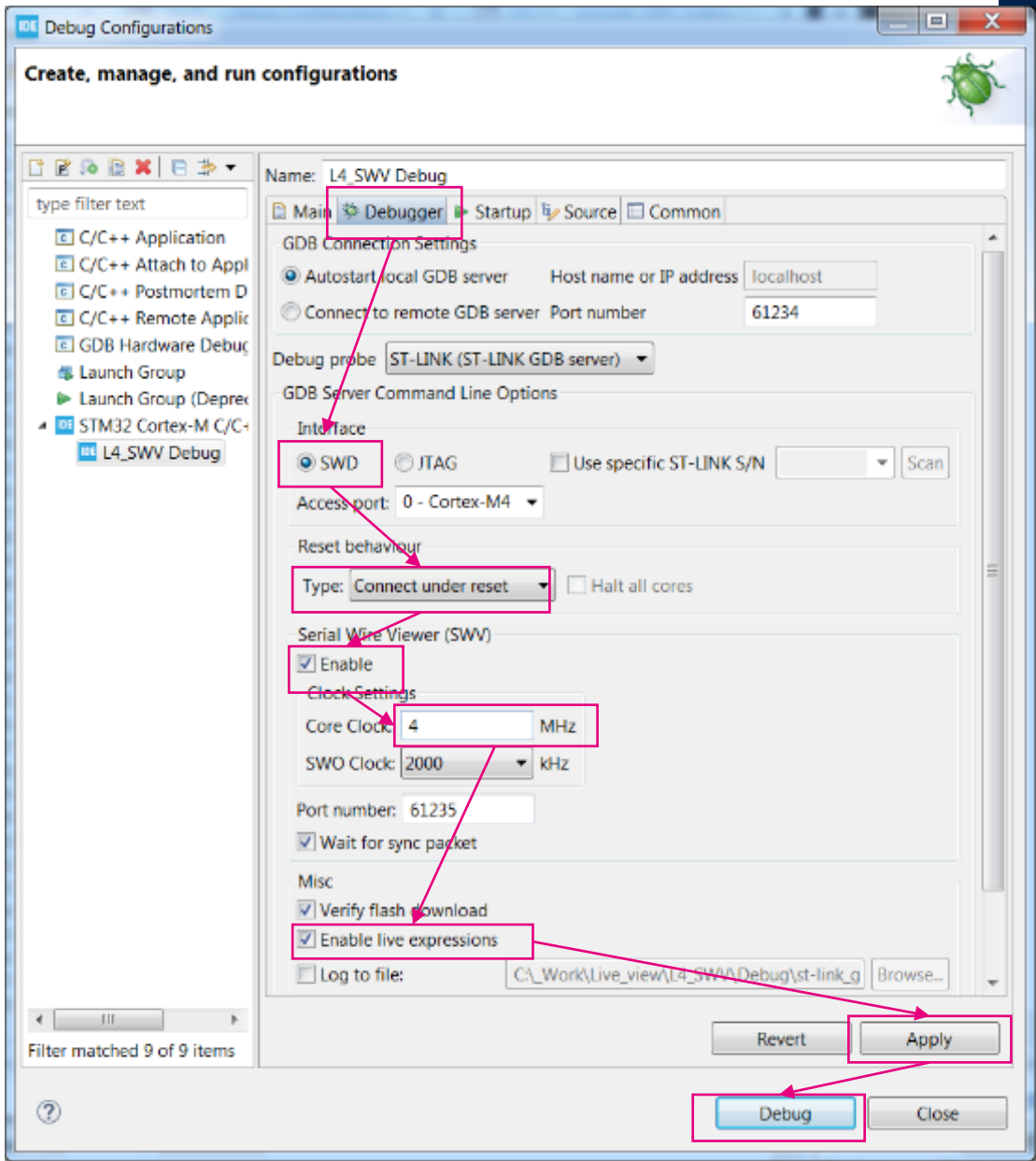
2x



life.augmented

Configuring debug session work with SWO/SWV 2/2

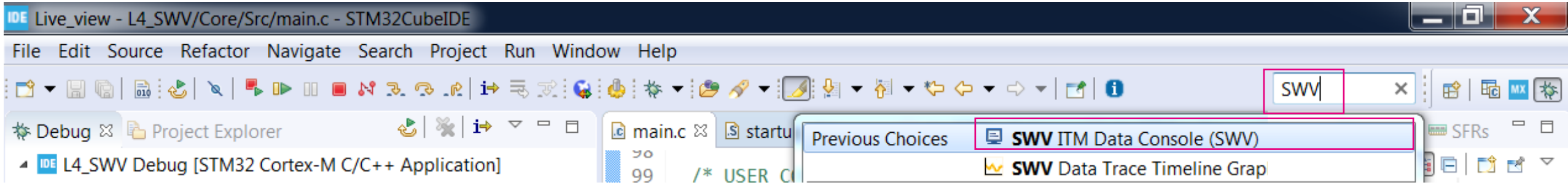
- Go to Debugger tab
- Select SWD interface
- Set Reset behavior to Connect under reset
- Enable SWV (Single Wire Viewer)
- Set Core Clock to 4MHz (used in our example)
- Enable live expressions
- Click Apply and Debug to start debug session



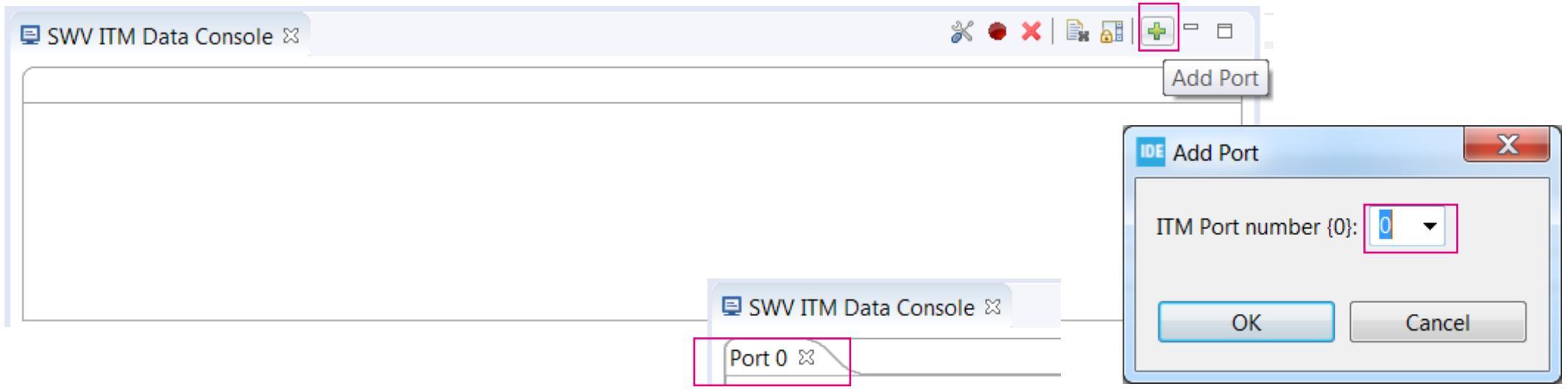
life.augmented

Strings display on SWV ITM data console ITM_SendChar() function result

- Within Quick Access window type SWV and select SWV ITM Data Console



- Add Port0 within SWV ITM Data Console (it is used by SWO)

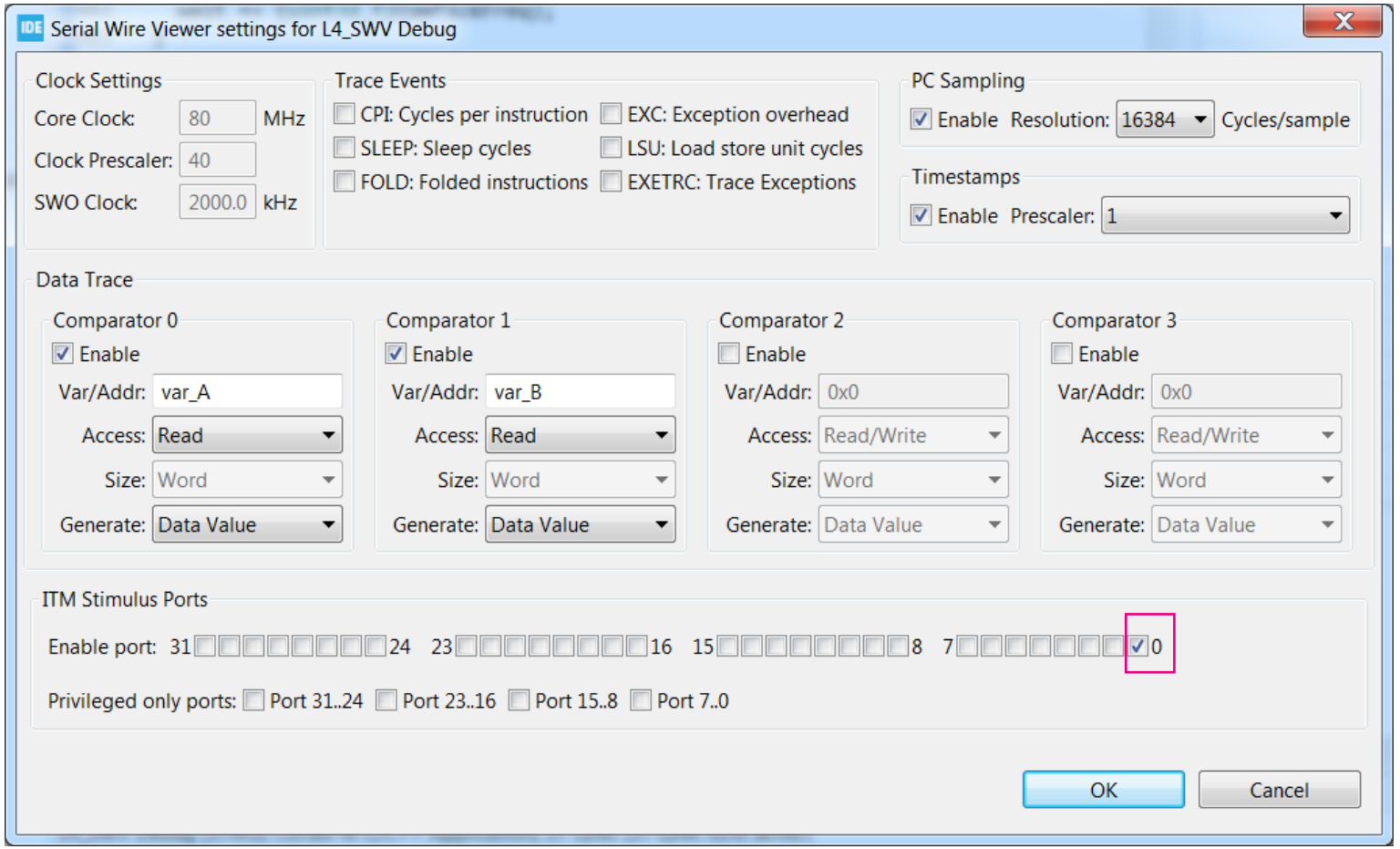


life.augmented

Strings display on SWV ITM data console ITM_SendChar() function result

- Enable port0 within trace configuration



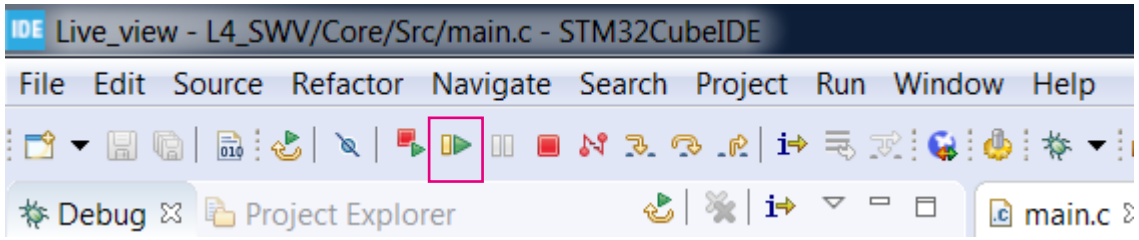


life.augmented

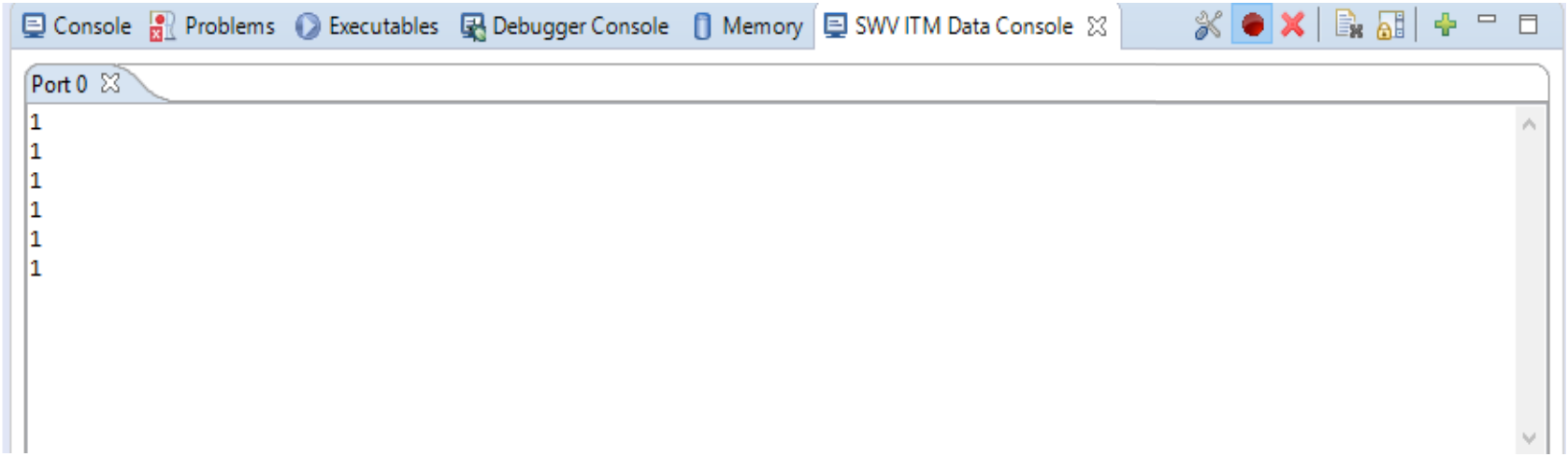
Strings display on SWV ITM data console ITM_SendChar() function result

- Start Trace
- Start the code





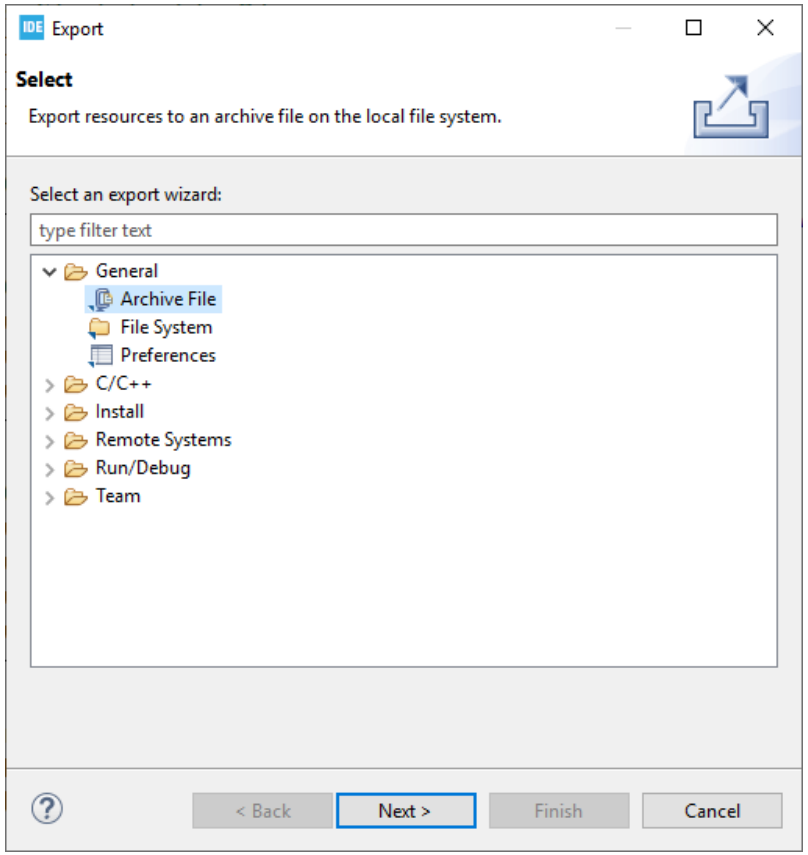
- Observe printf output on ITM data console

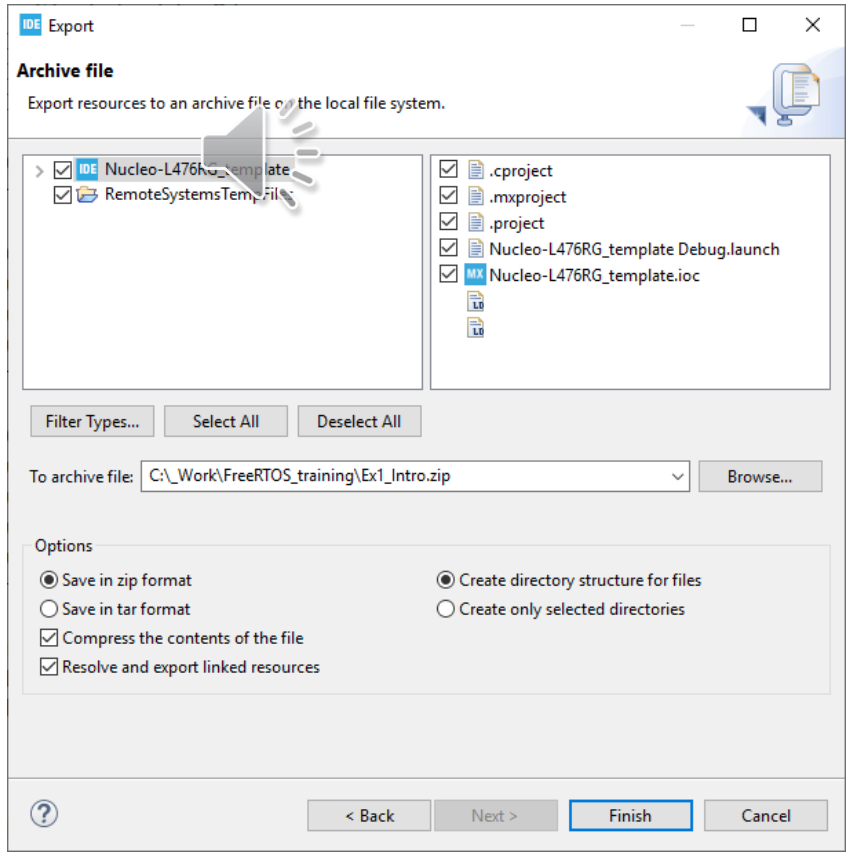


life.augmented

Project management – how to store an example

- To save some time needed for new project creation we will reuse this exercise during next hands-on parts
- In case you would like to store each exercise please use File -> Export option and store an example in dedicated zip file (example for Ex1_intro.zip below)





You can reuse stored examples using File -> Import option within any workspace

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Operating system what is … ?



life.augmented

What are the main components of OS ?

We can list the following main compoments of OS:

- Tasks or threads –mini programs run in endless loop
- Scheduler – to manage tasks and switch between them
- Queues – to pass information among tasks or among tasks and interrupts
- Semaphores – on/off type of queues
- Mutexes – semaphores with token
- Software timers – to manage tasks execution in particular moments

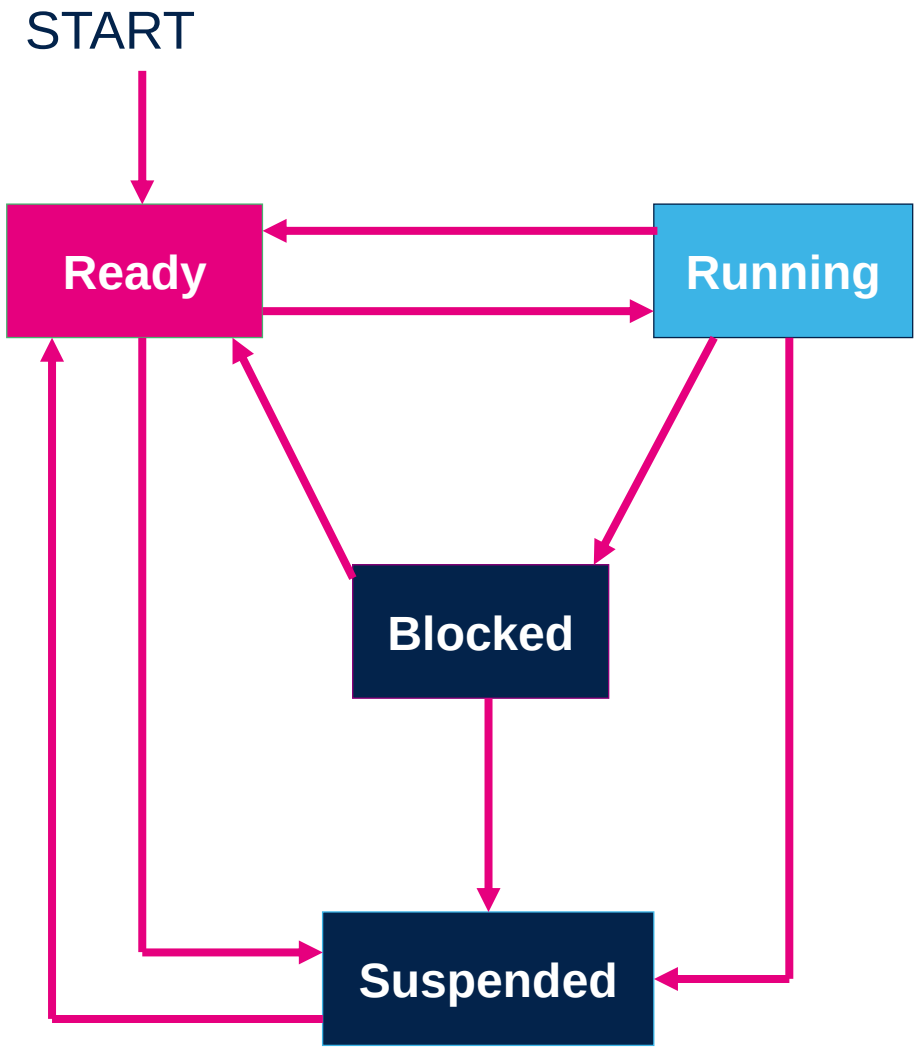
life.augmented

What is a Task?

- It is a C function:
- It should be run within infinite loop, like:

```
for(;;)
{
  /* Task code */
}
```

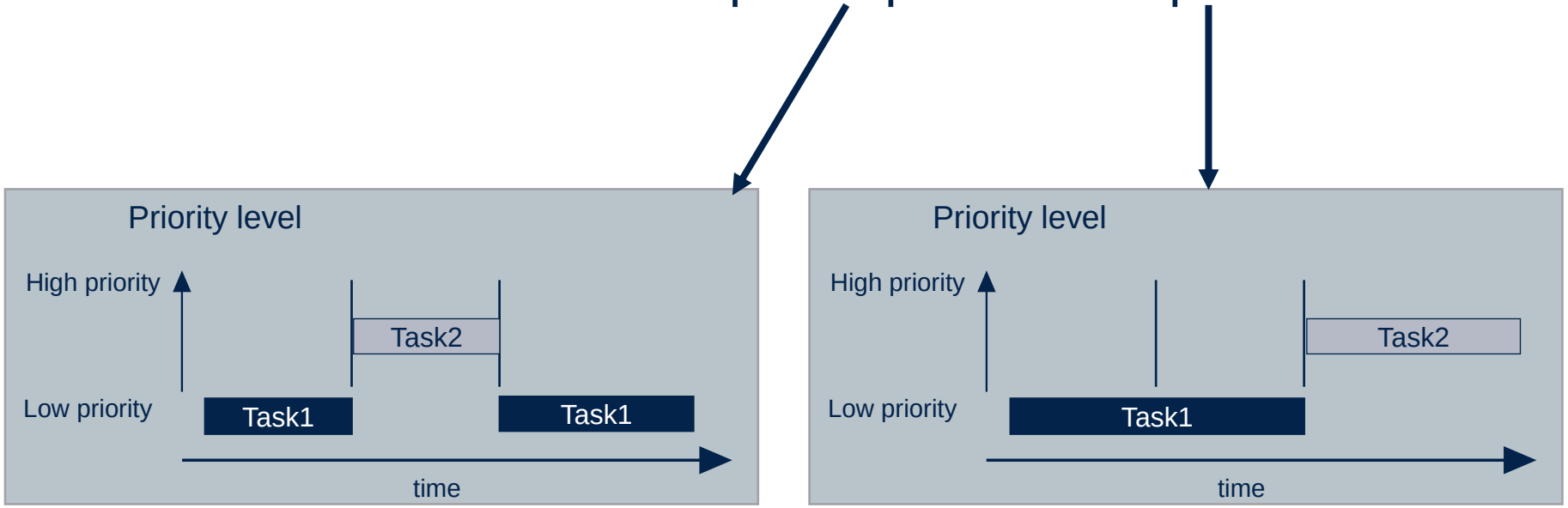
- It has its own part of stack, and priority
- It can be in one of 4 states (RUNNING, BLOCKED, SUSPENDED, READY)
- It is created and deleted by calling API functions



life.augmented

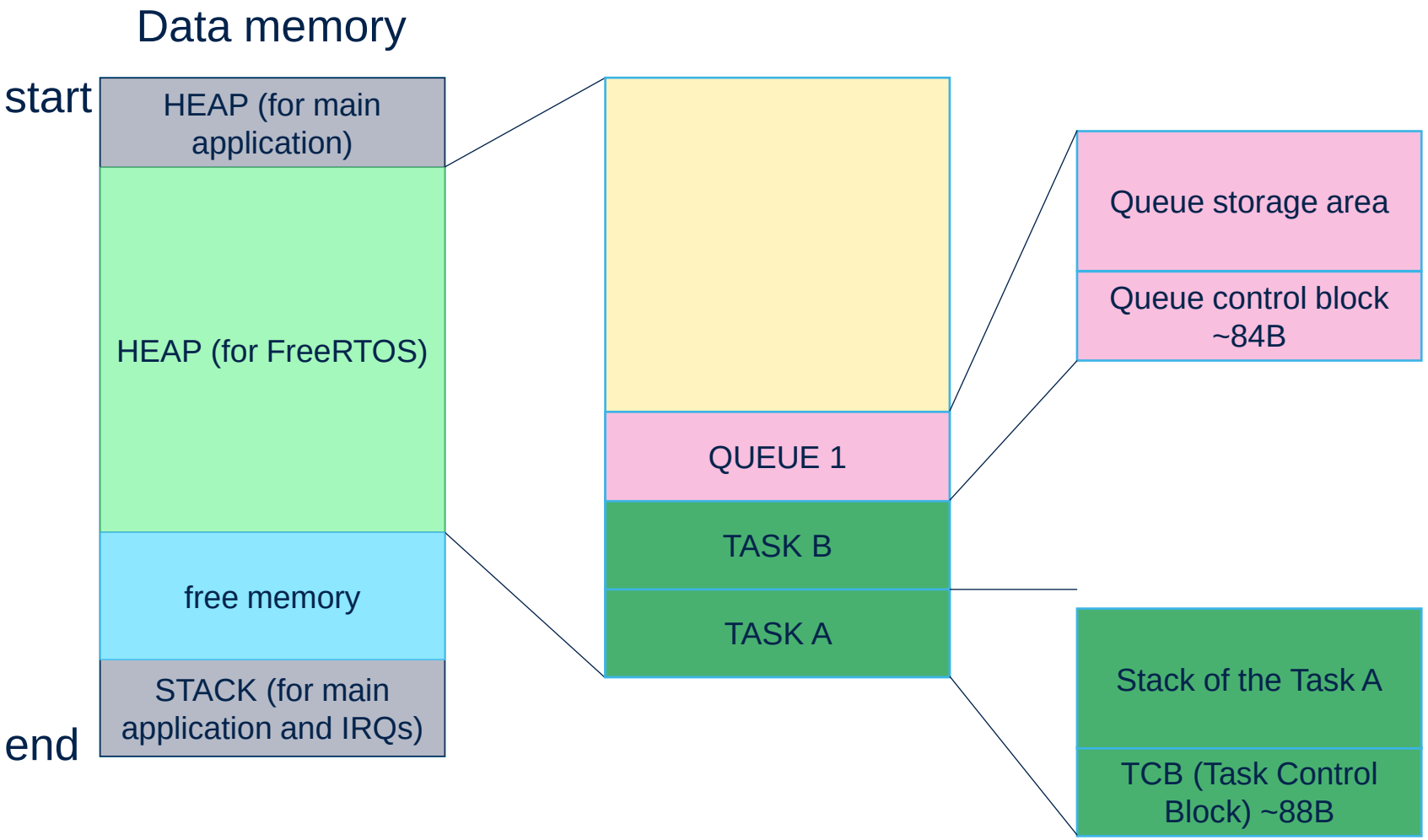
What is a scheduler?

- The scheduler is an algorithm determining which task to execute.
  - Is select one of the task being ready to be executed (in READY state)
  - There are few mechanisms controlling access to CPU for tasks (timeslice, preemption, idle)
- In FreeRTOS round-robin scheduling algorithm is implemented
- Round-robin can be used with either preemptive or cooperative multitasking



life.augmented

What is an OS heap?



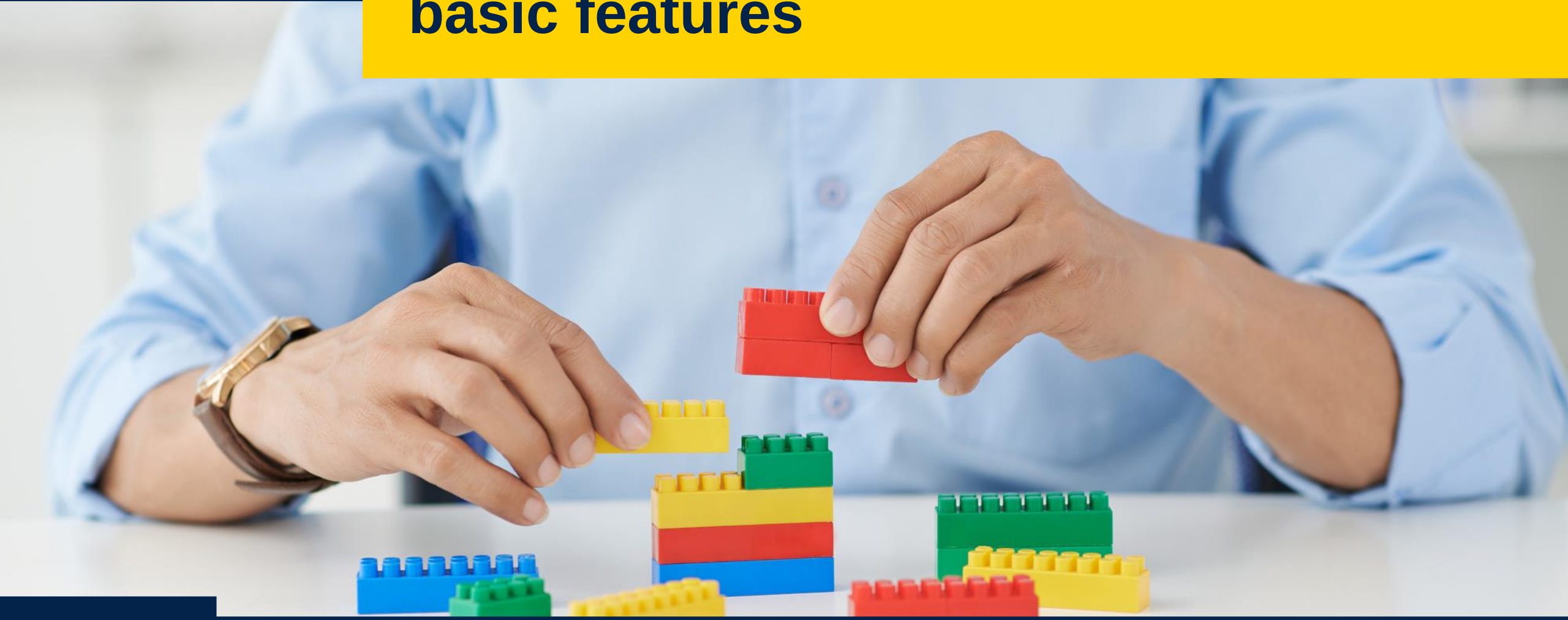
life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS
basic features



life.augmented

About FreeRTOS (1/2)

- Market leading RTOS by Real Time Engineers Ltd.
- Professionally developed with strict quality management
- Commercial versions available: OpenRTOS and SafeRTOS
- Documentation available on www.freertos.org
- Free support through forum (moderated by RTOS author Richard Barry)



About FreeRTOS (2/2)

- FreeRTOS is distributed under MIT open source license An alternative commercial license option is also available.
- FreeRTOS license details available on :
  - http://www.freertos.org/a00114.html
- In the STM32Cube firmware solution FreeRTOS is used as real time operating system through the generic CMSIS-OS wrapping layer provided by ARM. Examples and applications using the FreeRTOS can be directly ported on any other RTOS without modifying the high level APIs, only the CMSIS-OS wrapper has to be changed in this case.

FreeRTOS - main features

- Preemptive or cooperative real-time kernel
- Tiny memory footprint (less than 10kB ROM) and easy scalable
- Includes a tickless mode for low power applications
- Synchronization and inter-task communication using
  - message queues
  - binary and counting semaphores
  - mutexes
  - group events (flags)
- Software timers for tasks scheduling
- Execution trace functionality
- CMSIS-RTOS API port

life.augmented

FreeRTOS - resources used

Core resources:

- System timer (SysTick) – generate system time (time slice)
- Two stack pointers: **MSP**, **PSP**

Interrupt vectors:

- **SVC** – system service call (like SWI in ARM7)
- **PendSV** – pended system call (switching context)
- **SysTick** – System Timer

Flash memory:

- **6-10kB**

RAM memory:

- ~**0.5kB** + task stacks:

life.augmented

System service call (SVC)

- SVC – system service call / supervisor call
- It is an instruction and an exception. Once the svc instruction is executed, SVC IRQ is triggered immediately (unless there is higher priority IRQ active)
- SVC contains an 8bit immediate value what could help to determine which OS service is requested.
- Do not use SVC inside NMI or Hard Fault handler

Pended system call (PendSV)

- PendSV is a priority programmable exception triggered by SW (write to the in ICSR register @0xE000ED04)
  - SCB->ICSR |= (1<<28)
- It is not precise (in contrary to SVC). After set a pending bit, CPU can execute a number of instructions before the exception will start. Usually it is used like a subroutine called i.e. by the system timer in OS

System timer

- It is necessary to trigger a context switching in regular time slots.
- In CortexM architecture 24bit downcounting SysTick is used for this purpose (it can be changed)
- System timer is triggering PendSV SW interrupt to perform context switch.
- In case we are using HAL library it is strongly recommended to change its TimeBase timer from Systick to other timer available (i.e. TIM6)

FreeRTOS source files structure

File / header Directory	role
croutine.c / croutine.h .\Source	Co-routines functions definitions. Efficient in 8 and 16bit architecture. In 32bit architecture usage of tasks is suggested
event_groups.c / event_groups.h .\Source	Effective intertask communication
list.c / list.h .\Source	List implementation used by the scheduler.
queue.c / queue.h / semphr.h .\Source	Semaphores, mutexes functions definitions
stream_buffer.c / stream_buffer.h .\Source	Optimised for single writer and single reader intertask data stream communication (can be useful in dual core architectures)
tasks.c / task.h .\Source	Task functions and utilities definition
timers.c / timers.h .\Source	Software timers functions definitions
FreeRTOS.h .\Source\include	Configuration file which collect whole FreeRTOS sources
FreeRTOSConfig.h	Configuration of FreeRTOS system, system clock and irq parameters configuration
heap_x.c .\Source\portable\MemMang	Memory management functions (allocate and free memory segment, three different approaches in heap_1, heap_2, heap_3 and heap_4 files)
port.c / portmacro.h .\Source\portable\xxx\yyy	Low level functions supporting SysTick timer, context switch, interrupt management on low hw level – strongly depends on the platform (core and sw toolset). Mostly written in assembly. In portmacro.h file there are definitions of portTickType and portBASE_TYPE

FreeRTOS source files structure

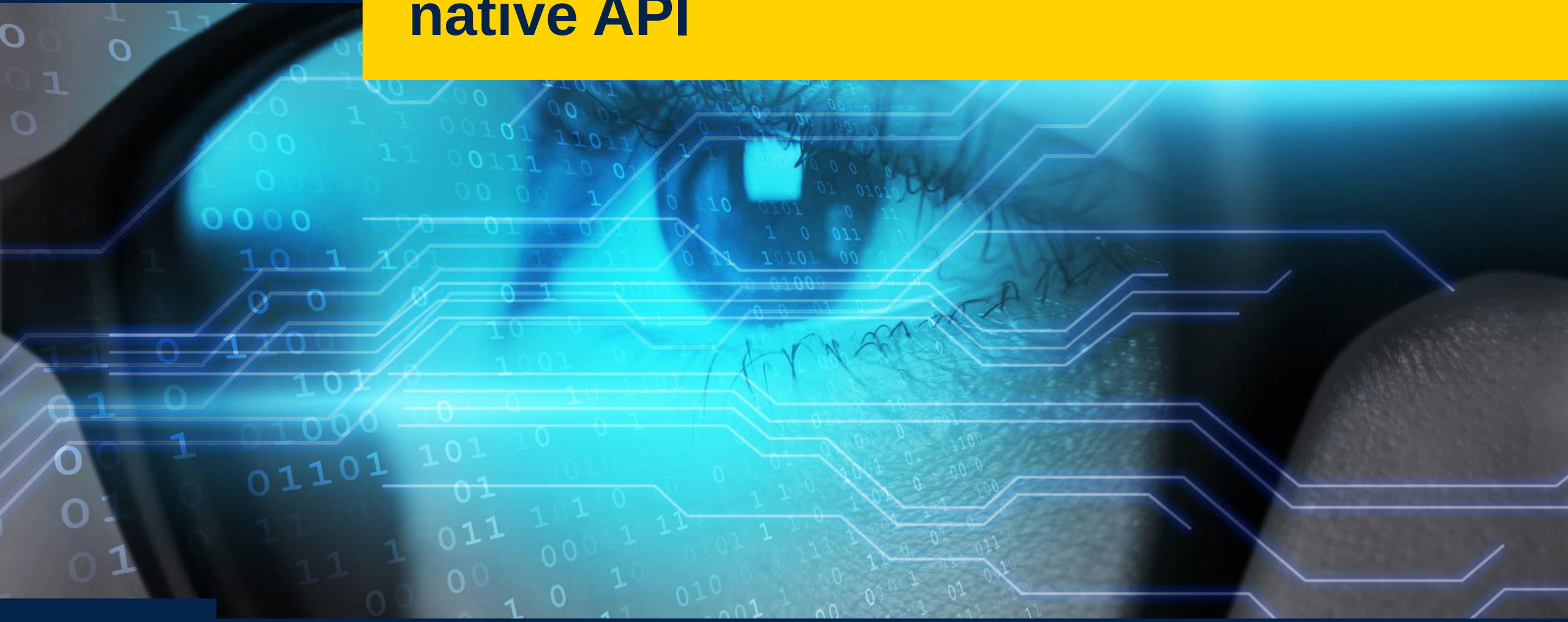
File / header Directory	role
event_groups.c / event_groups.h .\Source	
list.c / list.h .\Source	List implementation used by the scheduler.
queue.c / queue.h / semphr.h .\Source	Semaphores, mutexes functions definitions
tasks.c / task.h .\Source	Task functions and utilities definition
timers.c / timers.h .\Source	Software timers functions definitions
FreeRTOS.h .\Source\include	Configuration file which collect whole FreeRTOS sources
FreeRTOSConfig.h	Configuration of FreeRTOS system, system clock and irq parameters configuration
heap_x.c .\Source\portable\MemMang	Memory management functions (allocate and free memory segment, three different approaches in heap_1, heap_2, heap_3 and heap_4 files)
port.c / portmacro.h .\Source\portable\xxx\yyy	Low level functions supporting SysTick timer, context switch, interrupt management on low hw level – strongly depends on the platform (core and sw toolset). Mostly written in assembly. In portmacro.h file there are definitions of portTickType and portBASE_TYPE

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

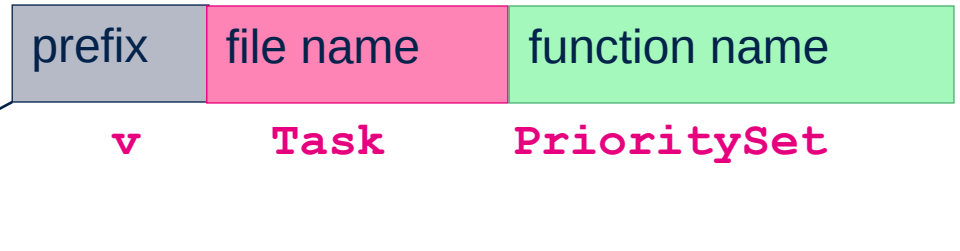
FreeRTOS native API



life.augmented

FreeRTOS API conventions

- Prefixes at variable names:
  - **`c`** – char
  - **`s`** – short
  - **`l`** – long
  - **`x`** – portBASE_TYPE defined in portmacro.h for each platform (in STM32 it is long)
  - **`u`** – unsigned
  - **`p`** - pointer
- Functions name structure (`vTaskPrioritySet()` is taken as example):



  - **`v`** – void
  - **`x`** – returns portBASE_TYPE
  - **`prv`** – private

life.augmented

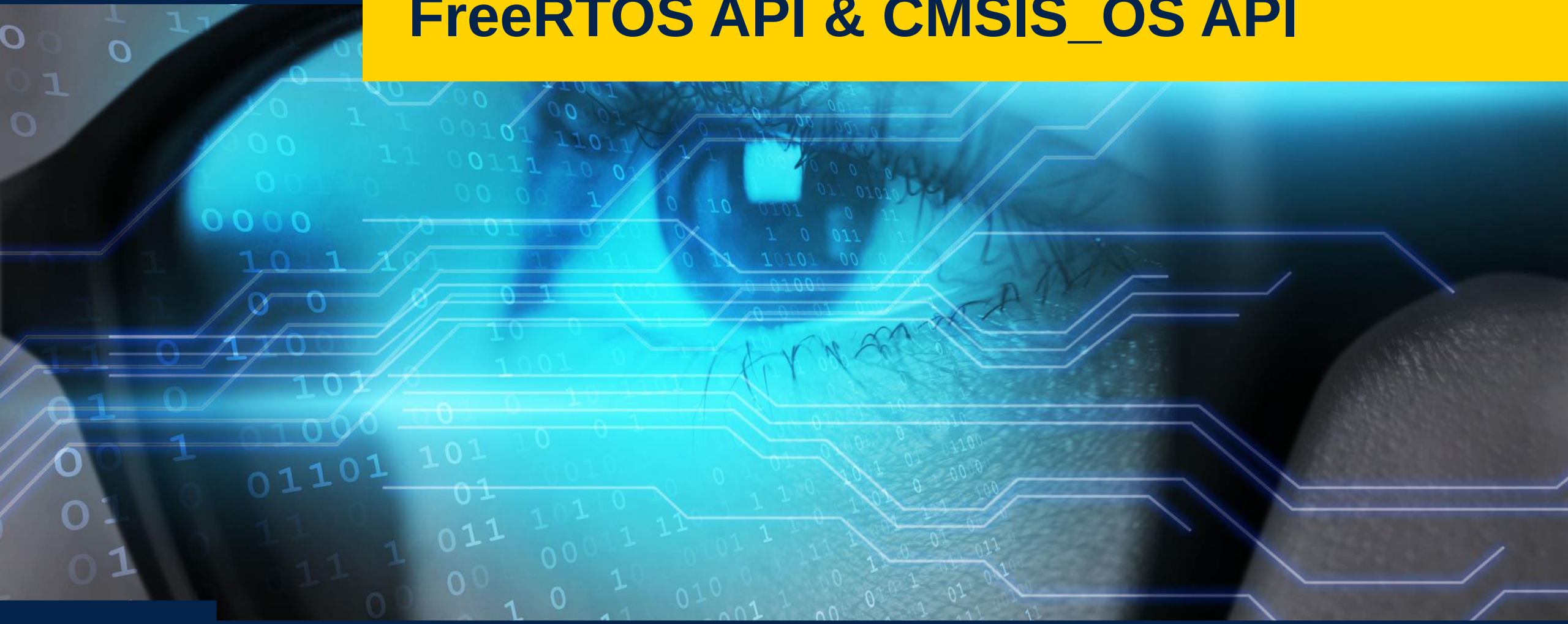
FreeRTOS API conventions - macros

- Prefixes at macros defines their definition location:
  - **port** – (ie. `portMAX_DELAY`) -> portable.h
  - **task** – (ie. `task_ENTER_CRITICAL`) -> task.h
  - **pd** – (ie. `pdTRUE`) -> projdefs.h
  - **config** – (ie. `configUSE_PREEMPTION`) -> FreeRTOSConfig.h
  - **err** – (ie. `errQUEUE_FULL`) -> projdefs.h

- Common macro definitions:
  - **`pdTRUE`** 1
  - **`pdFALSE`** 0
  - **`pdPASS`** 1
  - **`pdFAIL`** 0

life.augmented

FreeRTOS
FreeRTOS API & CMSIS_OS API



life.augmented

FreeRTOS
CMSIS-OS API and FreeRTOS API coexistence

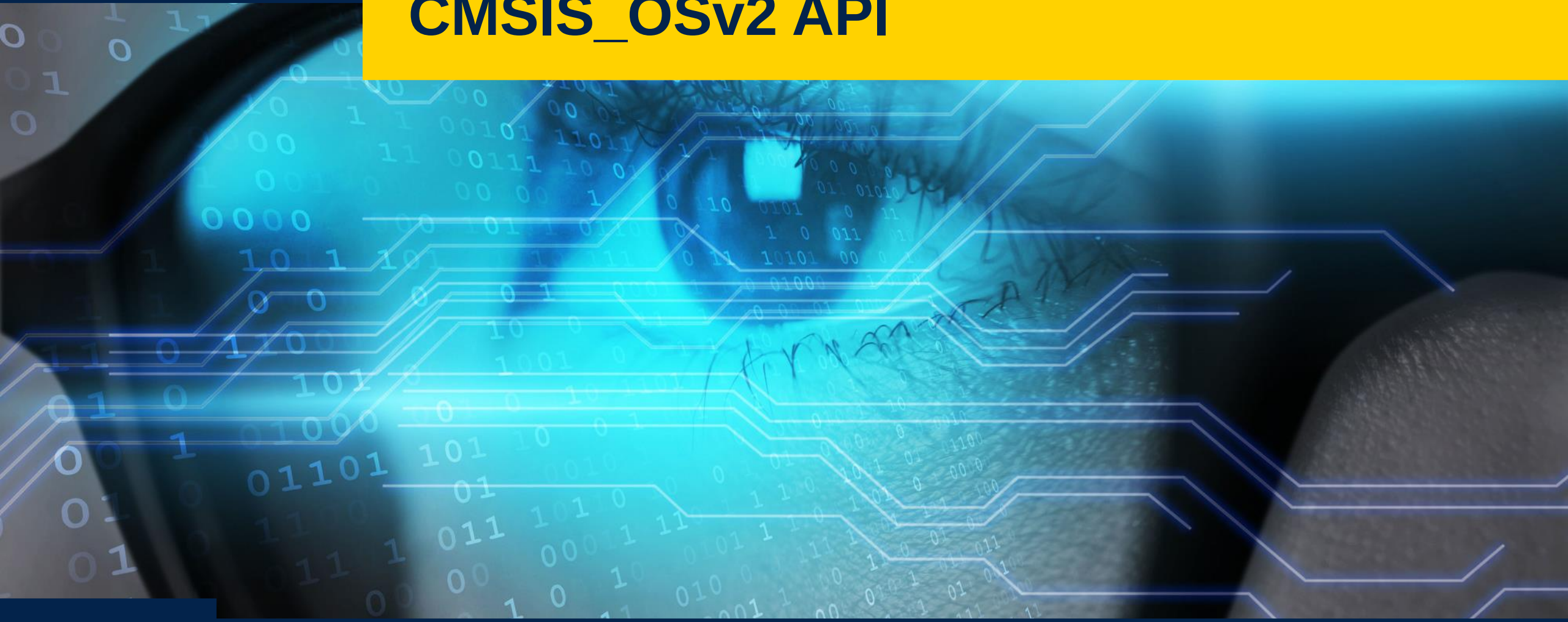
- CMSIS_OS API is a simple additional software layer over FreeRTOS API code
- Not all features of FreeRTOS API are available directly from CMSIS_OS API
- It is possible to use additional FreeRTOS API modules among CMSIS_OS code (like Task Notifications or Event Groups)
- There are big differences in Queues data structures, Task Notification handling (simplification) and priority names (limited to max 7)
- Main OS handlers are just typedef from FreeRTOS to CMSIS (cmsis_os.h file), i.e:
  - typedef TaskHandle_t osThreadId_t;
  - typedef TimerHandle_t osTimerId_t;
  - typedef SemaphoreHandle_t osMutexId_t;
  - typedef SemaphoreHandle_t osSemaphoreId_t;
  - typedef QueueHandle_t osMessageQId_t;

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS
CMSIS_OSv2 API



life.augmented

CMSIS_OS v1.x
CMSIS_OS v2.x

FreeRTOS CMSIS-OS API

- CMSIS-OS API is a generic RTOS interface for Cortex-M processor based devices
- Middleware components using the CMSIS-OS API are RTOS independent, this allows an easy linking to any third-party RTOS
- The CMSIS-OS API defines a minimum feature set including
  - Thread Management
  - Kernel control
  - Semaphore management
  - Message queue and mail queue
  - Memory management
- The STM32Cube comes with an implementation of the CMSIS-RTOS for FreeRTOS.
- For detailed documentation regarding CMSIS-OS refer to:

http://www.keil.com/pack/doc/CMSIS/RTOS/html/index.html **CMSIS_OS v1.x**

http://www.keil.com/pack/doc/CMSIS/RTOS2/html/index.html **CMSIS_OS v2.x**

FreeRTOS
CMSIS-OS API, main data structures

- Most of the functions returns osStatus (v1.x) / osStatus_t (v2.x) value, which allows to check whether the function is completed or there was some issue (cmsis_os.h file)
- Each OS component has its own ID:
  - Tasks: osThreadId (v1.x) / osThreadId_t (v2.x) (mapped to TaskHandle_t within FreeRTOS API)
  - Queues: osMessageQId (v1.x) / osMessageQId_t (v2.x) (mapped to QueueHandle_t within FreeRTOS API)
  - Semaphores: osSemaphoreId (v1.x) / osSemaphoreId_t (v2.x) (mapped to SemaphoreHandle_t within FreeRTOS API)
  - Mutexes: osMutexId (v1.x) / osMutexId_t (v2.x) (mapped to SemaphoreHandle_t within FreeRTOS API)
  - SW timers: osTimerId (v1.x) / osTimerId_t (v2.x) (mapped to TimerHandle_t within FreeRTOS API)
- Delays and timeouts are given in ms:
  - 0 – no delay
  - >0 – delay in ms
  - 0xFFFFFFFF – wait forever (defined in osWaitForever within cmsis_os.h file)

CMSIS_OS API v2.x - osStatus_t return values cmsis_os2.h

Most of the functions returns `osStatus_t` value (fully defined in cmsis_os2.h file)

osStatus	value	description
`osOK`	`0`	Operation completed successfully
`osError`	`-1`	Unspecified OS error: run-time error but no other error message fits.
`osErrorTimeout`	`-2`	Operation not completed within the timeout period.
`osErrorResource`	`-3`	Resource not available
`osErrorParameter`	`-4`	Parameter error
`osErrorNoMemory`	`-5`	System is out of memory: it was impossible to allocate or reserve memory for the operation.
`osErrorISR`	`-6`	Not allowed in ISR context: the function cannot be called from IRQ
`osStatusReserved`	`0x7FFFFFFF`	prevents from enum down-size compiler optimization

Task states (CMSIS_OS v2.x) cmsis_os2.h

- Tasks states are stored within `osThreadState_t` enum

State name	value	comment
`osThreadInactive`	`0`	Inactive
`osThreadReady`	`1`	READY
`osThreadRunning`	`2`	RUNNING
`osThreadBlocked`	`3`	BLOCKED, SUSPENDED
`osThreadTermimated`	`4`	TERMINATED
`osThreadError`	`-1`	Error
`osThreadReserved`	`0x7FFFFFFF`	Prevents enum down-size compiler optimization

Thread attributes (CMSIS_OS v2.x) cmsis_os2.h

Thread parameters are grouped within lists at `osThreadAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
const char * name	(CubeMX)	Name of the thread
uint32_t attr_bits		Attribute bits
void * cb_mem		Pointer to thread control block memory area
uint32_t cb_size		Thread control block (TCB) size
void * stack_mem		Pointer to thread stack memory area
uint32_t stack_size	(CubeMX),configMINIMAL_STACK_SIZE	Thread stack size (words)
osPriority_t priority	(CubeMX),osPriorityNormal	Initial thread priority (default: osPriorityNormal)
TZ_ModuleId_t tz_module		TrustZone module identifier
uint32_t reserved	0	For future use (must be set to 0)

Task priorities (CMSIS_OS v2.x) cmsis_os2.h

- Tasks priorities can be set within `osPriority_t` enum

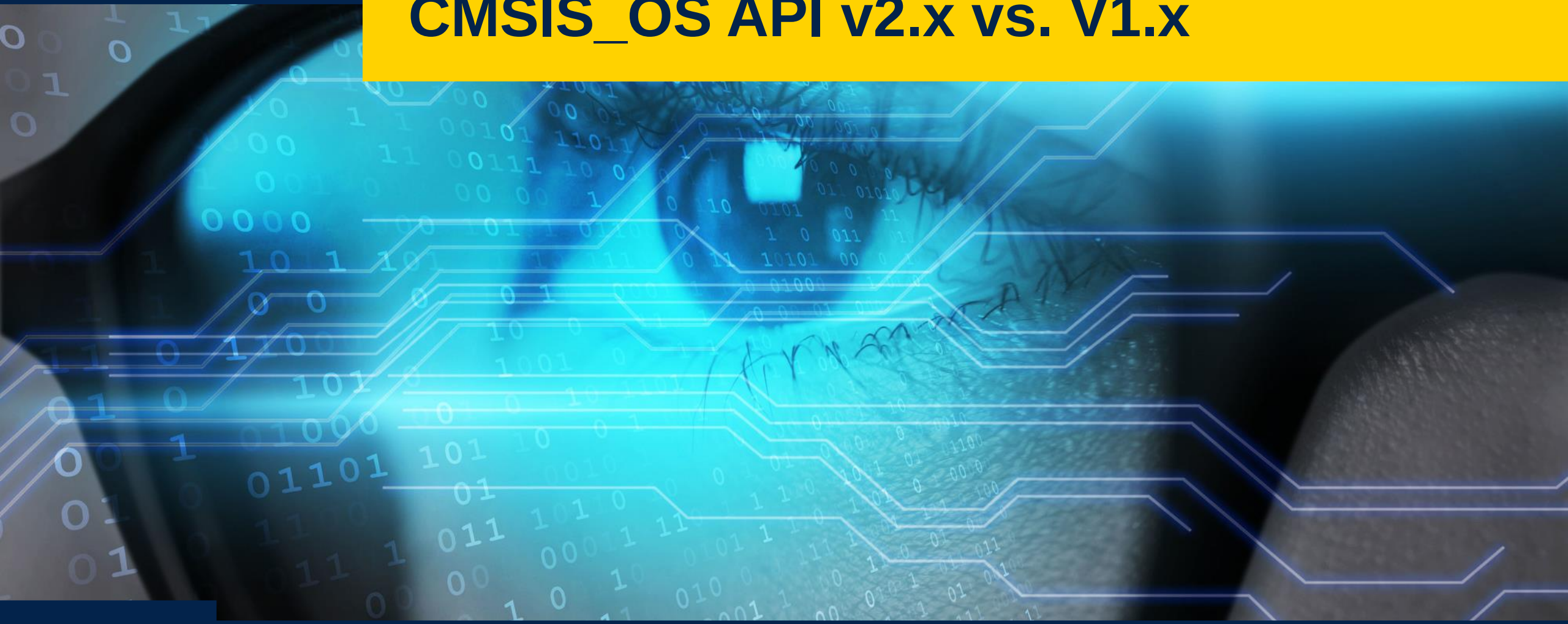
Priority name	value	comment
osPriorityNone	0	No priority (not initialized)
osPriorityIdle	1	Reserved for Idle thread
osPriorityLow	8	priority: low
osPriorityLow1..7	8+1..7	priority: low + 1..7
osPriorityBelowNormal	16	priority: below normal
osPriorityBelowNormal1..7	16+1..7	priority: below normal + 1..7
osPriorityNormal	24	priority: normal
osPriorityNormal1..7	24+1..7	priority: normal + 1..7
osPriorityAboveNormal	32	priority: above normal
osPriorityAboveNormal1..7	32+1..7	priority: above normal + 1..7
osPriorityHigh	40	priority: high
osPriorityHigh1..7	40+1..7	priority: high + 1..7
osPriorityRealtime	48	priority: realtime
osPriorityRealtime1..7	48+1..7	priority: realtime + 1..7
osPriorityISR	56	reserved for ISR deferred thread
osPriorityError	-1	system cannot determine priority or illegal priority
osPriorityReserved	0x7FFFFFFF	prevents enum down-size compiler optimization

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS
CMSIS_OS API v2.x vs. V1.x



life.augmented

FreeRTOS CMSIS-OS API v1.x -> v2.1

- OS objects creation without macros (dynamic creation and resource allocation):
  - added: osXxxxNew functions which replace osXxxxCreate
  - added: osXxxxAttr_t structures
  - deprecated: osXxxxCreate functions, osXxxxDef_t structures
  - deprecated: osXxxxDef and osXxxx macros
- osStatus codes simplified and renamed to osStatus_t
- osEvent return structure deprecated
- Updated Thread and Event Flags:
  - changed flags parameter and return type from int32_t to uint32_t

https://www.keil.com/pack/doc/CMSIS/RTOS2/html/os2MigrationFunctions.html

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Kernel:

- added: osKernelInfo_t and osKernelGetInfo
- added: osKernelState_t and osKernelGetState (replaces osKernelRunning)
- added: osKernelLock, osKernelUnlock
- added: osKernelSuspend, osKernelResume
- added: osKernelGetTickCount, osKernelGetTickFreq
- added: osKernelRestoreLock
- renamed osKernelSysTick to osKernelGetSysTimerCount
- replaced osKernelSysTickFrequency with osKernelGetSysTimerFreq
- deprecated osKernelSysTickMicroSec

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Thread:

- extended number of thread priorities
- renamed osPriority to osPriority_t
- replaced osThreadCreate with osThreadNew
- added: osThreadGetName
- added: osThreadState_t and osThreadGetState
- added: osThreadGetStackSize, osThreadGetStackSpace
- added: osThreadSuspend, osThreadResume
- added: osThreadJoin, osThreadDetach, osThreadExit
- added: osThreadGetCount, osThreadEnumerate
- added: Thread Flags (moved from Signals)

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Signals:

- renamed osSignals to osThreadFlags (moved to Thread Flags)
- changed return value of Set/Clear/Wait functions
- Clear function limited to current running thread
- extended Wait function (options)
- added: osThreadFlagsGet

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Event Flags:

- added new independent object for handling Event Flags

Delay and Wait functions:

- added: osDelayUntil
- deprecated: osWait

SW Timer:

- replaced osTimerCreate with osTimerNew
- added: osTimerGetName, osTimerIsRunning

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Mutex:

- extended: attributes (Recursive, Priority Inherit, Robust)
- replaced osMutexCreate with osMutexNew
- renamed osMutexWait to osMutexAcquire
- added: osMutexGetName, osMutexGetOwner

Semaphore:

- extended: maximum and initial token count
- replaced osSemaphoreCreate with osSemaphoreNew
- renamed osSemaphoreWait to osSemaphoreAcquire (changed return value)
- added: osSemaphoreGetName, osSemaphoreGetCount

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Memory Pool:

- using osMemoryPool prefix instead of osPool
- replaced osPoolCreate with osMemoryPoolNew
- extended osMemoryPoolAlloc (timeout)
- added: osMemoryPoolGetName
- added: osMemoryPoolGetCapacity, osMemoryPoolGetBlockSize
- added: osMemoryPoolGetCount, osMemoryPoolGetSpace
- added: osMemoryPoolDelete
- deprecated: osPoolCAlloc

FreeRTOS
CMSIS-OS API v1.x -> v2.1

Message Queue:

- extended: fixed size message instead of a single 32-bit value
- using osMessageQueue prefix instead of osMessage
- replaced osMessageCreate with osMessageQueueNew
- updated: osMessageQueuePut, osMessageQueueGet
- added: osMessageQueueGetName
- added: osMessageQueueGetCapacity, osMessageQueueGetMsgSize
- added: osMessageQueueGetCount, osMessageQueueGetSpace
- added: osMessageQueueReset, osMessageQueueDelete

Mail Queue:

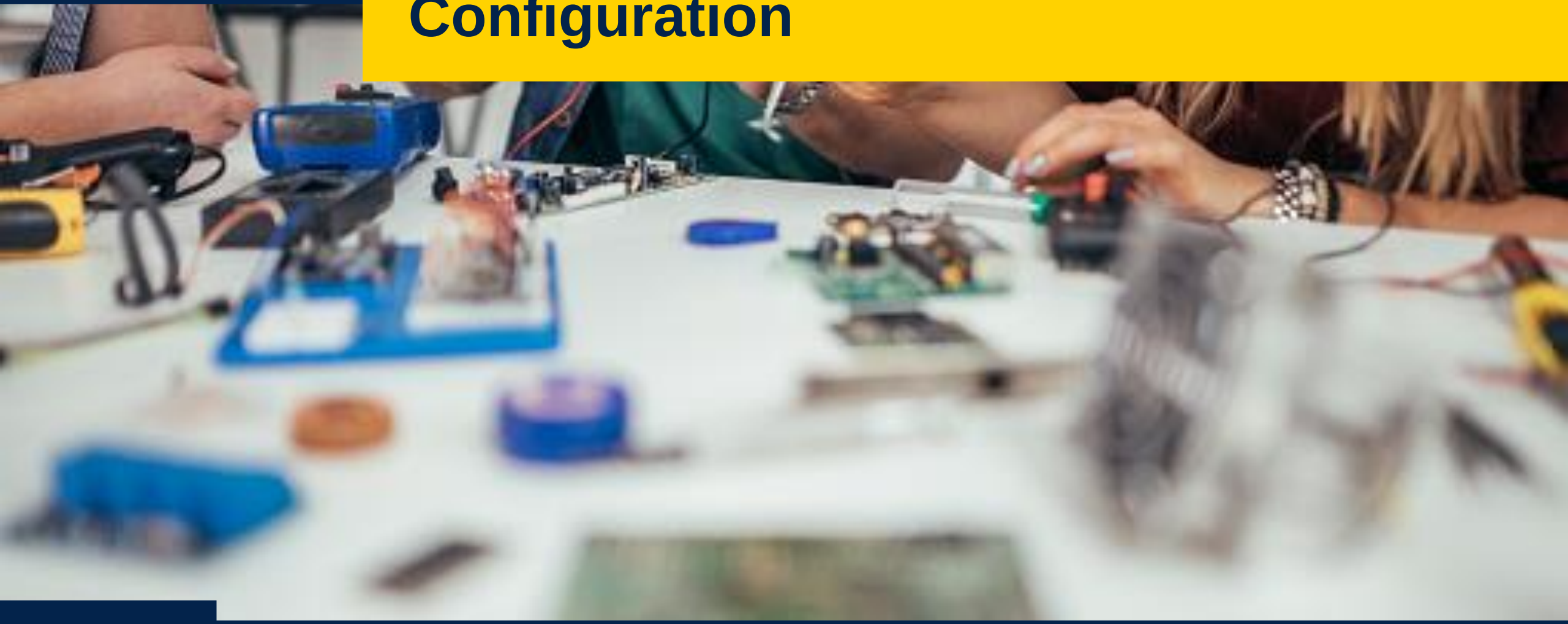
- deprecated (superseded by extended Message Queue functionality)

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

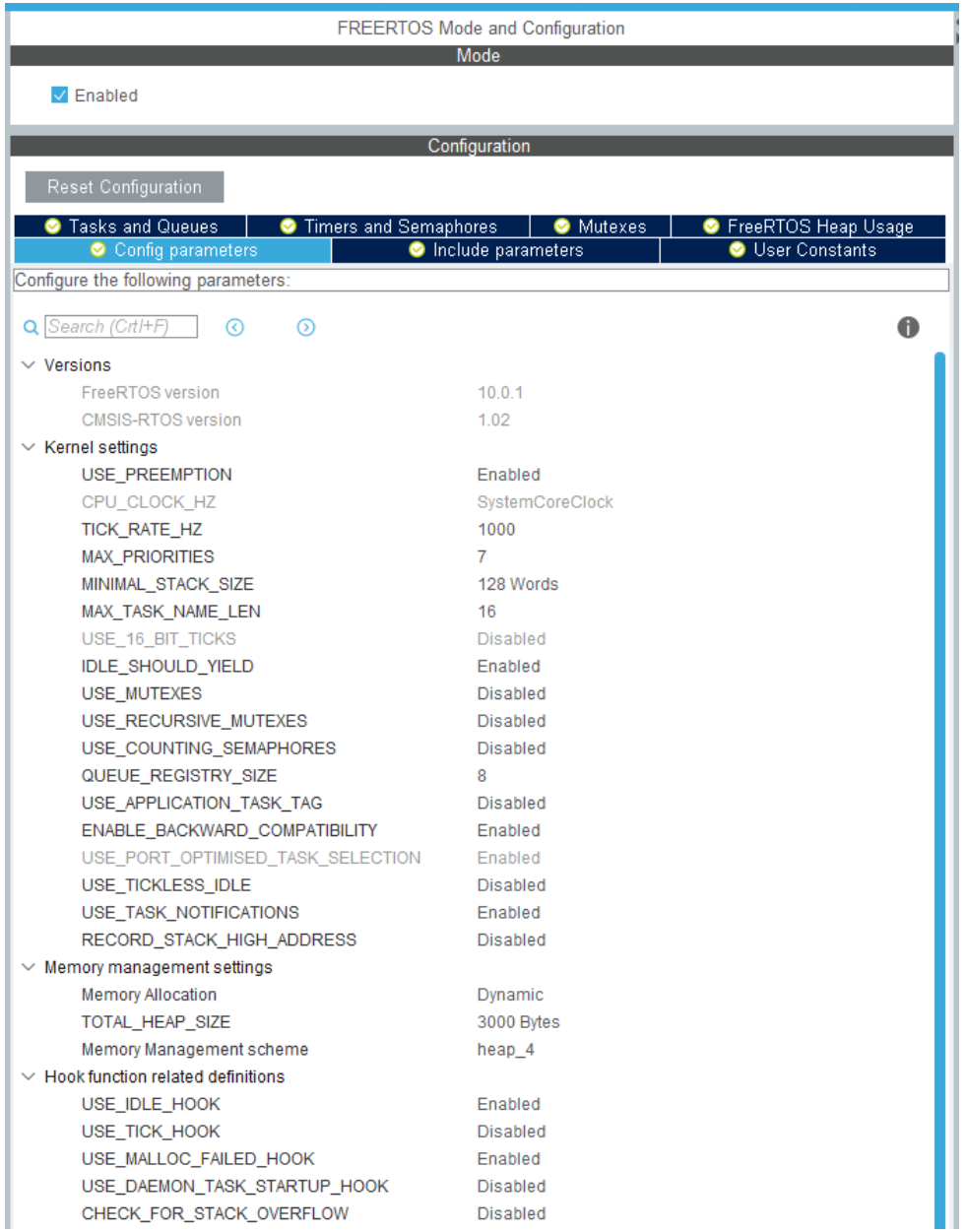
FreeRTOS Configuration



life.augmented

FreeRTOS configuration in STM32CubeMX

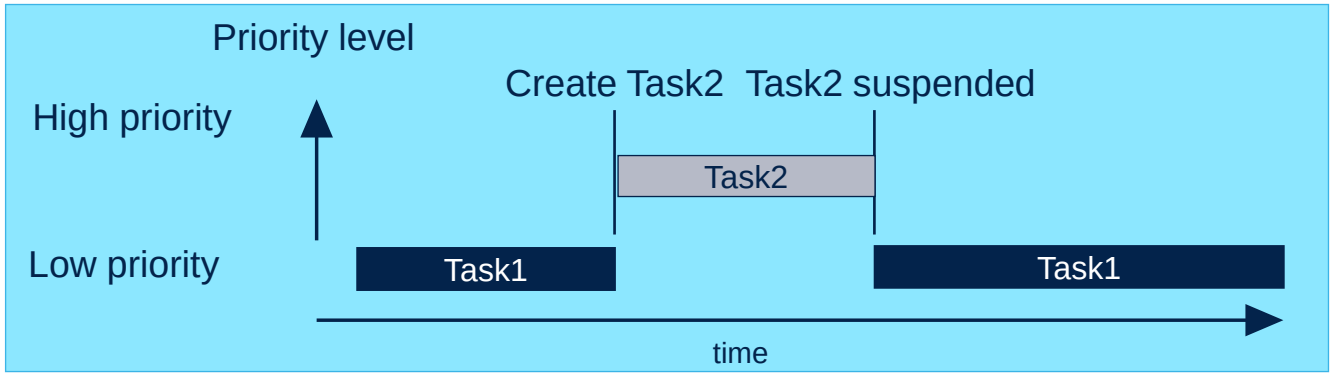
- Config parameters tab
  - Kernel settings
  - RTOS components settings
  - Memory setup
- Include parameters tab
  - Include some additional functions, not necessary for FreeRTOS run
- Tasks and Queues tab
  - Creation of tasks and queues
- Timers and Semaphores tab
  - Creation of timers and semaphores (binary, counting)
- Mutexes tab
  - Creation of mutexes



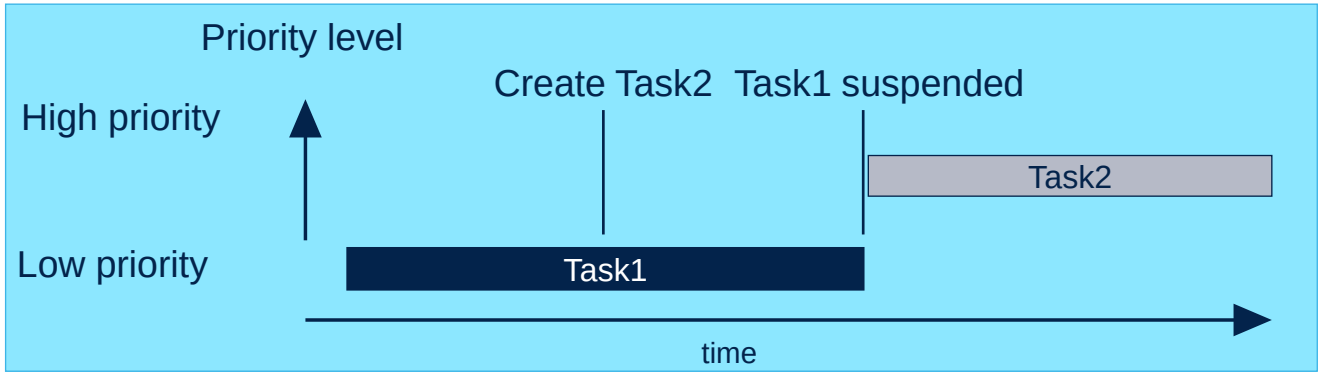
life.augmented

Kernel settings

- Use preemption
  - If enabled use pre-emptive scheduling



  - If disabled use co-operative scheduling



Kernel settings	
USE_PREEMPTION	Enabled
CPU_CLOCK_HZ	SystemCoreClock
TICK_RATE_HZ	1000
MAX_PRIORITIES	7
MINIMAL_STACK_SIZE	128 Words
MAX_TASK_NAME_LEN	16
USE_16_BIT_TICKS	Disabled
IDLE_SHOULD_YIELD	Enabled
USE_MUTEXES	Disabled
USE_RECURSIVE_MUTEXES	Disabled
USE_COUNTING_SEMAPHORES	Disabled
QUEUE_REGISTRY_SIZE	8
USE_APPLICATION_TASK_TAG	Disabled
ENABLE_BACKWARD_COMPATIBILITY	Enabled
USE_PORT_OPTIMISED_TASK_SELECTION	Enabled
USE_TICKLESS_IDLE	Disabled
USE_TASK_NOTIFICATIONS	Enabled
RECORD_STACK_HIGH_ADDRESS	Disabled
Memory management settings	
Memory Allocation	Dynamic
TOTAL_HEAP_SIZE	3000 Bytes
Memory Management scheme	heap_4
Hook function related definitions	
USE_IDLE_HOOK	Enabled
USE_TICK_HOOK	Disabled

life.augmented

FreeRTOS main configuration options

- Configuration options are declared in file FreeRTOSConfig.h
- Important configuration options are:

Config option	Description
configUSE_PREEMPTION	Enables Preemption
configCPU_CLOCK_HZ	CPU clock frequency in Hz
configTICK_RATE_HZ	Tick rate in Hz
configMAX_PRIORITIES	Maximum task priority
configTOTAL_HEAP_SIZE	Total heap size for dynamic allocation (**in bytes**)
configLIBRARY_LOWEST_INTERRUPT_PRIORITY	Lowest interrupt priority
configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY	Highest thread safe interrupt priority (higher priorities are lower numeric value)

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS memory allocation schemes



life.augmented

FreeRTOS dynamic memory management

- FreeRTOS manages own heap for:
  - Tasks
  - Queues
  - Semaphores
  - Mutexes
  - Dynamic memory allocation
- It is possible to select type of memory allocation

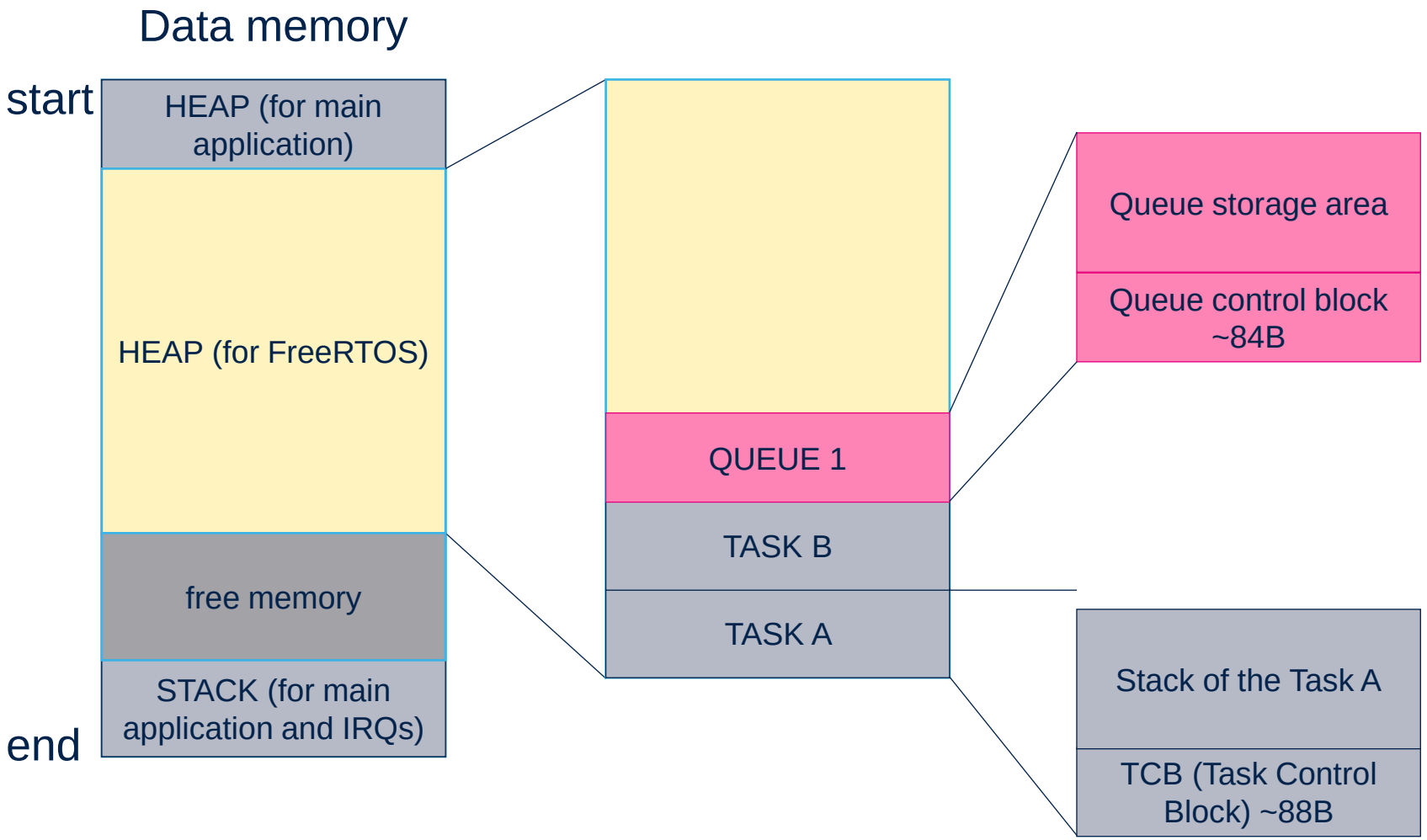


Total heap size for FreeRTOS

How memory is allocated and unalocated

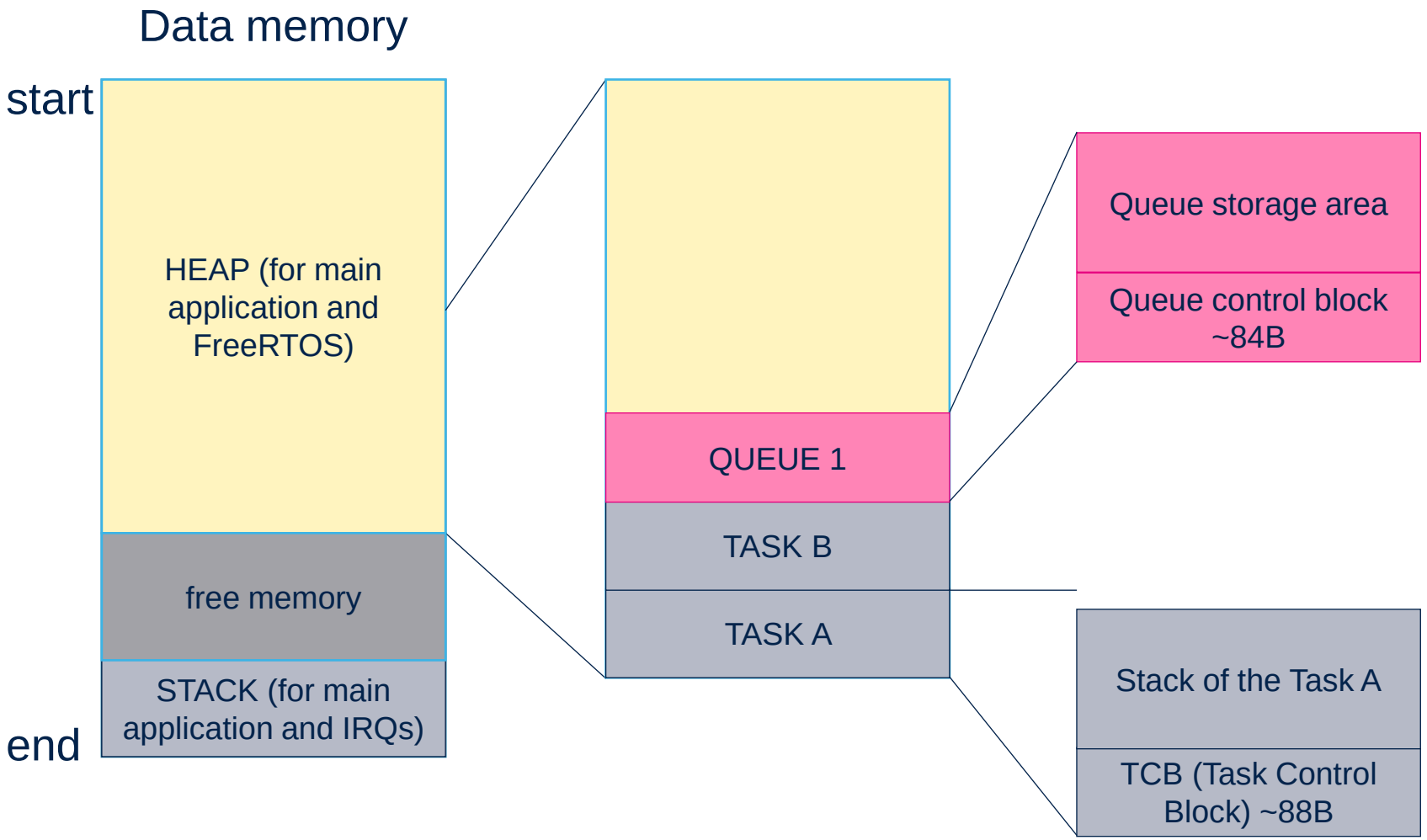
life.augmented

FreeRTOS in STM32 memory management (except heap_3.c model)



life.augmented

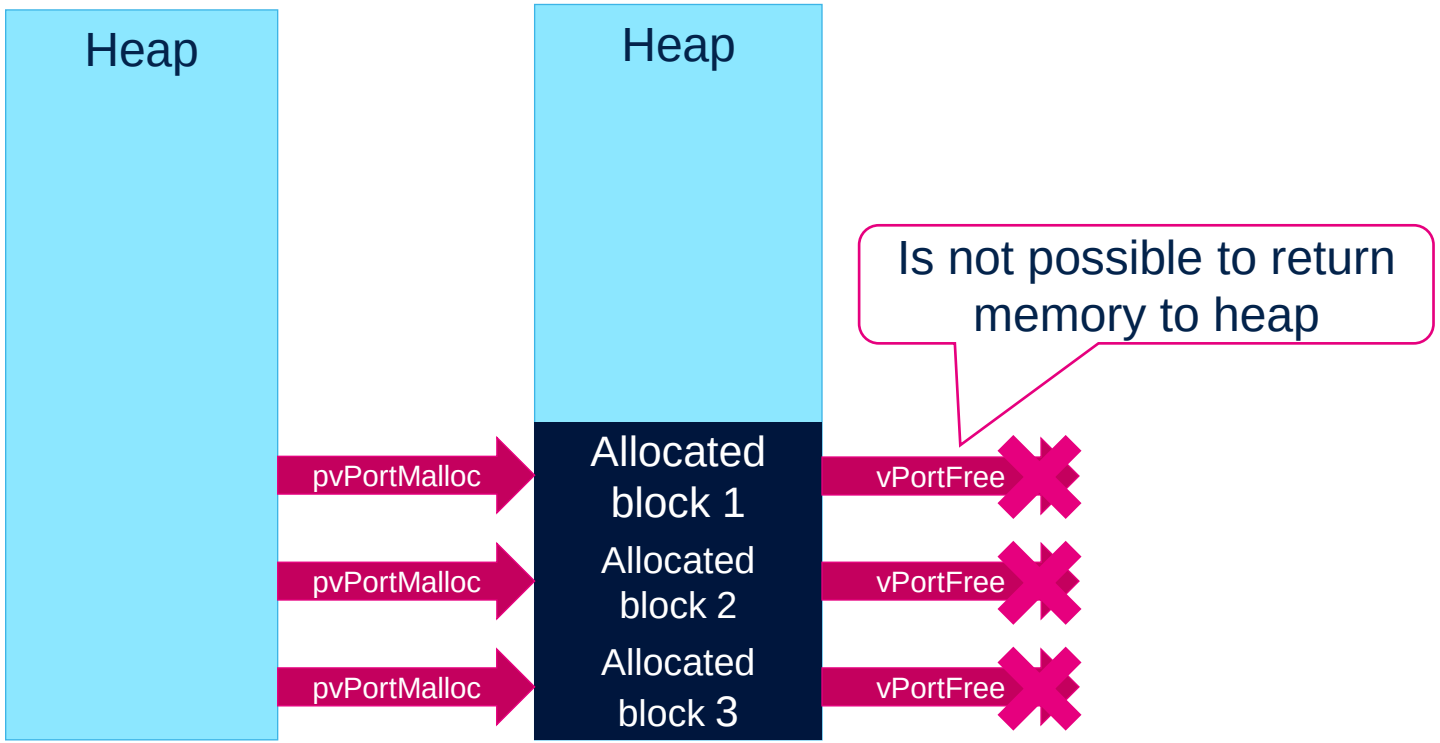
FreeRTOS in STM32 memory management (heap_3.c model)



life.augmented

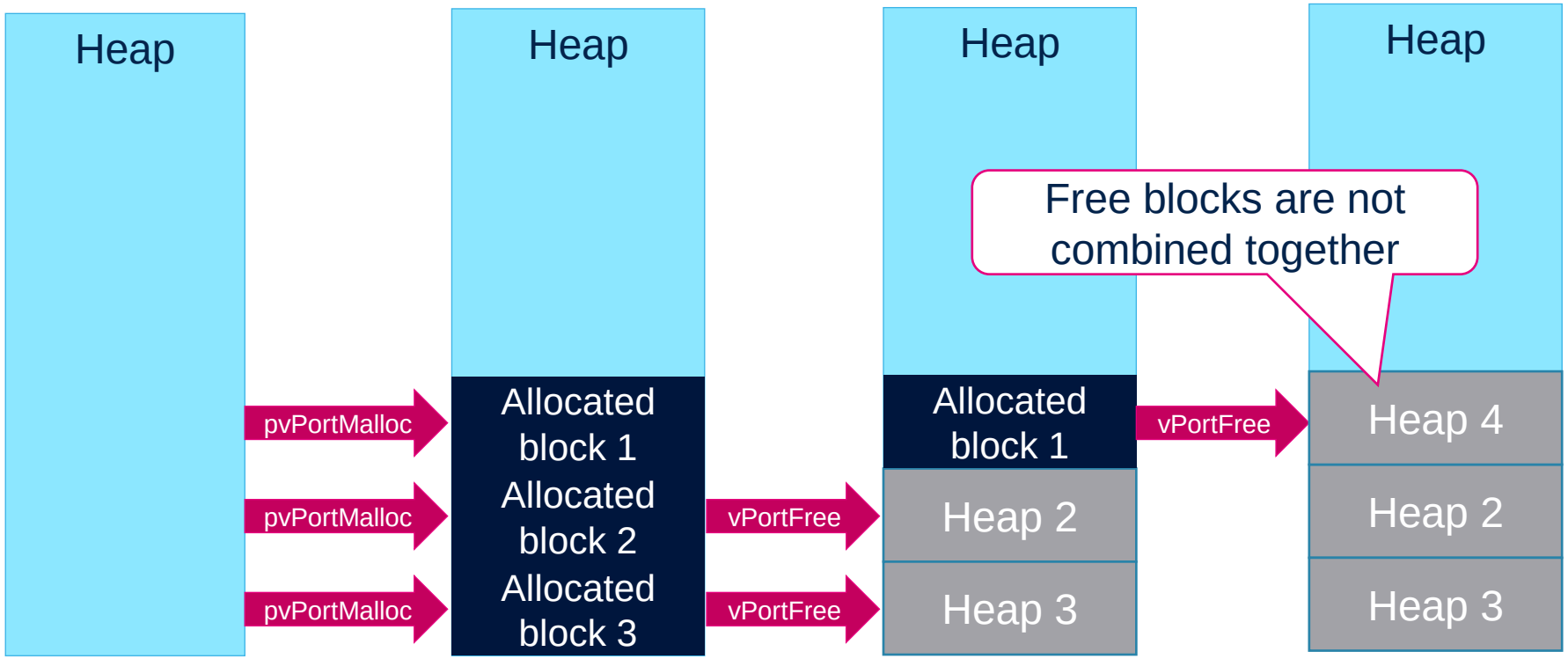
FreeRTOS dynamic memory management

- Heap_1.c
  - Uses first fit algorithm to allocate memory. Simplest allocation method (deterministic), but does not allow freeing of allocated memory => could be interesting when no memory freeing is necessary



FreeRTOS dynamic memory management

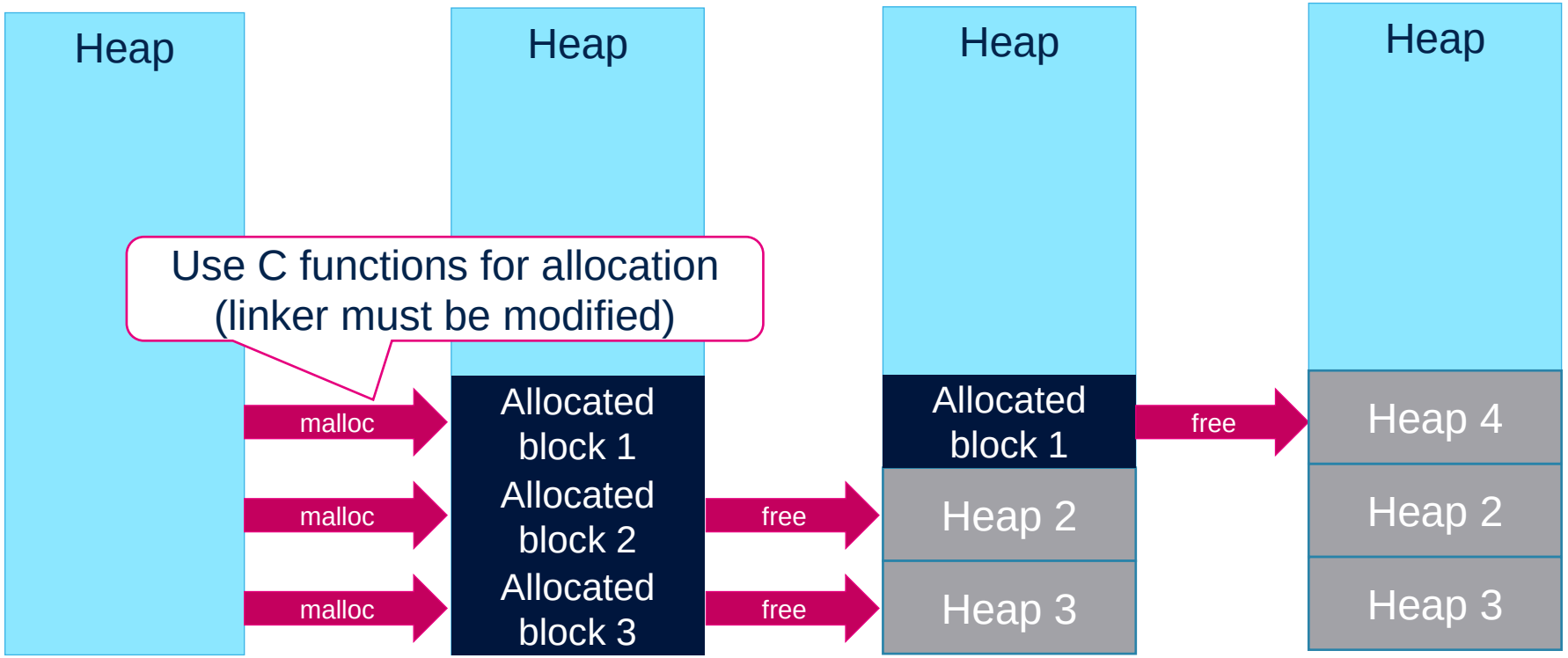
- Heap_2.c
  - Not recommended to new projects. Kept due to backward compatibility.
  - Implements the best fit algorithm for allocation
  - Allows memory free() operation but doesn't combine adjacent free blocks
    => risk of fragmentation



life.augmented

FreeRTOS dynamic memory management

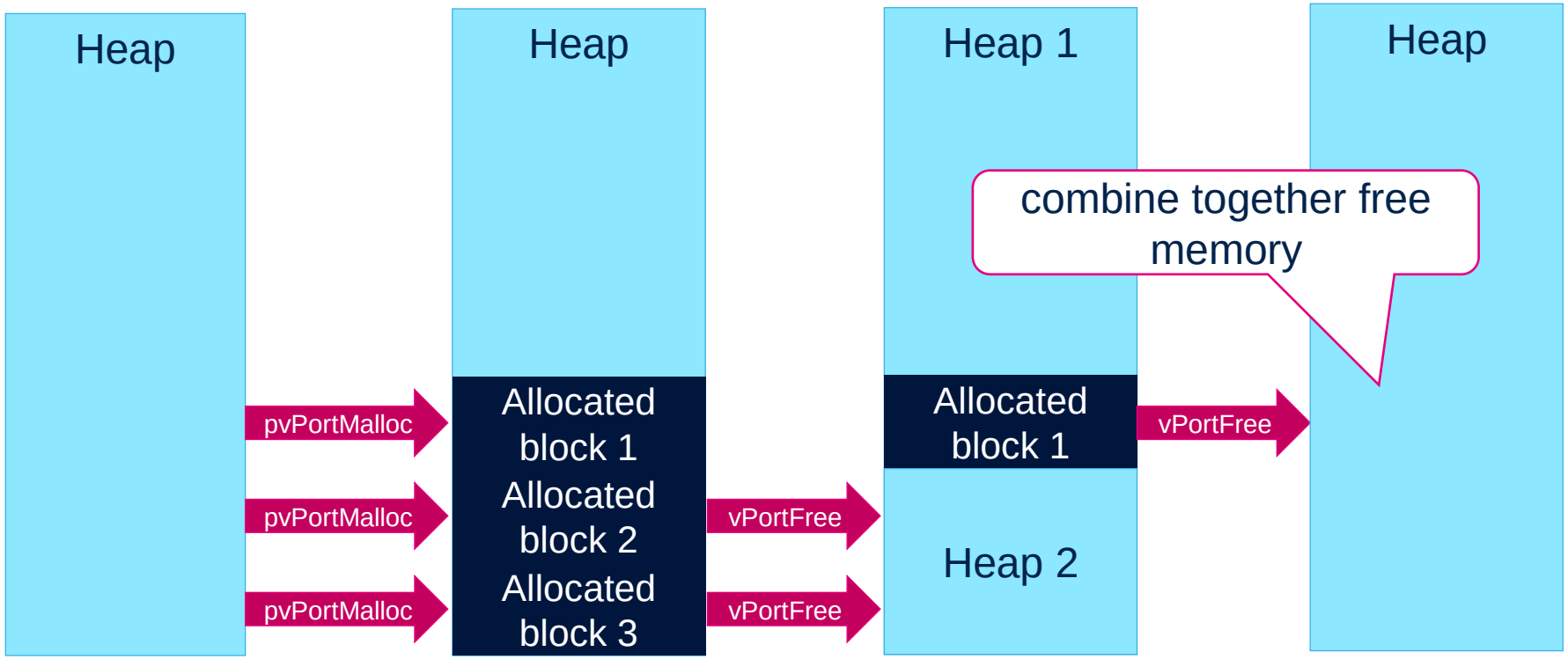
- Heap_3.c
  - Implements simple wrapper for standard C library malloc() and free(); wrapper makes these functions thread safe, but makes code increase and not deterministic
  - It uses linker heap region.
  - `configTOTAL_HEAP_SIZE` setting has no effect when this model is used.



life.augmented

FreeRTOS dynamic memory management

- Heap_4.c (1/2)
  - Uses first fit algorithm to allocate memory. It is able to combine adjacent free memory blocks into a single block
    => this model is used in STM32Cube examples



life.augmented

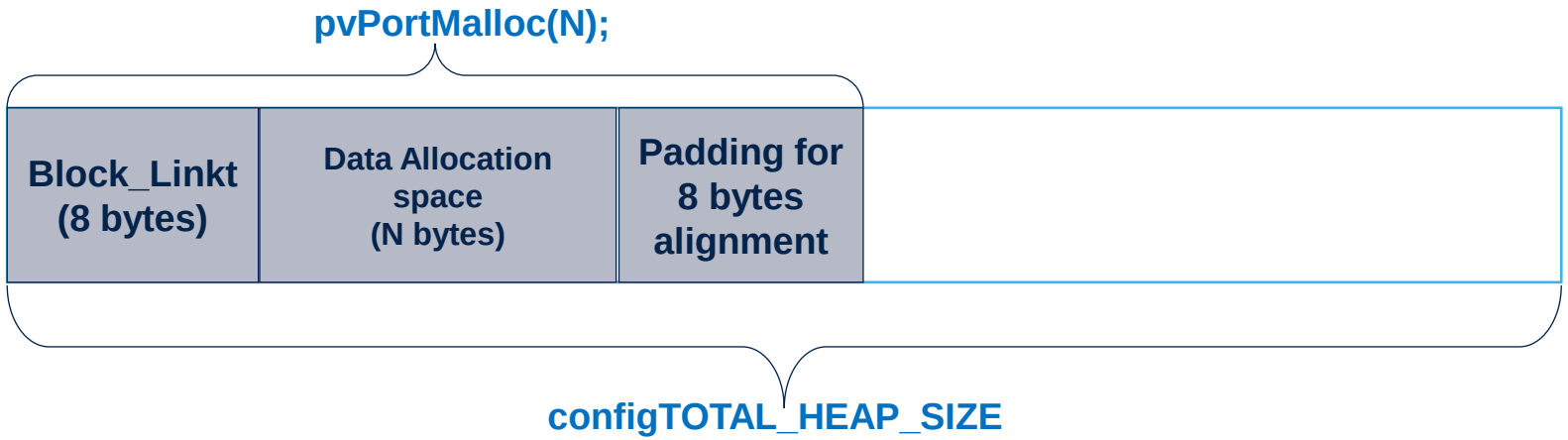
FreeRTOS dynamic memory management

- Heap_4.c (2/2) – place the heap in specific location
  - The memory array used by heap_4 is declared within heap_4.c file and its start address is configured by the linker automatically.
  - To use your own declaration `configAPPLICATION_ALLOCATED_HEAP` must be set to 1 (within FreeRTOSConfig.h file) and the array must be declared within user code with selected start address and size specified by `configTOTAL_HEAP_SIZE`.
  - Memory array used by heap_4 is specified as:

```
uint8_t ucHeap[configTOTAL_HEAP_SIZE];
```

FreeRTOS dynamic memory management

- Using heap_4.c : heap is organized as a linked list: for better efficiency when dynamically allocating/Freeing memory.
- As consequence when allocating “N” bytes in the heap memory using “pvPortMalloc” API it consumes:
  - Sizeof (BlockLink_t) (structure of the heap linked list) : 8 bytes.
  - Data to be allocated itself : N bytes.
  - Add padding to total allocated size (N + 8) to be 8 bytes aligned :
    - Example if trying to allocate 52 Bytes : it consumes from the heap : 52 + 8 = 60 bytes aligned to 8 bytes it gives 64 bytes consumed from the heap.

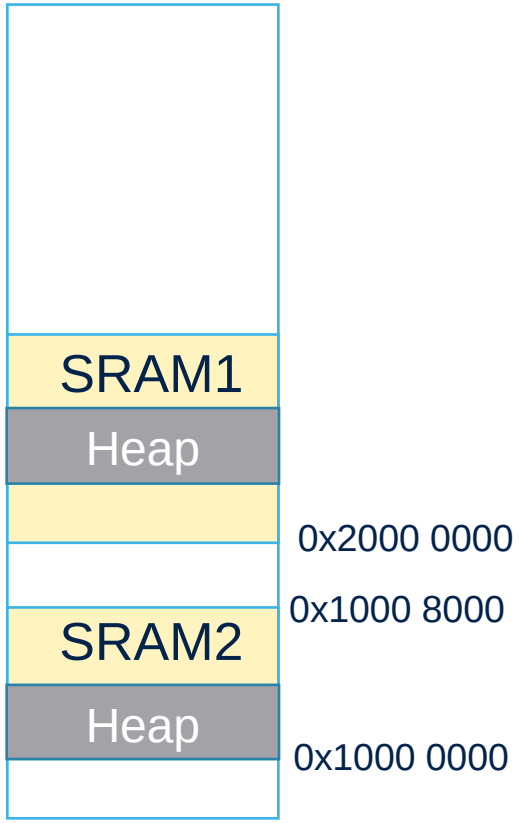


FreeRTOS dynamic memory management

- Heap_5.c (1/2)
  - Fit algorithm able to combine adjacent free memory blocks into a single block using the same algorithms like in heap_4, but supporting different memory regions (i.e. SRAM1, SRAM2) being not in linear memory space
  - It is the only memory allocation scheme that must be explicitly initialized before any OS object can be created (before first call of `pvPortMalloc()`).
  - To inialize this scheme `vPortDefineHeapRegions()` function should be called.
  - It specifies start address and size od each separate memory area.
  - An example for STM32L476 device with SRAM1 and SRAM2 areas is on the next slide

FreeRTOS dynamic memory management

- Heap_5.c (2/2)
  - An example for STM32L476 device with SRAM1 and SRAM2 areas.:




```
#define SRAM1_OS_START (uint8_t *)0x2000 1000
#define SRAM1_OS_SIZE  0x0800 //2kB
#define SRAM2_OS_START (uint8_t *)0x1000 0000
#define SRAM2_OS_SIZE  0x1000 //4kB

Const HeapRegion_t xHeapRegions[] =
{
 {SRAM2_OS_START, SRAM2_OS_SIZE},
 {SRAM1_OS_START, SRAM1_OS_SIZE},
 {NULL,0} /*terminates the array*/
}

/*before call of any OS create function*/
vPortDefineHeapRegions(HeapRegions);
```

Lower address appears in the array first.

life.augmented

FreeRTOS
dynamic memory management - details

- `pvPortMalloc()` (heap1, heap2, heap4, heap5)
  - `vTaskSuspedAll()` – freeze of all tasks
  - Initialization of OS heap in case of first call to malloc
  - Check if the requested block size is not bigger than heap top is set
  - Allocating the memory with proper alignment
  - `vTaskResumeAll()` – unfreeze the tasks
  - In case of memory error execute `vApplicationMallocFailedHook()`
- `pvPortFree()` (heap2, heap4, heap5)
  - `vTaskSuspedAll()` – freeze of all tasks
  - Add memory segment to available memory (pointer + memory segment size)
  - `vTaskResumeAll()` – unfreeze the tasks

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS memory management hints



life.augmented

FreeRTOS Heap

- FreeRTOS uses a region of memory called Heap (into the RAM) to allocate memory for tasks, queues, timers , semaphores, mutexes and when dynamically creating variables. FreeRTOS heap is different than the system heap defined at the compiler level.
- When FreeRTOS requires RAM instead of calling the standard malloc it calls `PvPortMalloc()`. When it needs to free memory it calls `PvPortFree()` instead of the standard free().
- FreeRTOS offers several heap management schemes that range in complexity and features. It includes five sample memory allocation implementations, each of which are described in the following link :
  - http://www.freertos.org/a00111.html
- The total amount of available heap space is set by `configTOTAL_HEAP_SIZE` which is defined in FreeRTOSConfig.h.
- The `xPortGetFreeHeapSize()` API function returns the total amount of heap space that remains unallocated (allowing the `configTOTAL_HEAP_SIZE` setting to be optimized). The total amount of heap space that remains unallocated is also available with `xFreeBytesRemaining` variable for heap management schemes 2 to 5.

Tasks - Heap usage 1/2

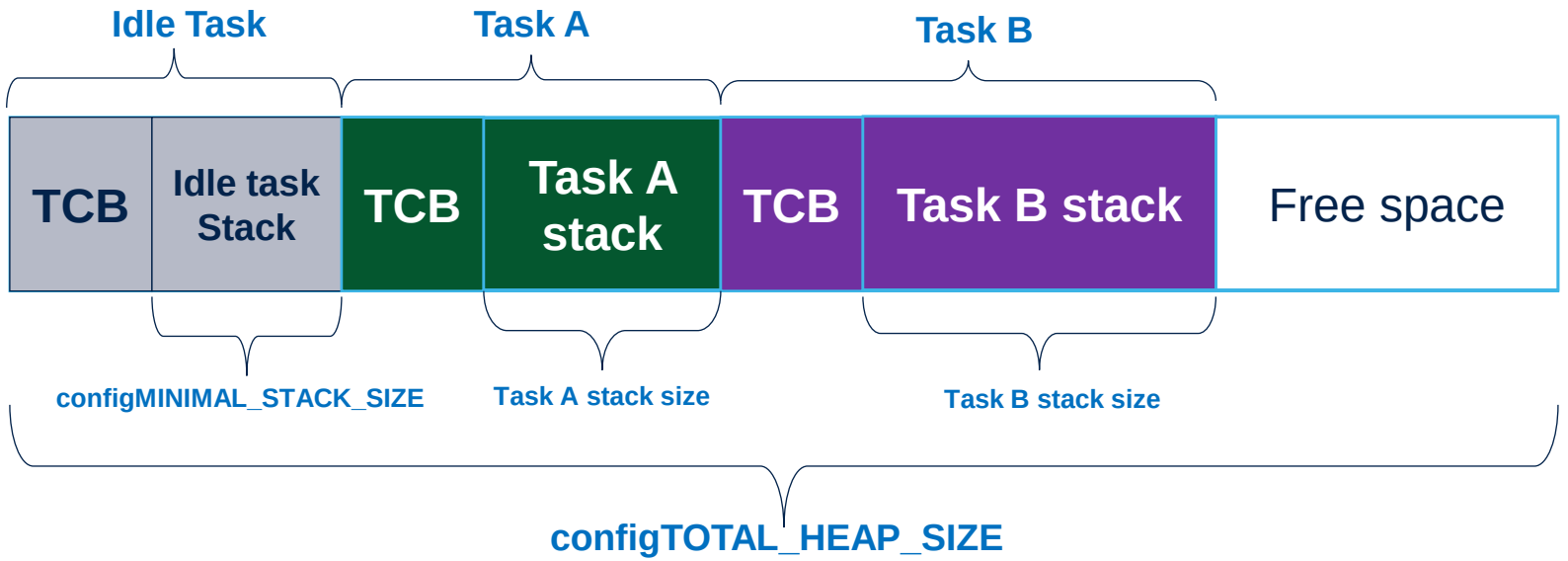
- Each created task (including the idle task) requires a Task Control Block (TCB) and a stack that are allocated in the heap.
- The TCB size in bytes depends of the options enabled in the FreeRTOSConfig.h.
  - With minimum configuration the TCB size is 24 words i.e 96 bytes.
  - if `configUSE_TASK_NOTIFICATIONS` enabled add 8 bytes (2 words)
  - if `configUSE_TRACE_FACILITY` enabled add 8 bytes (2 words)
  - if `configUSE_MUTEXES` enabled add 8 bytes (2 words).
- The task stack size is passed as argument when creating at task. The task stack size is defined in words of 32 bits not in bytes.

```
osThreadDef(Task_A, Task_A_Function, osPriorityNormal, 0, stacksize );
```

- FreeRTOS requires to allocate in the heap for each task :
  - number of bytes = TCB_size + (4 x task stack size)
- `configMINIMAL_STACK_SIZE` defines the minimum stack size that can be used in words. the idle task stack size takes automatically this value

Tasks - Heap usage 2/2

- The necessary task stack size can be fine-tuned using the API `uxTaskGetStackHighWaterMark()` as follow:
  - Use an initial large stack size allowing the task to run without issue (example 4KB)
  - The API `uxTaskGetStackHighWaterMark()` returns the minimum number of free bytes (ever encountered) in the task stack. Monitor the return of this function within the task.
  - Calculate the new stack size as the initial stack size minus the minimum stack free bytes.
  - The method requires that the task has been running enough to enter the worst path (in term of stack consumption).



Semaphores, Mutexes and Queues - Heap usage

- Each semaphore declared by the user application requires 88 bytes to be allocated in the heap.
- Each mutex declared by the user application requires 88 bytes to be allocated in the heap (+8 bytes within TCB of each task).
- To save heap size (i.e RAM footprint) it is recommended to disable the define `configUSE_MUTEXES` when mutexes are not used by the application (task TCB static size being reduced)
- FreeRTOS requires to allocate in the heap for each message queue (control block and queue storage area):
  - number of bytes = 76 + queue_storage_area.
  - queue_storage_area (in bytes) = (element_size * nb_elements) + 16

Software Timers - Heap usage 1/2

- When Software Timers are enabled (`configUSE_TIMERS` enabled) , the scheduler creates automatically the timers service task (daemon) when started. The timers service task is used to control and monitor (internally) all timers that user will create. The timers task parameters are set through the following defines :
  - `configTIMER_TASK_PRIORITY` : priority of the timers service task
  - `configTIMER_TASK_STACK_DEPTH` : timers task stack size (in words)

- The scheduler also creates automatically a message queue used to send commands to the timers service task (timer start, timer stop ...)
- The number of elements of this queue (number of messages that can be hold) are configurable through the define:

  `configTIMER_QUEUE_LENGTH`.

life.augmented

Software Timers - Heap usage 2/2

- FreeRTOS requires to allocate on the heap for software timers:

1. Timers Daemon Task (in bytes) :
   - TCB_size + (4 x `configTIMER_TASK_STACK_DEPTH`)

2. Timers message queue : number of bytes = 76 + queue_storage_area
   - With queue_storage_area = (12 * `configTIMER_QUEUE_LENGTH`) + 16

3. Each timer created by the user (by calling `osTimerCreate()`) needs 48 bytes

How to reduce RAM footprint (1/2)

- Optimize stack allocation for each task :
  - `uxTaskGetStackHighWaterMark()`. This API returns the minimum number of free bytes (ever encountered) in the task stack
  - `vApplicationStackOverflowHook()`. This API is a stack overflow callback called when a stack overflow is detected (available when activating the define `configCHECK_FOR_STACK_OVERFLOW`)
- Recover and minimize the stack used by main and rationalize the number of tasks.
- Use minimal number of priorities (`configMAX_PRIORITIES`)
  - `configMAX_PRIORITIES` defines the number of priorities available to the application tasks.
  - Any number of tasks can share the same priority.
  - The scheduler allocates statically the ready task list of size `configMAX_PRIORITIES` * list entry structure.

life.augmented

How to reduce RAM footprint (2/2)

- If the application doesn’t use any software timers then disable the define `configUSE_TIMERS`.
- If the application doesn’t use any mutexes then disable the define `configUSE_MUTEXES`.

- Adjust heap dimensioning :
  - `xPortGetFreeHeapSize()`. API that returns the total amount of heap space that remains unallocated. Must be used after created all tasks, message queues, semaphores, mutexes in order to check the heap consumption and eventually re-adjust the application define ” `configTOTAL_HEAP_SIZE`”.
  - The total amount of heap space that remains unallocated is also available with `xFreeBytesRemaining` variable for heap management schemes 2 to 5

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Scheduler



life.augmented

Multitasking 1/3

- Cooperative multitasking
  - Requires cooperation of all tasks
  - Context gets switched ONLY when RUNNING task:
    - goes to BLOCKED state (i.e. by call `osDelay()` function) or
    - goes to READY state (i.e. by call `osThreadYield()` function) or
    - is put into SUSPEND mode by the system (other task)
  - Tasks are not preempted with higher priority tasks
  - No time slice preemption as well
  - It requires the following setting in FreeRTOSConfig.h:

```
#define configUSE_PREEMPTION  0
```

Multitasking 2/3

- Preemptive multitasking (default in FreeRTOS)
  - Tasks with the same priority share CPU time
  - Context gets switched when:
    - Time slice has passed
    - Task with higher priority has come
    - Task goes to BLOCKED state (i.e. by call `osDelay()` function)
    - Task goes to READY state (i.e. by call `osThreadYield()` function)
  - It requires the following setting in FreeRTOSConfig.h:

```
#define configUSE_PREEMPTION  1
```

Multitasking 3/3

- Cooperative with preemption by IRQ multitasking
  - IRQs are used to trigger context switch by call a yield function directly or using macro portEND_SWITCHING_ISR or portYIELD_FROM_ISR
  - Preemptive system without time slice
  - It requires the following setting in FreeRTOSConfig.h:

```
#define configUSE_PREEMPTION  0
```

Scheduling

- The scheduler is an algorithm determining which task to execute.
  - Common point between schedulers is that they distinguish between tasks being ready to be executed (in READY state) and those being blocked for any reason (delayed, waiting for something)
  - The main difference between schedulers is how they distribute CPU time between the tasks in READY state.
- In FreeRTOS **round-robin** scheduling algorithm is implemented:
  - Round-robin can be used with either preemptive or cooperative multitasking (**configUSE_PREEMPTION** in FreeRTOSConfig.h).
  - It works well if response time is not an issue or all tasks have same priority.
  - The possession of the CPU changes periodically after a predefined execution time called timeslice (**configTICK_RATE_HZ** in FreeRTOSConfig.h)

API - Operations on scheduler CMSIS_OS v2.x

- Initialize the RTOS kernel (not working in irq mode)

  `osStatus_t osKernelInitialize()`

  - Initializes heap regions (vPortDefineHeapRegions()) if heap5 memory management scheme is selected
  - Sets kernel state to osKernelReady

- Get RTOS kernel information (can work from user code and irq)

  `osStatus_t osKernelGetInfo()`

  - Gets KERNEL_VERSION and KERNEL_ID values and stores them in memory

- Get the current RTOS Kernel state (can work from user code and irq)

  `osKernelState_t osKernelGetState()`

  - Reads (function xTaskGetSchedulerState()) and returns kernel current state (osKerelRunning, osKernelLocked, osKernelReady, osKernelInactive)

API - Operations on scheduler CMSIS_OS v2.x

- Start the scheduler (not working in irq mode)

  `osStatus_t osKernelStart()`

  - Set kernel state to osKernelRunning
  - Executing vTaskStartScheduler()
- Lock RTOS kernel scheduler (not working in irq mode)

  `int32_t osKernelLock()`

  - Checking scheduler state by calling xTaskGetSchedulerState() function
  - If scheduler is in run state it will be suspended, by calling vTaskSuspendAll() function
- Unlock RTOS kernel scheduler (not working in irq mode)

  `int32_t osKernelUnlock()`

  - Checking scheduler state by calling xTaskGetSchedulerState() function
  - If scheduler is in suspended state it will be unlocked, by calling vTaskResumeAll() function
- Restore RTOS kernel scheduler lock state (not working in irq mode)

  `int32_t osKernelRestoreLock()`

  - In case scheduler is in suspend or run mode it will confirm current state calling either vTaskSuspendAll() or vTaskResumeAll()
  - Otherwise (i.e. not started) it will return osError

API - Operations on scheduler
CMSIS_OS v2.x

- Get the RTOS kernel tick frequency (can work from user mode and irq)

  `osKernelGetTickFreq()`

  - Returns value of configTICK_RATE_HZ

- Get the RTOS kernel system timer count (can work from user mode and irq)

  `osKernelGetSysTimerCount`

  - Returns tick count by calling xTaskGetTickCount() function (in critical section) then: ticks * (configCPU_CLOCK_HZ / configTICK_RATE_HZ);

- Get the RTOS kernel system timer frequency (can work from user mode and irq)

  `osKernelSysTick()`

  - Returns value of configCPU_CLOCK_HZ

CMSIS_OS v1.x
CMSIS_OS v2.x

Scheduler APIs

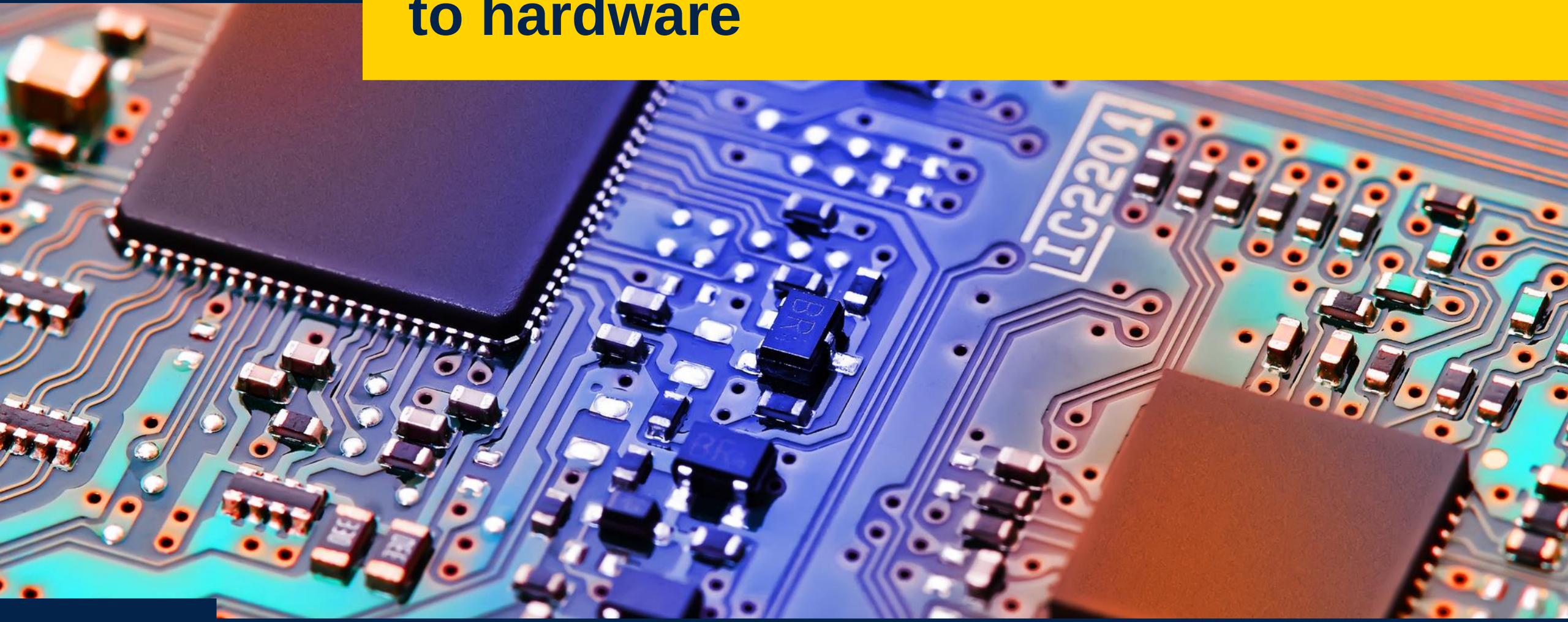
CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osKernelInitialize() - empty	osKernelInitialize()	-
osKernelStart()	osKernelStart()	vTaskStartScheduler()
osKernelRunning()	osKernelGetState()	xTaskGetSchedulerState()
osKernelSysTick()	osKernelGetTickCount()	xTaskGetTickCount() xTaskGetTickCountFromISR()
	osKernelLock()	vTaskSuspendAll()
	osKernelUnlock()	xTaskResumeAll()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

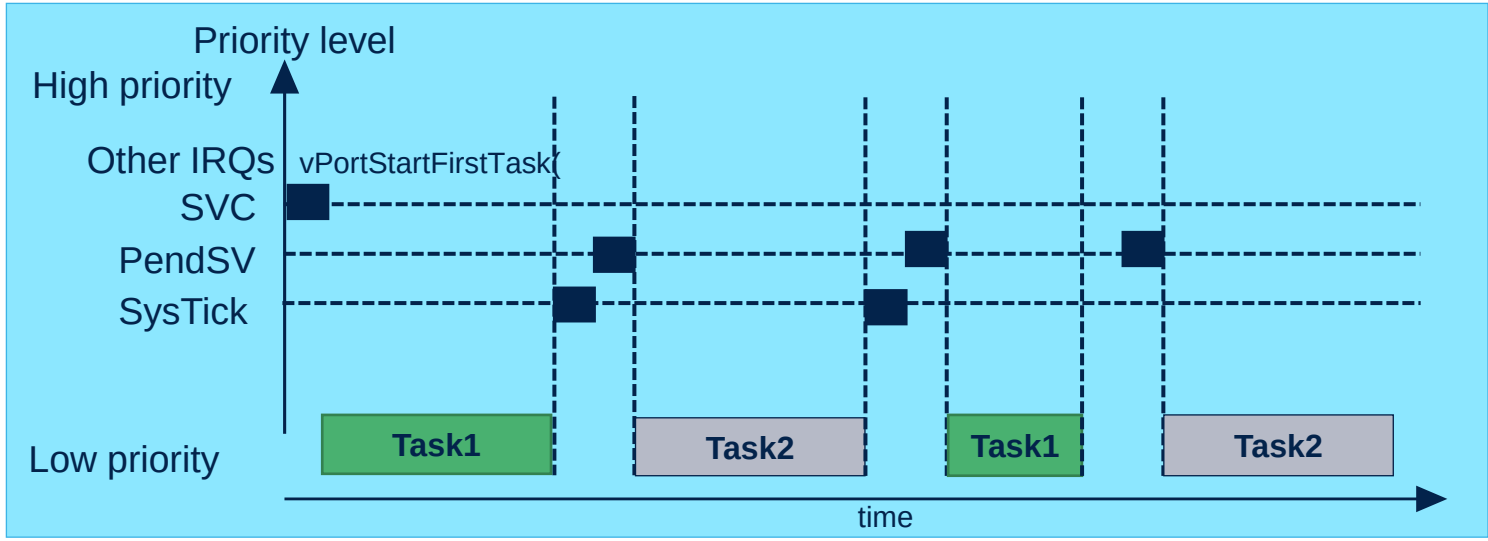
FreeRTOS – interrupts and connection to hardware



life.augmented

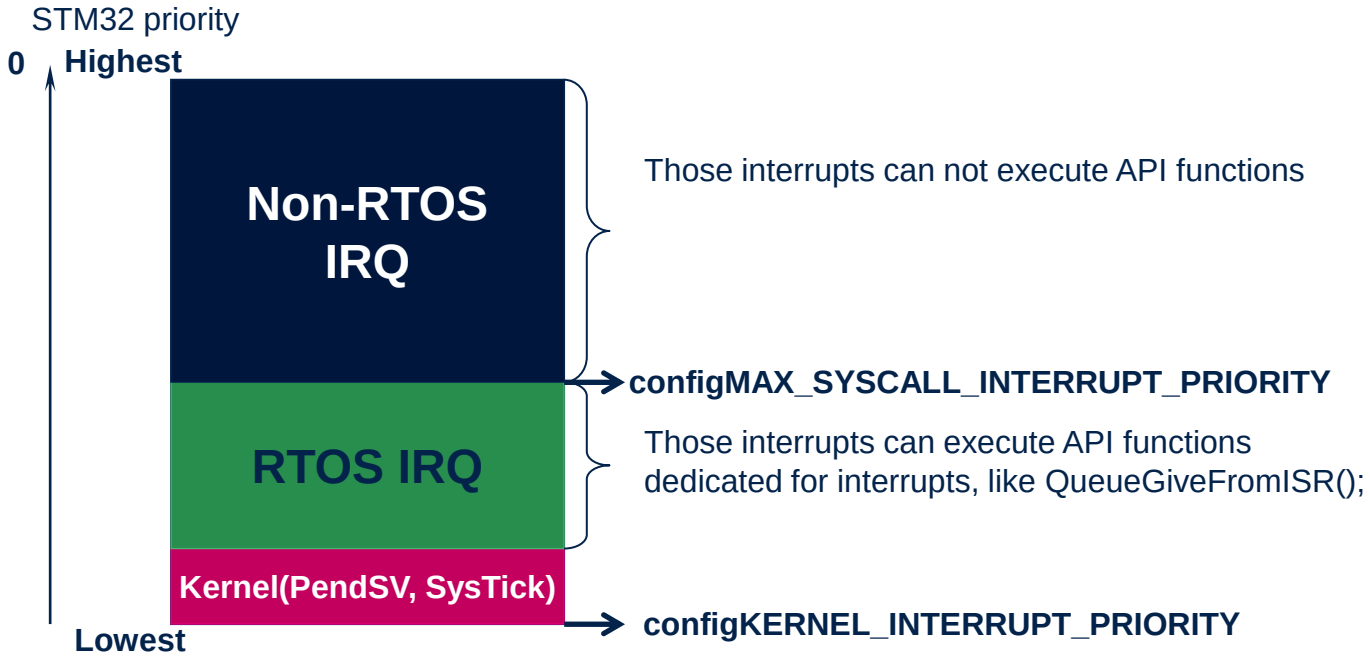
FreeRTOS OS interrupts

- PendSV interrupt
  - Used for task switching before tick rate
  - Lowest NVIC interrupt priority
  - Not triggered by any peripheral
- SVC interrupt
  - Interrupt risen by SVC instruction
  - `SVC 0` call used only once, to start the scheduler (within `vPortStartFirstTask()` which is used to start the kernel)
- SysTick timer
  - Lowest NVIC interrupt priority
  - Used for task switching on `configTICK_RATE_HZ` regular timebase
  - Set PendSV if context switch is necessary



life.augmented

NVIC configuration



- FreeRTOS kernel and its irq procedures (PendSV, SysTick) have lowest possible interrupt priority set in FreeRTOSConfig.h (**configKERNEL_INTERRUPT_PRIORITY**)
- There is a group of interrupts which can cooperate with FreeRTOS API by calling its functions. Maximum level for those peripherals (based on the position in vector table) is set in **configMAX_SYSCALL_INTERRUPT_PRIORITY**
- It is possible to use nested interrupts.

API functions in IRQ procedures

- Within **FreeRTOS API** there are dedicated functions to be executed within IRQ procedures. All of those functions has **FromISR** suffix in its names, like i.e.:

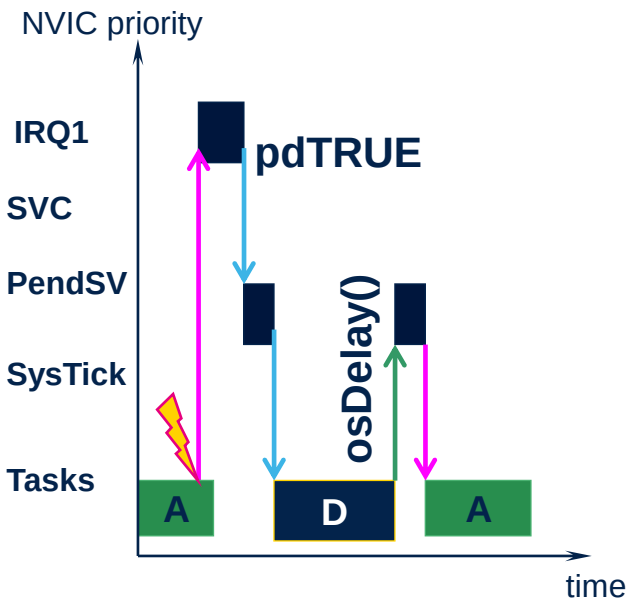
  xSemaphoreGiveFromISR(semaphore, *hp_task)

  vs

  xSemaphoreGive (semaphore)

- The only difference for the programmer is additional argument *hp_task. It is a pointer to the variable which is used to indicate whether operation on queue or semaphore within IRQ causes unblocking of the task with higher priority than currently running. If this parameter is pdTRUE, context switch (PendSV irq) should be requested by kernel before the interrupt exits.
- When using **CMSIS API**, this process is **automatically handled by the library** (by checking IPSR content) and is transparent for the programmer, i.e.:

```
osSemaphoreRelease(semaphore)
```



Example: Task A has been interrupted by IRQ1. During an interrupt, Task D with higher priority was unblocked, thus it will be executed once IRQ will finish

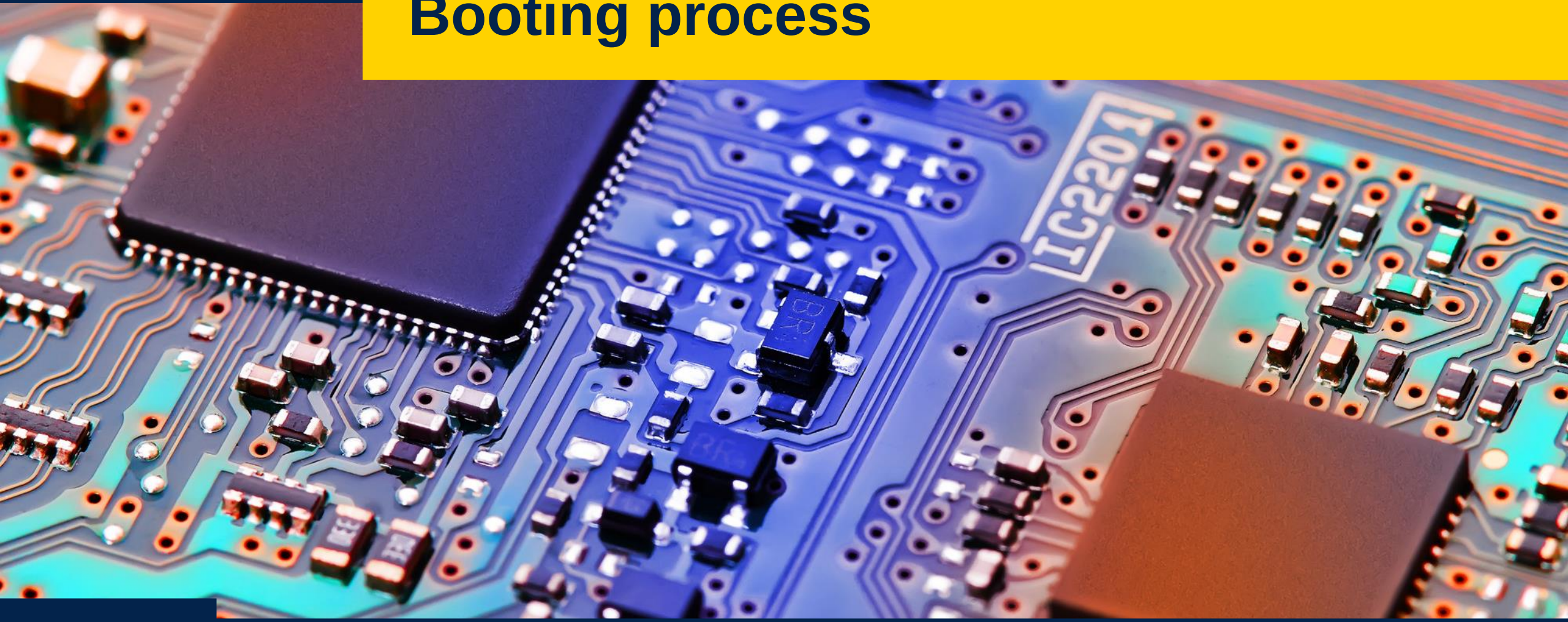
life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Booting process



life.augmented

FreeRTOS – boot sequence & timing

time

HW dependent:

- Configure the CPU clocks
- Initialize static and global variables that contain only the value zero (bss)
- Initialize variables that contain a value other than zero
- Perform any other hardware set up required

FreeRTOS related *)

- Create application queues, semaphores and mutexes (~500 CPU cycles/object)
- Create application tasks (~1100 CPU cycles/task)
- Start the RTOS scheduler (~1200 CPU cycles)
  - The RTOS scheduler is started by calling vTaskStartScheduler(). The start up process includes configuring the tick interrupt, creating the idle task, and then restoring the context of the first task to run

life.augmented

*) calculations based on ARM CortexM3 device, using ARM RVDS compiler with low optimization level (1)
Source: FreeRTOS FAQ – Memory Usage, Boot Time & Context Switch Times on www.freertos.org web page

Idle task code

- Idle task code is generated automatically when the scheduler is started
- It is `portTASK_FUNCTION()` function within task.c file
- It is performing the following operations (in endless loop):
  - Check for deleted tasks to clean the memory
  - `taskYIELD()` if we are not using preemption (configUSE_PREEMPTION=0)
  - Get yield if there is another task waiting and we set configIDLE_SHOULD_YIELD=1
  - Executes `vApplicationIdleHook()` if configUSE_IDLE_HOOK=1
  - Perform low power entrance if configUSE_TICKLESS_IDLE!=0

life.augmented

FreeRTOS start - step by step 1/2

- FreeRTOS is started by `osKernelStart()` function (main.c file) from CMSIS_OS API
- It is calling `vTaskStartScheduler()` function (cmsis_os.c file) from FreeRTOS API
- It is creating an IDLE task (`xTaskCreate()`), then disable OS interrupts (`portDISABLE_INTERRUPTS()`) to be sure that no tick will happened before or during call to `xPortStartScheduler()` function (task.c file)
- `xPortStartScheduler()` function (port.c file) is configuring lowest priority level for SysTick and PendSV interrupts, then it is starting the timer that generates the tick (in CortexM architecture usually it is SysTick), enables FPU if present (CortexM4) and starts the first task using `prvPortStartFirstTask()` function

life.augmented

FreeRTOS start - step by step 2/2

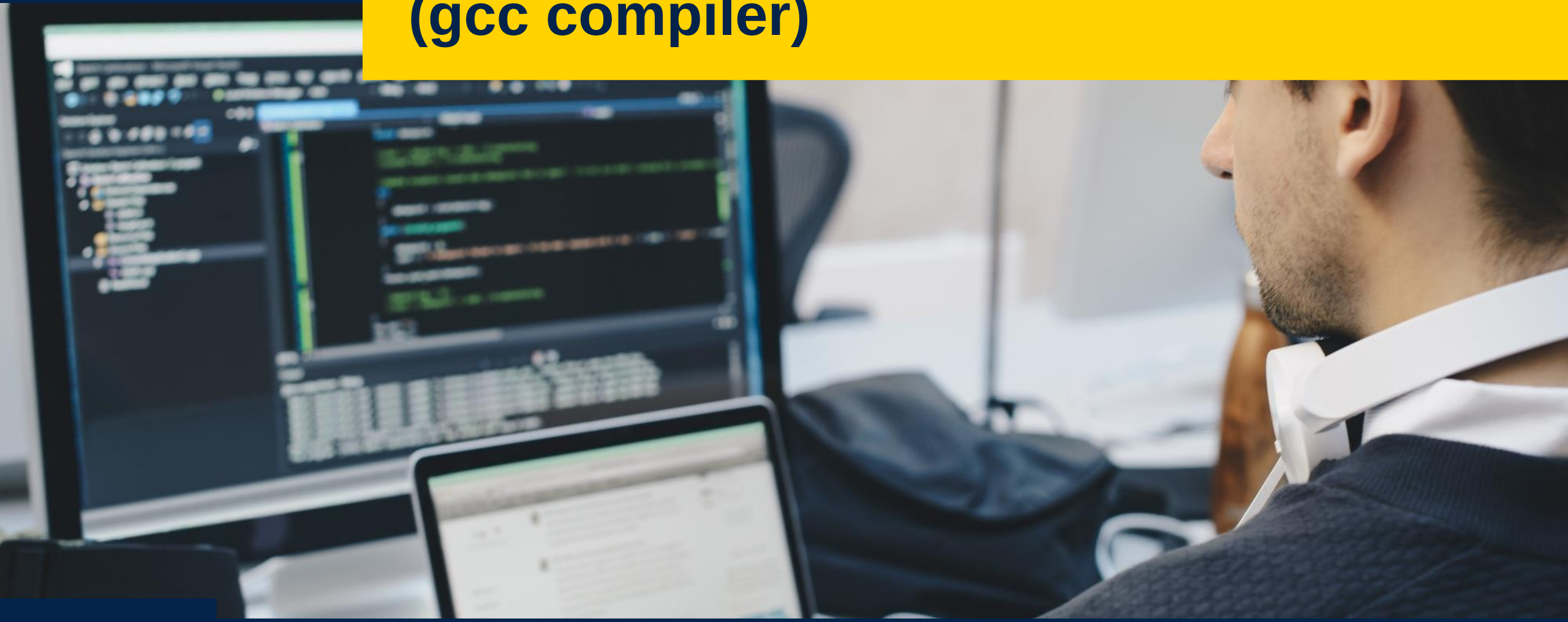
- `prvPortStartFirstTask()` function (port.c file, usually written in assembler) locates the stack and set MSP (used by the OS) to the start of the stack, then enables all interrupts. After this triggers software interrupt SVC
- As a result of SVC interrupt `vPortSVCHandler()` is called (port.c file)
- `vPortSVCHandler()` function (port.c file) restores the context, loads TCB (Task Control Block) for the first task (highest priority) form ready list and starts executing this task

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Examples of main interface procedures (gcc compiler)



life.augmented

Examples of main interface procedures prvPortStartFirstTask (port.c gcc compiler)

```
/* Use the NVIC offset register to locate the stack */
ldr    r0, =0xE000ED08
ldr    r0, [r0]
ldr    r0, [r0]
/* Set the msp back to the start of the stack */
lsr    msp, r0
/* Enable interrupts */
cpsie i
cpsie f
dsb
isb
/* Call SVC to start the first task */
svc    0
nop
```



see xPortSVCHandler()

life.augmented

Examples of main interface procedures vPortSVCHandler (port.c gcc compiler)

```
/* Get the location of the current TCB */
ldr    r3, =pxCurrentTCB
ldr    r1, [r3]
ldr    r0, [r1]

/* Pop the core registers (not stored automatically) */
ldmia r0!, {r4-r11, r14}
/* restore the task stack pointer */
msr    psp, r0
isb
mov    r0, #0
msr    basepri, r0
bx     r14
```

Examples of main interface procedures xPortPendSVHandler (port.c gcc compiler)

```
mrs       r0, psp
Isb

/* Get the location of the current TCB */
ldr       r3, =pxCurrentTCB
ldr       r2, [r3]

/* Is the task using the FPU context? If so, push high vfp registers. */
tst       r14, #0x10
it        eq
vstmdbeq  r0!, {s16-s31}

/* Save the core registers. */
stmdb     r0!, {r4-r11, r14}

/* Save the new top of stack into the first member of the TCB. */
str       r0, [r2]

stmdb     sp!, {r3}
mov       r0, #configMAX_SYSCALL_INTERRUPT_PRIORITY
msr       basepri, r0
dsb
isb
```

Save the data on the stack (of current task which needs to be removed from RUN state)

Freeze OS by blocking its interrupt

life.augmented

Examples of main interface procedures xPortPendSVHandler (port.c gcc compiler)

```
bl      vTaskSwitchContext
```

Switch the task (change pxCurrentTCB)

```
mov     r0, #0
msr     basepri, r0
ldmia   sp!, {r3}
/* The first item in pxCurrentTCB
   is the task top of stack. */
ldr     r1, [r3]
ldr     r0, [r1]

/* Pop the core registers. */
ldmia   r0!, {r4-r11, r14}

/* Is the task using the FPU context?
   If so, pop the high vfp registers too. */
tst     r14, #0x10
it      eq
vldmiaeq r0!, {s16-s31}
```

Restore the data from the stack (of new task)

```
/* return from PendSV irq
msr     psp, r0
isb
bx      r14
```

Go to new task execution

life.augmented

Critical sections

- Critical section mechanism allows to block all the interrupts during sensitive/atomic operation execution (like operations on queues)
- To enter into critical section **`portENTER_CRITICAL()`** should be used
- To exit from critical section **`portEXIT_CRITICAL()`** should be used

```
void function(void)
{
        portENTER_CRITICAL();

        ..sensitive code execution

        portEXIT_CRITICAL();
}
```

Storing current BASEPRI value (current level of interrupt masking) and programming BASEPRI with MASK value *)
It means masking of all interrupts below this value

restoring BASEPRI value and masking of all interrupts below this value

*) MASK = configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY<<(8- configPRIO_BITS)

defined in FreeRTOSConfig.h

= 4 for CortexM3, CortexM4 based STM32

life.augmented

Critical section in thread mode example of the implementation (portmacro.h)

- Beginning of the critical section is done using function `vPortEnterCritical(void)` (port.c file) which is disabling interrupts using `portDISABLE_INTERRUPTS()` macro
- `portDISABLE_INTERRUPTS()` (portmacro.h file) is defined as:

```
#define portDISABLE_INTERRUPTS()
{
  __set_BASEPRI( configMAX_SYSCALL_INTERRUPT_PRIORITY );
  __DSB();
  __ISB();
}
```

- End of the critical section is done using function `vPortExitCritical(void)` (port.c file) which is disabling interrupts using `portENABLE_INTERRUPTS()` macro
- portENABLE_INTERRUPTS() (portmacro.h file) is defined as:

```
#define portENABLE_INTERRUPTS()      __set_BASEPRI( 0 )
```

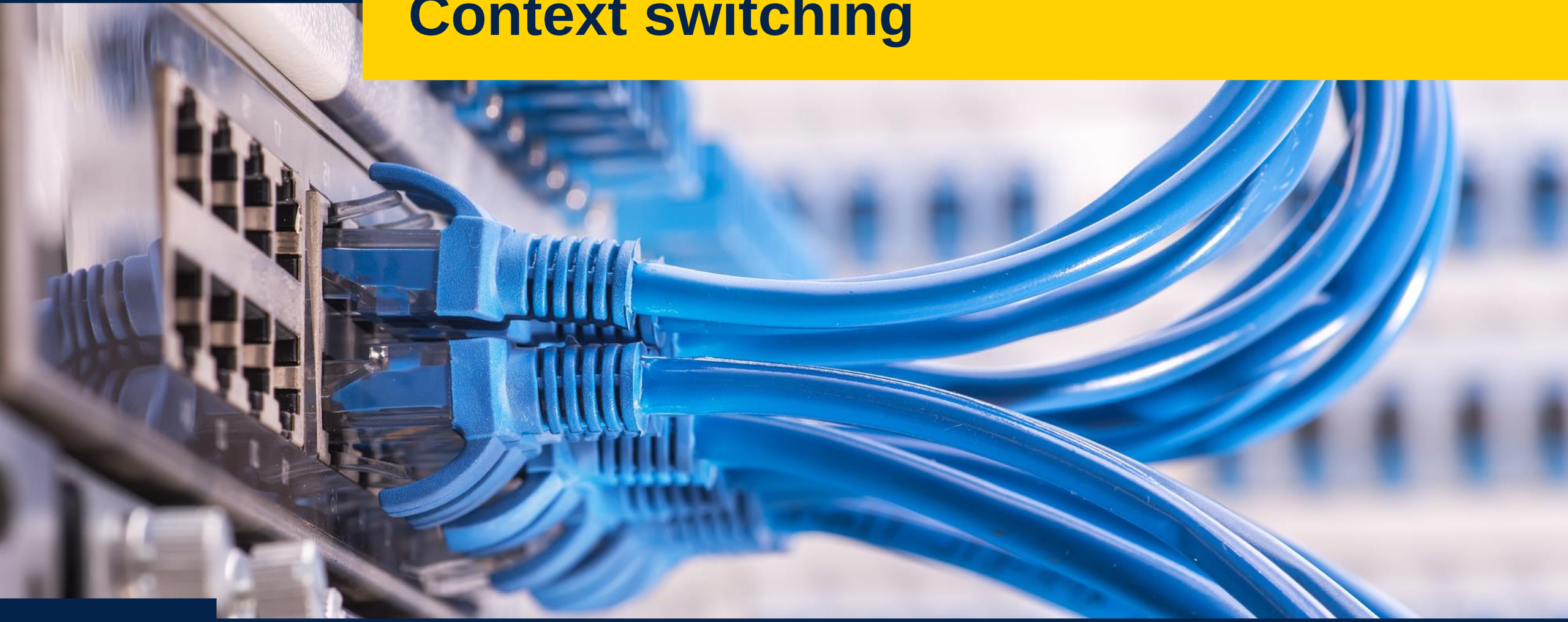
life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

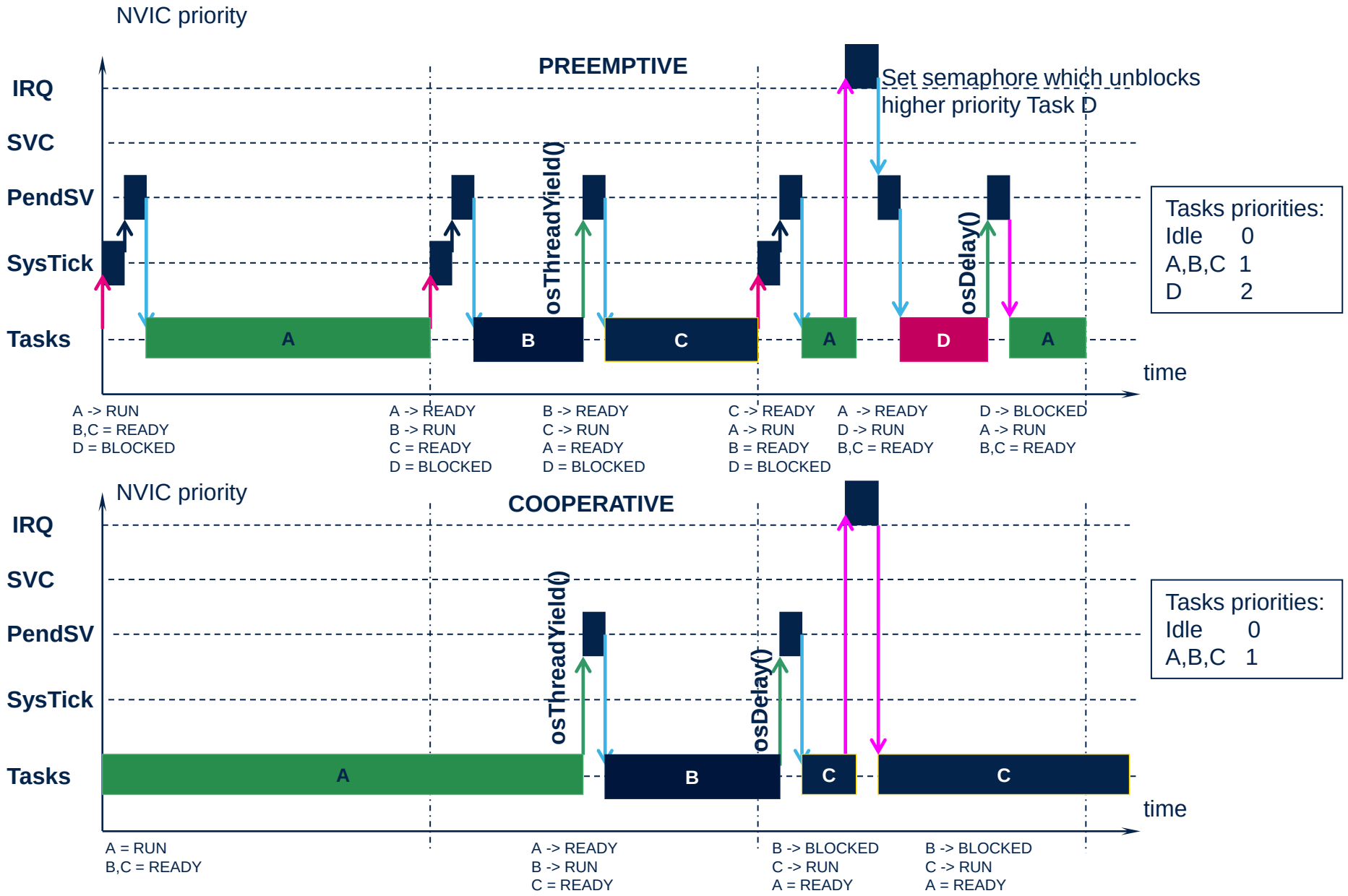
life.augmented

FreeRTOS – CMSIS_OS v2
Context switching



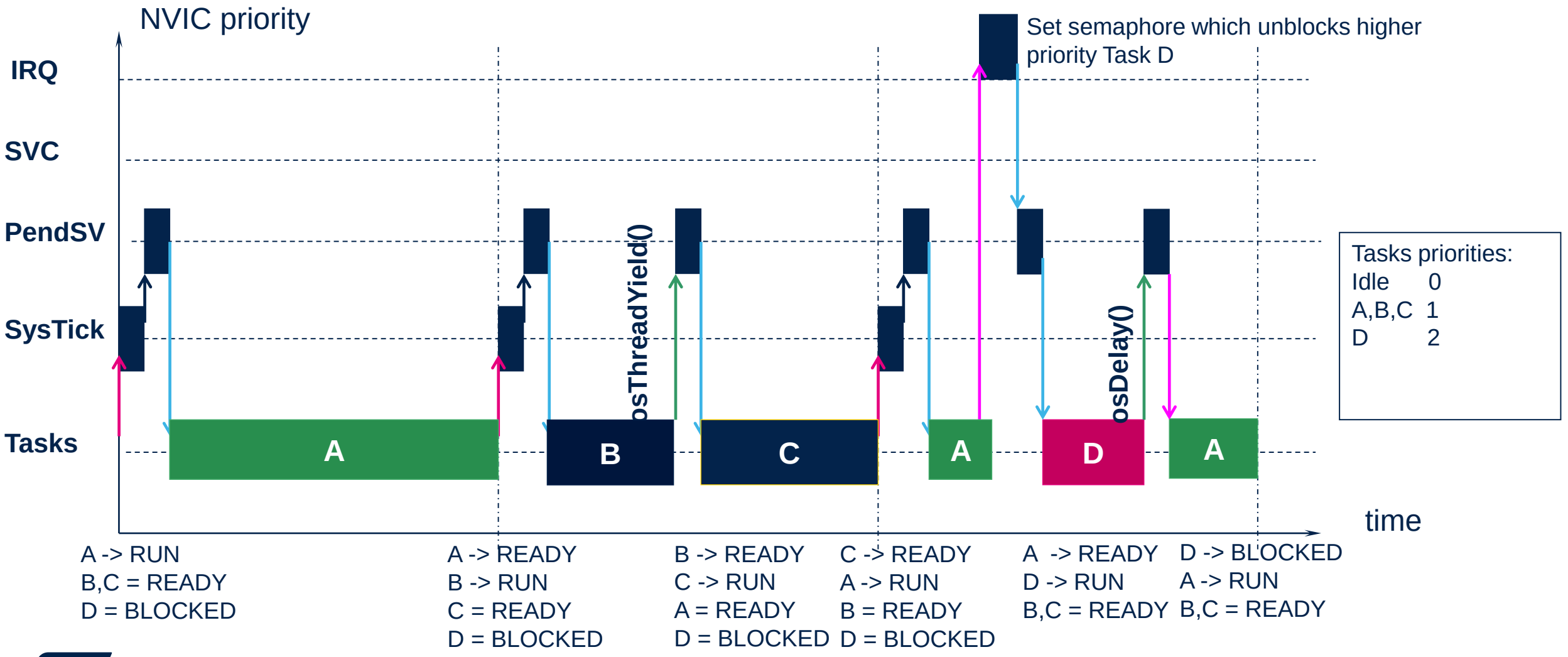
life.augmented

Context switching



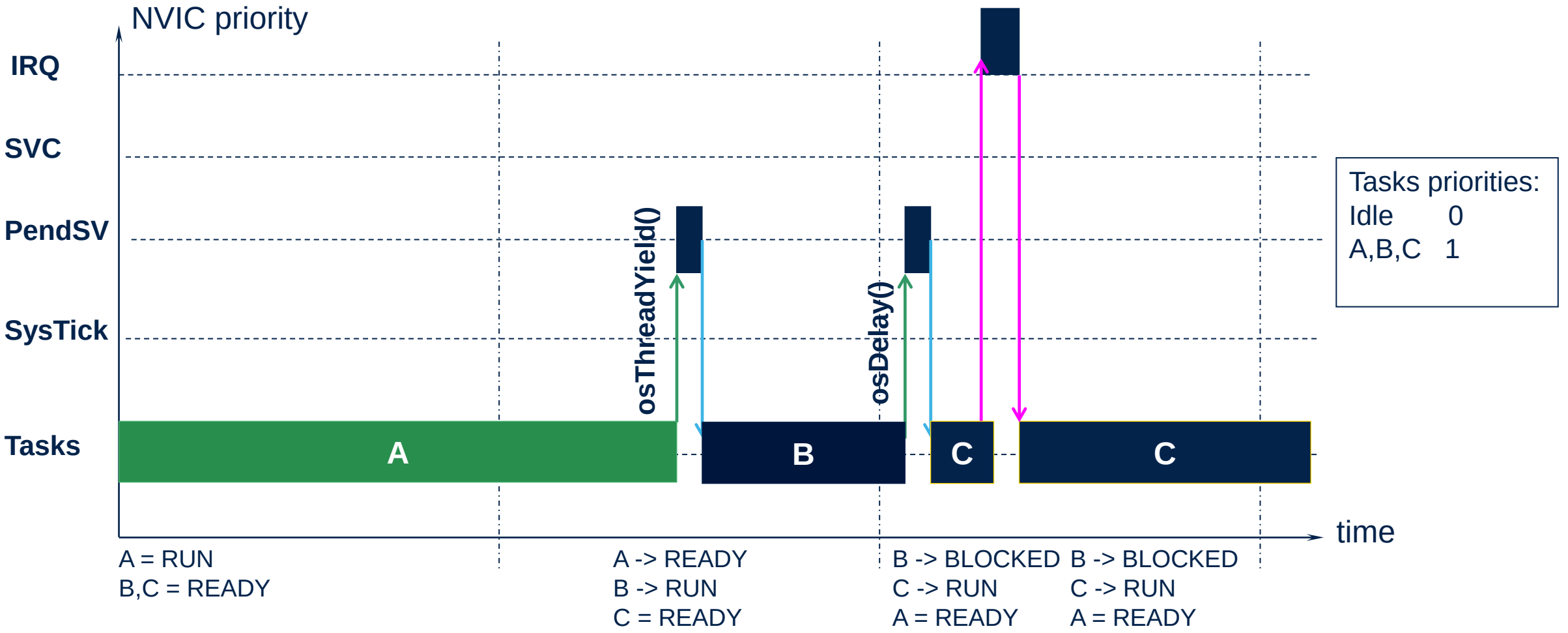
life.augmented

Context switching – Preemptive mode



life.augmented

Context switching – Cooperative mode



life.augmented

FreeRTOS – context switch time (1/2)

- Context switch time depends on the port, compiler and configuration. A context switch time of 84 CPU cycles was obtained under the following test conditions:
  - FreeRTOS ARM Cortex-M3 port for the Keil compiler
  - Stack overflow checking turned off
  - Trace features turned off
  - Compiler set to optimization for speed
  - configUSE_PORT_OPTIMISED_TASK_SELECTION set to 1 in FreeRTOSConfig.h

Remarks:

- Under these test conditions the context switch time is not dependent on whether a different task was selected to run or the same task was selected to continue running.
- The ARM Cortex-M port performs all task context switches in the PendSV interrupt. The quoted time does not include interrupt entry time.
- The quoted time includes a short section of C code. It has been determined that 12 CPU cycles could have been saved by providing the entire implementation in assembly code. It is considered that the benefit of maintaining a short section of generic C code (for reasons of maintenance, support, robustness, automatic inclusion of features such as tracing, etc.) outweighs the benefit of removing 12 CPU cycles from the context switch time.
- The Cortex-M CPU registers that are not automatically saved on interrupt entry can be saved with a single assembly instruction, then restored again with a further single assembly instruction. These two instructions on their own consume 12 CPU cycles.

*) source: FreeRTOS FAQ – Memory Usage, Boot Time & Context Switch Times on www.freertos.org web page

FreeRTOS – context switch time (2/2)

- Context switch time can be much longer in CortexM4 and CortexM7 based devices with Floating Point Unit due to necessity of stacking FPU registers (additional 17 32bit registers: S0-S15 and FPSCR).
- Rest of FPU registers (S16-S31) should be handled by software
- Within PendSV handler there is a check done whether floating point unit instruction has been used and based on this those registers are stacked/unstacked from/for current task or not:

```
/* Is the task using the FPU context?
   If so, push high vfp registers. */
tst     r14, #0x10
it      eq
vstmdbeq r0!, {s16-s31}
```

- And then on PendSV exit after the task switch:

```
/* Is the task using the FPU context?
   If so, pop the high vfp registers too. */
tst     r14, #0x10
it      eq
vldmiaeq r0!, {s16-s31}
```

- More information can be found in **Application note 298** from ARM.

FreeRTOS – context switching tick source - step by step

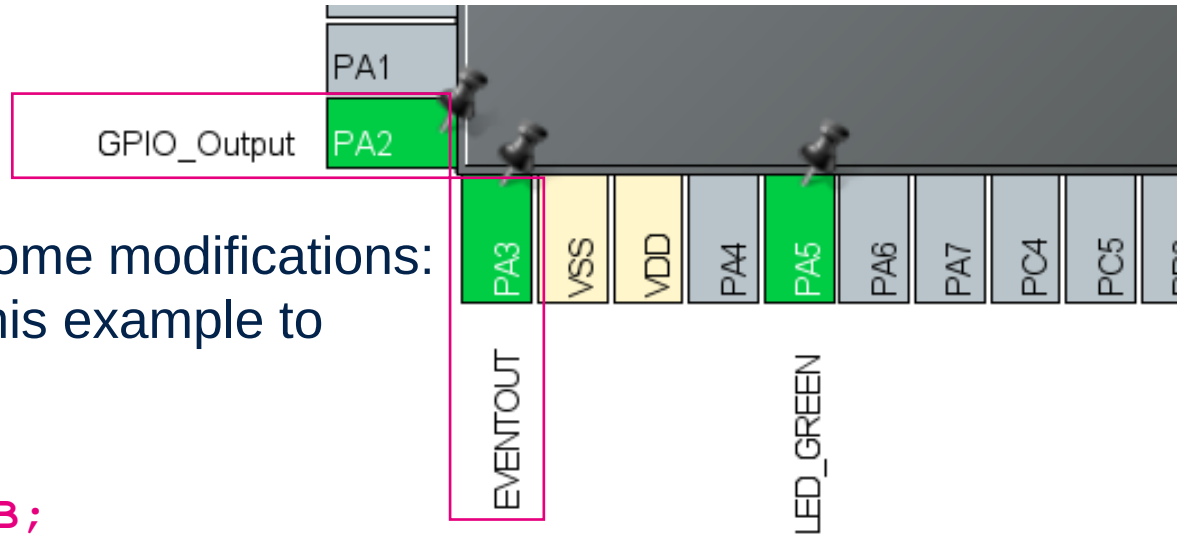
- Tick timer (CortexM architecture uses SysTick) interrupt causes execution of `xPortSysTickHandler()` (**port.c** file)
- `xPortSysTickHandler()` (usually written in assembly):
  - blocks all interrupts (as its own priority is the lowest possible) using `portDISABLE_INTERRUPTS()` macro (**portmacro.h** file)
  - Activates PendSV bit to run an interrupt what executes `xPortPendSVHandler()` function (**port.c** file):
    - Calls `vTaskSwitchContext()` function (**task.c** file), which is calling a macro `taskSELECT_HIGHEST_PRIORITY_TASK()` (**task.c** file) to select the READY task on the highest possible priority list.
  - Unblocks all interrupts using `portENABLE_INTERRUPT()` macro (**portmacro.h** file)

life.augmented

Context switching time STM32CubeMX modifications

Within STM32CubeMX, pinout tab:

- Configure PA2 as GPIO_Output
- Configure PA3 as EVENTOUT



Re-generate the code and within the code please add some modifications:

1. Configure speed for selected pins (PA2 and PA3 in this example to highest possilbe
2. To set pin PA2, please use `GPIOA->ODR |= 4;`
3. To reset pin PA2, please use `GPIOA->ODR &= 0xFFFB;`
4. To generate 1 sys clk long pulse on PA3 use `sev` (assembly code)

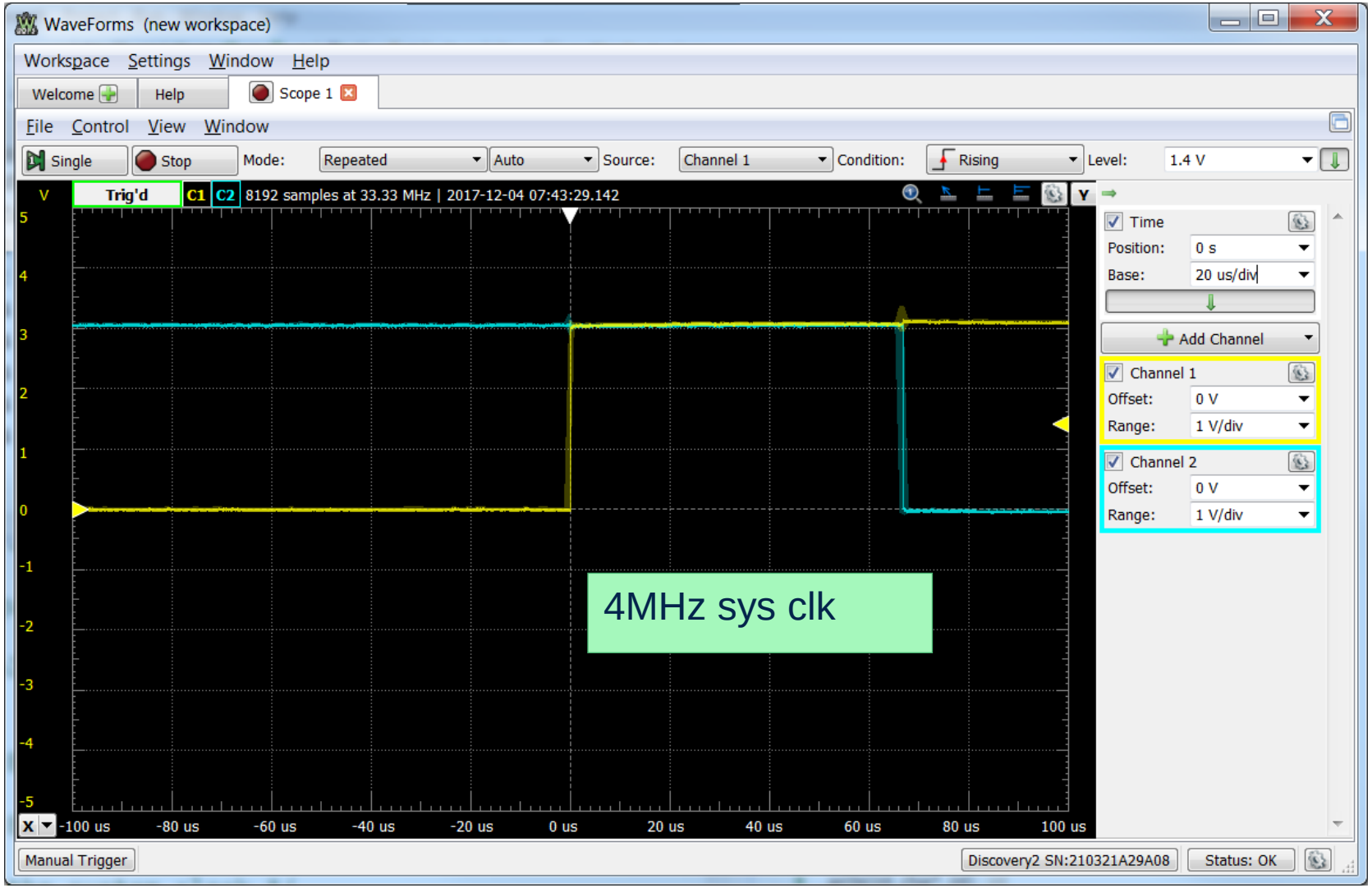
Put above lines in various places in the code to measure time intervals, i.e

- within `SysTick_Handler()` in **stm32L4xx_it.c**,
- in empty for(;;) loop within Task1 and Task2 accordingly (**main.c** file)
- within `xPortPendSVHandler()` function (**port.c** file)

life.augmented

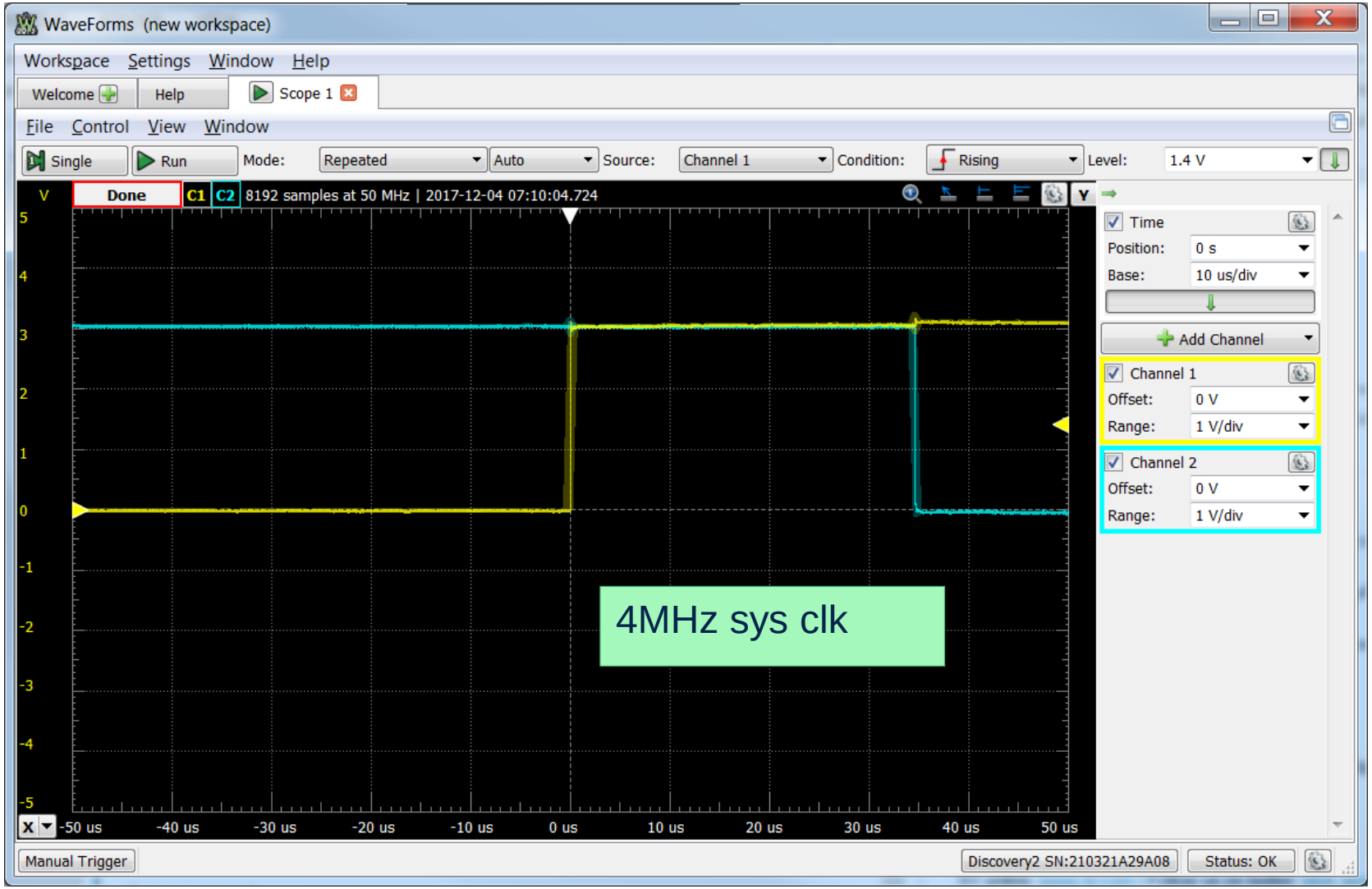
Context switching time

- Time between beginning of SysTick and user task code **~65us**



Context switching time

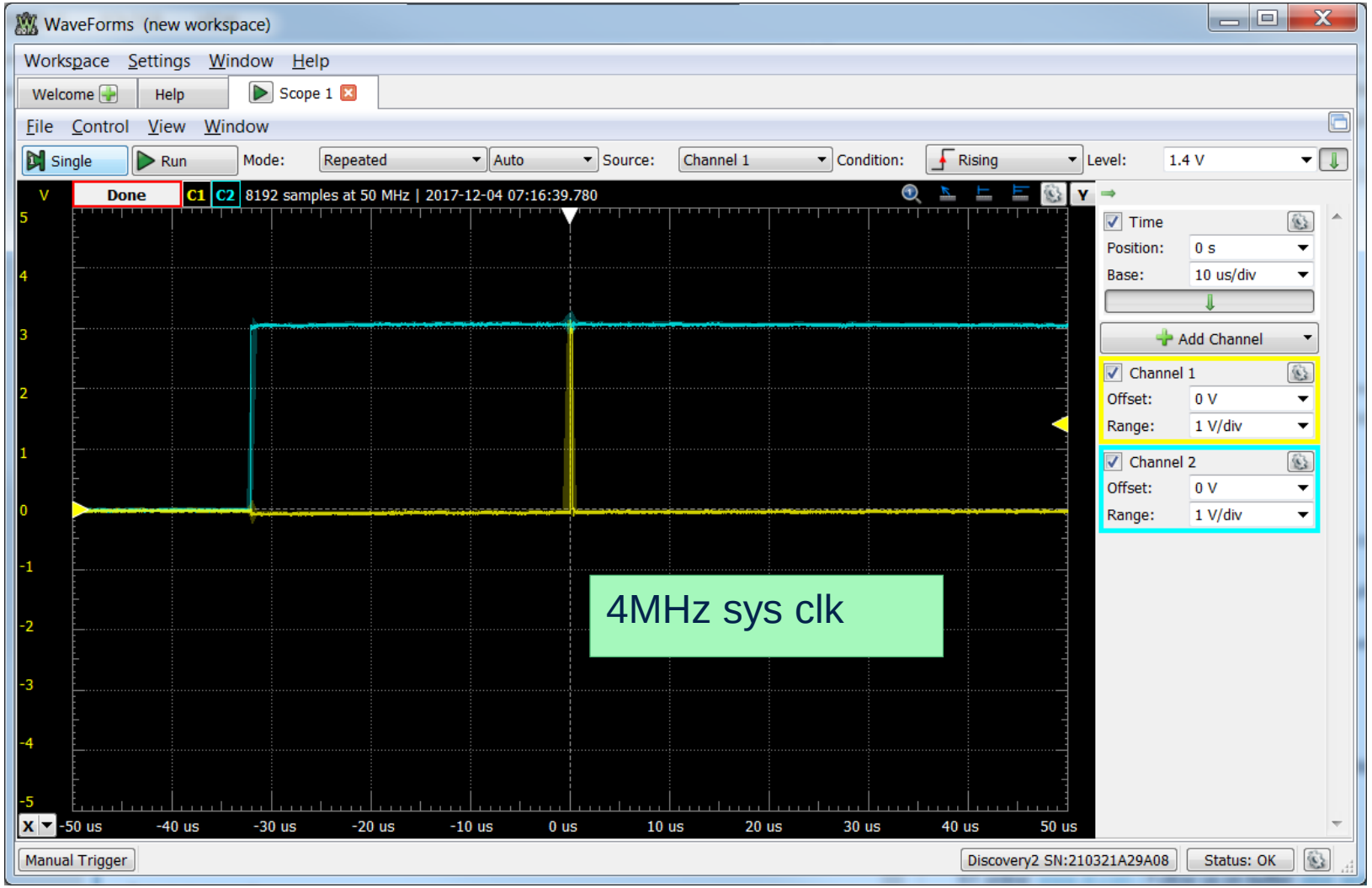
- Time between beginning of beginning of PendSV code and user task code **~37us**



gcc

Context switching time

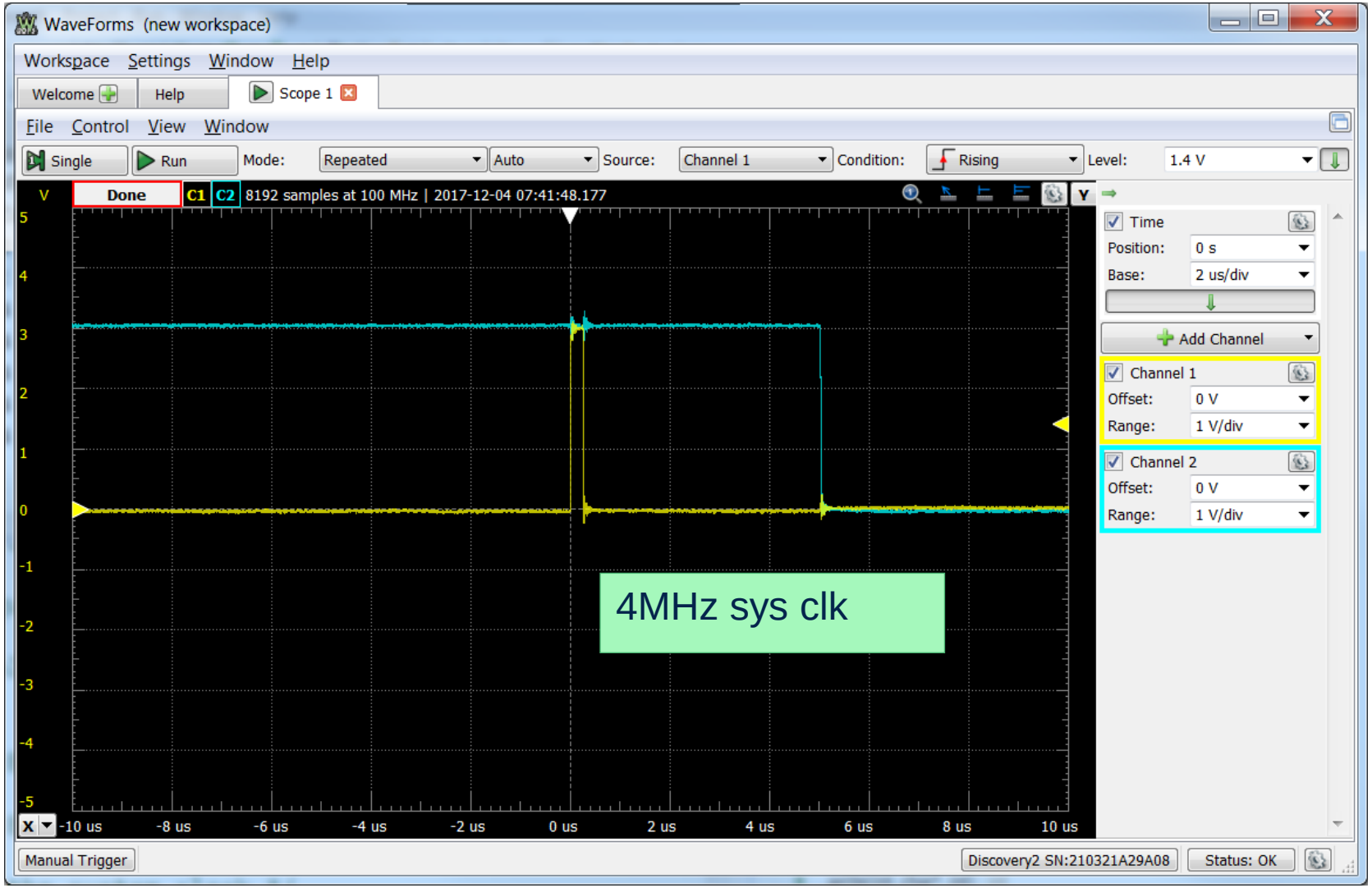
- Time between beginning of SysTick and jump to user task within PendSV **~30us**



life.augmented

Context switching time

- Time between jump to user task within PendSV and user task code **~5us**



Context switching time

- Length of the pulse generated by __sev() **~250ns** (1clk cycle @4MHz sys clk)



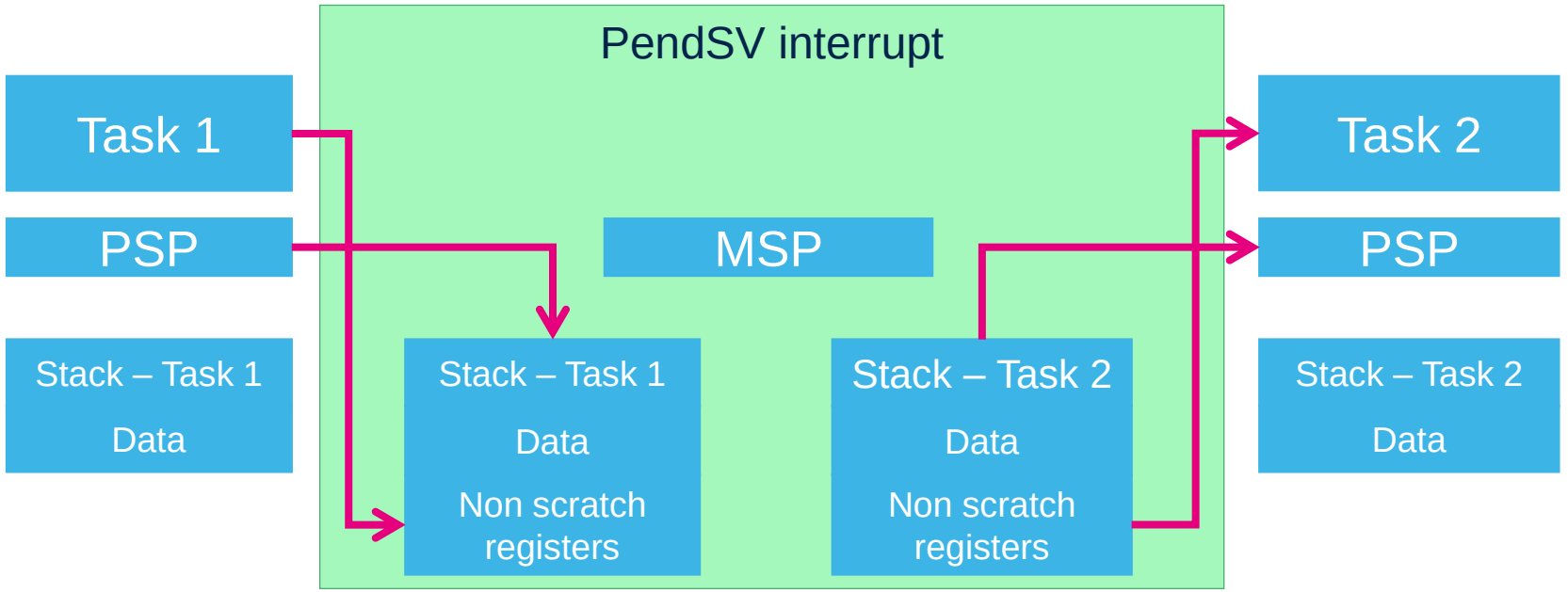
Dual stack

- There are two independent stack pointers in CortexM devices:
  - Main Stack Pointer (MSP) – enabled by default.
  - Process Stack Pointer (PSP) – could be enabled (bit 1 in CONTROL register)
- Both 32bit registers are visible as R13 register of the Core and only one can be used at one time.
- Dual stack architecture is used for OS:
  - MSP – OS kernel and exception handlers
  - PSP – application tasks

life.augmented

Stack pointers

- Main stack pointer (MSP)
  - Used in interrupts
  - Allocated by linker during compiling
- Process stack pointer (PSP)
  - Each task have own stack pointer
  - During context switch the stack pointer is initialized for correct task



life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Tasks



life.augmented

What is a Task?

- It is a C function:

```
FirstTask(void const * argument)
```

- It should be run within infinite loop, like:

```
for(;;)
{
 /* Task code */
}
```

- It can be used to generate any number of tasks (separate instances)
- It has its own part of stack (each instance), and priority
- It can be in one of 4 states (RUNNING, BLOCKED, SUSPENDED, READY)
- It is created and deleted by calling API functions of the CMSIS_OS:
  - `osThreadCreate()` and `osThreadDelete()` -> CMSIS_OS v1.x
  - `osThreadNew()` and `osThreadTerminate()` -> CMSIS_OS v2.x

life.augmented

Task function example

```
void FirstTask(void const * argument)
{
 /* task initialization */

 for(;;)
  {
  /* Task code */
  }

 /* we should never be here */

}
```

Run once at first run of each task instance

Run when task instance is in RUN mode

Should be never executed.

life.augmented

Task structure

- A task consists of three parts:
  - The **program code** (ROM)
  - A **stack**, residing in a RAM area that can be accessed by the stack pointer (The stack has the same function as in a single-task system: storage of return addresses of function calls, parameters and local variables, and temporary storage of intermediate calculation results and register values).
  - **TCB** - task control block (data structure assigned to a task when it is created. It contains status information of the task, including the stack pointer, task priority, current task status)
- Two calls to `pvPortMalloc()` are made during task creation. First one allocates TCB, second one allocates the task stack (it is taken from declared FreeRTOS heap area).
- The process of saving the context of a task that is being suspended and restoring the context of a task being resumed is called **context switching**.

life.augmented

Task Control Block (TCB)

Name	Description	condition
*pxTopOfStack	Points to the location of the last item placed on the tasks stack. THIS MUST BE THE FIRST MEMBER OF THE TCB STRUCT	
xMPUSettings	The MPU settings are defined as part of the port layer. THIS MUST BE THE SECOND MEMBER OF THE TCB STRUCT	portUSING_MPU_WRAPPERS == 1
xGenericListItem	The list that the state list item of a task is reference from denotes the state of that task (Ready, Blocked, Suspended).	
xEventListItem	Used to reference a task from an event list	
uxPriority	The priority of the task. 0 is the lowest priority	
*pxStack	Points to the start of the stack	
Task Name	Descriptive name given to the task when created. Facilitates debugging only	
*pxEndOfStack	Points to the end of the stack on architectures where the stack grows up from low memory	portSTACK_GROWTH > 0
uxCriticalNesting	Holds the critical section nesting depth for ports that do not maintain their own count in the port layer	portCRITICAL_NESTING_IN_TCB == 1
uxTCBNumber	Stores a number that increments each time a TCB is created. It allows debuggers to determine when a task has been deleted and then recreated.	configUSE_TRACE_FACILITY == 1
uxTaskNumber	Stores a number specifically for use by third party trace code	configUSE_TRACE_FACILITY == 1
uxBasePriority	The priority last assigned to the task - used by the priority inheritance mechanism	configUSE_MUTEXES == 1
uxMutexesHeld		configUSE_MUTEXES == 1
pxTaskTag		configUSE_APPLICATION_TASK_TAG == 1
ulRunTimeCounter	Stores the amount of time the task has spent in the Running state	configGENERATE_RUN_TIME_STATS == 1
_reent xNewLib_reent	Allocate a Newlib reent structure that is specific to this task. Note Newlib support has been included by popular demand, but is not used by the FreeRTOS maintainers themselves. FreeRTOS is not responsible for resulting newlib operation. User must be familiar with newlib and must provide system-wide implementations of the necessary stubs.	configUSE_NEWLIB_REENTRANT == 1

life.augmented

Task control block (TCB)

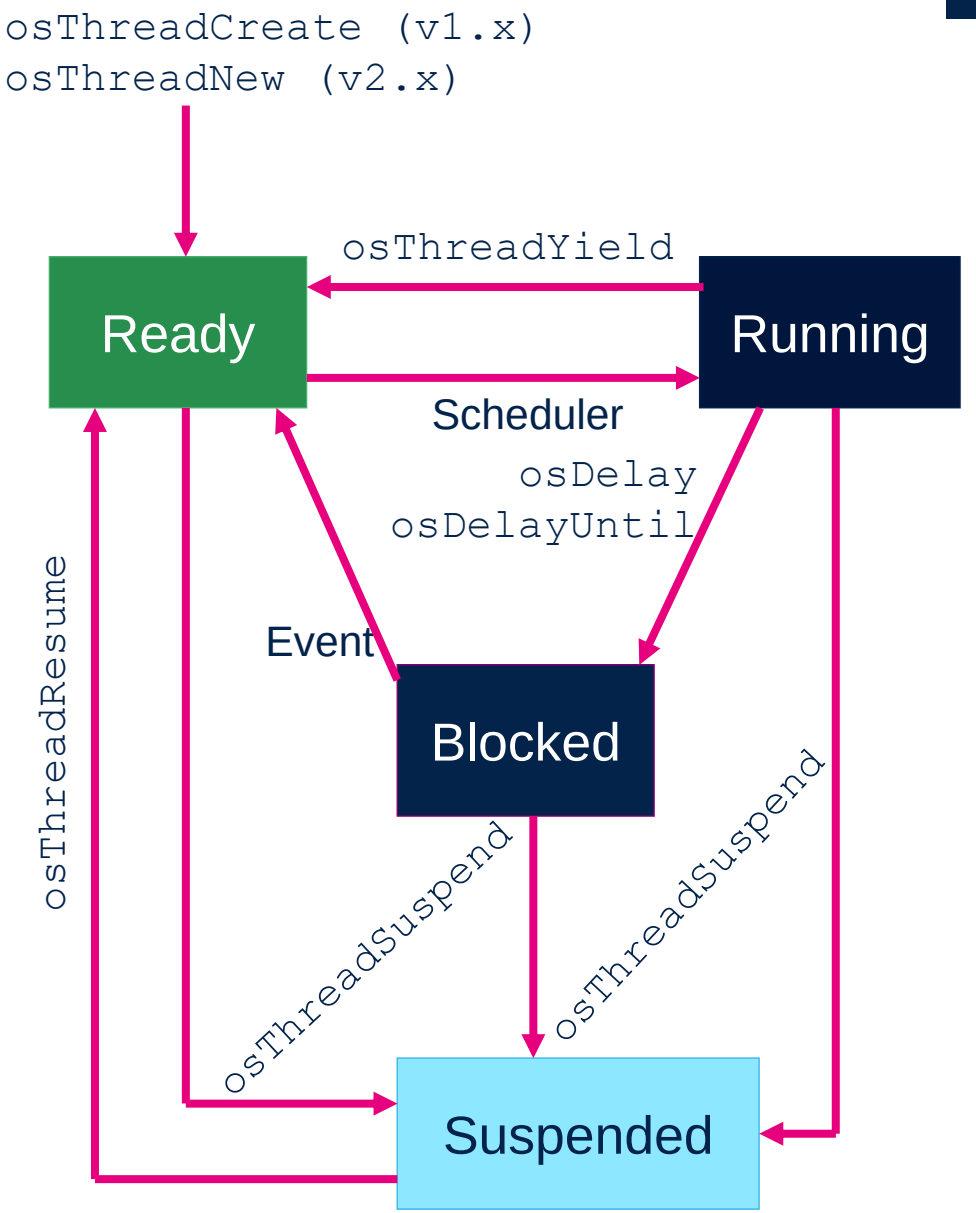
- Main fields within TCB (task.c file)

```
typedef struct tskTaskControlBlock
{
 volatile StackType_t     *pxTopOfStack; //Points to the location of the last item placed on the tasks
stack
        …
 ListItem_t xStateListItem;       //The list that the state list item of a task is reference from denotes
                                 //the state of that task (Ready, Blocked, Suspended )
 ListItem_t xEventListItem;       //Used to reference a task from an event list
 UBaseType_t uxPriority;  //The priority of the task.  0 is the lowest priority
 StackType_t *pxStack;    //Points to the start of the stack
 char pcTaskName[ configMAX_TASK_NAME_LEN ];//Descriptive name given to the task when created.
        …
#if ( configUSE_MUTEXES == 1 )
 UBaseType_t uxBasePriority; //The priority last assigned to the task - for priority inheritance
 UBaseType_t uxMutexesHeld;
#endif
        …
#if( configUSE_TASK_NOTIFICATIONS == 1 )
 volatile uint32_t ulNotifiedValue;
 volatile uint8_t ucNotifyState;
#endif
        …
} tskTCB;
```

life.augmented

Task states

- Ready
  - Task is ready to be executed but is not currently executing because a different task with equal or higher priority is running
- Running
  - Task is actually running (only one can be in this state at the moment)
- Blocked
  - Task is waiting for either a temporal or an external event
- Suspended
  - Task not available for scheduling, but still being kept in memory



life.augmented

Task states – CMSIS_OS v2.x

- Tasks states are stored within `osThreadState_t` enum (cmsis_os.h file)

State name	value	comment
`osThreadInactive`	`0`	Inactive
`osThreadReady`	`1`	READY
`osThreadRunning`	`2`	RUNNING
`osThreadBlocked`	`3`	BLOCKED, SUSPENDED
`osThreadTermimated`	`4`	TERMINATED
`osThreadError`	`-1`	Error
`osThreadReserved`	`0x7FFFFFFF`	Prevents enum down-size compiler optimization

Thread attributes (CMSIS_OS v2.x) cmsis_os2.h

Thread parameters are grouped within lists at `osThreadAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
const char * name	(CubeMX)	Name of the thread
uint32_t attr_bits		Attribute bits
void * cb_mem		Pointer to thread control block memory area
uint32_t cb_size		Thread control block (TCB) size
void * stack_mem		Pointer to thread stack memory area
uint32_t stack_size	(CubeMX),configMINIMAL_STACK_SIZE	Thread stack size (words)
osPriority_t priority	(CubeMX),osPriorityNormal	Initial thread priority (default: osPriorityNormal)
TZ_ModuleId_t tz_module		TrustZone module identifier
uint32_t reserved	0	For future use (must be set to 0)

FreeRTOS – lists management

name	Description	conditions

`ReadyTasksLists[0]`
…
`ReadyTasksList[configMAX_PRIORITIES]`
	Prioritized ready tasks lists separate for each task priority (up to configMAX_PRIORITIES
	Value stored in FreeRTOSConfig.h)
		configMAX_PRIORITIES

`TasksWaitingTermination`	List of tasks which have been deleted but their memory pools are not freed yet.	INCLUDE_vTaskDelete == 1
`SuspendedTaskList`	List of tasks currently suspended	INCLUDE_vTaskSuspend == 1
`PendingReadyTaskList`	Lists of tasks that have been read while the scheduler was suspended	-
`DelayedTaskList`	List of delayed tasks	-
`OverflowDelayedTaskList`	List of delayed tasks which have overflowed the current tick count	-

There is no dedicated list for task in Running mode (as we have only one task in this state at the moment), but the currently run task ID is stored in variable **pxCurrentTCB**

life.augmented

Task priorities

- Each task has assigned a priority from [tskIDLE_PRIORITY] (defined in task.h) to [MAX_PRIORITIES – 1] (defined in FreeRTOSConfig.h)
- The order of execution of tasks depends on this priority
- The scheduler activates the task that has the highest priority of all tasks in the READY state.
- Task with higher priority can preempt running task if configUSE_PREEMPTION (in FreeRTOSConfig.h) is set to 1
- Task priorities can be changed during work of the application

lower number = lower priority

life.augmented

Task priorities
CMSIS_OS v2.x

- Tasks priorities can be set within `osPriority_t` enum (cmsis_os.h file)

Priority name	value	comment
`osPriorityNone`	0	No priority (not initialized)
`osPriorityIdle`	1	Reserved for Idle thread
`osPriorityLow`	8	priority: low
`osPriorityLow1..7`	8+1..7	priority: low + 1..7
`osPriorityBelowNormal`	16	priority: below normal
`osPriorityBelowNormal1..7`	16+1..7	priority: below normal + 1..7
`osPriorityNormal`	24	priority: normal
`osPriorityNormal1..7`	24+1..7	priority: normal + 1..7
`osPriorityAboveNormal`	32	priority: above normal
`osPriorityAboveNormal1..7`	32+1..7	priority: above normal + 1..7
`osPriorityHigh`	40	priority: high
`osPriorityHigh1..7`	40+1..7	priority: high + 1..7
`osPriorityRealtime`	48	priority: realtime
`osPriorityRealtime1..7`	48+1..7	priority: realtime + 1..7
`osPriorityISR`	56	reserved for ISR deferred thread
`osPriorityError`	-1	system cannot determine priority or illegal priority
`osPriorityReserved`	0x7FFFFFFF	prevents enum down-size compiler optimization

Tasks API
CMSIS_OS v2.x

- Create Task example

```
/* Create the thread(s) */
/* definition and creation of Task1 */

  const osThreadAttr_t Task1_attr ={.name=name,
                       .stack_size = stacksize,
                       .priority=(osPriority_t)prio
                      };

 Task1Handle = osThreadNew(Thread, argument,&Task1_attr);
```

Name used for handler

Stack size in bytes

Priority of the task

Name of the function

(void *argument) to be passed to task function

Return value:
- **Task1Handle = NULL** -> error (i.e. lack of heap memory to allocate the stack)
- **Task1Handle != NULL** -> task ID for reference by other functions

Tasks API
CMSIS_OS v2.x

- Task handle definition:

```
/* Private variables -----------------------------------------*/
osThreadId_t Task1Handle;
```

- Create task

```
osThreadId_t osThreadNew (osThreadFunc_t func, void *argument, const osThreadAttr_t *attr)
```

- Delete task

```
osStatus_t osThreadTerminate (osThreadId_t thread_id)
```

- Get currently running task ID

```
osThreadId_t osThreadGetId (void)
```

- Get task name

```
const char *osThreadGetName (osThreadId_t thread_id)
```

Tasks API
CMSIS_OS v2.x

- Yield task

```
osStatus_t osThreadYield(void)
```

- Resume task

```
osStatus_t osThreadResume (osThreadId_t thread_id)
```

- Check state of task

```
osThreadState_t osThreadGetState(osThreadId_t thread_id)
```

- Suspend task

```
osStatus_t osThreadSuspend (osThreadId_t thread_id)
```

- Resume all tasks

```
osStatus_t osThreadResumeAll (void)
```

- Suspend all tasks

```
osStatus_t osThreadSuspendAll (void)
```

Tasks API
CMSIS_OS v2.x

- Get stack size of a task

```
uint32_t osThreadGetStackSize (osThreadId_t thread_id)
```

- Get available stack space of a thread based on stack watermark

```
uint32_t osThreadGetStackSpace(osThreadId_t thread_id)
```

- Change a priority of a task

```
osStatus_t osThreadSetPriority (osThreadId_t thread_id, osPriority_t priority)
```

- Get a current priority of a task

```
osPriority_t osThreadGetPriority (osThreadId_t thread_id)
```

- Delete of currently running task

```
__NO_RETURN void osThreadExit (void)
```

- Delete of a task

```
osStatus_t osThreadTerminate (osThreadId_t thread_id)
```

Threads / Tasks APIs examples

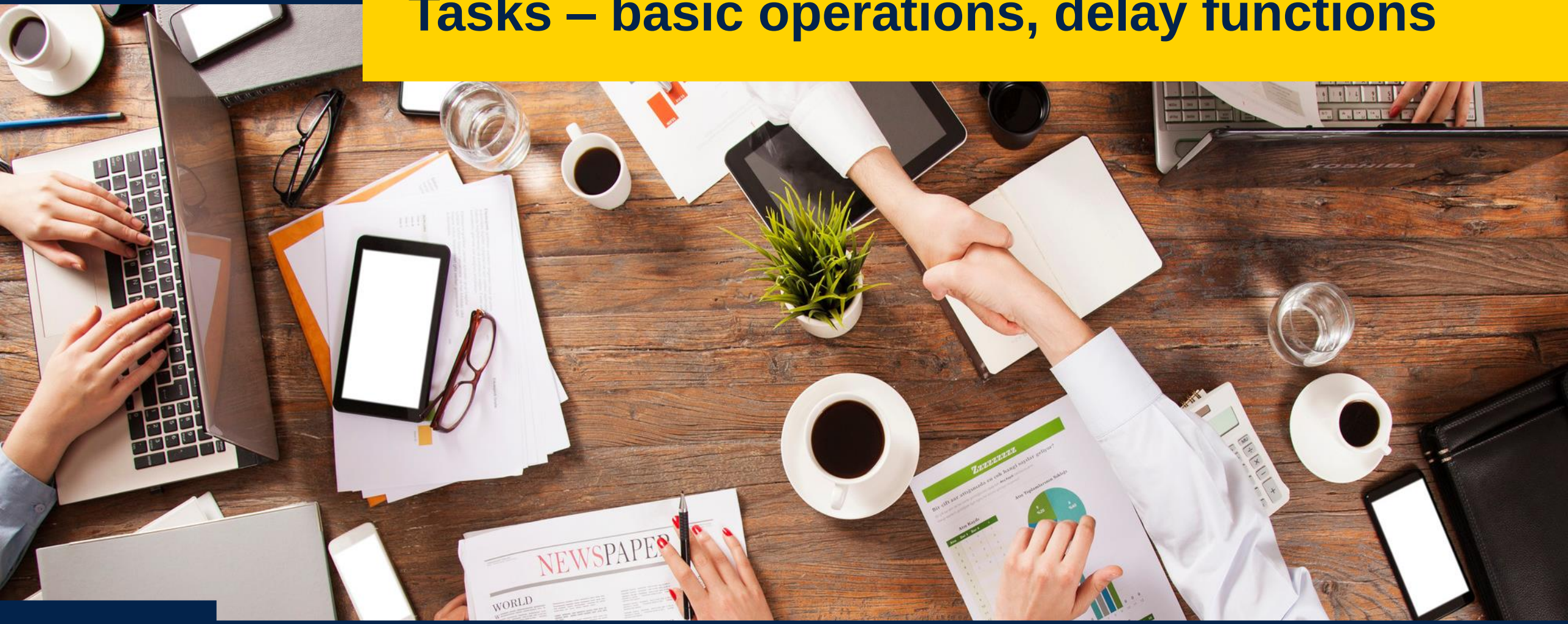
CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osThreadCreate()	osThreadNew()	xTaskCreate()
osThreadGetId()	osThreadGetId()	xTaskGetCurrentTaskHandle()
osThreadTerminate()	osThreadTerminate() osThreadExit()	vTaskDelete()
osThreadYield()	osThreadYield()	taskYIELD()
osThreadSetPriority()	osThreadSetPriority()	vTaskPrioritySet()
osThreadGetPriority()	osThreadGetPriority()	uxTaskPriorityGet() uxTaskPriorityGetFromISR()
osDelay()	osDelay()	vTaskDelay()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Tasks – basic operations, delay functions

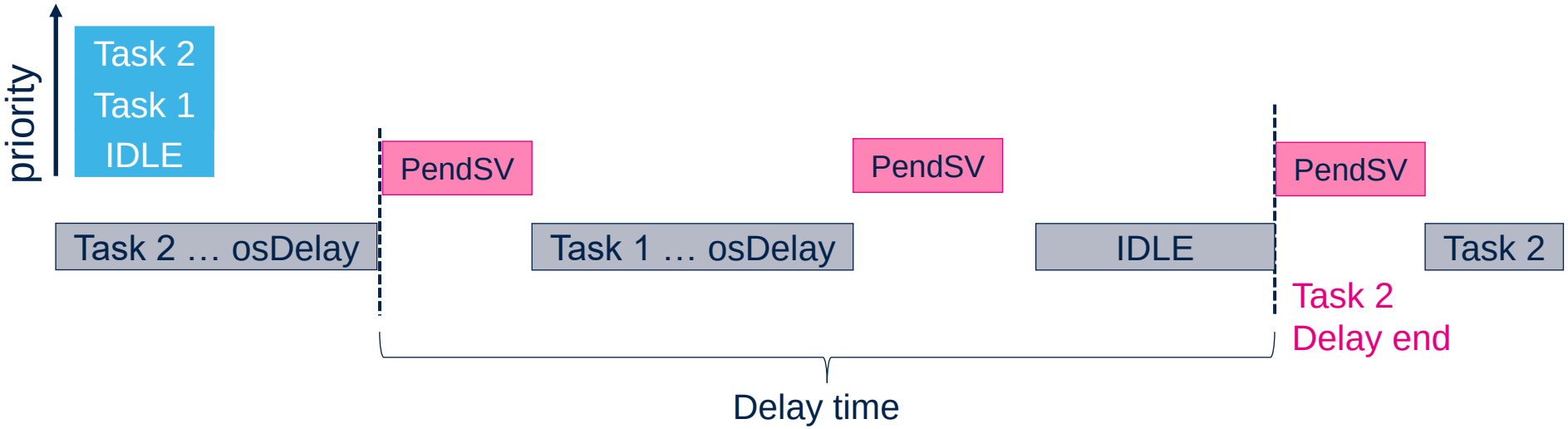


life.augmented

Include **vTaskDelay**

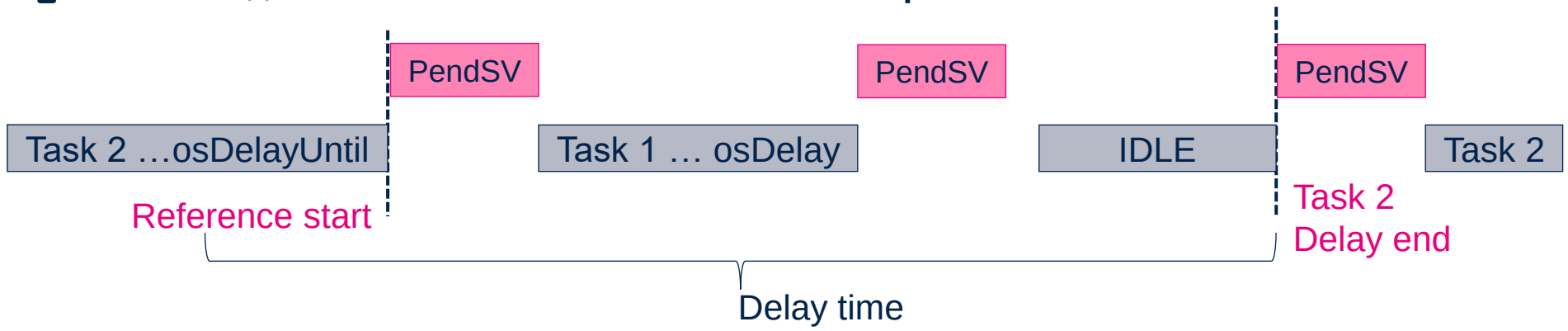
osDelay(), osDelayUntil functions

- **`osDelay()`** start measure time from osDelay call



Include **vTaskDelayUntil**

- **`osDelayUntil()`** starts measure time from point which we selected



life.augmented

Include **vTaskDelayUntil**

Include **vTaskDelay**

osDelay API

- Delay function

```
osStatus_t osDelay (uint32_t millisec)
```

- Delay function which measure time from which is delay measured

```
osStatus_t osDelayUntil (uint32_t millisec)
```

Include **vTaskDelay**

osDelay API step by step

- `osDelay()` calls `vTaskDelay()` (**tasks.c** file)
- `vTaskDelay()` is performing the following list of operations:
  - Calls `vTaskSuspendAll()` to pause the scheduler without disabling interrupts. RTOS tick will be held pending until the scheduler has been resumed.
  - Remove task from event list (running tasks) and move it to delayed list with given delay value using the function `prvAddCurrentTaskToDelayedList()`
  - Resume the scheduler using `xTaskResumeAll()` function
  - Trigger PendSV interrupt (using `portYIELD_WITHIN_API()` macro) to switch the context

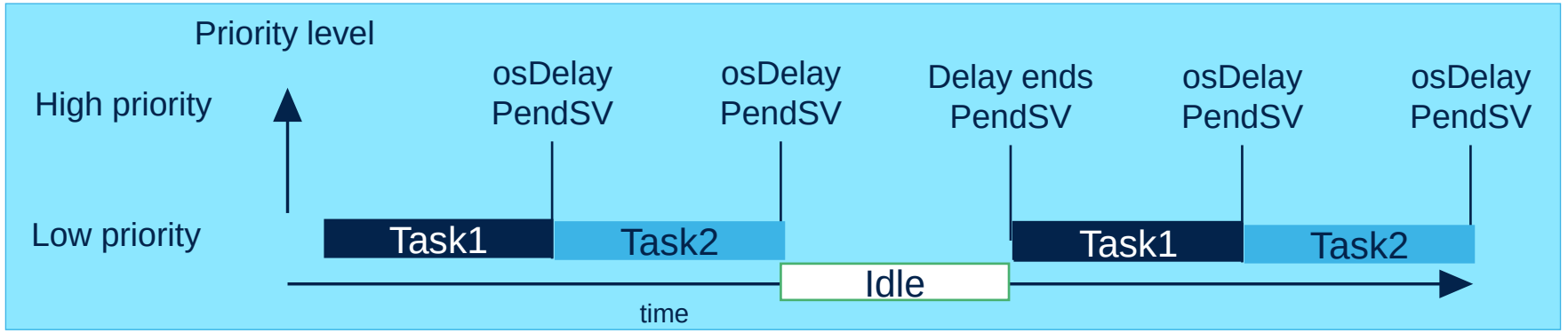
Tasks lab – the purpose

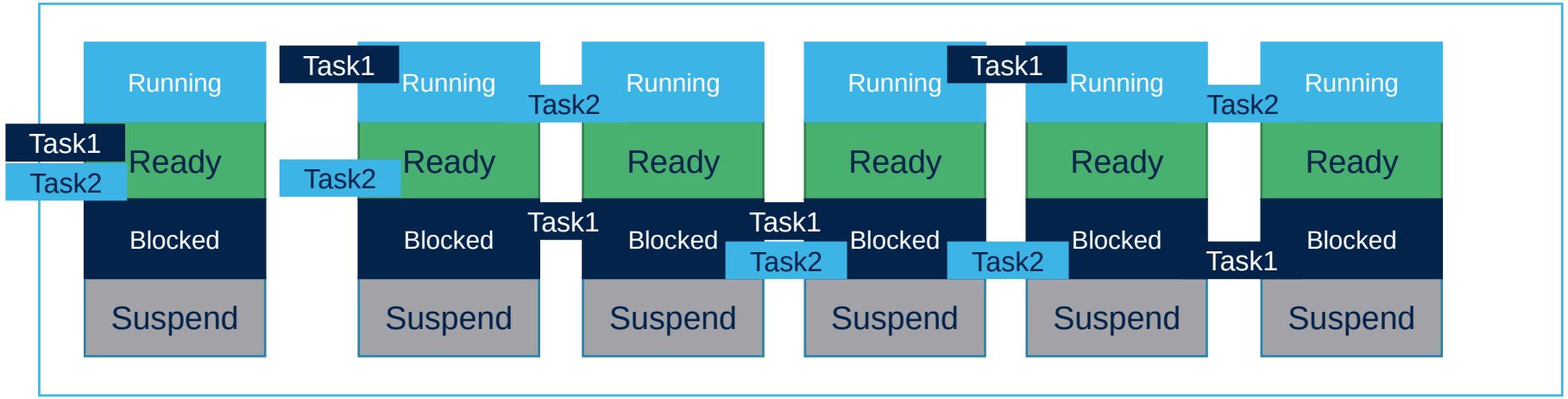
- **To save some time needed for new project creation we can reuse previous exercise done at the beginning of the session**

- In case you would like to store each exercise please use File -> Export option and use an example in dedicated zip file (Ex1_intro.zip)
- In case you would like to make an exercise from scratch for your STM32 platform, please create a new project within STM32CubeMX or STM32CubeIDE for your MCU, then:
  - Change timebase for HAL from SystTick -> TIM6 (it can be done within SYS peripherals group)
  - Enable FreeRTOS in version CMSIS_OS v2.x

Tasks lab

- If both Delays are processed the FreeRTOS is in idle state





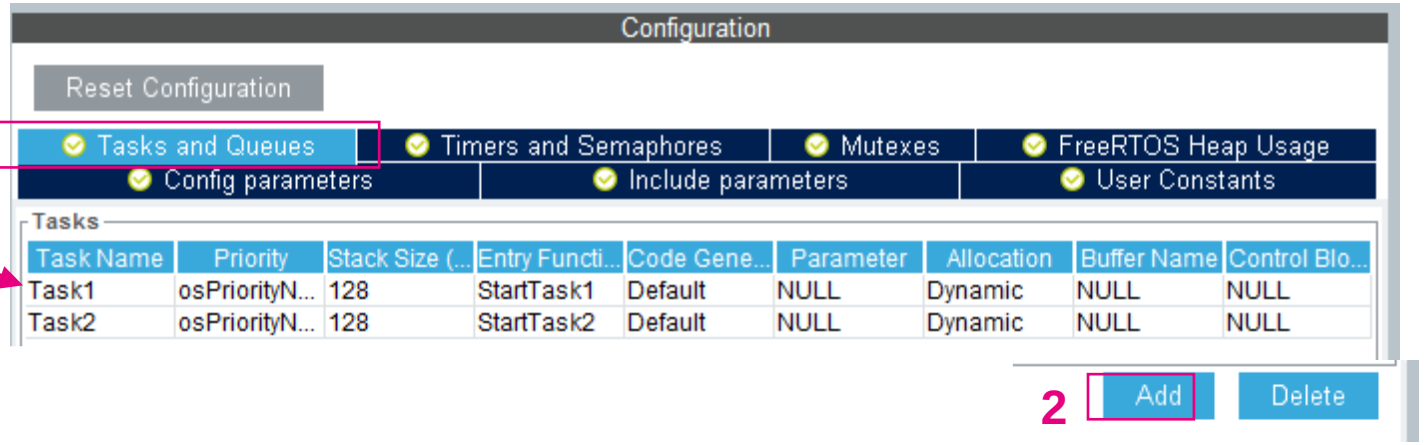
life.augmented

Tasks lab – adding tasks

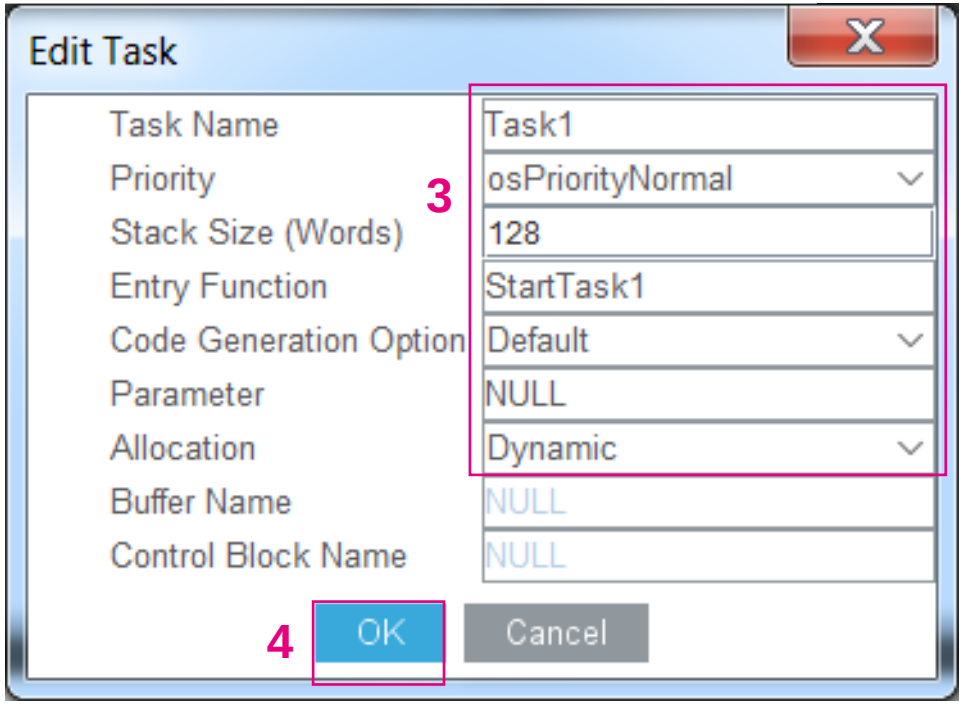
Press **FreeRTOS** button within Pinout&Configuration tab and select „Tasks and Queues”

To Edit existing task 2x





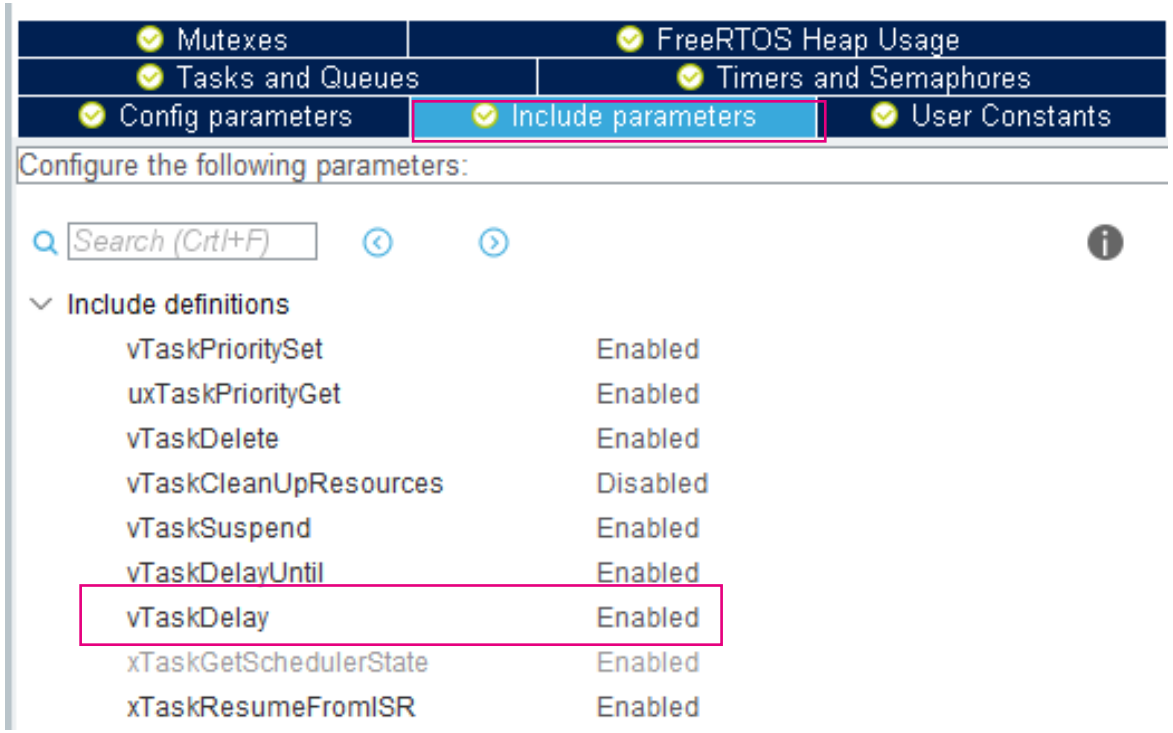
- We need to create 2 tasks:
  - **Task1 (can be edited from defaultTask)**:
    - Priority: osPriorityNormal
    - Stack Size: 128 Words
    - Entry Function: StartTask1
    - Code Generation: Default
    - Parameter: NULL
    - Allocation: Dynamic
  - **Task2**:
    - Priority: osPriorityNormal
    - Stack Size: 128 Words
    - Entry Function: StartTask2
    - Code Generation: Default
    - Parameter: NULL
    - Allocation: Dynamic



life.augmented

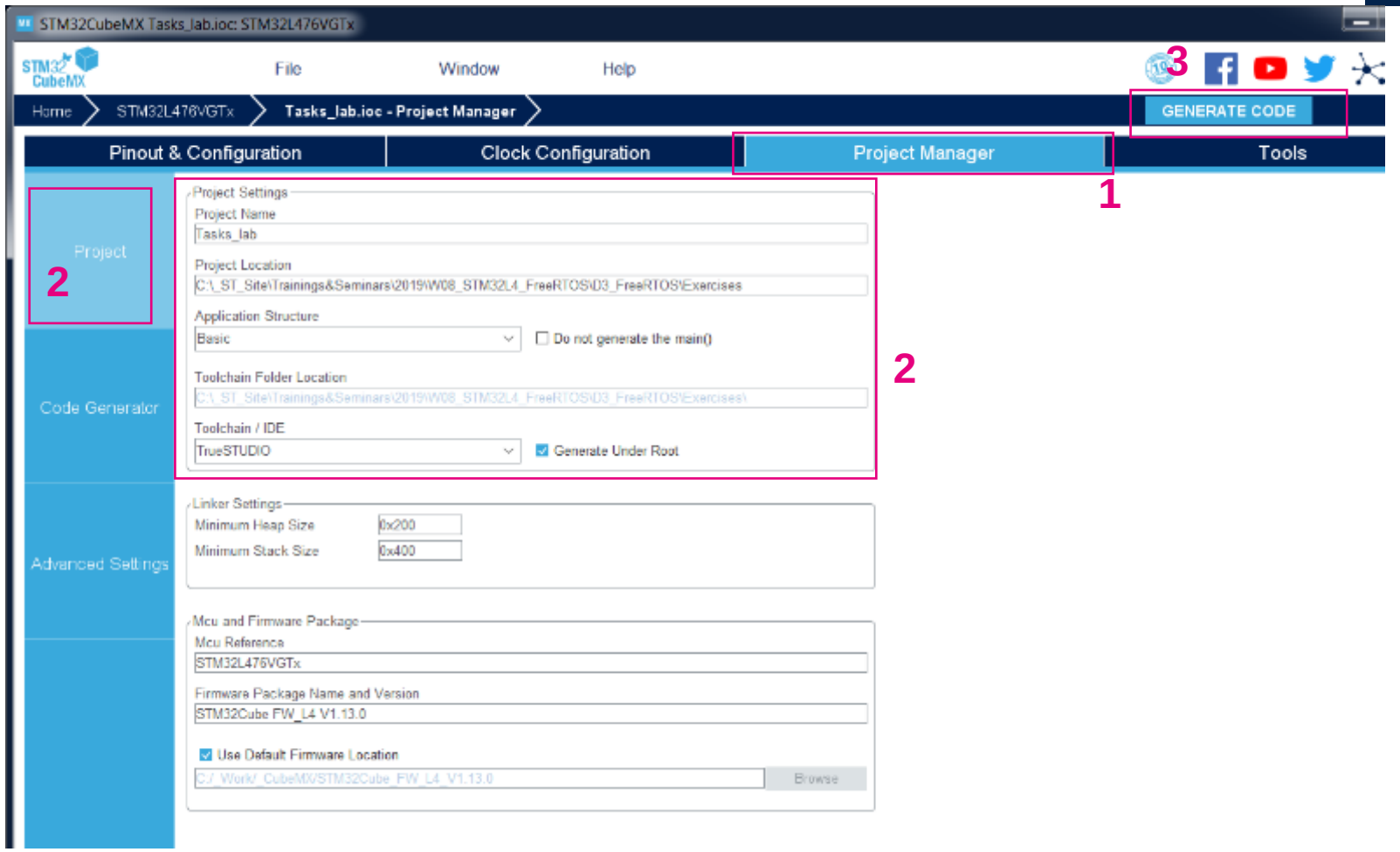
osDelay()

- Enable `vTaskDelay` in Include parameters



Task lab – code generation

- To configure the project
  - Select Project Manager tab
  - Within Project tab select:
    - project name
    - Project location
    - Type of toolchain (only in STM32CubeMX)
- To Generate Code
  - Select Generate Code button (in STM32CubeMX) or just press Ctrl+S in STM32CubeIDE



life.augmented

Tasks lab - code generated by STM32CubeMX (CMSIS_OS v2.x) – main.c file

- Any component in FreeRTOS need to have a handler

```
/* Private variables -----------------------------------------*/
osThreadId_t Task1Handle;
osThreadId_t Task2Handle;
```

- Task function prototypes

```
/* Private function prototypes -------------------------------*/
…
void StartTask1(void *argument);
void StartTask2(void *argument);
```

- Before any OS component creation we need to initialize the Kernel

```
/* USER CODE END 2 */

osKernelInitialize();
```

Tasks lab - code generated by STM32CubeMX (CMSIS_OS v2.x) – main.c file

- We can create already defined tasks and other OS components before the scheduler is started (it is possible to create and delete tasks within working OS as well).

```
 /* Create the thread(s) */
  /* definition and creation of Task1 */
const osThreadAttr_t Task1_attributes = {
    .name = "Task1",
    .priority = (osPriority_t) osPriorityNormal,
    .stack_size = 128
  };
  Task1Handle = osThreadNew(StartTask1, NULL, &Task1_attributes);

 /* definition and creation of Task2 */
const osThreadAttr_t Task2_attributes = {
    .name = "Task2",
    .priority = (osPriority_t) osPriorityNormal,
    .stack_size = 128
  };
  Task2Handle = osThreadNew(StartTask2, NULL, &Task2_attributes);
```

Define task parameters

Create task, allocate memory

Include **vTaskDelay**

Tasks lab
some code modifications – main.c file

- Start the scheduler. This function should never end *)

```
/* Start scheduler */
 osKernelStart();
```

- Once scheduler selects task for execution, an assigned function is called (in case of Task1 it is StartTask1()). Please add there Task_action() and osDelay() functions like below.

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    Task_action('1');
    osDelay(1000);
  }
  /* USER CODE END 5 */
}
```

Endless loop

osDelay will start context switch

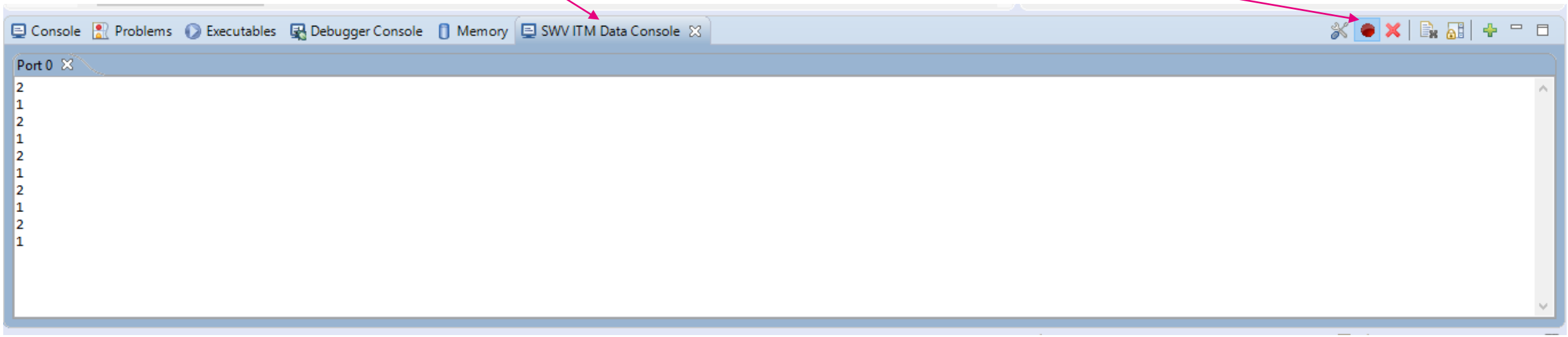
- Similar code prepare for Task2 function (i.e. with Task_action(’2’) instead).

life.augmented

*) if it ends, it means that we are out of declared heap size and there was not enough memory space to create a new task

Tasks lab
run the example

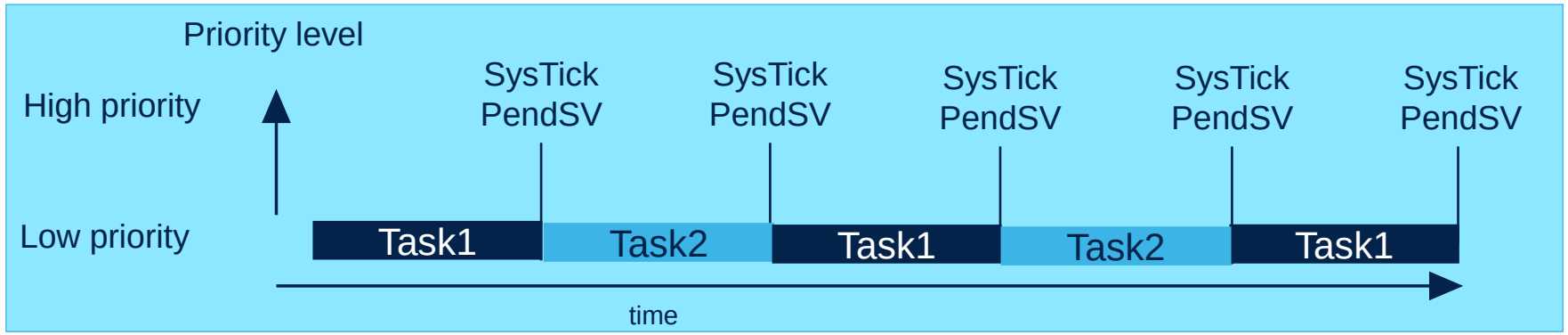
- Compile the code 
- Run debug session 
- Turn on SWV ITM Data Console and Start Trace (in case it is used within Task_action())

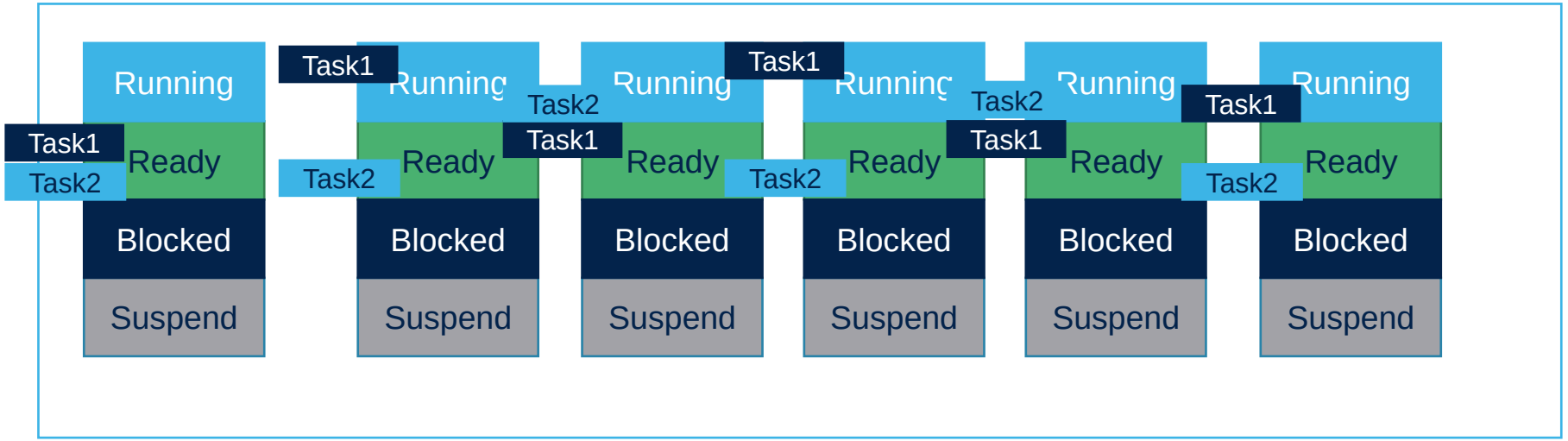


life.augmented

Tasks lab

- Without Delays the threads will be in Running state or in Ready state
- Use `HAL_Delay()`





life.augmented

Tasks lab
task priorities

- We will continue with previous example
- Double click on task for change
- Increase the priority of Task1 (to `osPriorityNormal2`)
- Button OK
- Regenerate the code and compile it
- Is there any difference in during debug?
- What could be done to see the difference?



Tasks and Queues | Timers and Semaphores | Mutexes | FreeRTOS Heap Usage

Config parameters | Include parameters | User Constants

Tasks

Task Name	Priority	Stack Siz...	Entry Function	Code Generat...	Parameter	Allocation	Buffer Name	Control Block...
Task1	osPriorityNormal2	128	StartTask1	Default	NULL	Dynamic	NULL	NULL
Task2	osPriorityNormal	128	StartTask2	Default	NULL	Dynamic	NULL	NULL

life.augmented

Tasks lab
task priorities

- If higher priority task is not running we can print text from this task

```
/* StartTask2 function */
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  /* Infinite loop */
  for(;;)
  {
    Task action('2');
    HAL_Delay(50);
  }
  /* USER CODE END StartTask2 */
}
```

Helps not spam terminal

life.augmented

Include **vTaskDelay**

Tasks lab task priorities

- Send text 5 times and put task to block state for 1 second
- Because task have high priority it allows to run lower priority task

```
/* USER CODE END 4 */
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  uint32_t i = 0;
  /* Infinite loop */
  for(;;)
  {
    for (i = 0; i < 5; i++){
      Task_action('1');
      HAL_Delay(50);
    }
    osDelay(1000);
  }
  /* USER CODE END 5 */
}
```

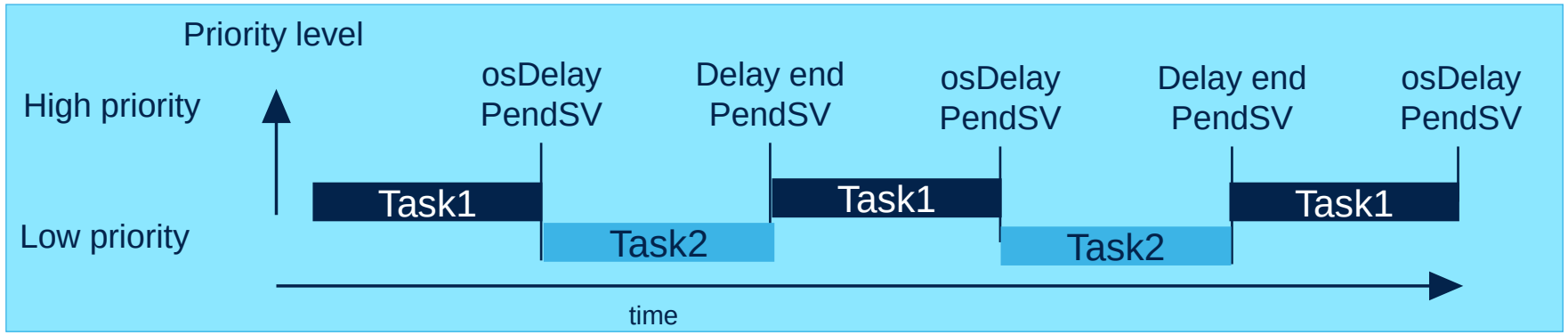
Helps not spam terminal

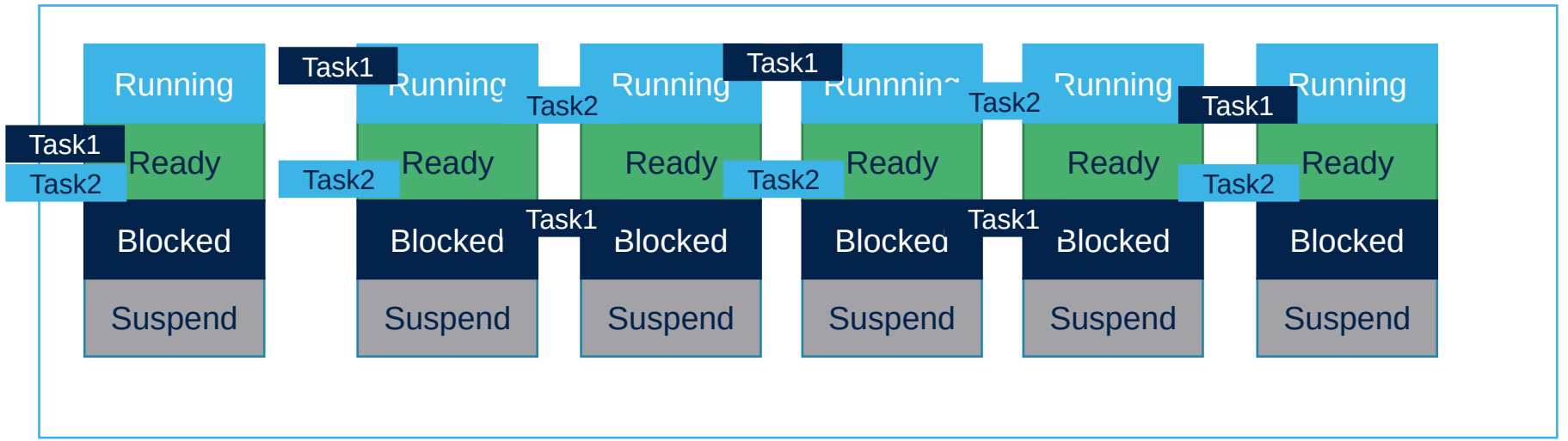
Block task

life.augmented

Tasks lab

- There is a time for Task2 due to `osDelay()` within Task1 service function ?





life.augmented

Tasks lab
task priorities

- Send text 5 times and put task to block state for 1 second
- Because task have high priority it allows to run lower priority task
- Comment `osDelay()` line in the code below and observe the difference

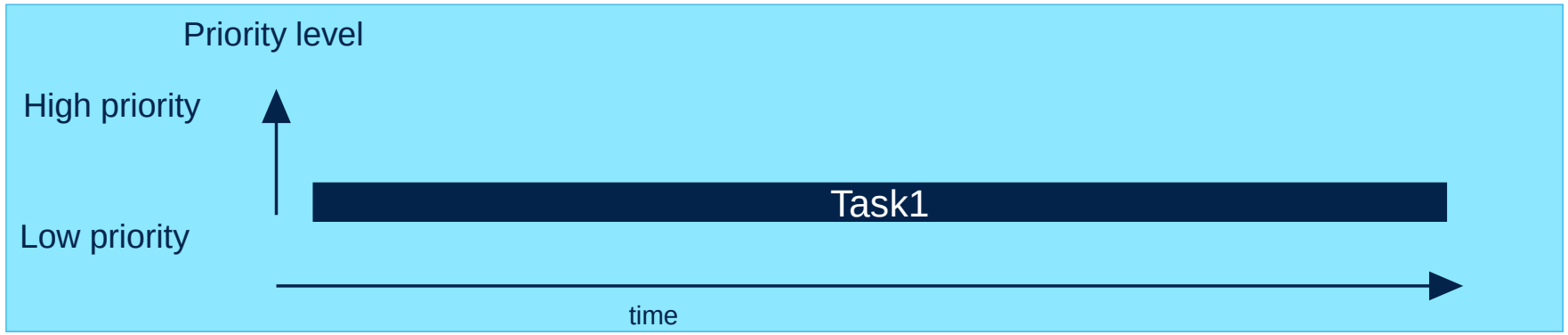
```
/* USER CODE END 4 */
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  uint32_t i = 0;
  /* Infinite loop */
  for(;;)
  {
    for (i = 0; i < 5; i++){
      Task_action('1');
      HAL_Delay(50);
    }
    // osDelay(1000);
  }
  /* USER CODE END 5 */
}
```

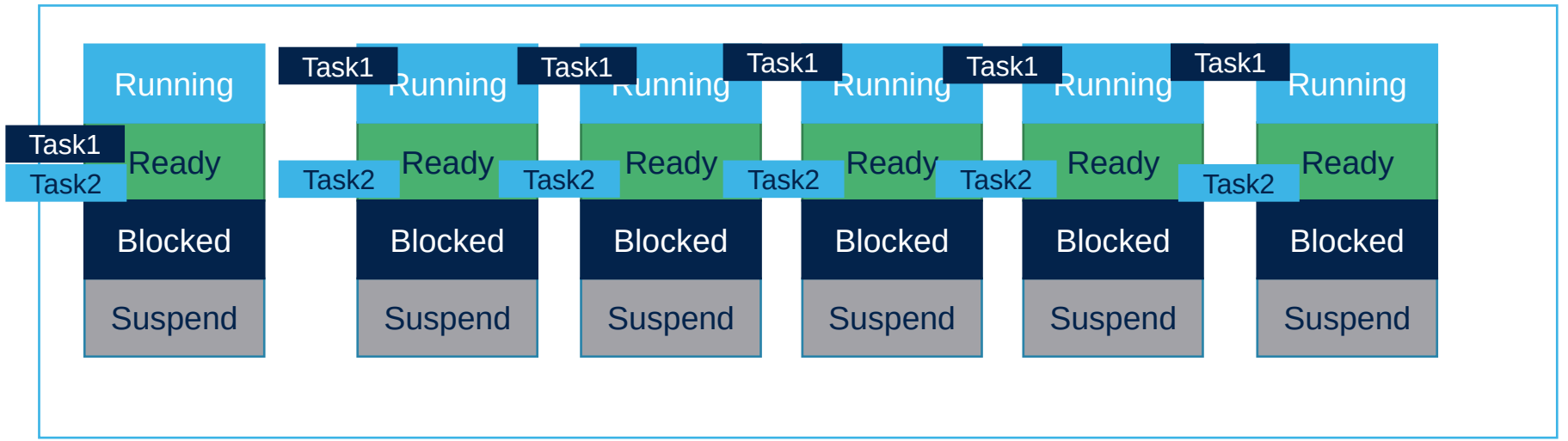
Helps not spam terminal

life.augmented

Tasks lab

- If we remove `osDelay()` Task1 will be executed continuously





life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Tasks – manipulating on its priority



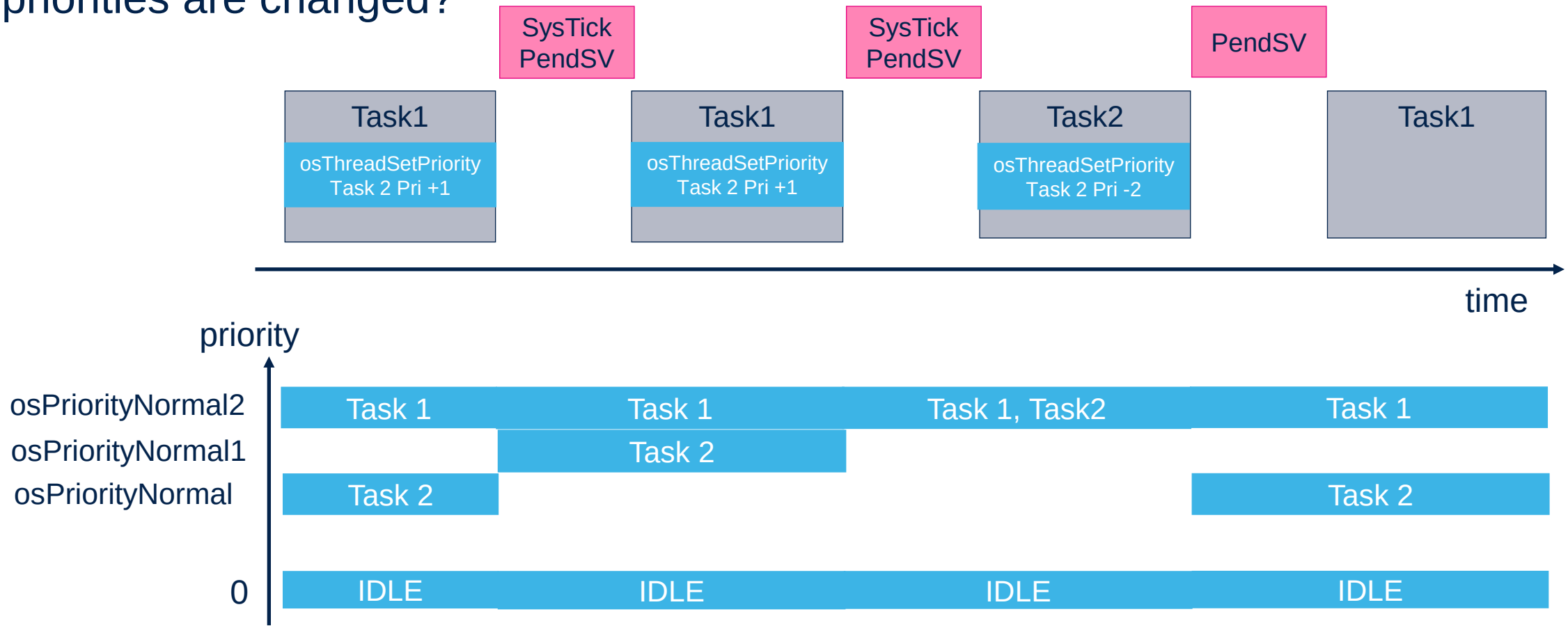
life.augmented

Include **vTaskPrioritySet**

Include **uxTaskPriorityGet**

Changing priority of the task

- How priorities are changed?



life.augmented

Include **uxTaskPriorityGet**

Thread priority get API step by step

- `osThreadGetPriority()` calls `uxTaskPriorityGet()` or `uxTaskPriorityGetFromISR()` (**tasks.c** file)

- `uxTaskPriorityGet()` is performing the following list of operations:
  - Entering into critical section (to avoid any parallel operations on OS) using `taskENTER_CRITICAL()` in case of executing from thread mode or `portSET_INTERRUPT_MASK_FROM_ISR()` in case of interrupt mode
  - Read priority value from TCB of the given task using function `prvGetTCBFromHandle(TCB_t xTask)`
  - extract Priority value from the TCB structure (`uxPriority` field)
  - Exit from critical section using `taskEXIT_CRITICAL()` in case of executing from thread mode or `portCLEAR_INTERRUPT_MASK_FROM_ISR()` in case of interrupt mode

Include **vTaskPrioritySet**

Thread priority set API step by step

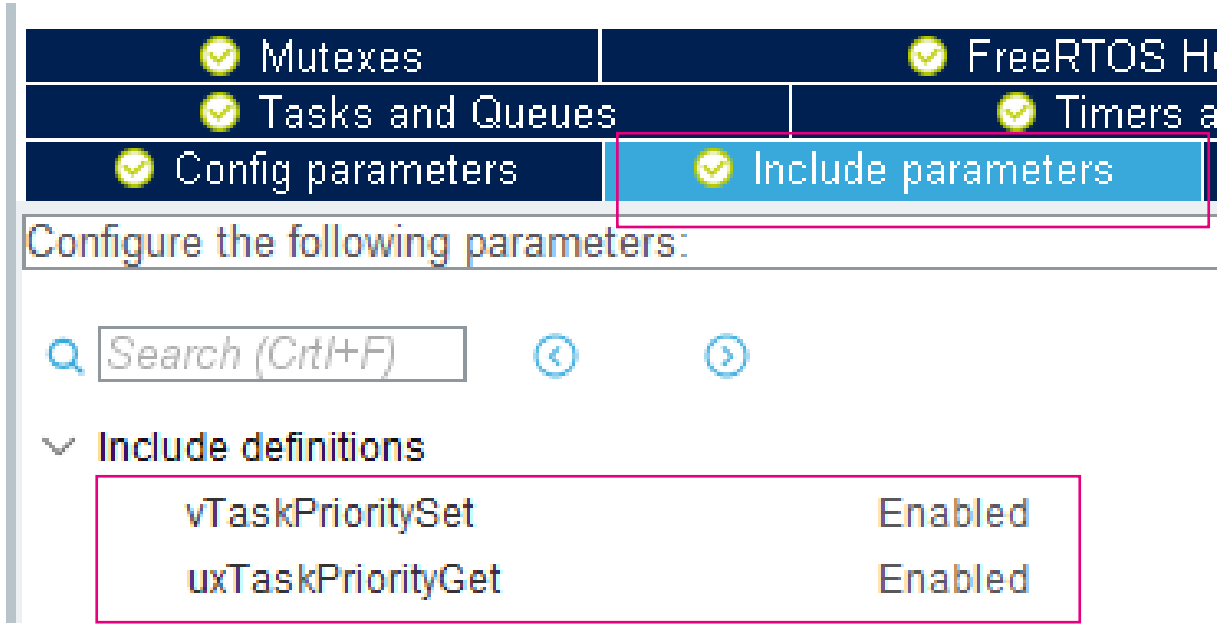
- `osThreadSetPriority()` calls `vTaskPrioritySet()` (**tasks.c** file)
- `vTaskPrioritySet()` is performing the following list of operations:
  - Entering into critical section (to avoid any parallel operations on OS) using `taskENTER_CRITICAL()`
  - Set given priority value to TCB of the given task
  - Checks whether task should not be moved to different task list due to new priority
  - Exit from critical section using `taskEXIT_CRITICAL()`

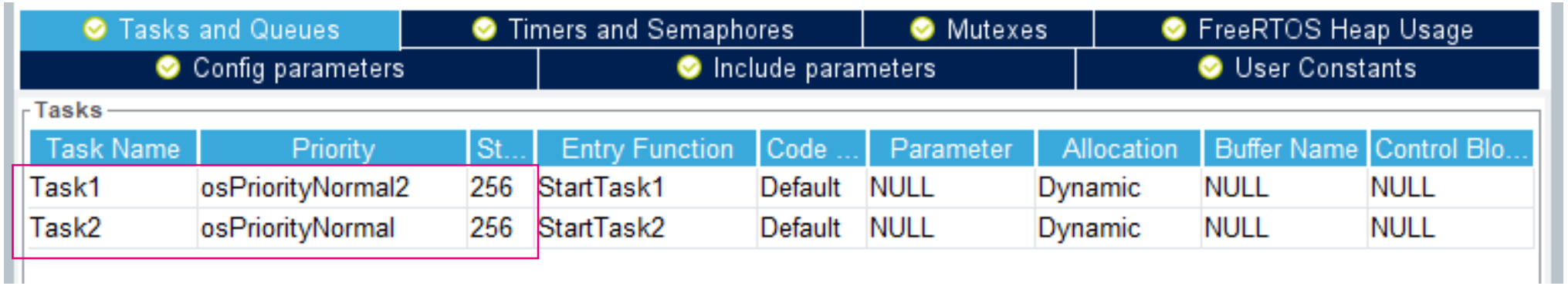
Include **vTaskPrioritySet**

Include **uxTaskPriorityGet**

Priority change lab

- Let's reuse previous exercise where Task1 has higher priority than Task2
- Increase stack size of both tasks to 256
- Enable `vTaskPriorityGet` and `uxTaskPrioritySet` in IncludeParameters





Task Name	Priority	St...	Entry Function	Code ...	Parameter	Allocation	Buffer Name	Control Blo...
Task1	osPriorityNormal2	256	StartTask1	Default	NULL	Dynamic	NULL	NULL
Task2	osPriorityNormal	256	StartTask2	Default	NULL	Dynamic	NULL	NULL

life.augmented

Include **vTaskPrioritySet**

Include **uxTaskPriorityGet**

Priority change lab CMSIS_OS v2.x

- Modify Task1 service function in order to increase Task2 priority by 1:

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  osPriority_t priority;
  /* Infinite loop */
  for(;;)
  {
    priority=osThreadGetPriority(Task2Handle);
    Task_action('1');
    osThreadSetPriority(Task2Handle,priority+1);
    HAL_Delay(1000);
  }
  /* USER CODE END 5 */
}
```

Read Task2 priority

Increase Task2 priority

life.augmented

Include **vTaskPrioritySet**

Include **uxTaskPriorityGet**

Priority change lab CMSIS_OS v2.x

- Modify Task2 service function in order to decrease its priority by 2:

```
/* StartTask2 function */
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  osPriority_t priority;
  /* Infinite loop */
  for(;;)
  {
    priority=osThreadGetPriority(Task2Handle);
    Task_action('2');
    osThreadSetPriority(Task2Handle,priority-2);
  }
  /* USER CODE END StartTask2 */
}
```

Read priority of current task

Decrease task priority

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

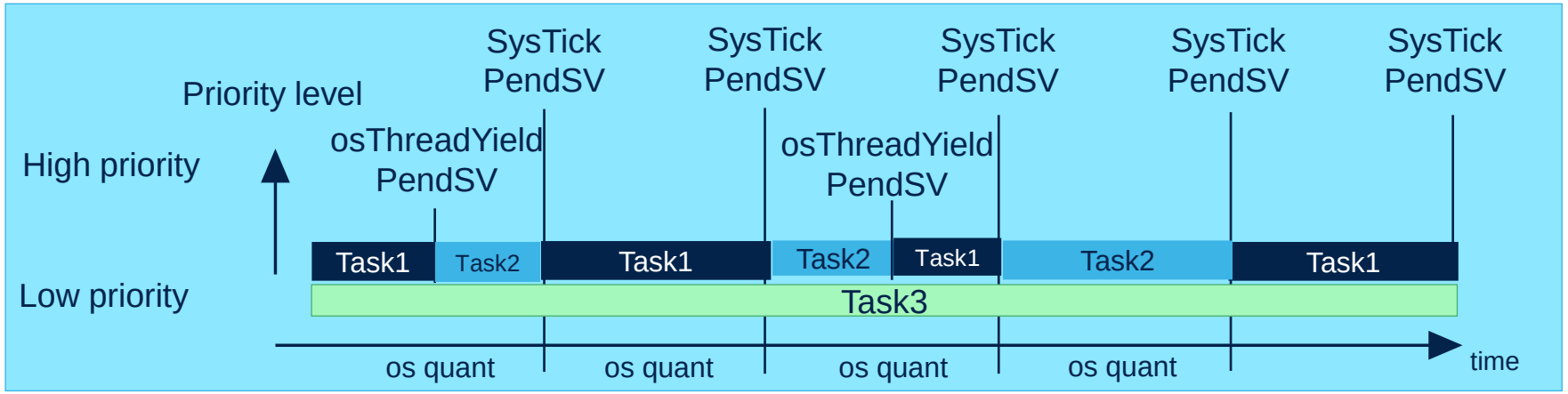
FreeRTOS – CMSIS_OS v2
Tasks – finishing job earlier - TaskYield

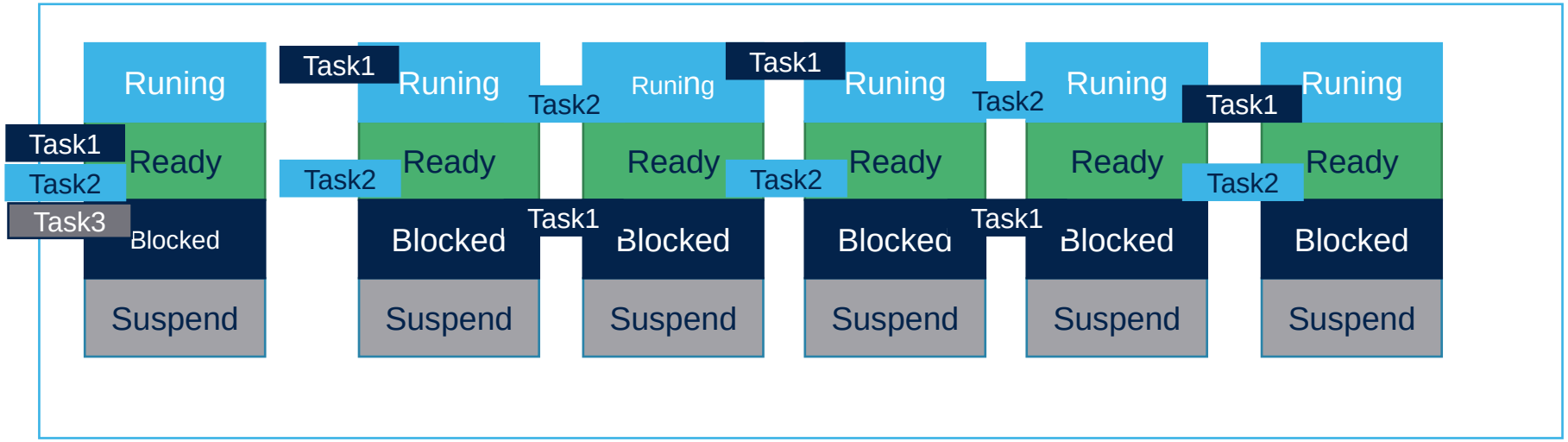


life.augmented

If the task has finished its job earlier…

- `osThreadYield()` – moves the task from Run to Ready state. Next task with the same priority will be executed.

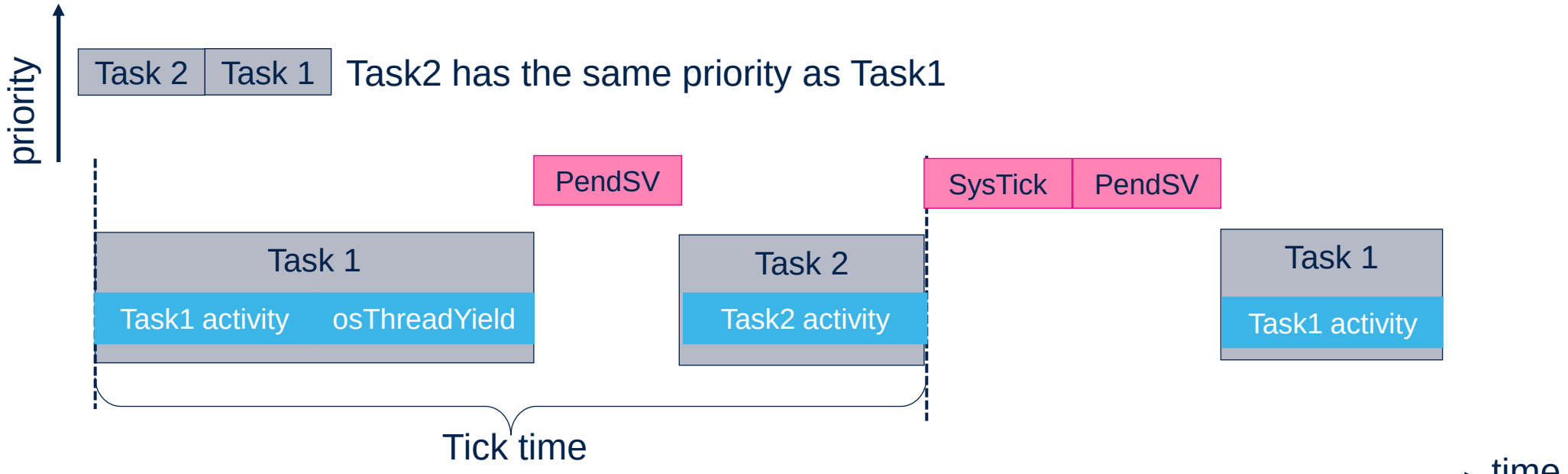




life.augmented

osThreadYield() function

- **`osThreadYield()`** function is used to end task activity once the job is done to not wait for the tick.
- It moves task from **RUN** mode to **READY**
- It makes sense if we have few tasks on the same priority otherwise yielded task will be executed again



osThreadYield API step by step

- `osThreadYield()` calls `taskYIELD()` (**cmsis_os.c** file) which is defined as `portYIELD()` (**task.h** file)

- `portYIELD()` function (**portmacro.h** file) triggers PendSV interrupt to request a context switch to the next task from ready list

An example (version for IAR C compiler):

```
#define portYIELD()
{
 /* Set a PendSV to request a context switch. */
 portNVIC_INT_CTRL_REG = portNVIC_PENDSVSET_BIT;
 __DSB();
 __ISB();
}
```

life.augmented

Task yield lab
CMSIS_OS v2.x

- Let's use previous lab, configure the same priority (i.e. `osPriorityNormal`) for both tasks
- Modify body of Task1 and Task2 service functions like below

```
/* USER CODE END Header_StartTask1*/
void StartTask1(void *argument)
{
  /* USER CODE BEGIN 5 */

  /* Infinite loop */
  for(;;)
  {
    Task_action('1');
    osThreadYield();
  }
  /* USER CODE END 5 */
}
```

```
/* USER CODE END Header_StartTask2*/
void StartTask2(void *argument)
{
  /* USER CODE BEGIN StartTask2 */

  /* Infinite loop */
  for(;;)
  {
    Task_action('2');
  }
  /* USER CODE END StartTask2 */
}
```

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Tasks – creating and deteleting them



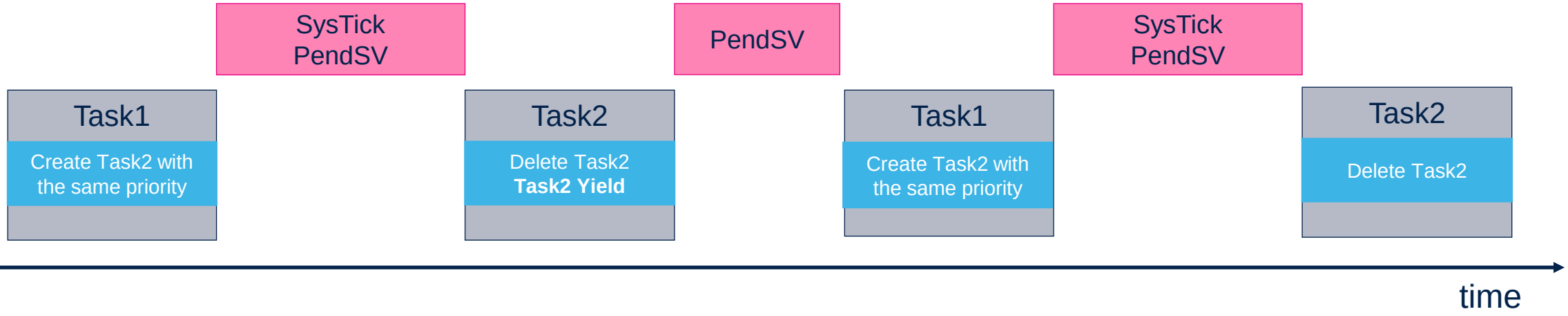
life.augmented

Include **vTaskDelete**

Creating and deleting tasks

- Example how to delete tasks
- Modify StartTask2() to delete Task2

- The result should be the following:



life.augmented

Creating and deleting tasks lab CMSIS_OS v2.x

- Example how to create tasks (based on previous exercise)
- Change priority for Task1 to `osPriorityNormal` (to have the same for both tasks)
- Comment Task2 creation part in main.c

```
 /* definition and creation of Task2 */
//const osThreadAttr_t Task2_attributes = {.name = "Task2", .priority = (osPriority_t)
//osPriorityNormal,.stack_size = 128 };
//  Task2Handle = osThreadNew(StartTask2, NULL, &Task2_attributes);
```

- Modify Task1 to create Task2

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
   const osThreadAttr_t Task2_attributes = { .name = "Task2",
   .priority = (osPriority_t) osPriorityNormal, .stack_size = 128 };
  /* Infinite loop */
  for(;;)
  {
    Task_action('1');
    Task2Handle = osThreadNew(StartTask2, NULL, &Task2_attributes);
    osDelay(1000);
  }
  /* USER CODE END 5 */
}
```

Task 2 creation

Include **vTaskDelete**

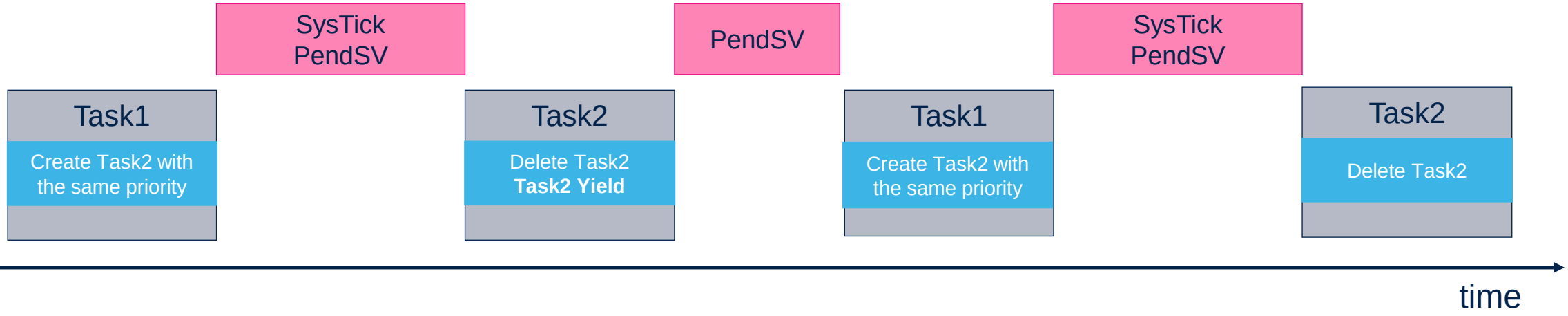
Creating and deleting tasks lab

- Example how to delete tasks
- Modify StartTask2() to delete Task2

```
/* StartTask2 function */
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  /* Infinite loop */
  for(;;)
  {
    Task_action('2');
    osThreadTerminate(Task2Handle);
    Task_action('x');
  }
  /* USER CODE END StartTask2 */
}
```

Delete Task2

- The result should be the following:



life.augmented

Include **vTaskDelete**

osThreadTerminate API - step by step

- `osThreadTerminate()` calls `vTaskDelete()` (**cmsis_os.c** file)
- The only argument specifies the ID of the task to be deleted (NULL argument, as indication to currently executed task is not supported in CMSIS_OS v2).
- `vTaskDelete()` function (**task.c** file):
  - Within critical section (started by `taskENTER_CRITICAL()` macro which is running `vPortEnterCritical()` defined in **port.c** file) removes the task from the ready list using function `uxListRemove()` and removes the task from waiting on an event tasks list.
  - In case the task is deleting itself function is switching execution to the next task calling function `portYIELD_WITHIN_API()` which could be in fact `portYIELD()` function (default setting, **FreeRTOS.h** file)

Memory allocated by the task code is not automatically freed and should be freed before the task is deleted, TCB and its original stack are freed by IDLE Task.

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Intertask communication



life.augmented

CMSIS OS inter-task communication

- Queues. Allows to pass more information between the tasks. Suspend task if tries to “put” to full queue or “get” from empty one.
- Semaphores are used to communication between the tasks without specifying the ID of the thread who can accept it. It allows counting multiple events and can be accepted by many threads.
- Event Flags are used to synchronize task with multiple events (OR-ed together). There could be 8 or 24 bit value used here (depends on configUSE_16_BIT_TICKS settings) – not implemented in CMSIS_OS API v1.x
- Direct to task notifications are used to precise communication between the tasks. It is necessary to specify within signal thread id.
- Mutexes are used to guard the shared resources. It must be taken and released always in that order by each task that uses the shared resource.

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Queues



life.augmented

Queues (1/2)

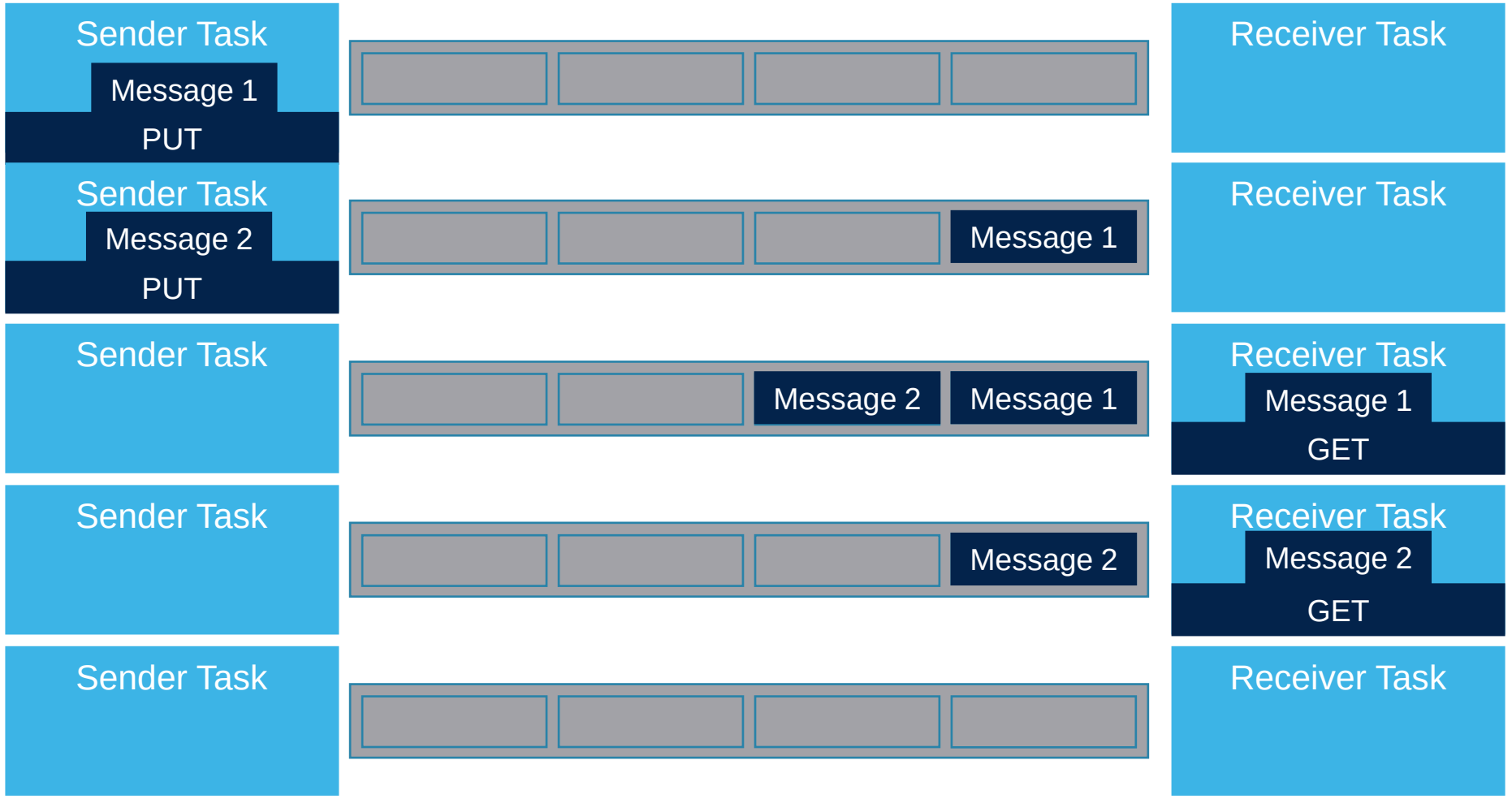
- Queues are pipes to transfer data between tasks in RTOS
- By default queue is behaving as FIFO (First In - First Out); can be redefined to perform as LIFO (Last In - First Out) structure by using **xQueueSendToFront()** function (not available in current CMSIS-RTOS API).
- All data send by queue must be of the same type, declared during queue creation phase. It can be simple variable or structure.
- Within CMSIS-RTOS API there are two types of queues:
  - **Message** where one can send only integer type data or a pointer
  - **Mail** where one can send memory blocks

Queues (2/2)

- Length of queue is declared during creation phase and is defined as a number of items which will be send via queue.
- Operations within queues are performed in **critical sections** (blocking interrupts by programming BASEPRI register for the time of operation on queue).
- Tasks can block on queue sending or receiving data with a timeout or infinitely.
- If multiple tasks are blocked waiting for receiving/sending data from/to a queue then only the task with the highest priority will be unblocked when a data/space is available. If both tasks have equal priority the task that has been waiting the longest time will be unblocked.

life.augmented

Queue



life.augmented

Queue
CMSIS_OS v2.x

```
const osMessageQueueAttr_t attr = { .name = "Queue1" };
```

Queue name

Queue attributes

- Create Queue:

```
osMessageQueueId_t osMessageQueueNew(
                        uint32_t msg_cnt,
                        uint32_t msg_size,
                        const osMessageQueueAttr_t *attr);
```

Queue Handle

Max number of messages in the queue

Message size in bytes

Pointer to message queue attributes

- Delete the queue

```
osStatus_t osMessageQueueDelete (osMessageQueueId_t queue_id )
```

Queue handle

Queue
CMSIS_OS v2.x

- Put data into Queue

```
osStatus_t osMessageQueuePut (osMessageQueueId_t queue_id,
                              const void *msg_ptr,
                              uint8_t msg_prio,
                              uint32_t timeout);
```

Queue handle

Pointer to buffer with message to put into queue

Message priority – not used

Status of the operation (0 – ok)

Time in ms to wait till operation will be completed

- Receive data from Queue

```
osStatus_t osMessageQueueGet (osMessageQueueId_t queue_id,
                              const void *msg_ptr,
                              uint8_t *msg_prio,
                              uint32_t timeout);
```

Queue handle

Pointer to buffer for message to get from queue

Pointer to buffer for message priority -> NULL

Status of the operation (0 – ok)

Receiving timeout (in ms)

Queue
CMSIS_OS v2.x

- Reset a message queue to initial empty state

```
osStatus_t osMessageQueueReset (osMessageQueueId_t queue_id )
```

status of the operation

Queue handle

- Get the number of messages stored in a queue

```
uint32_t osMessageQueueGetCount (osMessageQueueId_t queue_id )
```

Number of the messages stored in a queue

Queue handle

- Get the available space in a message queue

```
uint32_t osMessageQueueGetSpace(osMessageQueueId_t queue_id )
```

Available space in a message queue

Queue handle

Queue parameters - cmsis_os2.h

Queue parameters are grouped within lists at `osMessageQueueAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
**const char ***name	(CubeMX)	Name of the queue
uint32_t attr_bits		Attribute bits
**void ***cb_mem		Pointer to queue control block memory area
uint32_t cb_size		Queue control block (QCB) size
**void ***mq_mem		Pointer to queue data storage area
uint32_t mq_size		Queue data storage area size

Queue structure management - queue.c

Name	Description	condition
*pcHead	Points to the beginning of the queue storage area	
*pcTail	Points to the byte at the end of the queue storage area. Once more byte is allocated than necessary to store the queue items, this is used as a marker	
*pcWriteTo	Points to the free next place in the storage area	
*pcReadFrom	Points to the last place that a queued item was read from when the structure is used as a queue	Use of a union is an exception to the coding standard to ensure two mutually exclusive structure members don't appear simultaneously (wasting RAM)
uxRecursiveCallCount	Maintains a count of the number of times a recursive mutex has been recursively 'taken' when the structure is used as a mutex	Use of a union is an exception to the coding standard to ensure two mutually exclusive structure members don't appear simultaneously (wasting RAM)
xTasksWaitingToSend	List of tasks that are blocked waiting to post onto this queue. Stored in priority order	
xTasksWaitingToReceive	List of tasks that are blocked waiting to read from this queue. Stored in priority order	
uxMessagesWaiting	The number of items currently in the queue	
uxLength	The length of the queue defined as the number of items it will hold, not the number of bytes.	
uxItemSize	The size of each items that the queue will hold.	
xRxLock	Stores the number of items received from the queue (removed from the queue) while the queue was locked. Set to queueUNLOCKED when the queue is not locked	
xTxLock	Stores the number of items transmitted to the queue (added to the queue) while the queue was locked. Set to queueUNLOCKED when the queue is not locked.	
uxQueueNumber		configUSE_TRACE_FACILITY == 1
ucQueueType		configUSE_TRACE_FACILITY == 1
*pxQueueSetContainer		configUSE_QUEUE_SETS == 1

CMSIS_OS v1.x
CMSIS_OS v2.x

Queues APIs

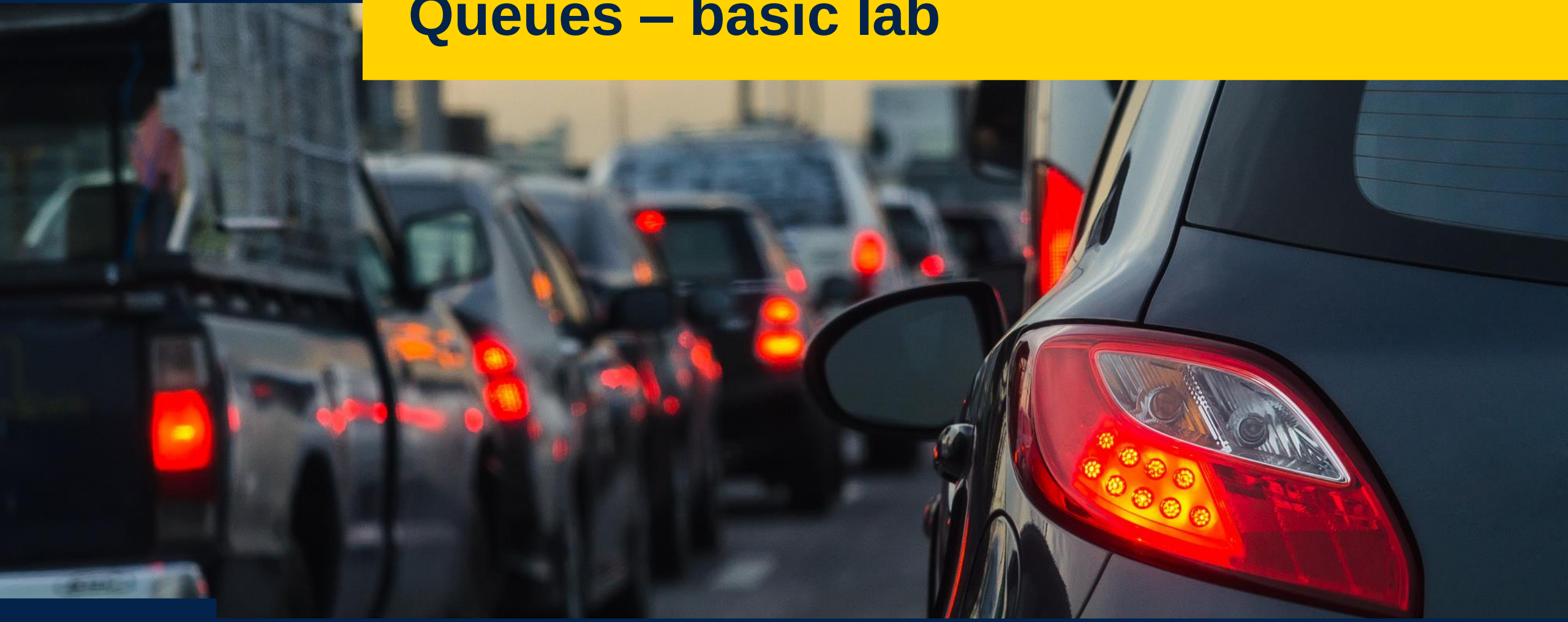
CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osMessageCreate()	osMessageQueueNew()	xQueueCreateStatic() xQueueCreate()
osMessagePut()	osMessageQueuePut()	xQueueSend() xQueueSendFromISR()
osMessageGet()	osMessageQueueGet()	xQueueReceive() xQueueReceiveFromISR()
osMessageDelete()	osMessageQueueDelete()	vQueueDelete(queue_handler)
osMessageWaiting()	osMessageQueueGetCount()	**uxQueueMessagesWaiting(queue_handler)** **uxQueueMessagesWaitingFromISR(queue_handler)**
-	osMessageQueuePut()	**xQueueSendToBack**(queue_handle,*to_queue,block_time) **xQueueSendToBackFromISR**(queue_handle,*to_queue,block_time)
-	-	**xQueueSendToFront**(queue_handle,*to_queue,block_time) **xQueueSendToFrontFromISR**(queue_handle,*to_queue,block_time)
osMessagePeek()	-	**xQueuePeek**(queue_handle,*to_queue,block_time)
osMessageAvailableSpace()	osMessageQueueGetSpace()	Returns uxQueueSpacesAvailable
-	osMessageQueueReset()	xQueueReset()
-	osMessageQueueGetMsgSize()	-
-	osMessageQueueGetCapacity()	-

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Queues – basic lab

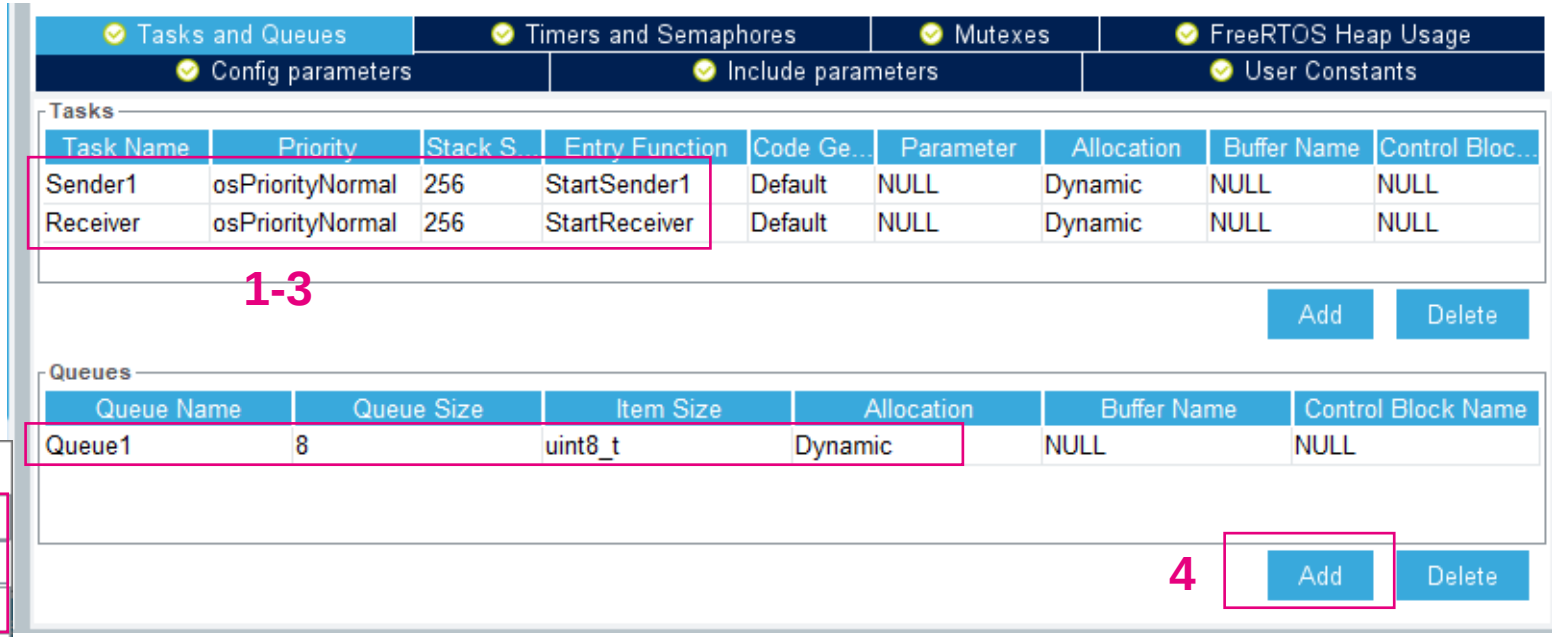


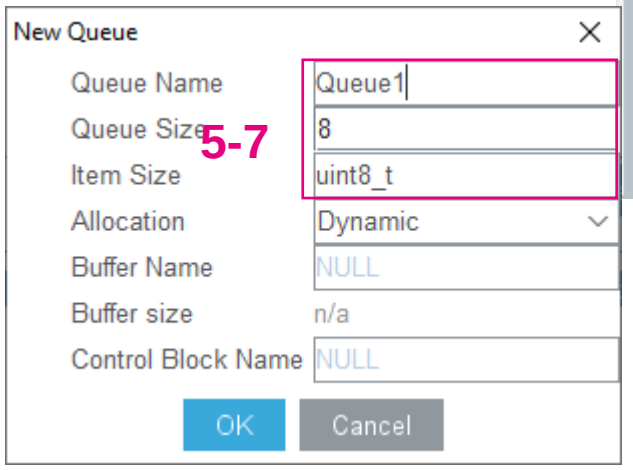
life.augmented

Queue lab

- We can reuse one of previous labs (with two tasks)
- Tasks part:
  - Rename tasks to Sender1 and Receiver and its functions.
  - If deleted, old tasks will be removed (with USER CODE !!!) from the code. To keep them just rename the task.
  - Set both tasks to normal priority
- Queue part
  - Button Add
  - Set queue size to 8
  - Queue type to uint8_t
  - Button OK

8. Ctrl+s to regenerate the code





life.augmented

Queue lab - code processing – main.c file

- Queue handle is now defined

```
/* Private variables -----------------------------------------------------*/
osThreadId_t Sender1Handle;
osThreadId_t ReceiverHandle;
osMessageQueueId_t Queue1Handle;
```

- Queue item type initialization, length definition and create of queue and memory allocation

```
/* Create the queue(s) */
 /* definition and creation of Queue1 */
 const osMessageQueueAttr_t Queue1_attributes = {.name = "Queue1" };

 Queue1Handle = osMessageQueueNew (8, sizeof(uint8_t), &Queue1_attributes);
```

Queue name

Queue handler

Queue size

Queue item size

Queue lab - code processing – main.c file

- Let's implement Sender1 task function

```
void StartSender1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  uint8_t x=0;
  /* Infinite loop */
  for(;;)
  {
    Task_action('S');
    osMessageQueuePut(Queue1Handle, &x,0,200);
    if(++x>9)
       x=0;
    osDelay(1000);
  }
  /* USER CODE END 5 */
}
```

Item to send

Not used priority

Queue handle

Timeout for send in ms

Queue lab - code processing – main.c file

- Let's implement Receiver task function

```
/* StartReceiver function */
void StartReceiver(void const * argument)
{
  /* USER CODE BEGIN StartReceiver */
  uint8_t res=0;
  /* Infinite loop */
  for(;;)
  {
    Task_action('R');
    osMessageQueueGet(Queue1Handle, &res, NULL, 2000);
    Task_action(res+48);
  }
  /* USER CODE END StartReceiver */
}
```

Get item from queue

How long we wait for data in queue It will move the task to blocked state for 2000ms and then go to next line

+48 to have ASCII codes of 0..9

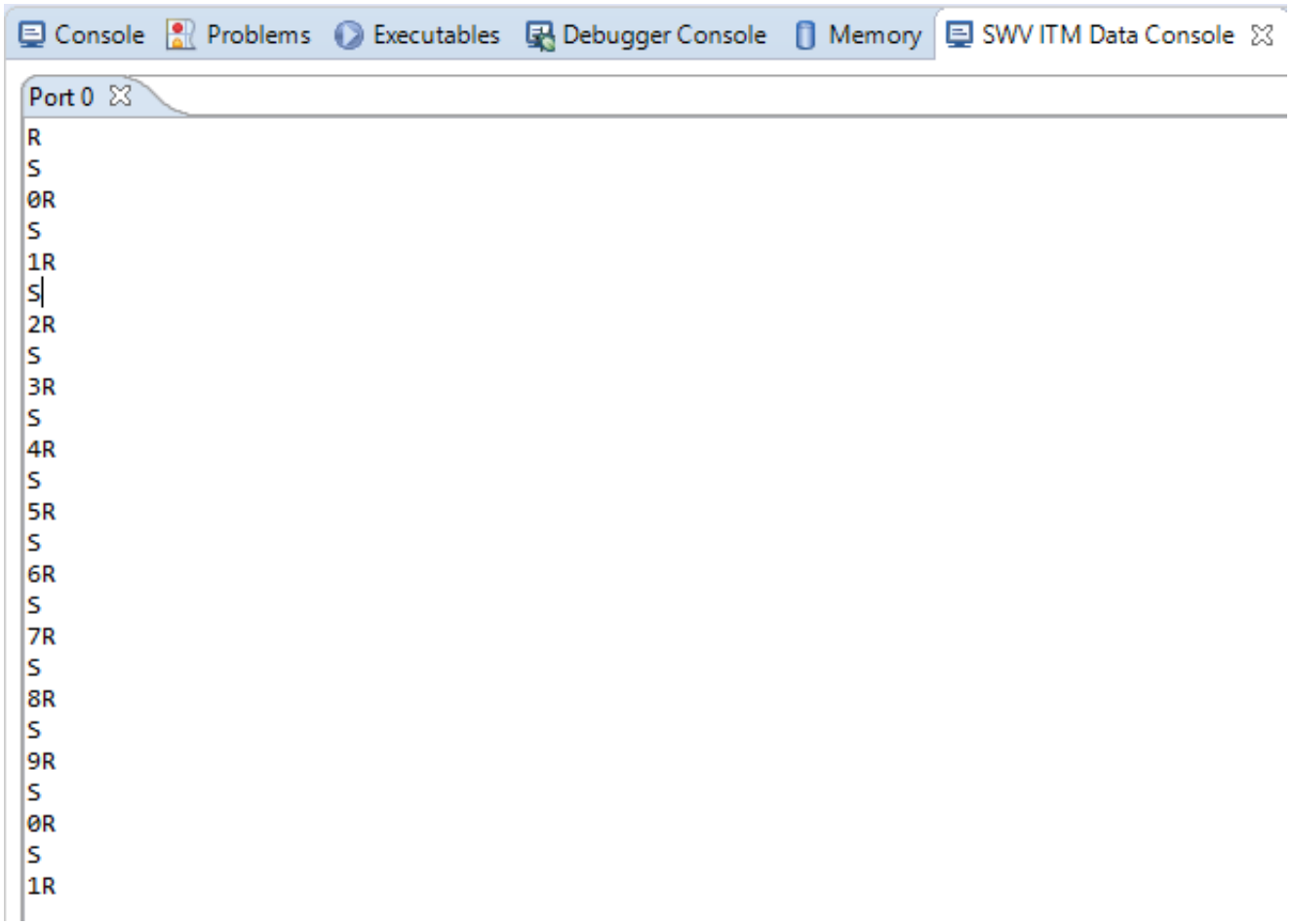
Queue handle

Not used priority

- Compile the project and start debug session

Queue lab

- Example of the result (using Task_action() to print simple messages over SWO)



life.augmented

Queue lab - code processing – main.c file

- Let's implement more robust Receiver task function

```
/* StartReceiver function */
void StartReceiver(void const * argument)
{
  /* USER CODE BEGIN StartReceiver */
  uint8_t res=0;
  osStatus_t r_state;
  /* Infinite loop */
  for(;;)
  {
    Task_action('R');
    r_state=osMessageQueueGet(Queue1Handle, &res, NULL, osWaitForever);
    Task_action(res+48);
  }
  /* USER CODE END StartReceiver */
}
```

Should be global to watch it

How long we wait for data in queue
It will move the task to blocked state till there will be new data in the queue

Get item from queue

Function return value osStatus_t

Queue handle

Not used priority

- Compile the project and start debug session

We can add **r_state** global variable to „Live view" and monitor it within debug session

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

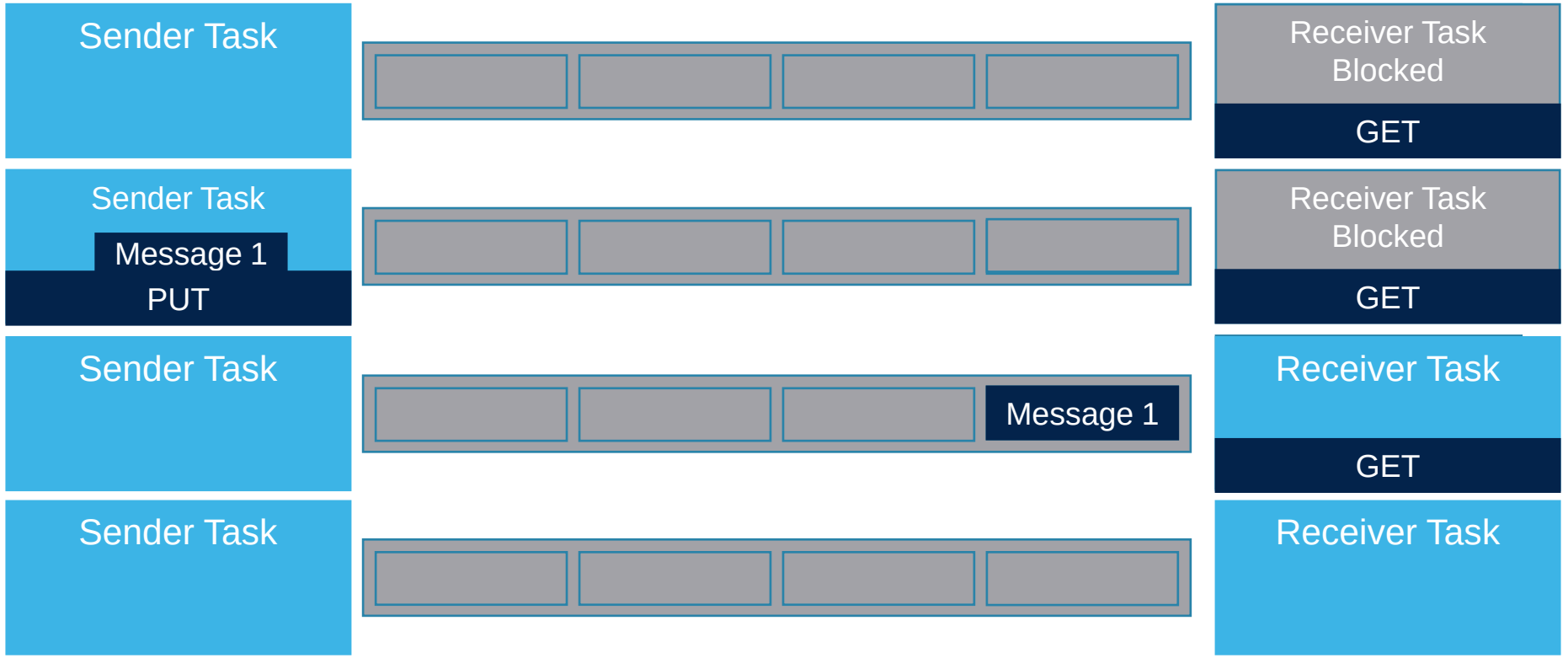
life.augmented

FreeRTOS – CMSIS_OS v2
Blocking on queue lab



life.augmented

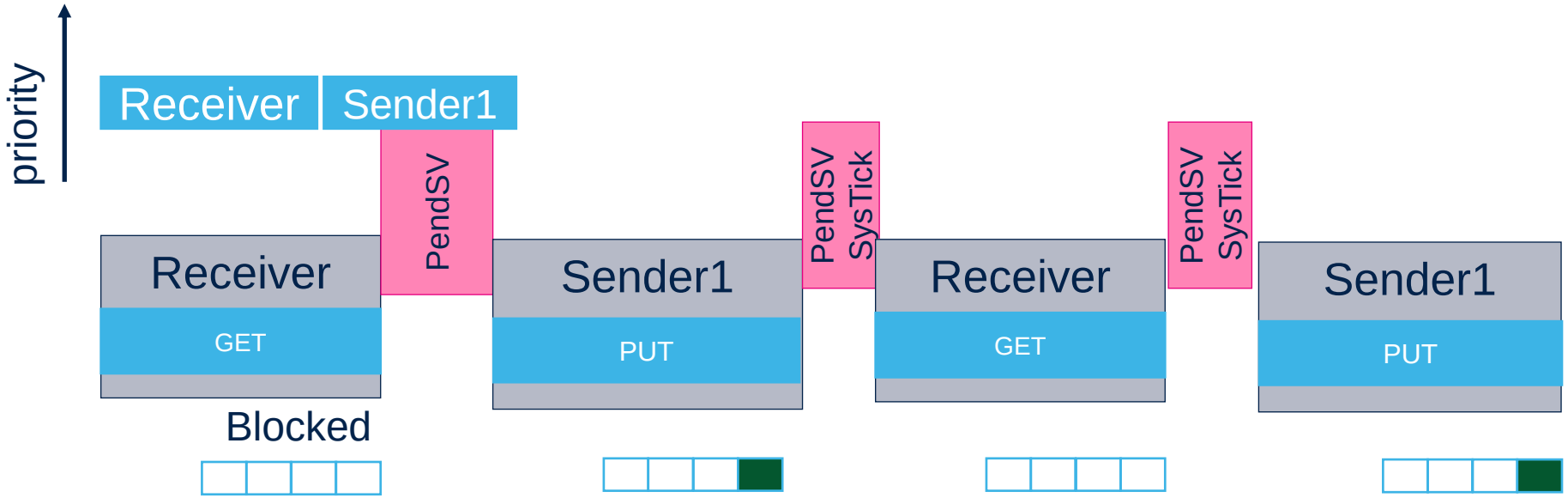
Queue blocking



life.augmented

Queue blocking

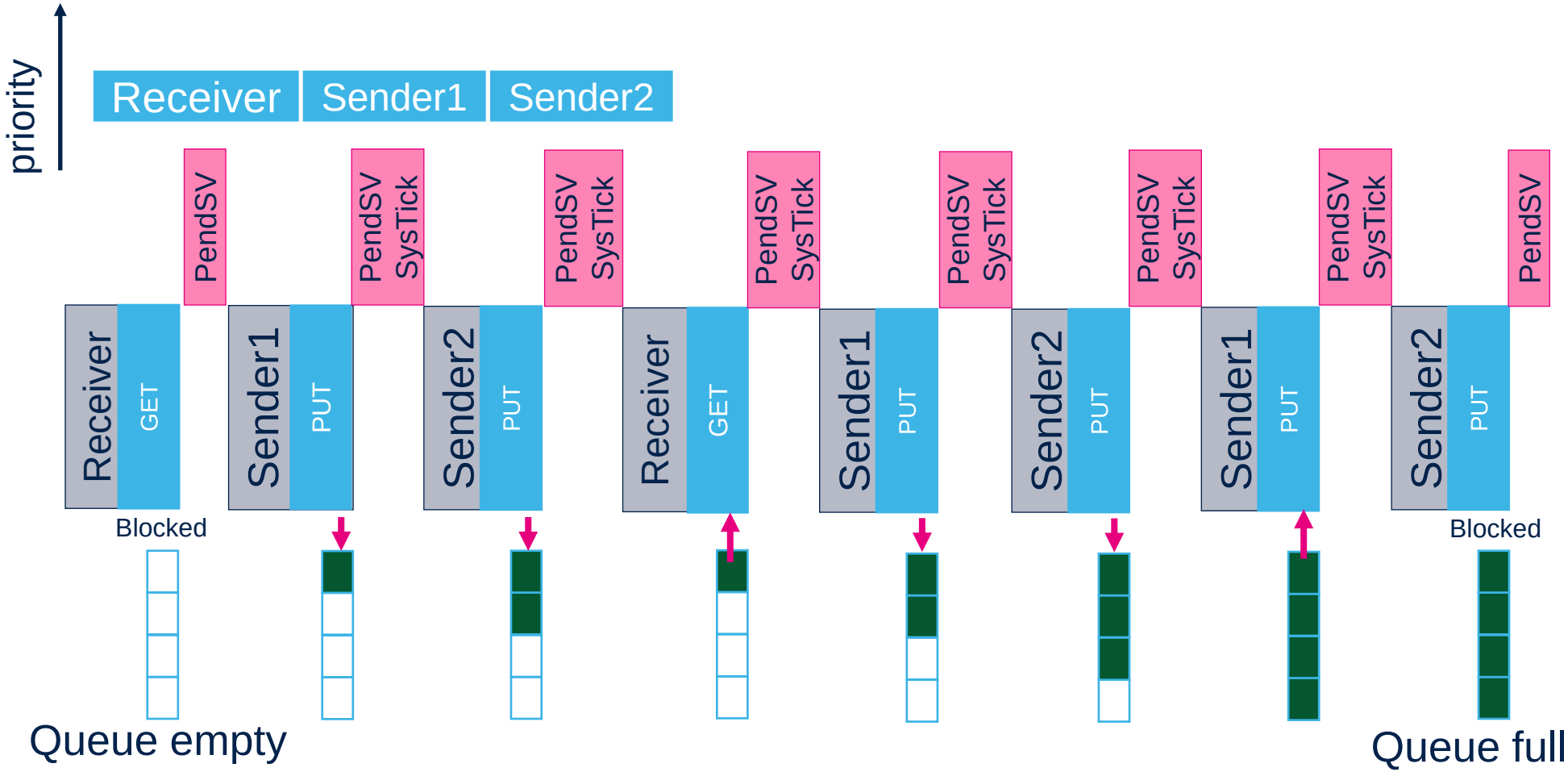
- After calling `osMessageQueuePut()`
  - If there is no free space in queue the Sender task is blocked for settable time then it will continue (without sending the data)
  - If there is free space in queue the Sender task will continue just after data send
- After calling `osMessageQueueGet()`
  - If any data are not in queue the Receiver task is blocked for settable time then it will continue (without data reception)
  - If the data are in queue the task will continue just after data reception



life.augmented

Multiple senders, one receiver

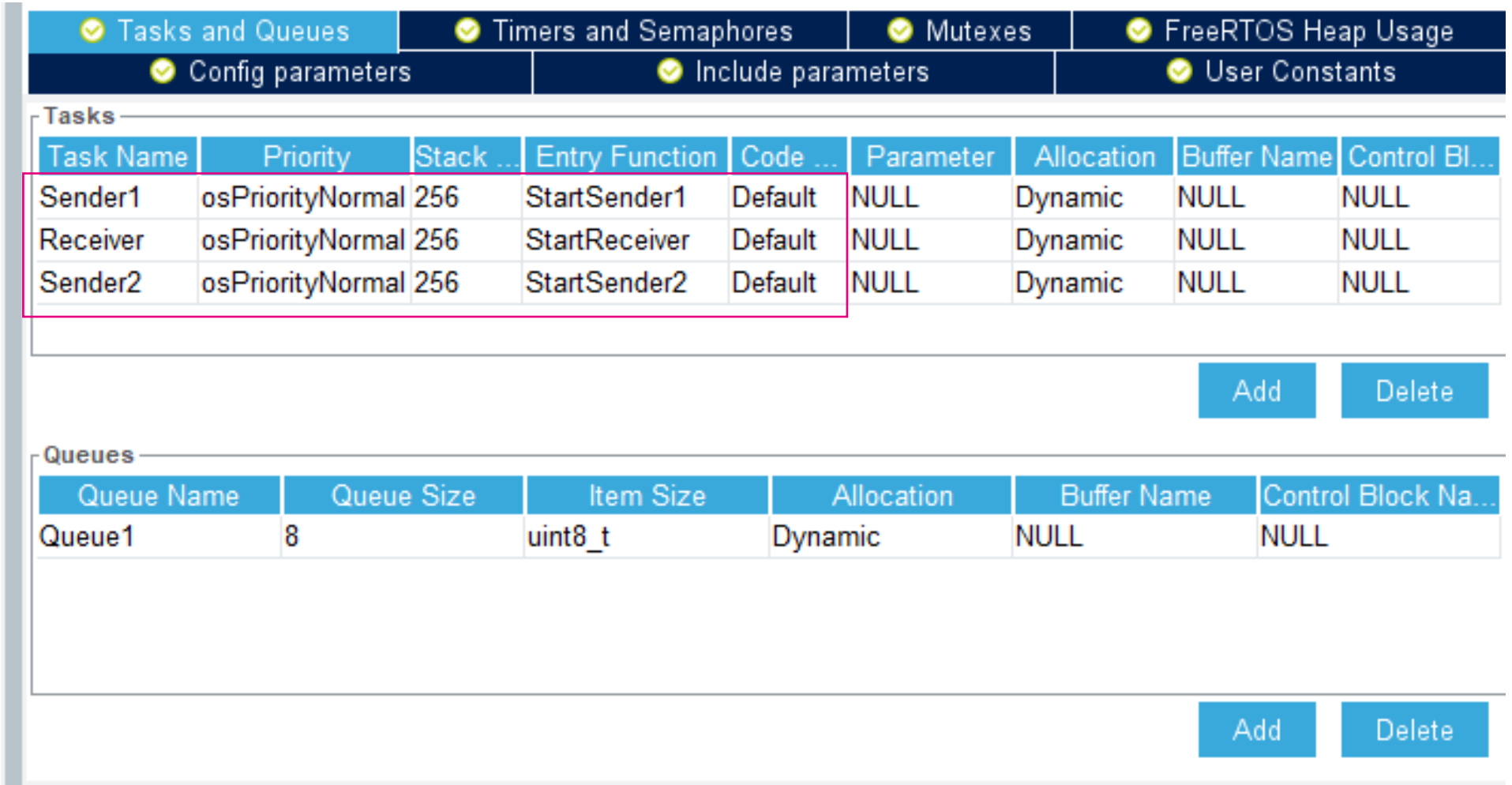
- Because tasks have same priority, receiver will get data from queue after both task put data into queue
- What would happened if will be more tasks?



life.augmented

Two senders queue lab

- We can reuse previous lab
- Let’s create two sending tasks: Sender1, Sender2 and one Receiver task with the same priorities.



life.augmented

Two senders queue lab code processing – main.c file

- Two sending tasks

```
void StartSender1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  uint8_t x=1;
  /* Infinite loop */
  for(;;)
  {
    Task_action('s');
    osMessageQueuePut(Queue1Handle, &x, 0,200);

    osDelay(2000);
  }
  /* USER CODE END 5 */
}
```

```
void StartSender2(void const * argument)
{
  /* USER CODE BEGIN StartSender2 */
  uint8_t x=2;
  /* Infinite loop */
  for(;;)
  {
    Task_action('S');
    osMessageQueuePut(Queue1Handle, &x, 0,200);

    osDelay(2000);
  }
  /* USER CODE END StartSender2 */
}
```

Two senders queue lab code processing – main.c file

- Simple receiver

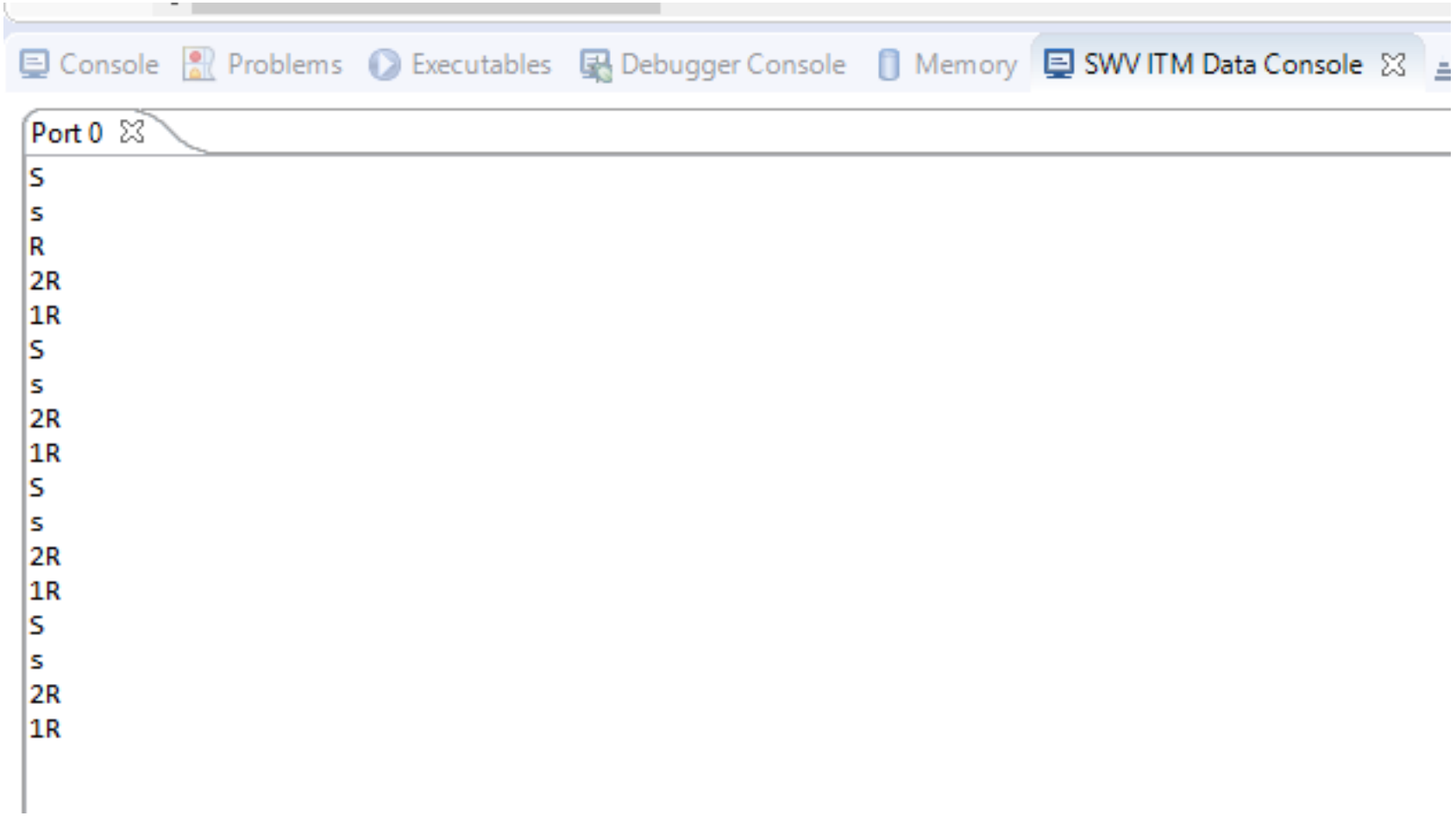
```
/* StartReceiver function */
void StartReceiver(void const * argument)
{
  /* USER CODE BEGIN StartReceiver */
  uint8_t res=0;
  /* Infinite loop */
  for(;;)
  {
    Task_action('R');

    if( osMessageQueueGet(Queue1Handle,&res, NULL, 4000) == osOK)
            Task_action(res+48);

    osDelay(2000);
  }
  /* USER CODE END StartReceiver */
}
```

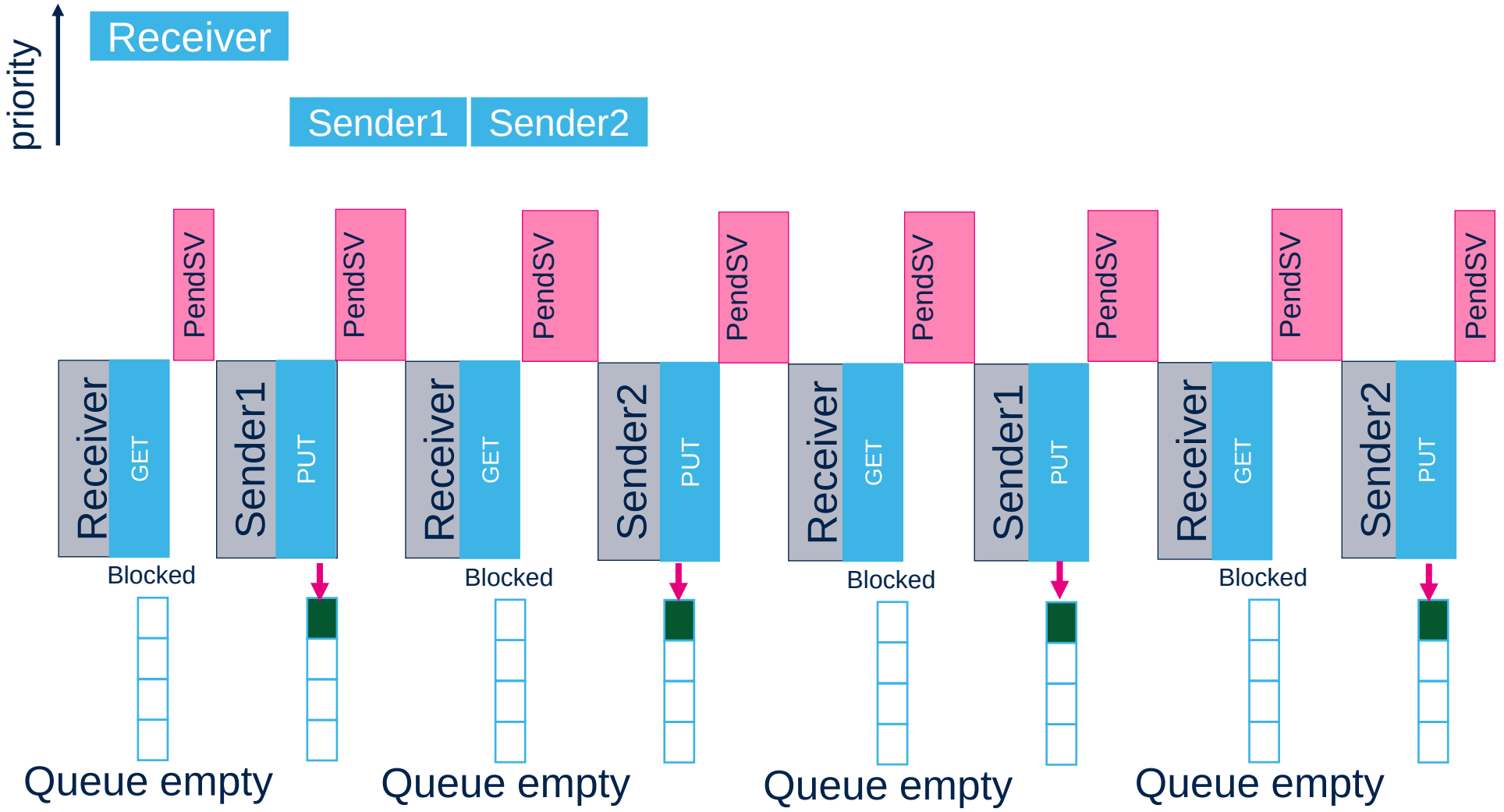
Two senders queue lab

- Example of the result (using Task_action() to print simple messages over SWO)



Receiver with higher priority lab

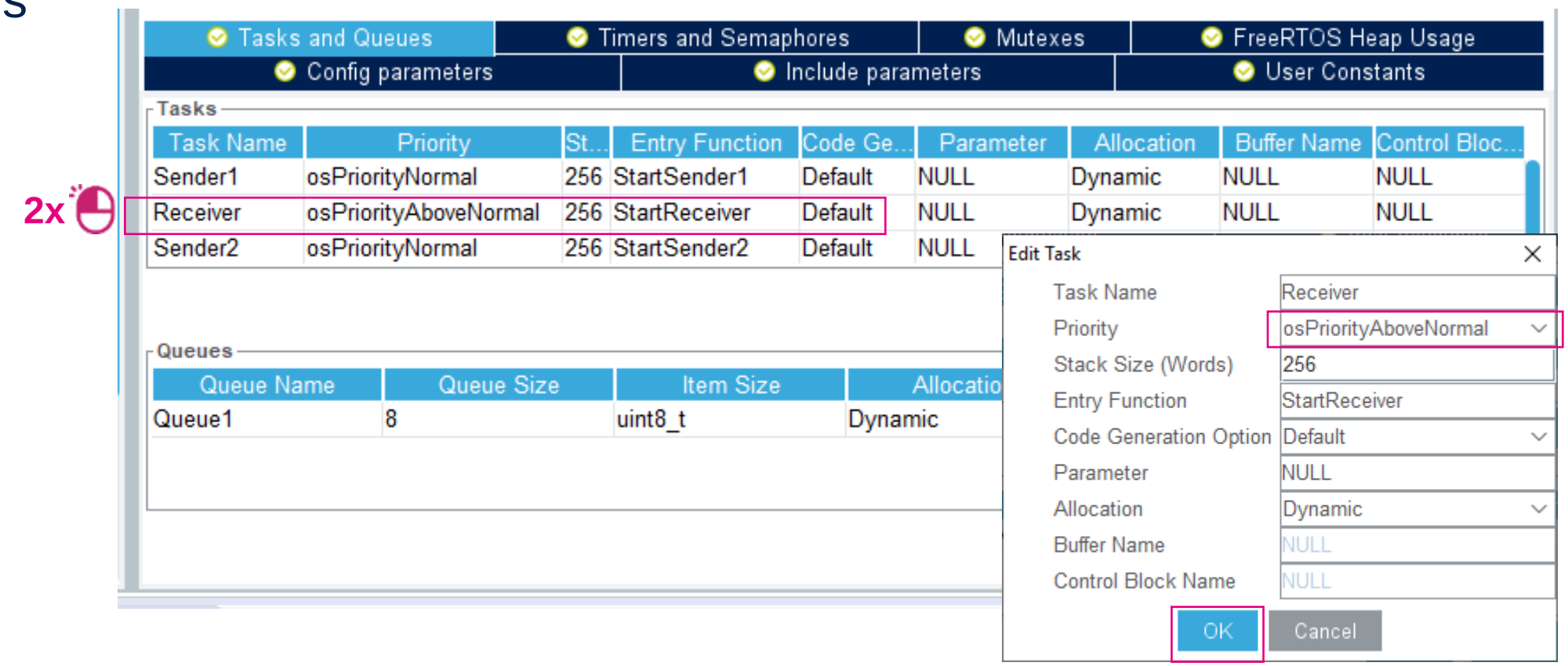
- Receiver is now unblocked every time when sender tasks put data into queue



life.augmented

Receiver with higher priority lab

- Let’s continue with previous lab, doing only one small modification
- Receiver have higher priority than senders
- Please build the code, start debug and verify whether behavior is inline with expectations



life.augmented

Receiver with higher priority lab

- Example of the result (using Task_action() to print simple messages over SWO)



life.augmented

Single sender, two receivers

- Message from the queue is taken by the task with higher priority
- In case of equal priorities the task which is waiting longer time is selected. It is not deterministic.

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

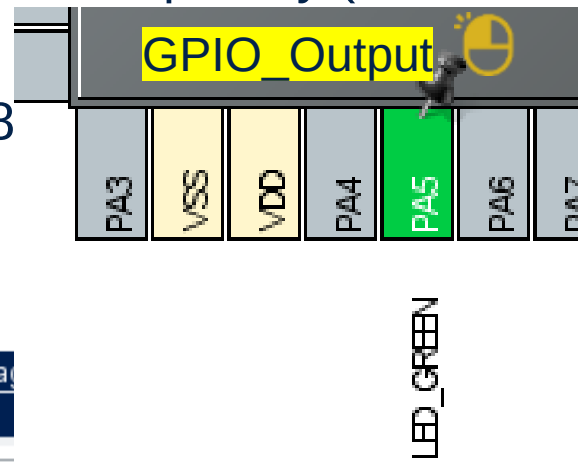
FreeRTOS – CMSIS_OS v2
Passing the structure over the queue lab



life.augmented

Queue items lab

- Queues allow to define type (different variables or structures) which the queue use.
- We can reuse previous lab with two Senders and one Receiver – all with the same priority (i.e. osPriorityNormal)
- Within Queue1 Item size put a structure called Data (instead of existing uint8
- We will configure PA5 as GPIO_Output and label it as LED_GREEN
- Regenerate project (i.e. Ctrl+s)



Tasks and Queues | Timers and Semaphores | Mutexes | FreeRTOS Heap Usag
Config parameters | Include parameters | User Constants

Tasks

Task Name	Priority	St...	Entry Function	Code Ge...	Parameter	Allocation	Buffer Name	Control Bloc...
Sender1	osPriorityNormal	256	StartSender1	Default	NULL	Dynamic	NULL	NULL
Receiver	osPriorityNormal	256	StartReceiver	Default	NULL	Dynamic	NULL	NULL
Sender2	osPriorityNormal	256	StartSender2	Default	NULL	Dynamic	NULL	NULL

Add | D

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block
Queue1	8	Data	Dynamic	NULL	NULL

Add | D

2x

Edit Queue

Queue Name	Queue1
Queue Size	8
Item Size	Data
Allocation	Dynamic
Buffer Name	NULL
Buffer size	n/a
Control Block Name	NULL

OK | Cancel

life.augmented

Queue items lab code processing – main.c file

- Create new structure type for data

```
/* USER CODE BEGIN PV */
/* Define the structure type that will be passed on the queue. */
typedef struct
{
  uint16_t Value;
  uint8_t Source;
} Data;
```

- Define structures which will be sent from sender task

```
/* Declare two variables of type Data that will be passed on the queue. */
Data DataToSend1={'a',1};
Data DataToSend2={'b',2};
```

Queue items lab code processing – main.c file

- Send data from Sender1 task

```
void StartSender1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    Task_action('s');
    osMessageQueuePut(Queue1Handle, &DataToSend1, 0, 200);
    osDelay(2000);
  }
  /* USER CODE END 5 */
}
```

Put data into queue

- **Prepare similar code for Sender2 to send DataToSend2 and Task action with ‚S’ as an argument**

Queue items lab code processing – main.c file

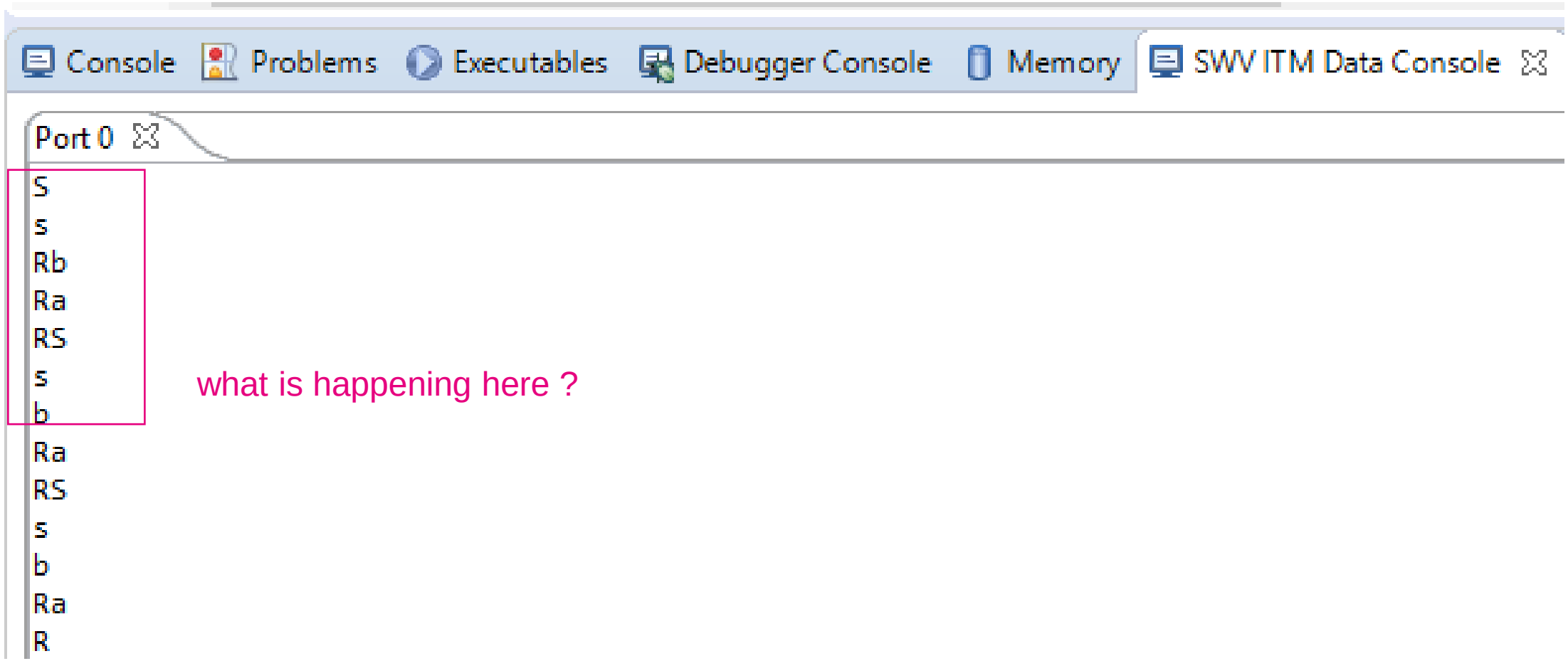
- Receiver data from sender task

```
/* StartReceiver function */
void StartReceiver(void const * argument)
{
  /* USER CODE BEGIN StartReceiver */
  Data retvalue;
  /* Infinite loop */
  for(;;)
  {
    ITM_SendChar('R'); //to avoid extra /n at the end
    osMessageQueueGet(Queue1Handle, &retvalue, NULL, osWaitForever);
    if(retvalue.Source==1){
      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET);
    }else{
      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET);
    }
    Task_action(retvalue.Value);
  }
  /* USER CODE END StartReceiver */
}
```

Get data from queue

Queue items lab

- Example of the result (using Task_action() to print simple messages over SWO)



Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Semaphores

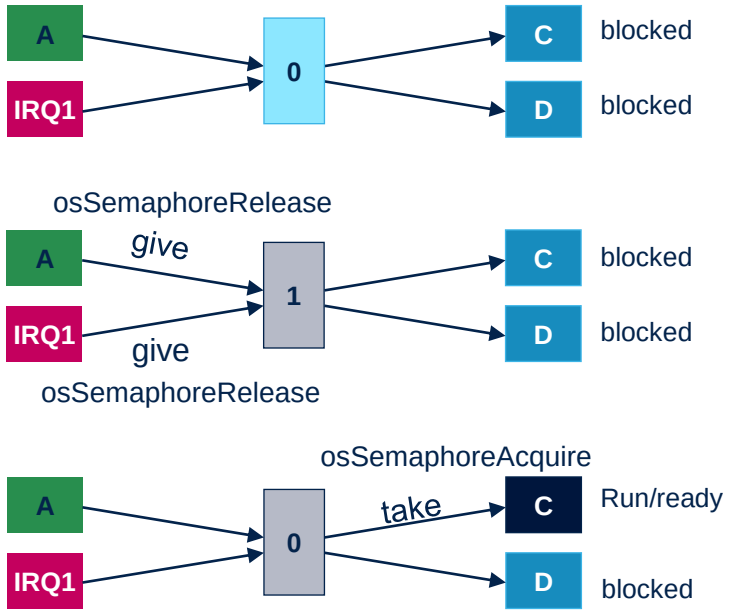


life.augmented

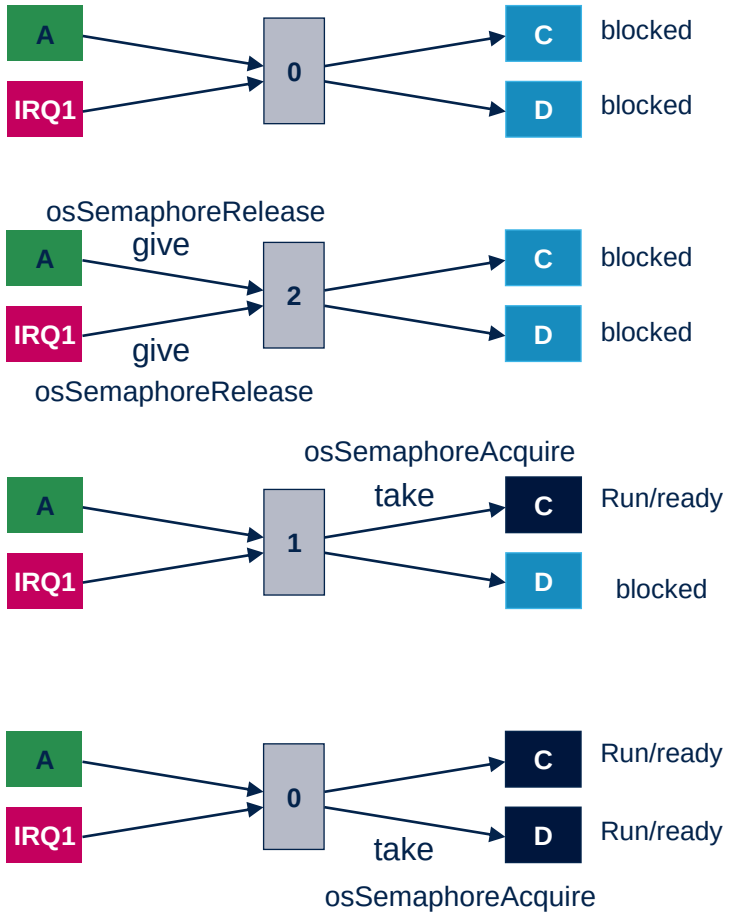
Semaphores

- Semaphores are used to synchronize tasks with other events in the system (especially IRQs)
- Waiting for semaphore is equal to wait() procedure, task is in blocked state not taking CPU time
- Semaphore should be created before usage
- In FreeRTOS implementation semaphores are based on queue mechanism
- In fact those are queues with length 1 and data size 0
- There are following types of semaphores in FreeRTOS:
  - **Binary** – simple on/off mechanism
  - **Counting** – counts multiple **give** and multiple **take**
  - **Mutex** – Mutual Exclusion type semaphores (explained later on)
  - **Recursive** (in CMSIS FreeRTOS used only for Mutexes)
- **Turn on** semaphore = **give** a semaphore can be done from other task or from interrupt subroutine
- **Turn off** semaphore = **take** a semaphore can be done from the task

Semaphores: binary vs counting



Binary



Counting

life.augmented

CMSIS_OS v1.x
CMSIS_OS v2.x

Semaphores APIs

CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osSemaphoreCreate()	osSemaphoreNew()	vSemaphoreCreateBinaryStatic() vSemaphoreCreateCountingStatic() vSemaphoreCreateBinary() xSemaphoreCreateCounting()
osSemaphoreWait()	osSemaphoreAcquire()	xSemaphoreTake() xSemaphoreTakeFromISR()
osSemaphoreRelease()	osSemaphoreRelease()	xSemaphoreGive() xSemaphoreGiveFromISR()
osSemaphoreDelete()	osSemaphoreDelete()	vSemaphoreDelete()
-	osSemaphoreGetName()	-
-	osSemaphoreGetCount()	uxSemaphoreGetCount uxQueueMessagesWaitingFromISR()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

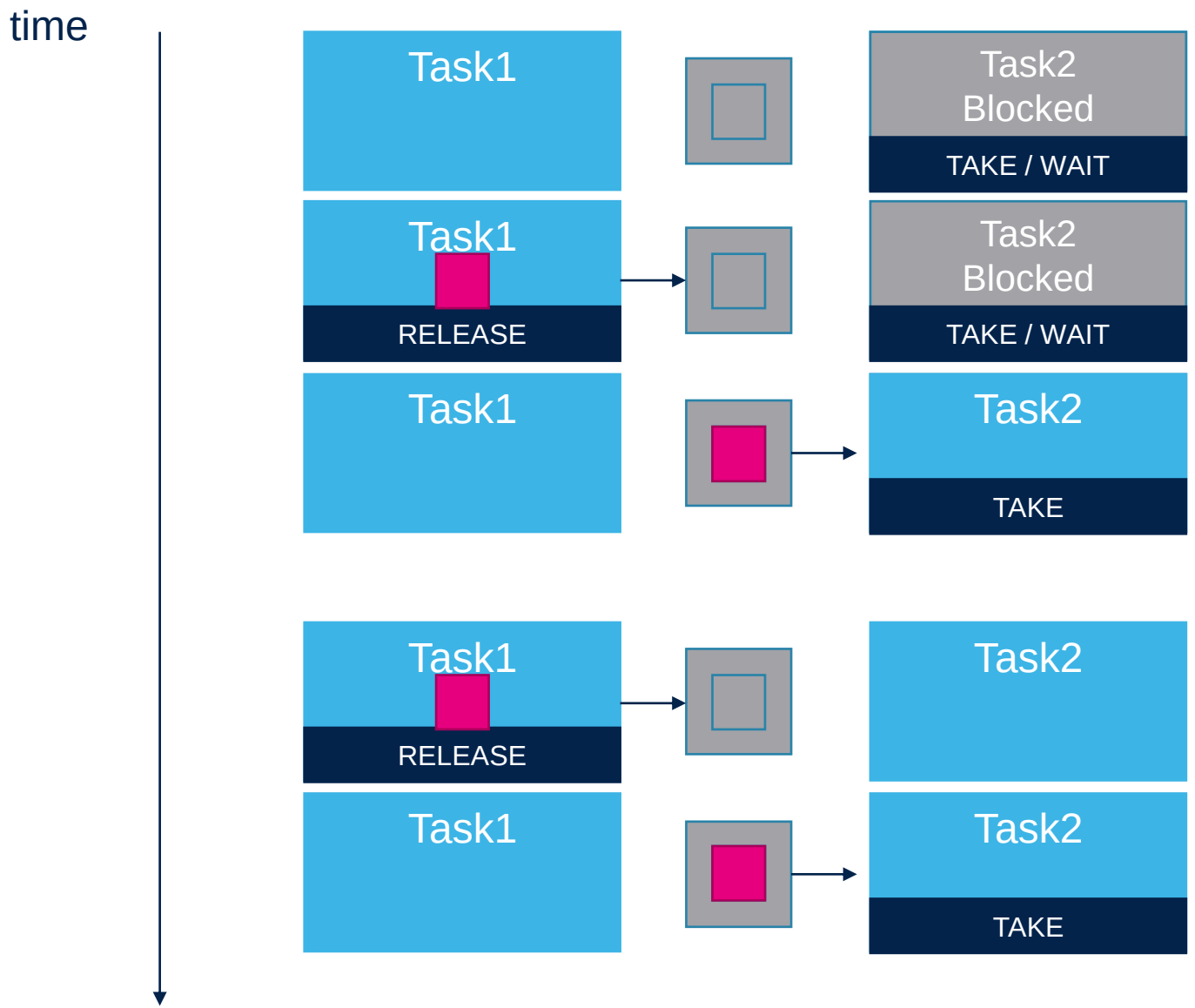
life.augmented

FreeRTOS – CMSIS_OS v2
Binary semaphores



life.augmented

Binary semaphore



life.augmented

Semaphore parameters (CMSIS_OS v2.x) cmsis_os2.h

Semaphore parameters are grouped within lists at `osSemaphoreAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
`const char *``name`	`(CubeMX)`	Name of the semaphore
`uint32_t` `attr_bits`		Attribute bits
`void *``cb_mem`		Pointer to semaphore control block memory area
`uint32_t` `cb_size`		semaphore control block (TCB) size

Binary semaphore CMSIS_OS v2.x

- Semaphore creation

Semaphore attributes definition

```
const osSemaphoreAttr_t Sem_attr = { .name = “Sem_name” };
```

Semaphore handle

Max number of tokens (1 for binary semaphore)

Initial number of tokens (0 or 1 for binary semaphore, 0 to n for counting one)

```
osSemaphoreId osSemaphoreNew (uint32_t max_count, uint32_t initial_count,
const osSemaphoreAttr_t *Sem_attr)
```

Semaphore attributes or NULL for default values

- Semaphore delete

```
osStatus_t osSemaphoreDelete (osSemaphoreId_t semaphore_id)
```

Return status

Semaphore handle

Binary semaphore CMSIS_OS v2.x

- Wait for Semaphore release (acquire semaphore)

```
osStatus_t osSemaphoreAcquire (osSemaphoreId semaphore_id, uint32_t millisec)
```

Return status

Semaphore handle

How long wait for semaphore release
- 0 – no wait
- >0 – delay in ms
- 0xFFFFFFFF - forever

- Semaphore release

```
osStatus_t osSemaphoreRelease (osSemaphoreId semaphore_id)
```

Return status

Semaphore handle

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

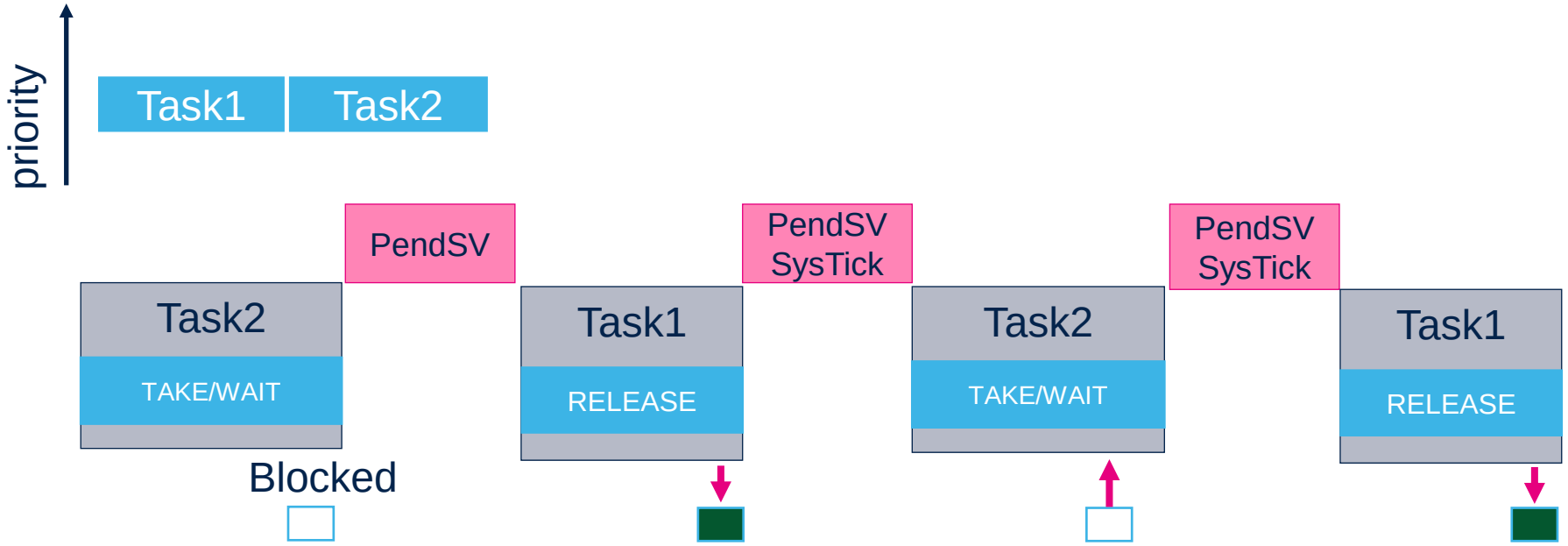
FreeRTOS – CMSIS_OS v2
Binary semaphores - lab



life.augmented

Binary semaphore lab

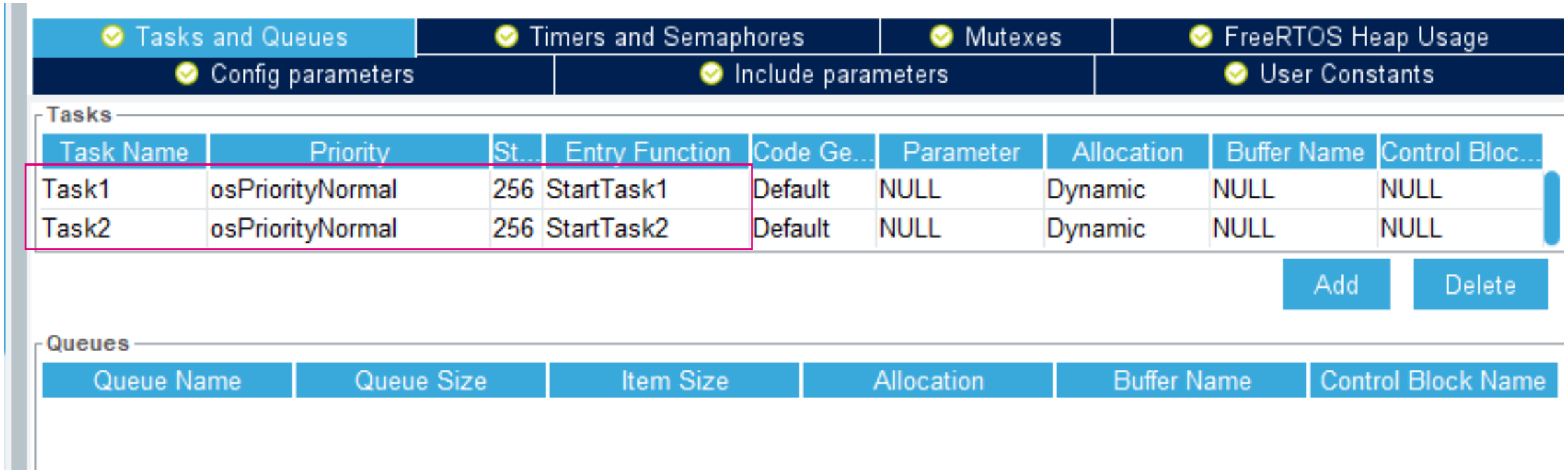
- Task1 is synchronized with Task2
- Both tasks have the same priorities
- Task2 is waiting for semaphore (with 4seconds delay)
- Task1 is releasing the semaphore each 2 seconds



life.augmented

Binary semaphore lab

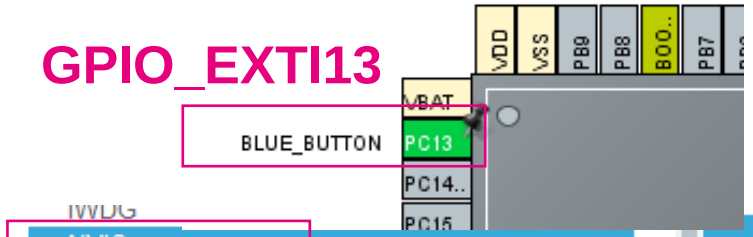
- Using one of previous exercises, please remove all OS components (you can use Reset Configuration button)
- Create two tasks Task1, Task2 with the same priority value (osPriorityNormal)



life.augmented

Binary semaphore lab preparation

- Within this lab we will use an external interrupts to release semaphore
- Please check whether in your project you have the following settings:



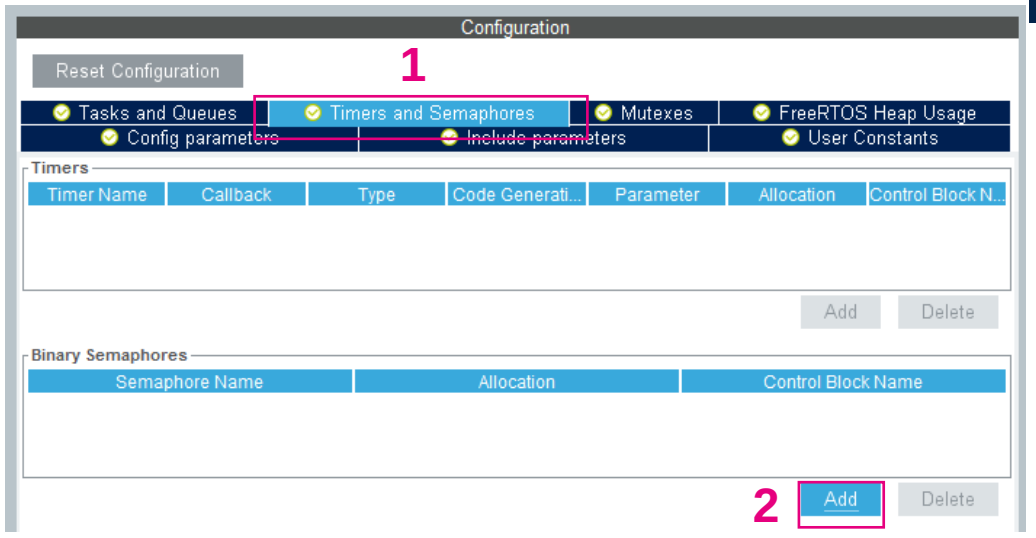
IWDG
NVIC
RCC
SYS
TSC
WWDG

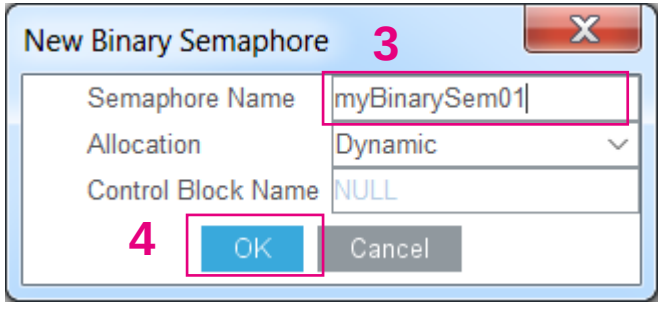
Analog
Timers
Connectivity
Multimedia
Security

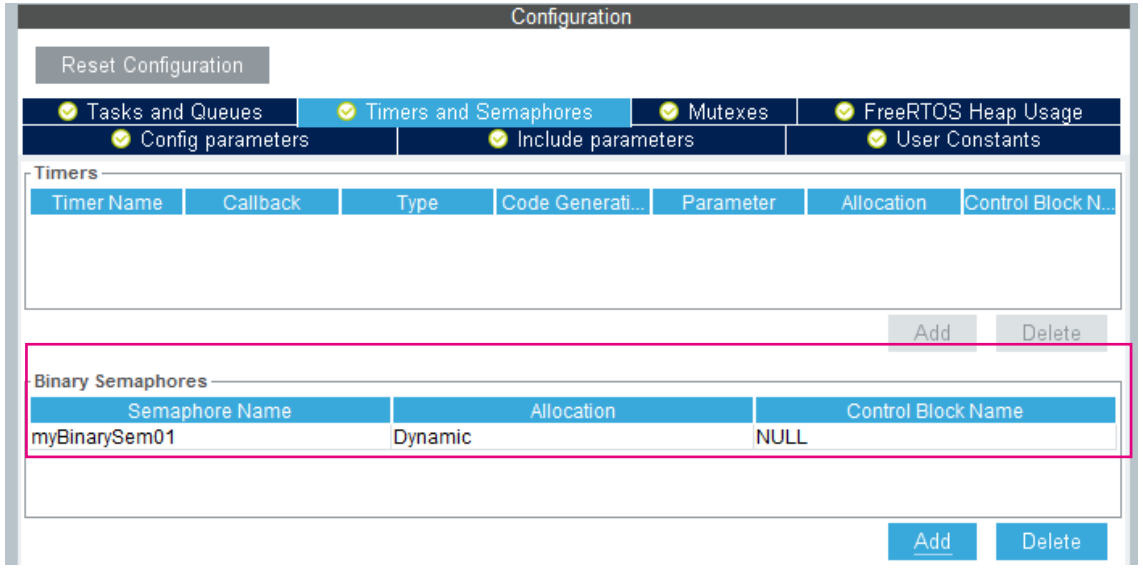
NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	Managed by RTOS	Managed by RTOS
Debug monitor	☑	0	0
Pendable request for system service	☑	Managed by RTOS	Managed by RTOS
System tick timer	☑	Managed by RTOS	Managed by RTOS
PVD/PVM1/PVM2/PVM3/PVM4 interrupts through EXTI line...	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
EXTI line[15:10] interrupts	☑	5	0
Time base: TIM6 global interrupt, DAC channel1 and channe...	☑	0	0
FPU global interrupt	☐	0	0

Binary semaphore lab

- Create binary semaphore
- Select Timers and Semaphore tab
- Click Add button in Binary Semaphores section
- Set name: myBinarySem01
- Click OK button
- Generate the project by Ctrl+s and open main.c file







life.augmented

Binary semaphore lab code processing – main.c file

- Semaphore handle definition

```
/* Private variables ---------------------------------------------------*/
osThreadId_t Task1Handle;
osThreadId_t Task2Handle;
osSemaphoreId_t myBinarySem01Handle;
```

- Semaphore creation

```
  /* Create the semaphores(s) */
  /* definition and creation of myBinarySem01 */
  const osSemaphoreAttr_t myBinarySem01_attributes = { .name = “myBinarySem01”};
  myBinarySem01Handle = osSemaphoreNew(1, 0, &myBinarySem01_attributes);
```

Binary semaphore lab code processing – main.c file

- Semaphore release usage
- If tasks/interrupt is done the semaphore is released

```
void StartTask1(void const * argument)
{

  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    osDelay(2000);
    Task_action('1'); //Task1 release semaphore
    osSemaphoreRelease(myBinarySem01Handle);
  }
  /* USER CODE END 5 */
}
```

Binary semaphore lab code processing – main.c file

- Semaphore wait usage
- Second task waits on semaphore release
  After release task is unblocked and continue in work

```
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  /* Infinite loop */
  for(;;)
  {
    osSemaphoreAcquire(myBinarySem01Handle,4000);
    Task_action('2'); //Task2 synchronized
  }
  /* USER CODE END StartTask2 */
}
```

Binary semaphore lab code processing – main.c file

- Semaphore can be released from interrupt (if interrupt priority is below – higher number in CortexM cores - configured `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`)
- Using HAL libraries we can release semaphore in the callback (BLUE_BUTTON button press):

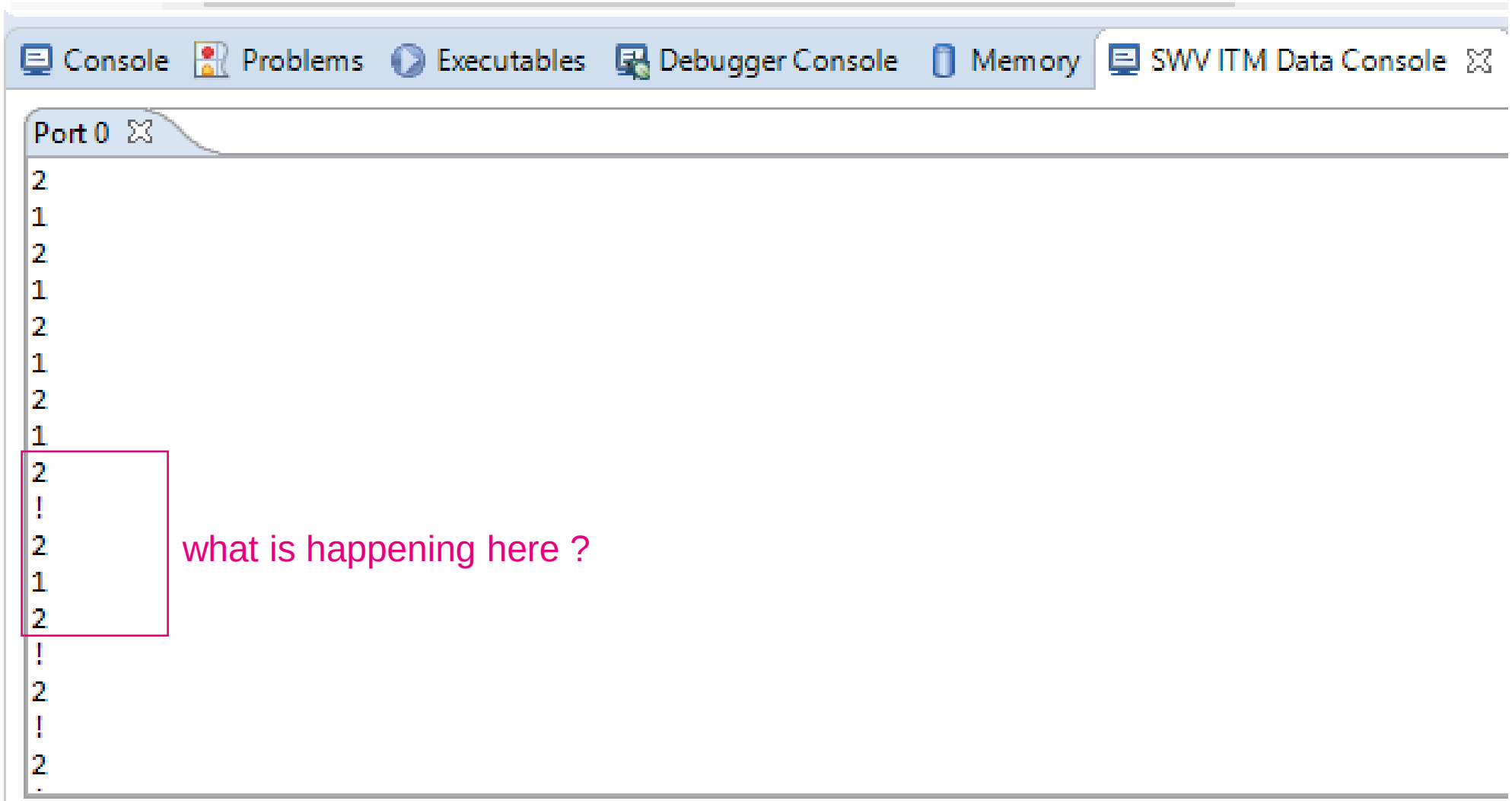
```
/* USER CODE BEGIN 4 */

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  Task_action('!');
  osSemaphoreRelease(myBinarySem01Handle);
}
```

life.augmented

Binary semaphore lab

- Example of the result (using Task_action() to print simple messages over SWO)



Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Counting semaphores

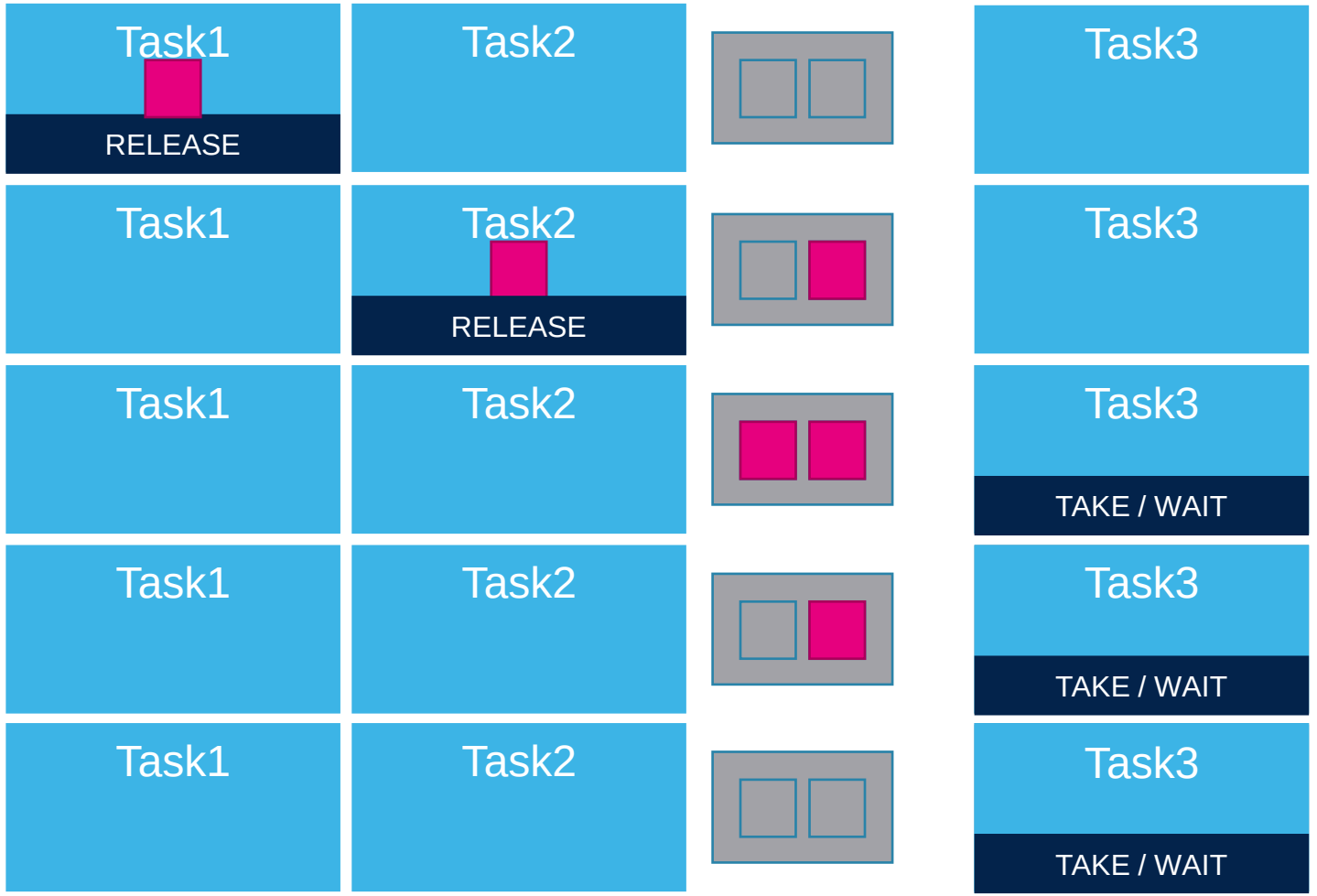


life.augmented

Counting semaphores

- Counting semaphores can be seen as a as queues of length greater than one. users of the semaphore (Tasks, IT) are not interested in the data that is stored in the queue, just whether the queue is empty or not.
- Counting semaphores are typically used for two purposes:
  - Counting events : an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is the difference between the number of events that have occurred and the number that have been processed. In this case it is important to set the count value to be zero when the semaphore is created.
  - Resource management : the count value indicates the number of resources available. To obtain control of a resource a task must first obtain a semaphore decrementing the semaphore count value. When the count value reaches zero there are no free resources. When a task finishes with the resource it releases (gives) the semaphore back incrementing the semaphore count value. In this case it is important to set the count value to the maximum count value when the semaphore is created.

Counting semaphore



life.augmented

Counting semaphore CMSIS_OS v2.x

- Semaphore creation

```
const osSemaphoreAttr_t Sem_attr = { .name = “Sem_name” };
```

Semaphore attributes definition

```
osSemaphoreId osSemaphoreNew (uint32_t max_count, uint32_t initial_count,
const osSemaphoreAttr_t *Sem_attr)
```

Semaphore handle

Max number of tokens (1 for binary semaphore)

Initial number of tokens (0 or 1 for binary semaphore, 0 to n for counting one)

Semaphore attributes or NULL for default values

- Semaphore delete

```
osStatus_t osSemaphoreDelete (osSemaphoreId_t semaphore_id)
```

Return status

Semaphore handle

Counting semaphore CMSIS_OS v2.x

- Wait for Semaphore release (acquire semaphore)

```
osStatus_t osSemaphoreAcquire (osSemaphoreId semaphore_id, uint32_t millisec)
```

Return status

Semaphore handle

How long wait for semaphore release
- 0 – no wait
- >0 – delay in ms
- 0xFFFFFFFF - forever

- Semaphore release

```
osStatus_t osSemaphoreRelease (osSemaphoreId semaphore_id)
```

Return status

Semaphore handle

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

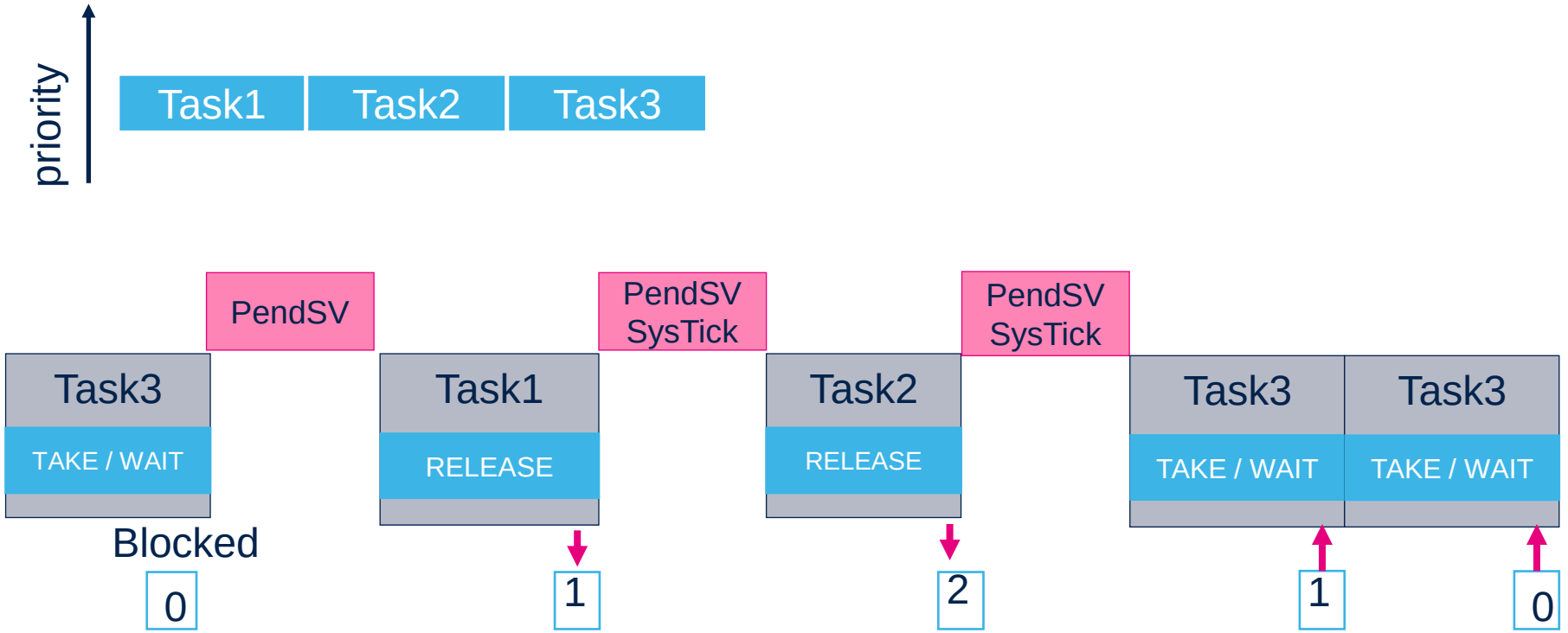
FreeRTOS – CMSIS_OS v2
Counting semaphores lab



life.augmented

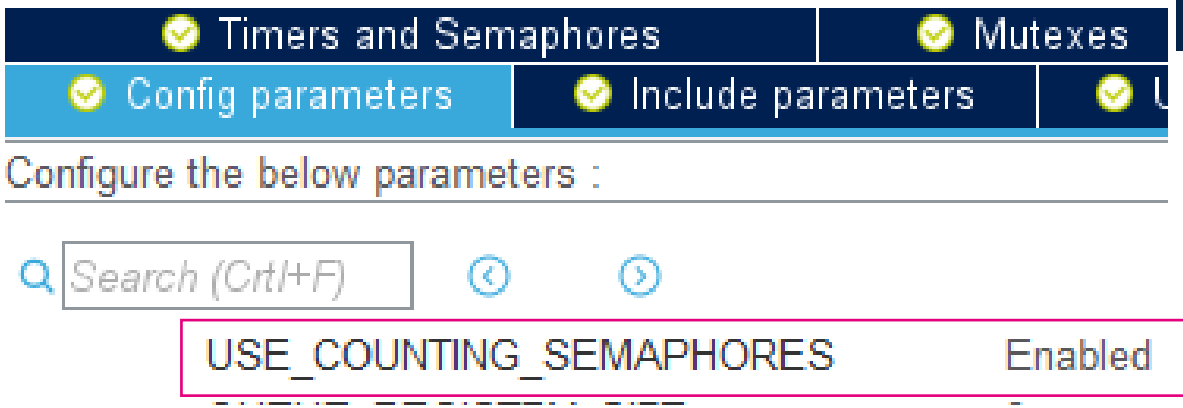
Counting semaphore lab

- Task1 and Task2 release semaphore
- Task 3 wait for two tokens

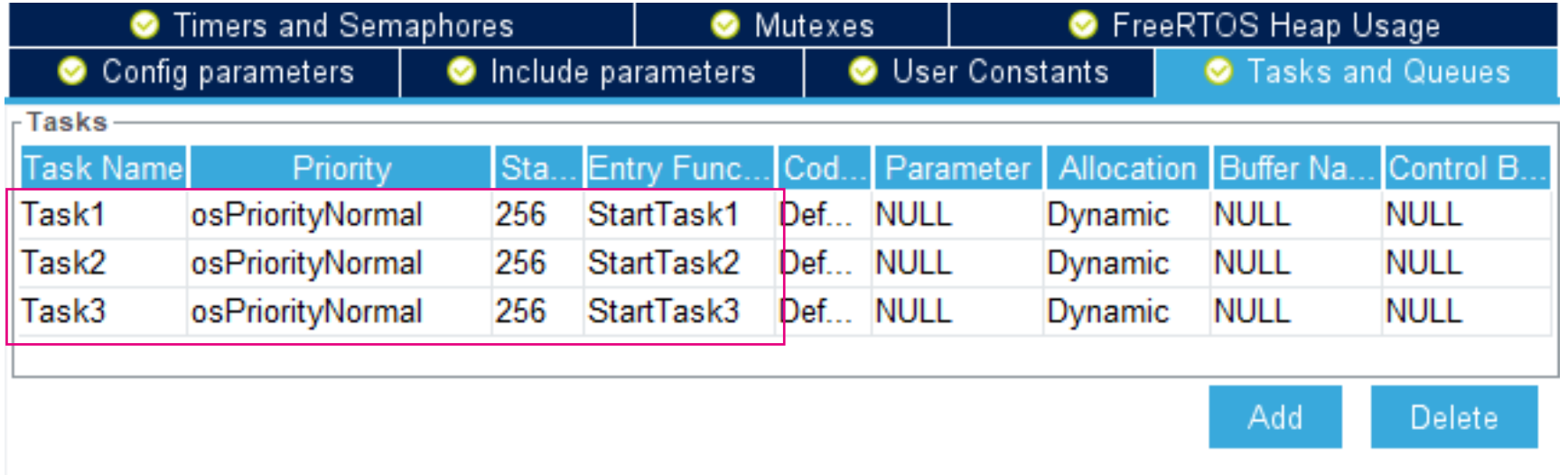


life.augmented

Counting semaphore lab



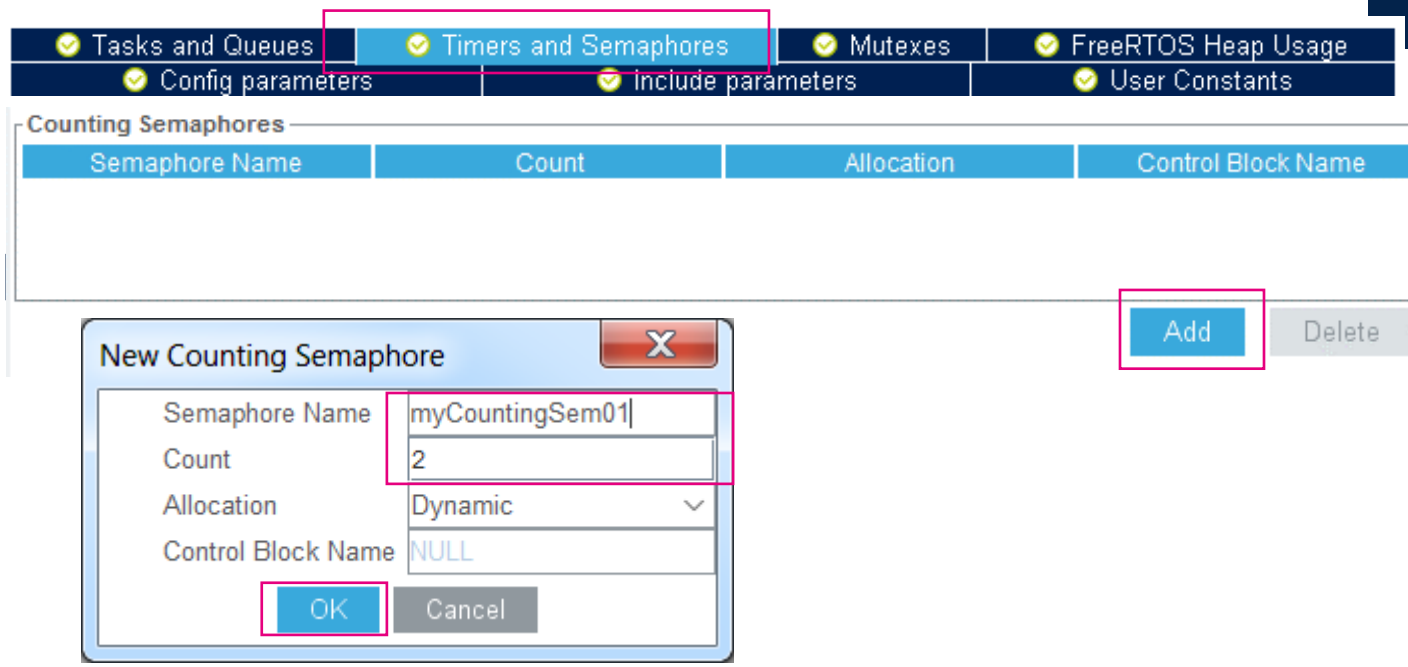
- We can reuse previous exercise
- Enable counting semaphores within FreeRTOS config parameters
- Create three tasks (Task1, Task2, Task3) with same priority
- Set entry function to StartTask1,2,3 respectively

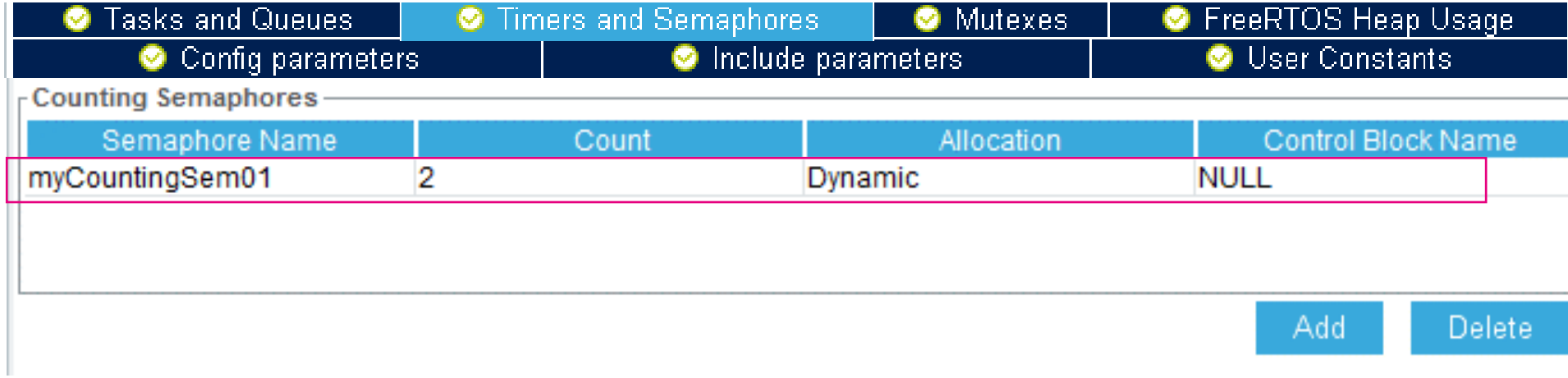


life.augmented

Counting semaphore lab

- Create Counting semaphore
- Select Timers and Semaphores tab
- Click Add button in Counting Semap
- Set name to myCountingSem01
- Set count of tokens to 2
- Click OK button
- Generate the code (Ctrl+s) and open main.c file

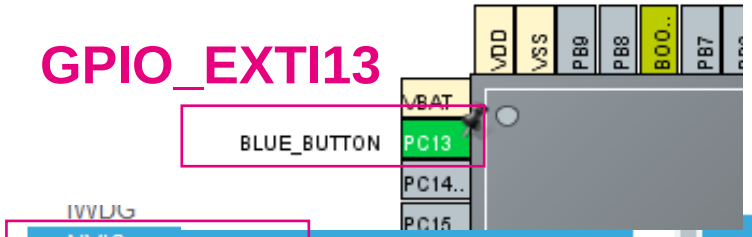




life.augmented

Counting semaphore lab preparation

- Within this lab we will use an external interrupts to release semaphore
- Please check whether in your project you have the following settings:



IWDG
NVIC
RCC
SYS
TSC
WWDG

Analog
Timers
Connectivity
Multimedia
Security

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	Managed by RTOS	Managed by RTOS
Debug monitor	☑	0	0
Pendable request for system service	☑	Managed by RTOS	Managed by RTOS
System tick timer	☑	Managed by RTOS	Managed by RTOS
PVD/PVM1/PVM2/PVM3/PVM4 interrupts through EXTI line...	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
EXTI line[15:10] interrupts	☑	5	0
Time base: TIM6 global interrupt, DAC channel1 and channe...	☑	0	0
FPU global interrupt	☐	0	0

Counting semaphore lab code processing – main.c file

- Create Counting semaphore

```
/* Create the semaphores(s) */
 /* definition and creation of myCountingSem01 */
 const osSemaphoreAttr_t myCountingSem01_attributes = { .name = “myCountingSem01”};
 myCountingSem01Handle = osSemaphoreNew(2, 0, &myCountingSem01_attributes);
```

- Task1 and Task2 will be almost the same

```
void StartTask1(void const * argument)
{

  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
   osSemaphoreRelease(myCountingSem01Handle);
   Task_action('1');
   osDelay(2000);
  }
  /* USER CODE END 5 */
}
```

```
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  /* Infinite loop */
  for(;;)
  {
   osSemaphoreRelease(myCountingSem01Handle);
   Task_action('2');
   osDelay(2000);
  }
  /* USER CODE END StartTask2 */
}
```

Counting semaphore lab code processing – main.c file

- Task3 will wait until semaphore will be 2 times released

```
void StartTask3(void const * argument)
{
  /* USER CODE BEGIN StartTask3 */
  /* Infinite loop */
  for(;;)
  {
  osSemaphoreAcquire(myCountingSem01Handle, 4000);
  osSemaphoreAcquire(myCountingSem01Handle, 4000);
  Task_action('3');
  }
  /* USER CODE END StartTask3 */
}
```

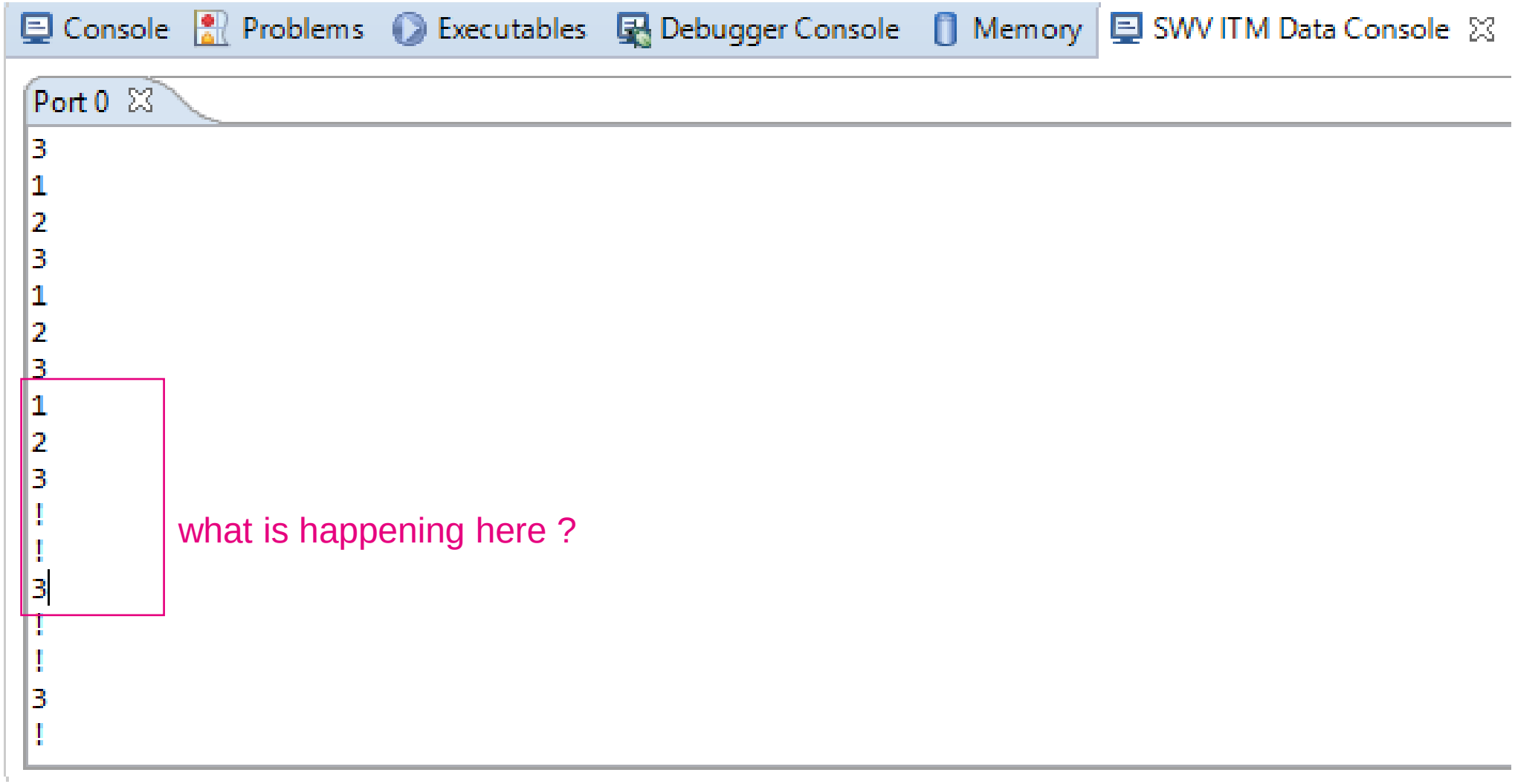
- BLUE_BUTTON interrupt callback will release counting semaphore as well

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  Task_action('!');
  osSemaphoreRelease(myCountingSem01Handle);
}
```

Build the project and start debug session

Counting semaphore lab

- Example of the result (using Task_action() to print simple messages over SWO)



life.augmented

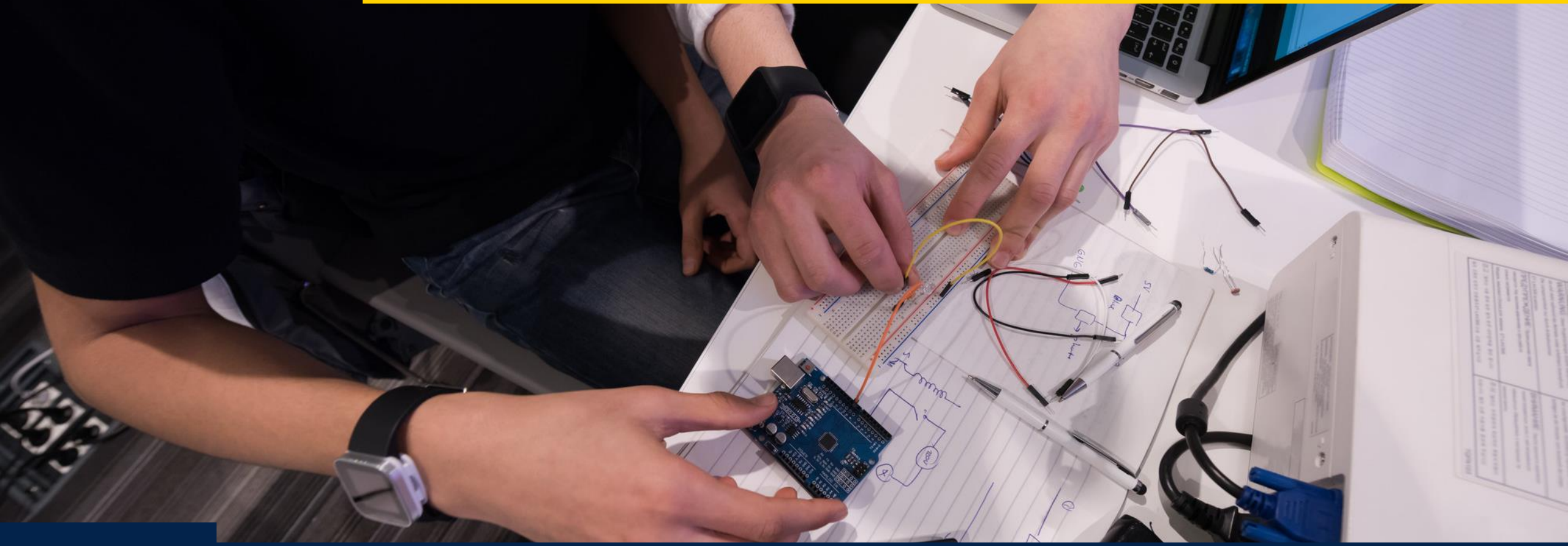
Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Events:

- Event Groups (FreeRTOS API)
- Event Flags (CMSIS_OS v2.x API)



life.augmented

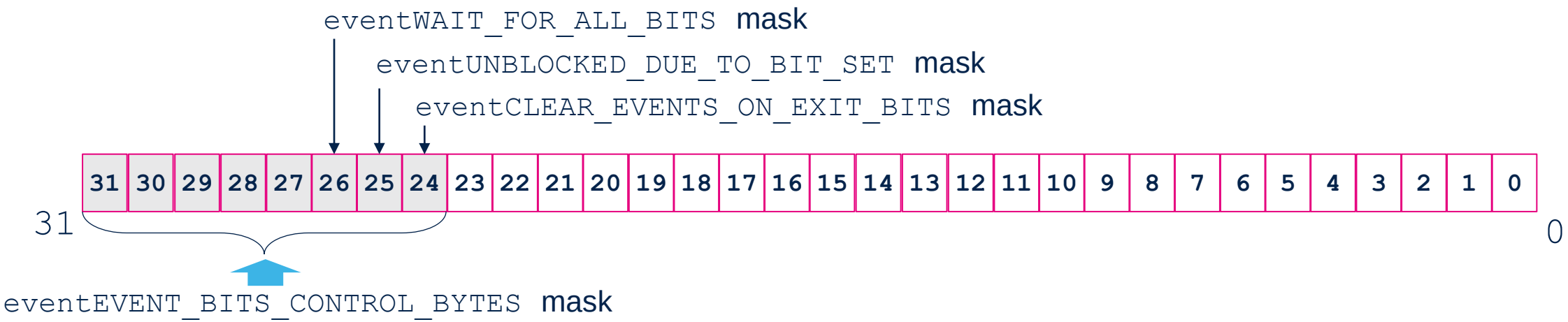
Events

- It allows a task to wait (in blocked state) for a combination of one or more events to occur
- It unblocks all the tasks that are waiting for the same event, combination of events, when the event occurs
- Event group functionality is optional and requires to add **event_groups.c** and **event_groups.h** files to the project (only when using CMSIS_OS API < v2.x)
- It allows to reduce RAM usage by replacing various combination of semaphores
- It allows to use broadcasting mechanism within OS
- Event groups can be set / clear from other tasks or interrupt procedures
- It is supported within CMSIS_OS API from version v2.x and requires usage of RTOS agent for proper functionality (set by default within code generation using CMSIS_V2) – more comments on next slides.

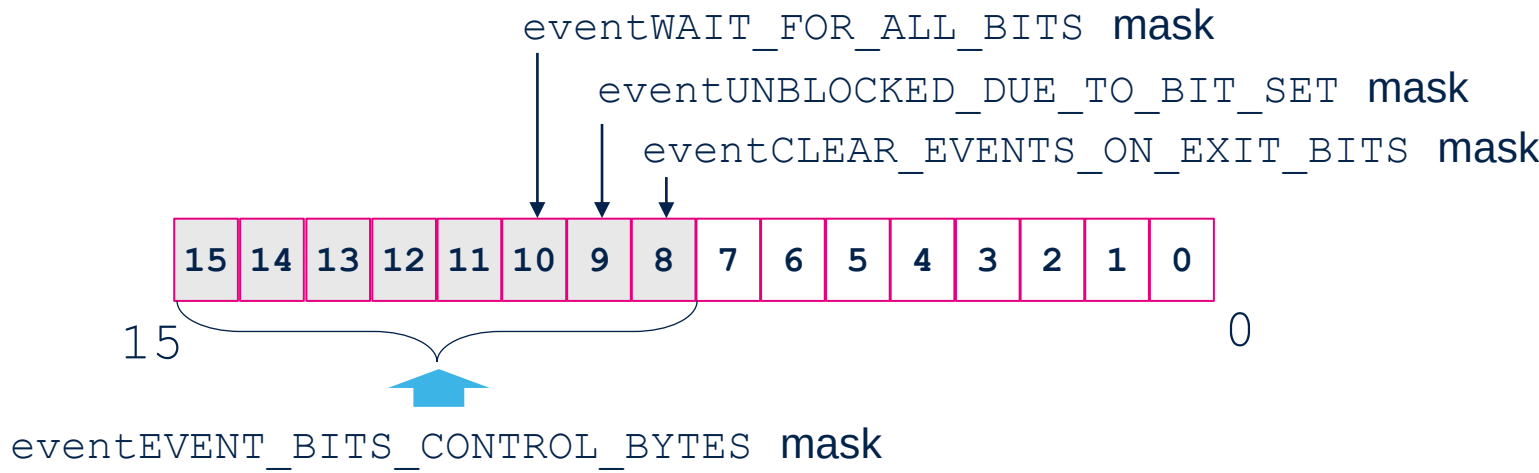
life.augmented

Event Groups bit mask structure

- Bit mask structure structure:



- Bit mask structure (if `configUSE_16_BIT_TICKS` is set):



life.augmented

Event Group / Event Flags APIs

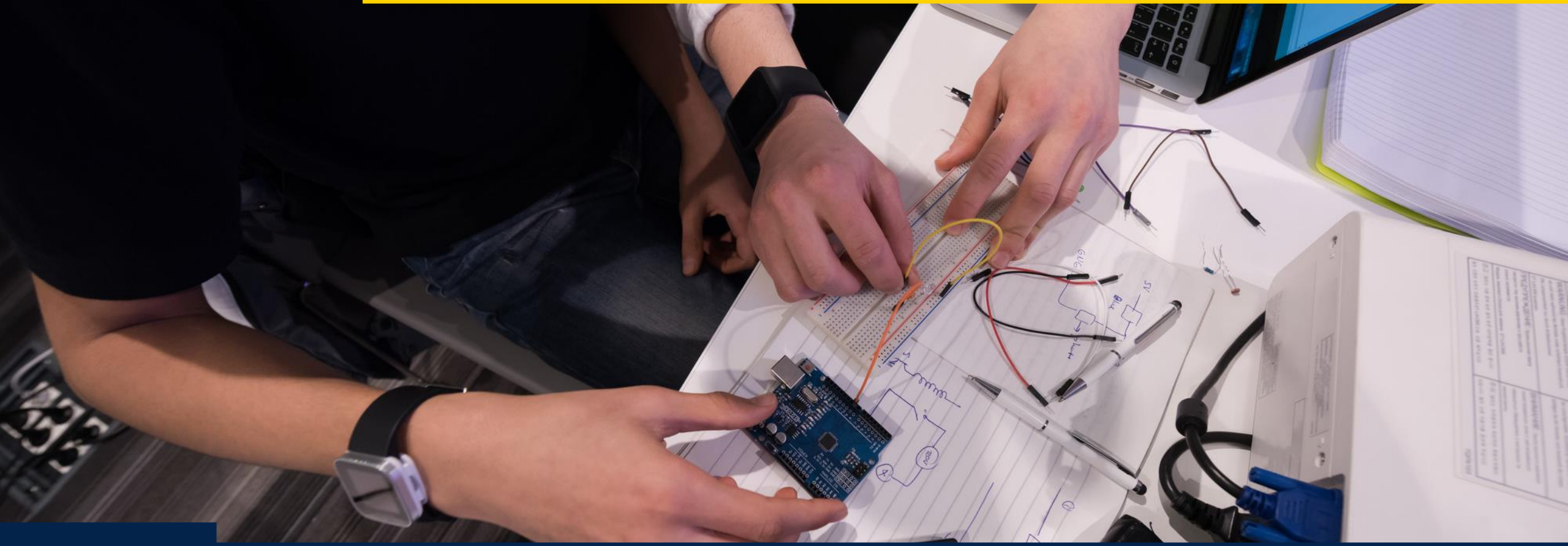
CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
-	osEventFlagsNew()	xEventGroupCreate()
-	osEventFlagsGetName() - empty	-
-	osEventFlagsSet()	xEventGroupSetBits() xEventGroupSetBitsFromISR()
-	osEventFlagsClear()	xEventGroupClearBits() xEventGroupClearBitsFromISR()
-	osEventFlagsGet()	xEventGroupGetBits() xEventGroupGetBitsFromISR()
-	osEventFlagsWait()	xEventGroupWaitBits()
-	osEventFlagsDelete()	vEventGroupDelete()
-	-	vEventGroupSync()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Event Flags



life.augmented

Event Flags parameters (CMSIS_OS v2.x)
cmsis_os2.h

Event Flags parameters are grouped within lists at `osEventFlagsAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
`const char *name`		Name of the event flags
`uint32_t attr_bits`		Attribute bits
`void *cb_mem`		Pointer to event flags control block memory area
`uint32_t cb_size`		Event flags control block size

Event Flags available functions

Creating Event Flags

```
const osEventFlagsAttr_t Flags_attr ={.name=name };

osEventFlagsId_t osEventFlagsNew(const osEventFlagsAttr_t *attr)
```

Event Flags handler

NULL: default values

Delete Event Flags

```
osStatus_t osEventFlagsDelete(osEventFlagsId_t ef_id)
```

Event Flags handler

Event Flags available functions

Set **bit mask** within Event Flags

```
uint32_t osEventFlagsSet(osEventFlagsId_t ef_id, uint32_t flags)
```

Event Flags handler

Flags status after operation or error code from osStatus_t group casted to uint32_t

32b value with event bit mask to be set

Event Flags **bit mask** clear

```
uint32_t osEventFlagsClear(osEventFlagsId_t ef_id, uint32_t flags)
```

Flags status after operation or error code from osStatus_t group casted to uint32_t

Event Flags handler

Bit mask to be cleared

Event Flags available functions

Wait for particular **bit mask** within Event Flags

```
uint32_t osEventFlagsWait(osEventFlagsId_t ef_id,
uint32_t flags,
uint32_t options,
uint32_t timeout )
```

Flags status after operation or error code from osStatus_t group casted to uint32_t

Event Flags handler

Bit mask to be wait for

Clear all bits on exit?

Wait for all bits from the mask?

Max timeout

RTOS Agent task

- The agent task is a standard FreeRTOS task that is created automatically at scheduler start
- Originally it was used to execute software timer callback functions, now it is used by other components (like EVENT Flags)
- Its priority is set within `configTIMER_TASK_PRIORITY`
- Its stack size is set within `configTIMER_TASK_STACK_DEPTH`
- It is necessary to define `configUSE_TIMERS` and `INCLUDE_xTimerPendFunctionCall` within FreeRTOSConfig.h file to generate this function

RTOS Agent task example of FreeRTOSConfig.h settings

- Below there is an example of additional configuration of FreeRTOSConfig.h file in order to use Event Flags from interrupts:

```
#define configUSE_TIMERS                    1 (Enabled)
#define configTIMER_TASK_PRIORITY           ( 2 )
#define configTIMER_QUEUE_LENGTH            10
#define configTIMER_TASK_STACK_DEPTH        256
#define INCLUDE_xTimerPendFunctionCall      1 (Enabled)
```

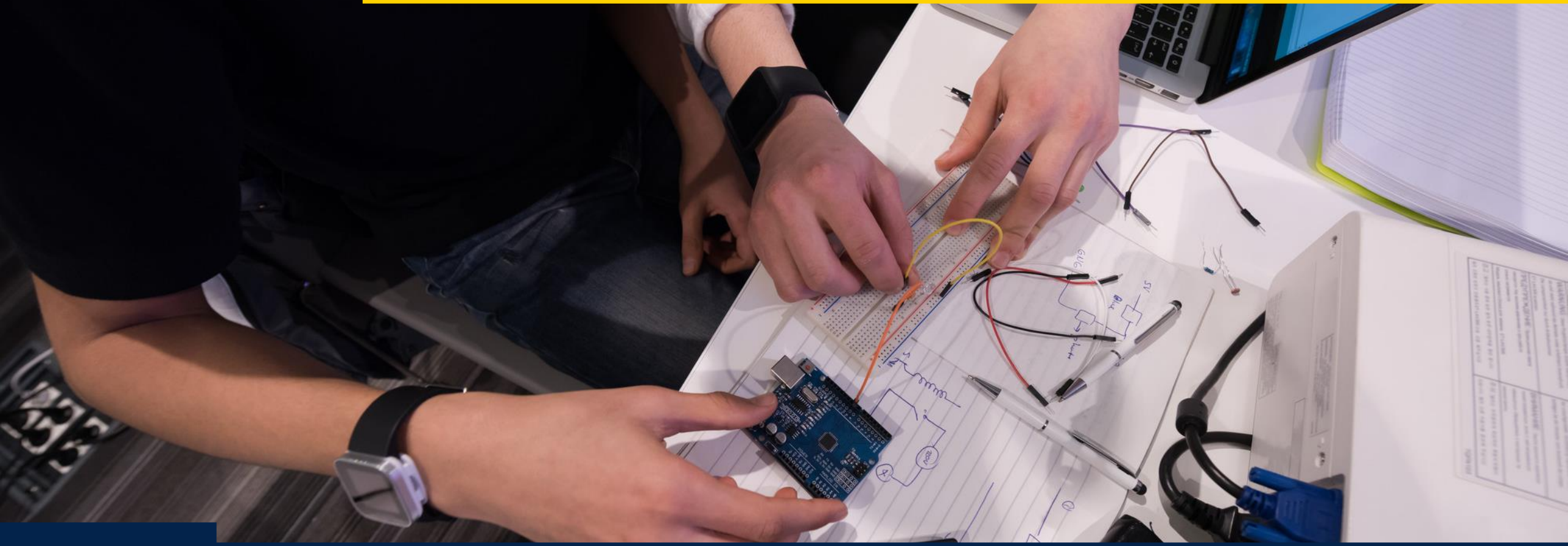
- Above configuration is enabled (and forced) by default when CMSIS_OS v2.x option of FreeRTOS has been used

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

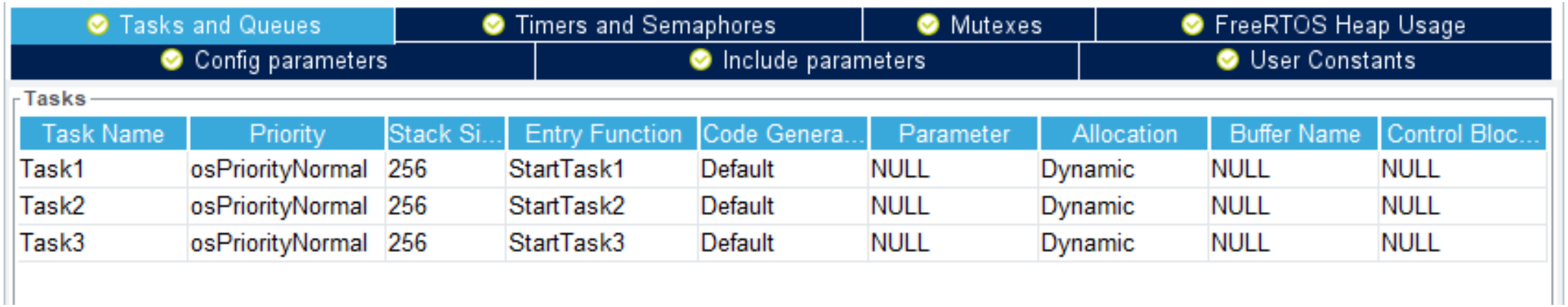
Event Flags lab



life.augmented

Event Flags lab example1 description

- We can reuse any project from previous lab sessions
- We would need one interrupt source (i.e. BLUE BUTTON) and three tasks on equal priorities



Task Name	Priority	Stack Si...	Entry Function	Code Genera...	Parameter	Allocation	Buffer Name	Control Bloc...
Task1	osPriorityNormal	256	StartTask1	Default	NULL	Dynamic	NULL	NULL
Task2	osPriorityNormal	256	StartTask2	Default	NULL	Dynamic	NULL	NULL
Task3	osPriorityNormal	256	StartTask3	Default	NULL	Dynamic	NULL	NULL

- Task1 will wait for an external interrupt and Task2 action occurrences (order not important) by waiting for 0x51 within EventGroup1
- Task2 will set 0x1 within EventGroup1
- Task3 will be used later
- BLUE_BUTTON interrupt will cause setting of 0x50 within EventGroup1

Event Flags lab
example1 code processing – main.c file

- Let's declare Event Flags variable

```
/* USER CODE BEGIN PV */
osEventFlagsId_t EventGroup1;
```

- Let's initialize it

```
/* USER CODE BEGIN RTOS_THREADS */
EventGroup1 = osEventFlagsNew(NULL);
```

- Task1 is waiting (being in blocked mode) for an external interrupt and Task2 action occurrence.

```
void StartTask1(void const * argument)
{
 for(;;)
 {
  osEventFlagsWait(EventGroup1, 0x51, osFlagsWaitAll, osWaitForever);
  Task_action('1');
  }
}
```

Event Flags lab
example1 code processing – main.c file

- Task2 is setting 0x1 within EventGroup1.

```
void StartTask2(void const * argument)
{
 for(;;)
 {
  osEventFlagsSet(EventGroup1, 1);
  Task_action('2');
   osDelay(3000);
 }
}
```

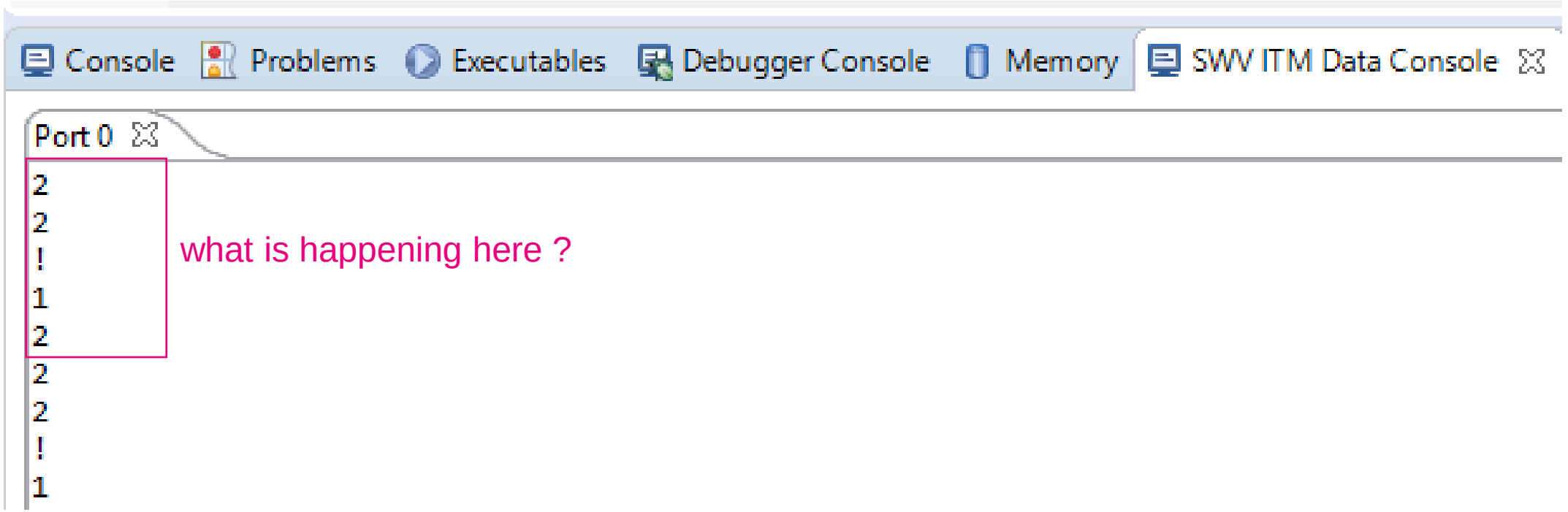
- Within external interrupt callback we send 0x50 within EventGroup1

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
 osEventFlagsSet(EventGroup1, 0x50);
 Task_action('!');
}
```

- Build the project and start debug session
- Task_action('1') will be invoked once on each button press and activation of Task2 (each 3 seconds).

Event Flags lab1

- Example of the result (using Task_action() to print simple messages over SWO)



Event Flags lab2
example2 code processing – main.c file

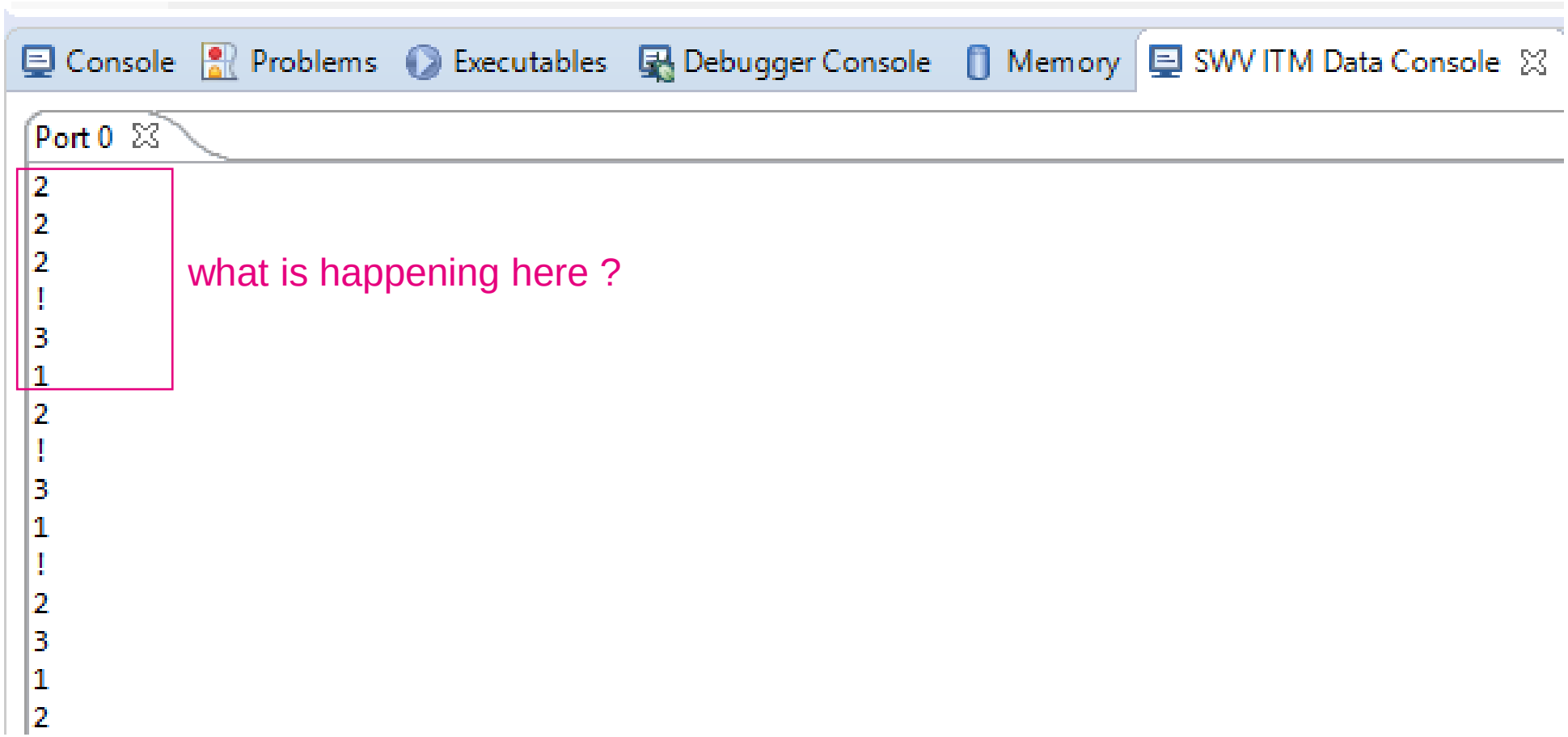
- Can we unblock more tasks on the same event flags set?
- We will continue with our application from previous example
- Let's modify our Task3 application to wait for 0x51 EventGroup1 occurrence as well

```
void StartTask3(void const * argument)
{
  for(;;)
  {
  osEventFlagsWait(EventGroup1, 0x51, osFlagsWaitAll, osWaitForever);
  Task_action('3');
  }
}
```

- Compile the project and start debug session
- Both (Task_action('1') and Task_action('3')) functions will be invoked once on each button press and activation of Task2 (each 3 seconds).

Event Flags lab2

- Example of the result (using Task_action() to print simple messages over SWO)



Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Direct to task notification – Thread Flags



life.augmented

configUSE_TASK_NOTIFICATIONS

Direct to task notification

- FreeRTOS Direct Task notifications feature is available starting from release 8.2.0.
- Within CMSIS_OS it is covered by less featured Signals (v1.x) and Thread Flags (v2.x).
- Each FreeRTOS task/thread has a 31-bit notification value (MSB is not used). An RTOS task notification is an event sent directly to a task that can unblock the receiving task.
- Task notifications can be used where previously it would have been necessary to create a separate queue, binary semaphore, counting semaphore or event group. Unblocking an RTOS task with a direct notification is **45% faster** and **uses less RAM** than unblocking a task with a binary semaphore.
- Task notification RAM footprint and speed advantage over other FreeRTOS feature (performing equivalent functionalities). Nevertheless It presents following limitations:
  - Task notifications can only be used to notify only one Task at a time : i.e only one task can be the recipient of the event. This condition is however met in the majority of real world applications.
  - If Task notification is used in place of a message queue then the receiving task (waiting for the notification) is set to the blocked state. However The sending task (sending the notification) cannot wait in the Blocked state for a send to complete if the send cannot complete immediately

Thread Flags
CMSIS_OS v2.x

- Thread Flags are used to trigger execution states between the threads and from IRQ to thread.
- The maximum number of Thread Flags is defined in **cmsis_os.h** (`osFeature_Signals`). It is set to 24. It is not possible to configure it from STM32CubeMX.
- Main functions:
  - `osThreadFlagsSet()` - set specified flags of an active thread

```
uint32_t osThreadFlagsSet (osThreadId_t thread_id, uint32_t signals)
```

Task handler

Signal bits which would be set (OR operation with already set signals @given task

  - `osThreadFlagsWait()` - wait for one or more flags for running thread

Flags options:
- osFlagsWaitAny (0) – default value – waits for any combination of expected flag set
- osFlagsWaitAll (1) – waits for complete flags/signals set)
- osFlagsNoClear (2) – flags are not cleared; any other value will clear the flag after

```
uint32_t osThreadFlagsWait (uint32_t signals, uint32_t options, uint32_t milisec)
```

Signals bits we are waiting for. Matching bits could be cleared (it depends on options)

Max delay we will wait for function completion

configUSE_TASK_NOTIFICATIONS

CMSIS_OS v2.x

Signals / Thread Flags

CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osSignalSet() (given value is OR-ed with current notification value of given task – eSetBits action set in cmsis_os.c)	**osThreadFlagsSet()**	**xTaskGenericNotify()** **xTaskGenericNotifyFromISR()**
osSignalClear() (empty declaration)	osThreadFlagsClear()	Not available
osSignalWait() (it is clearing notification value)	**osThreadFlagsWait()**	**xTaskNotifyWait()**
osSignalGet() (removed in CMSIS_OS v1.02)	-	Not available
-	-	xTaskNotifyGive() vTaskNotifyGiveFromISR()
-	-	ulTaskNotifyTake()
-	-	xTaskNotifyStateClear()
	osThreadFlagsGet()	xTaskNotifyAndQuery()

Signals (task notifications) cannot be used:

- To send an event or data to IRQ
- To communicate with more than one task (thread)
- To buffer multiple data items

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Direct to task notification
CMSIS-OS v2.x – Thread Flags lab

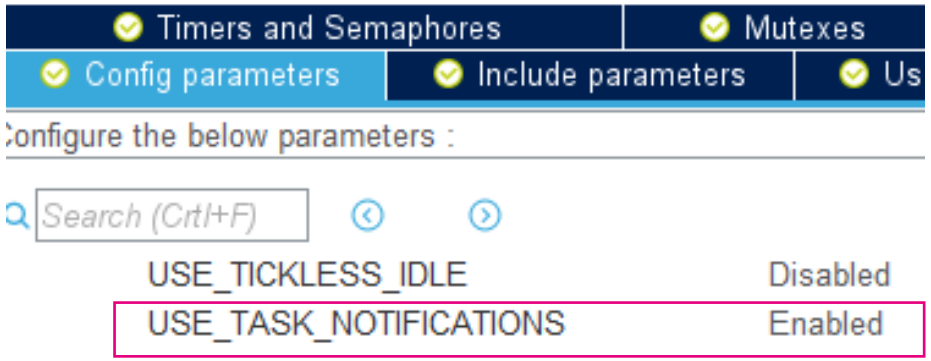


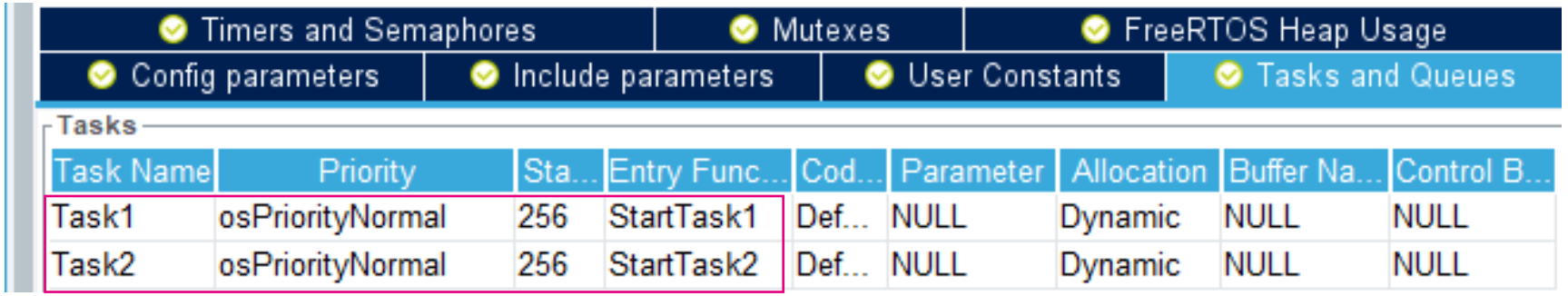
life.augmented

Thread Flags lab

CMSIS_OS v2.x

- We can reuse any project from previous lab sessions
- We would need one interrupt source (i.e. BLUE BUTTON) and two tasks on equal priorities
- Additionally `configUSE_TASK_NOTIFICATIONS` should be enabled within FreeRTOS configuration
- Generate the code by Ctrl+s and open main.c file





life.augmented

Thread Flags lab
example1 code processing – main.c file

- Within external interrupt callback flag 1 (bit 0) is set @Task1

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    osThreadFlagsSet(Task1Handle,1);
    Task_action('!');
}
```

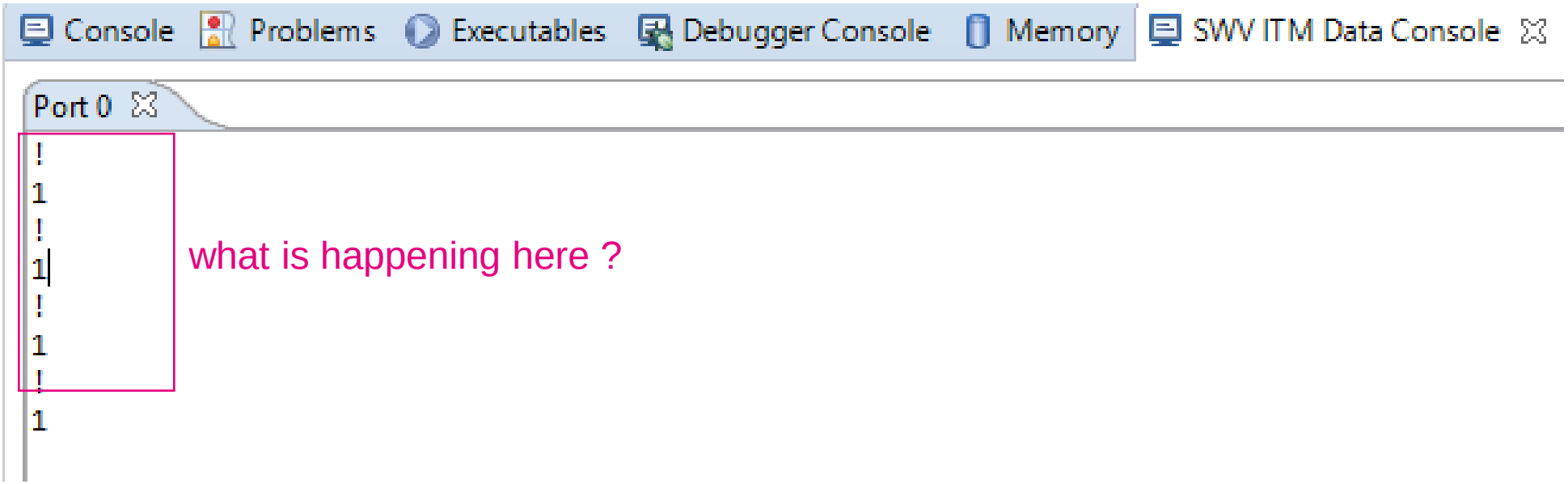
- Task1 is waiting (being in blocked mode) for a flag 0 set (value 1), flags will be cleared after

```
void StartTask1(void const * argument)
{
 for(;;)
  {
    osThreadFlagsWait(1,0,osWaitForever);
    Task_action('1');
  }
}
```

- Task_action('1') will be invoked once on each button press.

Thread Flags lab

- Example of the result (using Task_action() to print simple messages over SWO)



Thread Flags lab example2 code processing – main.c file

- Within Task2 function value 0x50 is stored within flags field of Task1 each 3 seconds

```
void StartTask2(void const * argument)
{
 for(;;)
  {
    osThreadFlagsSet(Task1Handle,0x50);
    Task_action('2');
    osDelay(3000);
  }
}
```

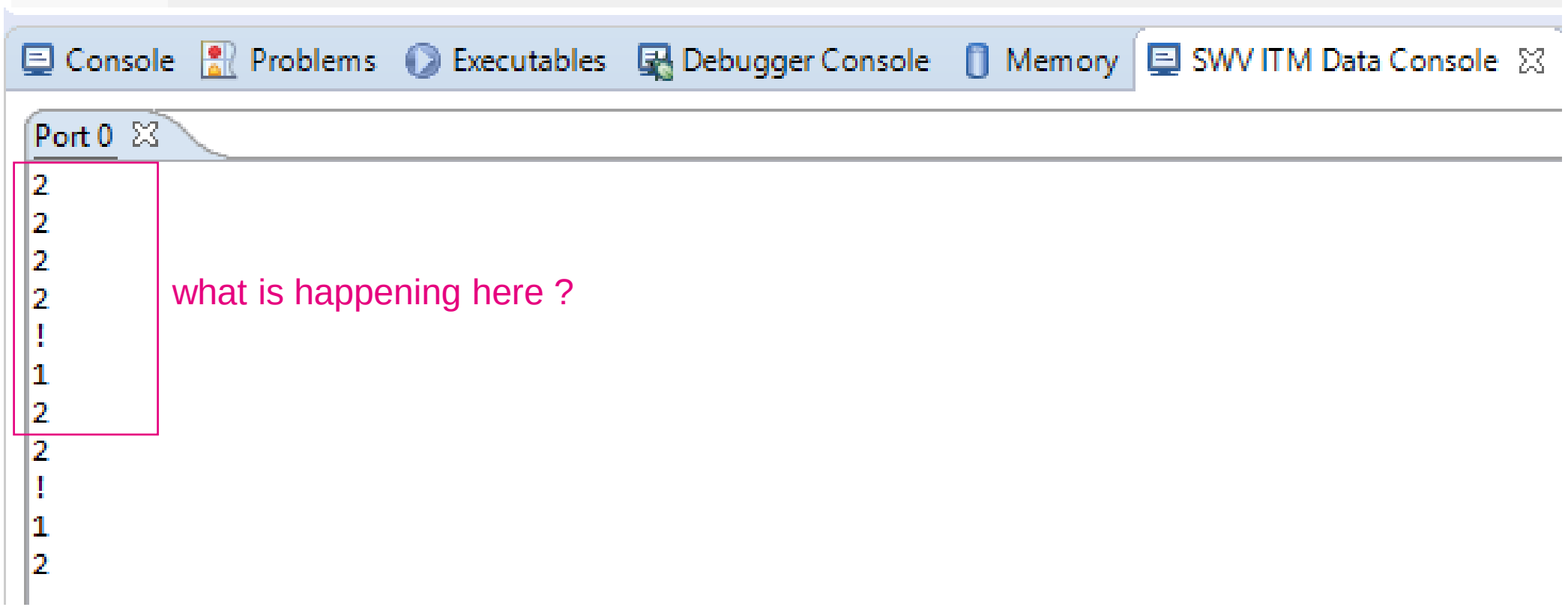
- Task1 is waiting (being in blocked mode) for a flag 0 set (value 1), flags will be cleared after

```
void StartTask1(void const * argument)
{
 for(;;)
  {
    osThreadFlagsWait(0x51, osFlagsWaitAll,osWaitForever);
    Task_action('1');
  }
}
```

- Task_action('1') will be invoked once each 3 seconds if the button has been pressed.

Thread Flags lab

- Example of the result (using Task_action() to print simple messages over SWO)

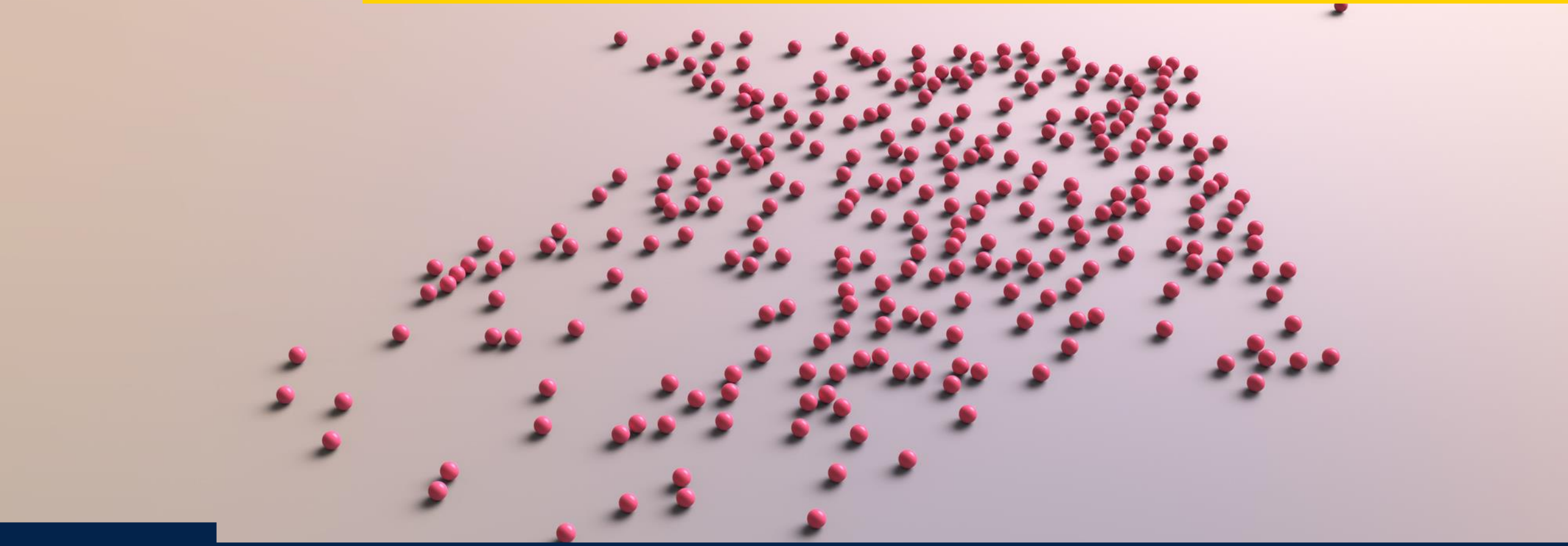


Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2 resources management



life.augmented

Resource management

- **Critical sections** – when it is necessary to block small piece of code inside the task against task switching or interrupts. This section should start with macro **`taskENTER_CRITICAL()`**, and should end with macro **`taskEXIT_CRITICAL()`**
- **Suspendig the scheduler** – when waiting on interrupt and no task switching allowed. Function **`vTaskSuspendAll()`** block context switching with interrupts enabled. Unblock the tasks is done by **`xTaskResumeAll()`** function.

  It is not allowed to run any FreeRTOS API function when scheduler is suspended.
- **Gatekeeper task**
  - Dedicated procedure managing selected resource (i.e. peripheral)
  - No risk of priority inversion and deadlock
  - It has ownership of a resource and can access it directly
  - Other tasks can access protected resource indirectly via gatekeeper task
  - Example: standard out access
- **Mutexes**
  - Kind of binary semaphore shared between tasks
  - Require set **`configUSE_MUTEXES`** at 1 in FreeRTOSConfig.h

life.augmented

Critical sections

- Critical section mechanism allows to block all the interrupts during sensitive/atomic operation execution (like operations on queues)
- To enter into critical section **`portENTER_CRITICAL()`** should be used
- To exit from critical section **`portEXIT_CRITICAL()`** should be used

```
void function(void)
{
        portENTER_CRITICAL();

        ..sensitive code execution

        portEXIT_CRITICAL();
}
```

Storing current BASEPRI value (current level of interrupt masking) and programming BASEPRI with MASK value *)
It means masking of all interrupts below this value

restoring BASEPRI value and masking of all interrupts below this value

*) MASK = configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY<<(8- configPRIO_BITS)

defined in FreeRTOSConfig.h

= 4 for CortexM3, CortexM4 based STM32

life.augmented

Gatekeeper task

- Gatekeeper is a task being the only allowed to access certain resources (i.e. peripheral).
  - It owns selected resource and only it can access it directly; other tasks can do it indirectly by using services provided by the gatekeeper task.
    - There is nothing physically preventing other tasks from accessing the resource → it is on the designer side to program it proper way
  - It spends most of the time in the blocked state waiting for the requests on the owned resources
  - It is up to the designer to set the priority of the gatekeeper and its name.
  - It is providing clean method to implement mutual exclusion without risk of priority inversion or deadlock.

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Mutexes



life.augmented

Mutex 1/2

- Mutex is a binary semaphore that include a priority inheritance mechanism.
  - binary semaphore is the better choice for implementing synchronization (between tasks or between tasks and an interrupt),
  - mutex is the better choice for implementing simple mutual exclusion (hence 'MUT'ual 'EX'clusion).

- When used for mutual exclusion the mutex acts like a token that is used to guard a resource.
  - When a task wishes to access the resource it must first obtain ('take') the token.
  - When it has finished with the resource it must 'give' the token back - allowing other tasks the opportunity to access the same resource.
  - In case of **recursive mutex** it should be given as many times as it was successfully taken (like counting semaphores) to release it for another task.

Mutex 2/2

- Mutexes use the same access API functions as semaphores
- The block time indicates the maximum number of 'ticks' that a task should enter the Blocked state when attempting to 'take' a mutex if the mutex is not available immediately.
- Unlike binary semaphores mutexes employ priority inheritance. This means that if a high priority task is blocked while attempting to obtain a mutex (token) that is currently held by a lower priority task, then the priority of the task holding the token is temporarily raised to that of the blocked task.
- A task must not be deleted while it is controlling a Mutex. Otherwise, the Mutex resource will be locked out to all other tasks
- Mutex Management functions cannot be called from interrupts (IRQ).

life.augmented

Mutex, Semaphore – threats 1/3

Priority inversion

- This is the situation where a higher priority task is waiting for a lower priority task to give a control of the mutex and low priority task is not able to execute.

Mutex, Semaphore – threats 2/3

Priority inheritance

- It is temporary raise of the priority of the mutex holder to that of the highest priority task that is attempting to obtain the same mutex. The low priority task that holds the mutex inherits the priority of the task waiting for the mutex. The priority of the mutex holder is reset automatically to its original value when it gives the mutex back.
- It is a mechanism that minimizes the negative effects of priority inversion
- It is complicating system timing analysis and it is not a good practice to rely on it for correct system operation

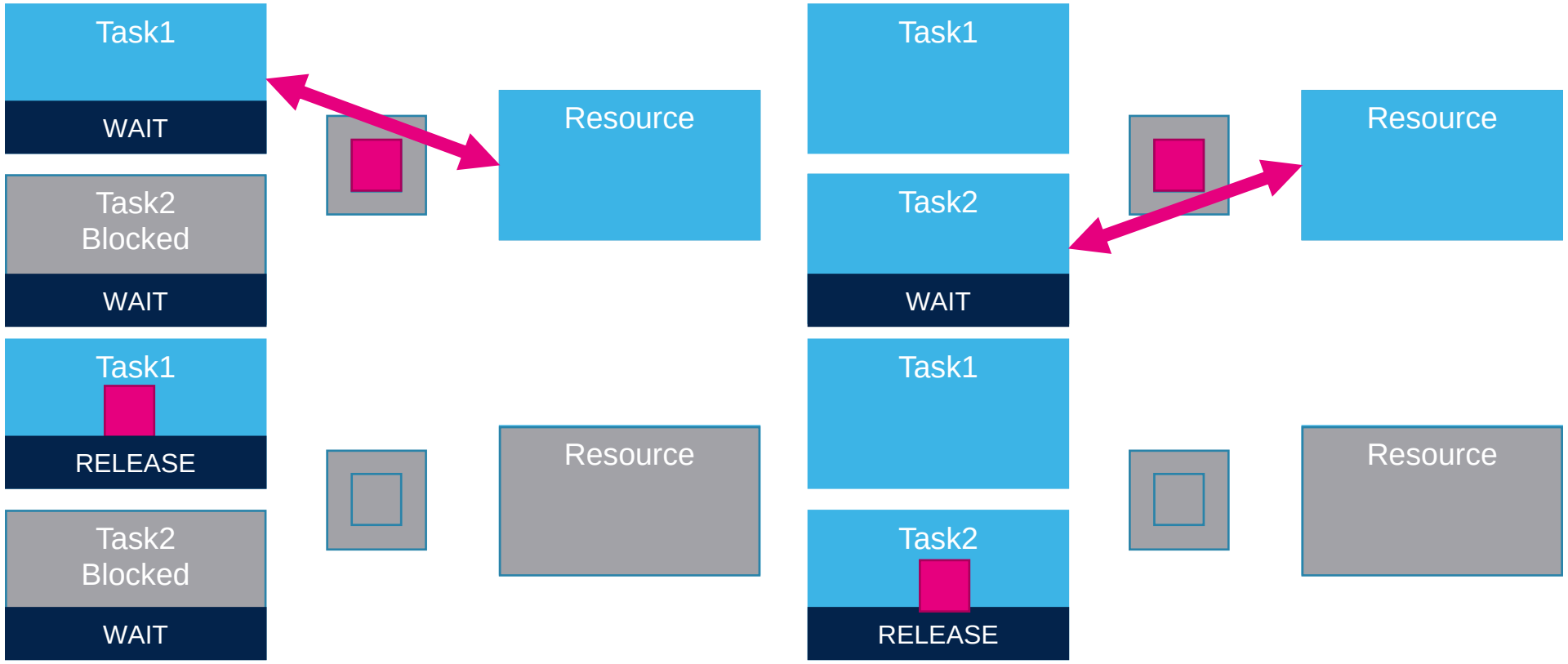
Mutex, Semaphore – threats 3/3

Deadlock (Deadly Embrace)

- It occurs when two tasks cannot work because they are both waiting for a resource held by each other
- The best way to avoid deadlock is to consider them at design time and design the system to be sure that the deadlock cannot occur.

Mutex

- Used to guard access to limited recourses
- Works very similar as semaphores



life.augmented

Mutex attributes (CMSIS_OS v2.x) - cmsis_os2.h

Mutex attributes are grouped within lists at `osMutexAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
**const char ***name	(CubeMX)	Name of the mutex
uint32_t attr_bits		Attribute bits
**void ***cb_mem		Pointer to mutex control block memory area
uint32_t cb_size		Mutex control block (TCB) size

Mutex
CMSIS_OS v2.x

- Mutex creation

```
const osMutexAttr_t attr = { .name = “name” };
osMutexId_t osMutexNew (const osMutexAttr_t *attr)
```

Mutex handle

Mutex attributes

- Wait for Mutex release (Mutex Acquire)

```
osStatus_t osMutexAcquire (osMutexId_t mutex_id, uint32_t millisec)
```

Return status

Mutex handle

How long wait for mutex release

- Mutex release

```
osStatus_t osMutexRelease (osMutexId_t mutex_id)
```

Return status

Mutex handle

CMSIS_OS v1.x
CMSIS_OS v2.x

Mutex APIs

CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osMutexCreate()	osMutexNew()	xSemaphoreCreateMutexStatic() xSemaphoreCreateMutex()
osMutexRelease()	osMutexRelease()	xSemaphoreGive() xSemaphoreGiveFromISR()
osMutexWait()	osMutexAcquire()	xSemaphoreTake() xSemaphoreTakeFromISR()
osMutexDelete()	osMutexDelete()	vQueueDelete()
osRecursiveMutexCreate()	osMutexNew() with .attr_bits = osMytexRecursive within attrbutes	xSemaphoreCreateRecursiveMutexStatic() xSemaphoreCreateRecursiveMutex()
osRecursiveMutexRelease()	osMutexRelease()	xSemaphoreGiveRecursive()
osMutexAcquire()	osMutexAcquire()	xSemaphoreTakeRecursive()
-	osMutexGetName()	-
-	osMutexGetOwner()	xSemaphoreGetMutexHolder()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

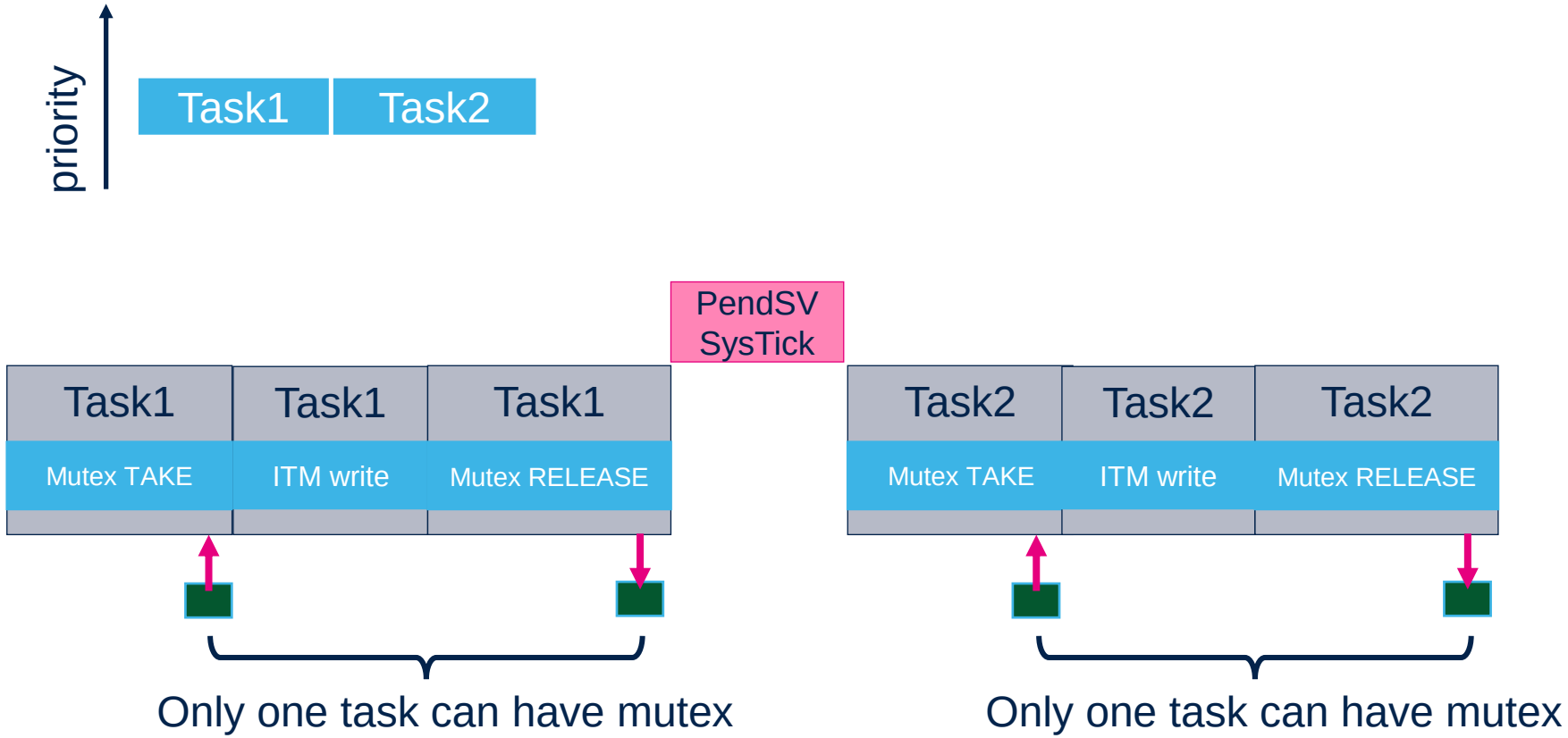
FreeRTOS – CMSIS_OS v2
Mutexes - lab



life.augmented

Mutex lab - purpose

- Both tasks use ITM to send data using Task_action()
- Mutex is used to avoid collisions, so only one task can pass its data over ITM in the same time

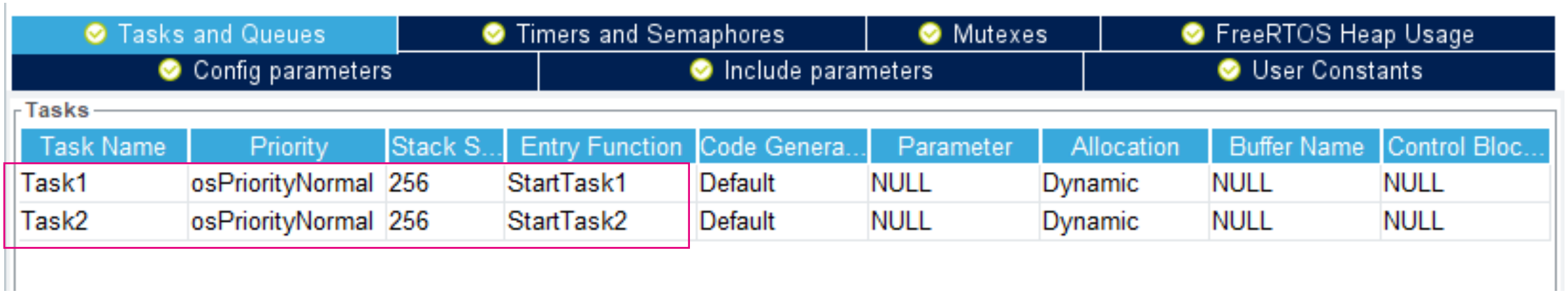


life.augmented

configUSE_MUTEXES

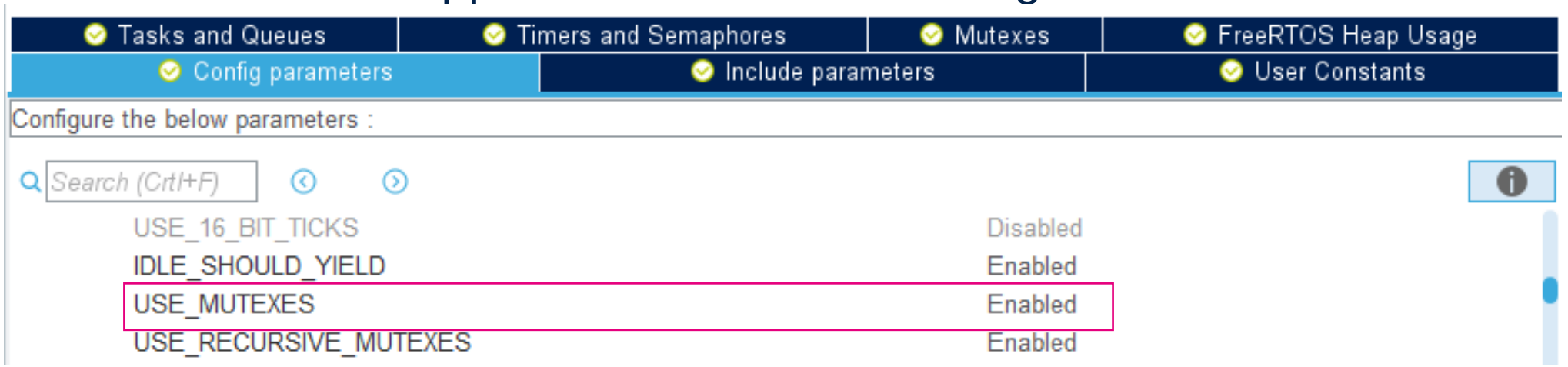
Mutex lab
basic configuration and code generation

- We need two tasks with the same priority level



Task Name	Priority	Stack S...	Entry Function	Code Genera...	Parameter	Allocation	Buffer Name	Control Bloc...
Task1	osPriorityNormal	256	StartTask1	Default	NULL	Dynamic	NULL	NULL
Task2	osPriorityNormal	256	StartTask2	Default	NULL	Dynamic	NULL	NULL

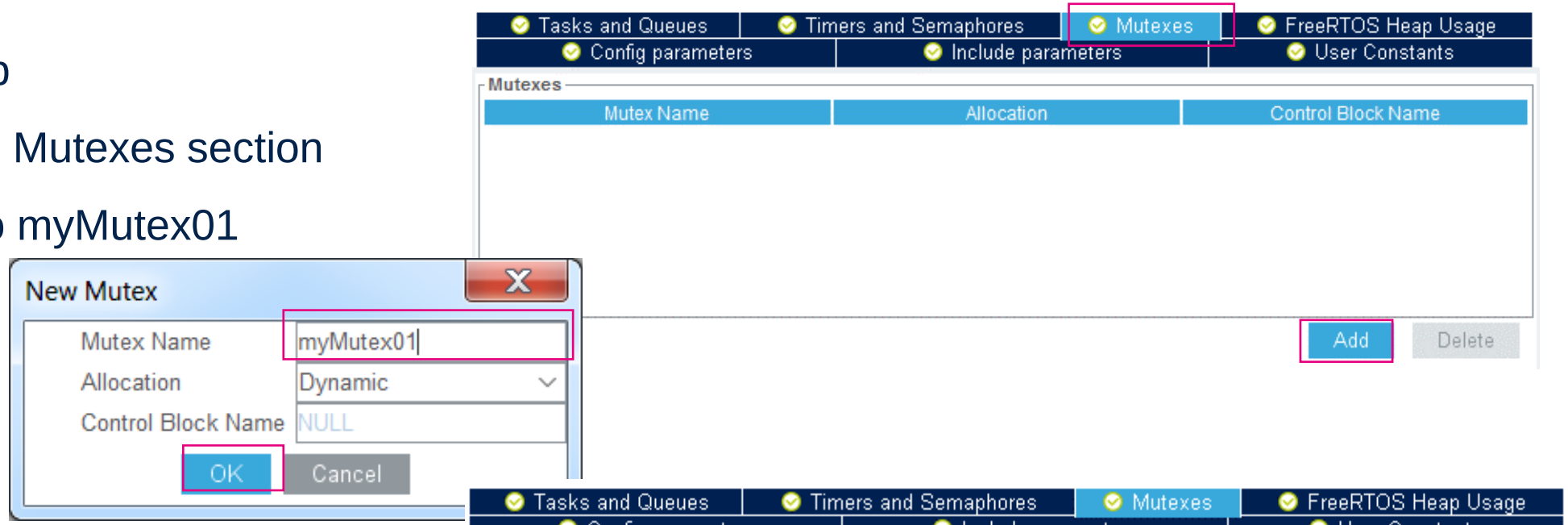
- We need to add Mutex support into FreeRTOS configuration

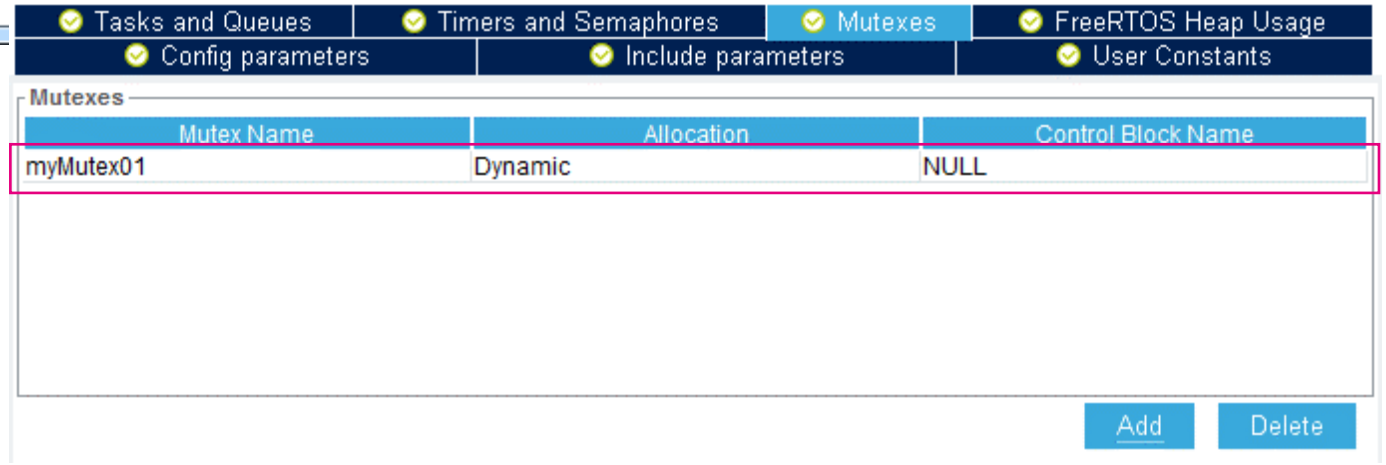


life.augmented

Mutex lab
basic configuration and code generation

- Add Mutex
- Select Mutexes tab
- Click Add button in Mutexes section
- Set Mutex name to myMutex01
- Click OK button





- Generate the code by Ctrl+s and open main.c file

life.augmented

Mutex lab
code processing – main.c file

- Mutex handle definition

```
/* Private variables -----------------------------------------*/
osThreadId_t Task1Handle;
osThreadId_t Task2Handle;
osMutexId_t myMutex01Handle;
```

- Mutex creation

```
 /* Create the mutex(es) */
  /* definition and creation of myMutex01 */
  const osMutexAttr_t myMutex01_attributes = {.name = "myMutex01" };
  myMutex01Handle = osMutexNew(&myMutex01_attributes);
```

Mutex lab
code processing – main.c file

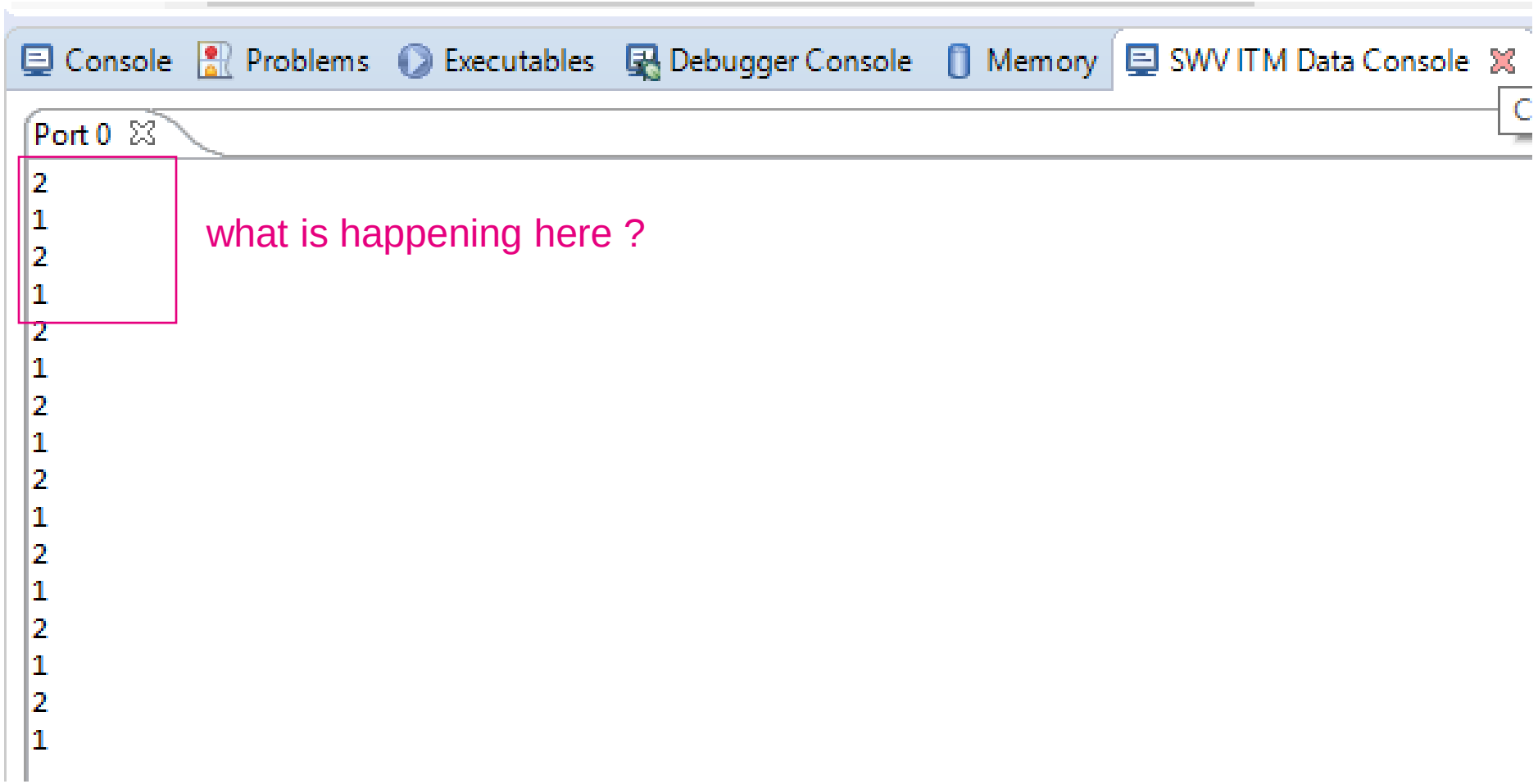
- Task1 and Task2 using of Mutex

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    osDelay(2000);
    osMutexAcquire(myMutex01Handle,1000);
    Task_action('1');
    osMutexRelease(myMutex01Handle);
  }
  /* USER CODE END 5 */
}
```

```
void StartTask2(void const * argument)
{
  /* USER CODE BEGIN StartTask2 */
  /* Infinite loop */
  for(;;)
  {
    osDelay(2000);
    osMutexAcquire(myMutex01Handle,1000);
    Task_action('2');
    osMutexRelease(myMutex01Handle);
  }
  /* USER CODE END StartTask2 */
}
```

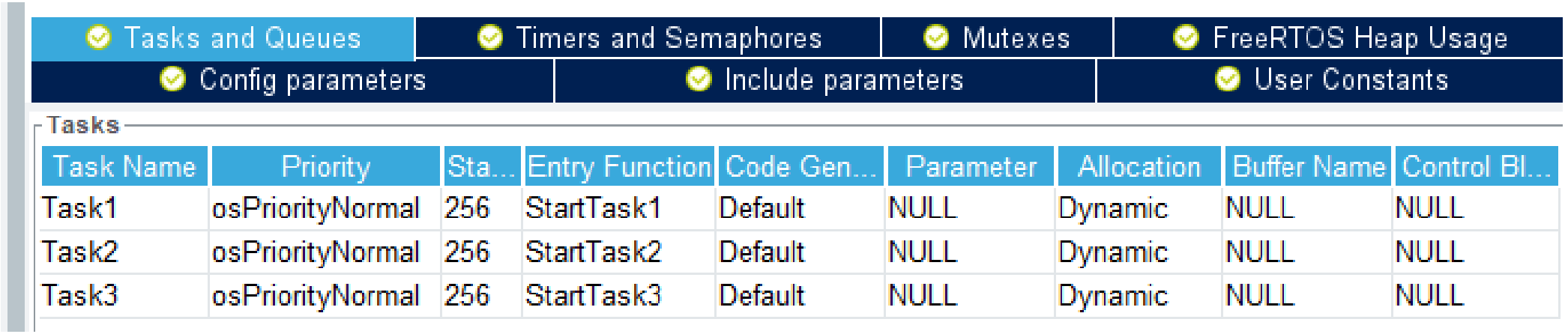
Mutex lab

- Example of the result (using Task_action() to print simple messages over SWO)



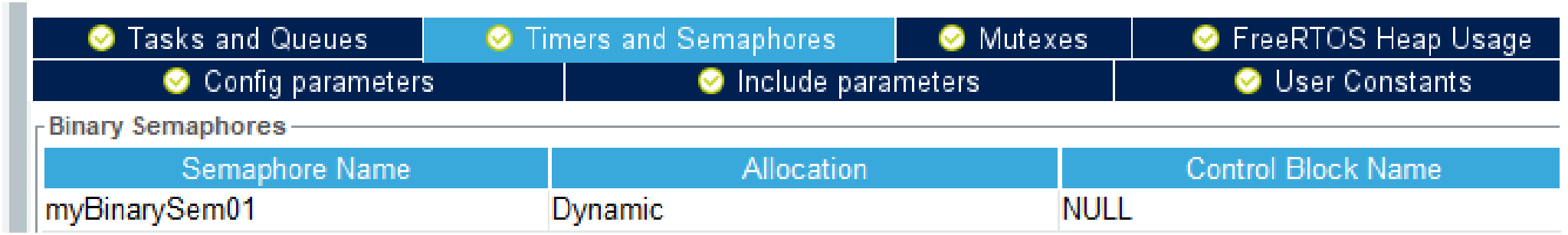
Mutex vs semaphore lab

- What would happened if at some moment Task1 will have lower priority than Task2 and it will hold Mutex which will be requested by Task2 ?
- Let's add a new Task3 with `osPriorityNormal`



Task Name	Priority	Sta...	Entry Function	Code Gen...	Parameter	Allocation	Buffer Name	Control Bl...
Task1	osPriorityNormal	256	StartTask1	Default	NULL	Dynamic	NULL	NULL
Task2	osPriorityNormal	256	StartTask2	Default	NULL	Dynamic	NULL	NULL
Task3	osPriorityNormal	256	StartTask3	Default	NULL	Dynamic	NULL	NULL

- Let's add one binary semaphore myBinarySem01 for future use:



Semaphore Name	Allocation	Control Block Name
myBinarySem01	Dynamic	NULL

life.augmented

Mutex vs semaphore lab

- We can add some code into StartTask1 procedure:

```
void StartTask1(void *argument)
{
  uint8_t idx=0;
  for(;;)
  {
   osMutexAcquire(myMutex01Handle,osWaitForever);
   Task_action('1');
   if(3==idx)
   {
     osThreadSetPriority(Task1Handle, osPriorityLow);
     //osThreadYield();
    }
   idx++;
   osMutexRelease(myMutex01Handle);
   HAL_Delay(500);
  }
}
```

life.augmented

Mutex vs semaphore lab

- We can add some code into StartTask3:

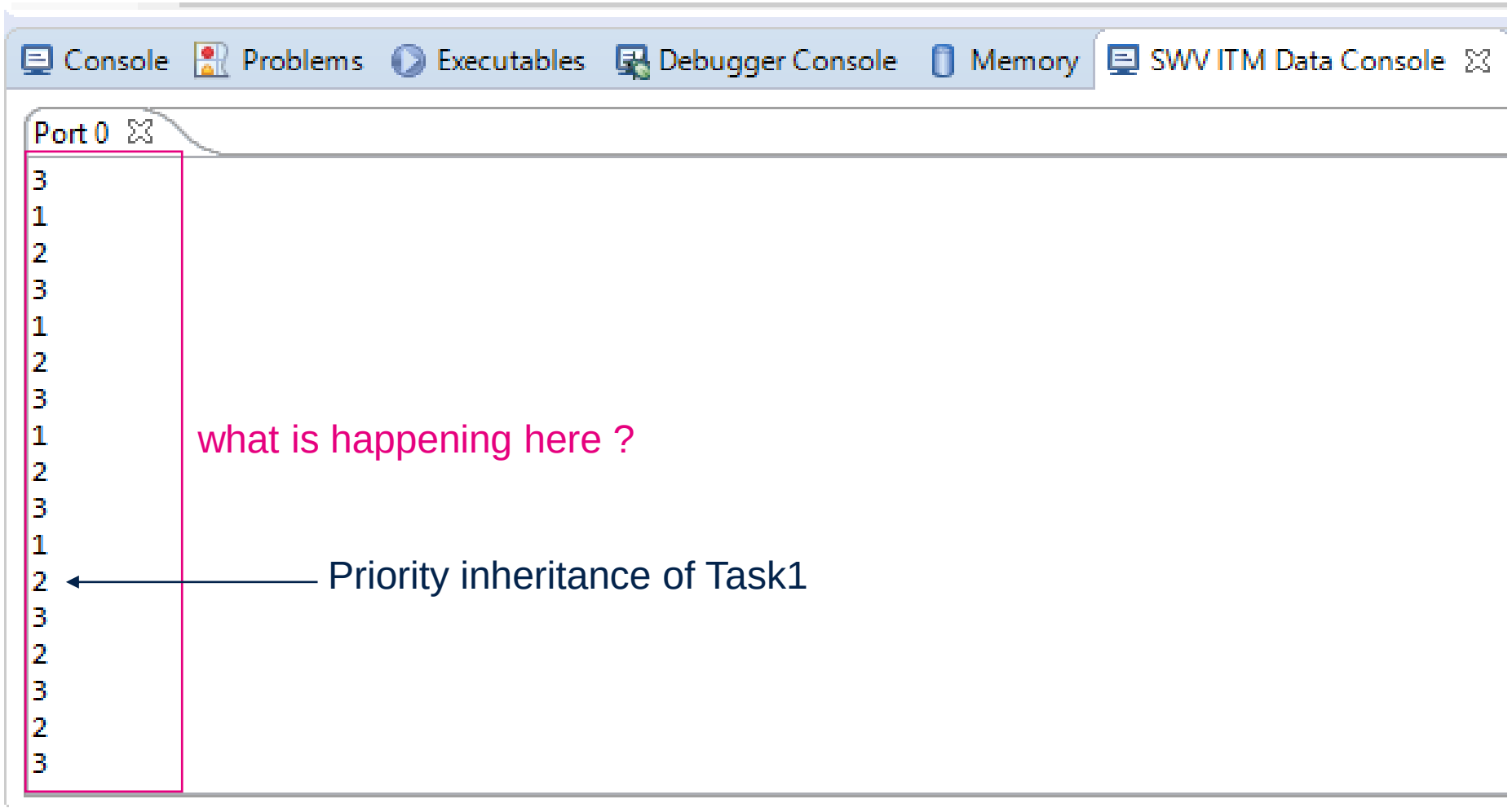
```
void StartTask3(void *argument)
{
 for(;;)
  {
   Task_action('3');
   HAL_Delay(500);
  }
}
```

- StartTask2 procedure remains not changed
- Let’s build the project and start debug session

life.augmented

Mutex lab

- Example of the result (using Task_action() to print simple messages over SWO)



life.augmented

Mutex vs semaphore lab

- Let’s change Mutex into Binary Semaphore within StartTask1 function

```
void StartTask1(void *argument)
{
  uint8_t idx=0;
  for(;;)
  {
   osSemaphoreAcquire(myBinarySem01Handle, osWaitForever);
   Task_action('1');
   if(3==idx)
   {
     osThreadSetPriority(Task1Handle, osPriorityLow);
     osThreadYield();
    }
   idx++;
   osSemaphoreRelease(myBinarySem01Handle);
   HAL_Delay(500);
  }
}
```

life.augmented

Mutex vs semaphore lab

- Let’s change Mutex into Binary Semaphore within StartTask2 function

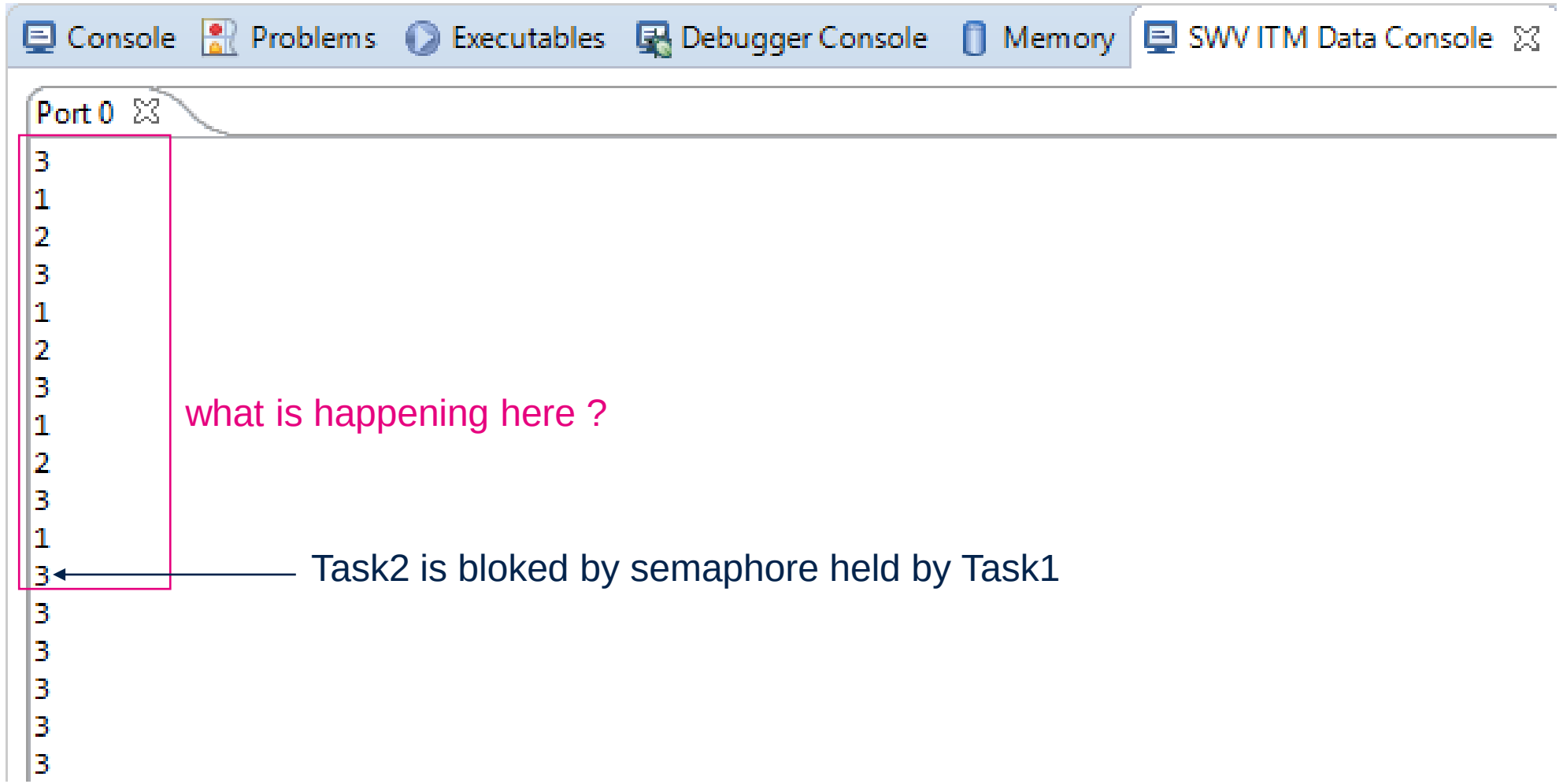
```
void StartTask2(void *argument)
{
 for(;;)
  {
   osSemaphoreAcquire(myBinarySem01Handle, osWaitForever);
   Task_action('2');
   osSemaphoreRelease(myBinarySem01Handle);
   HAL_Delay(500);
  }
}
```

- No change within StartTask3 function
- Build the project and start debug session

life.augmented

Mutex lab – semaphore case

- Example of the result (using Task_action() to print simple messages over SWO)



life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Software timers



life.augmented

Software timers (1/3)

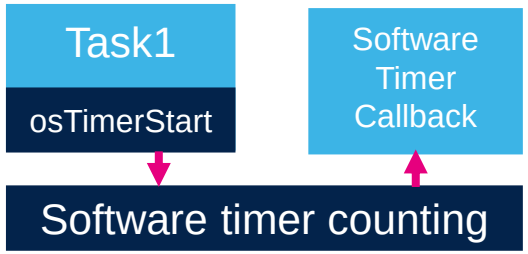
- Software timer is one of standard component of every RTOS
- FreeRTOS “software” Timers allows to execute a callback at a set of time (timer period). Timer callback functions execute in the context of the timer service task.
- It is therefore essential that timer callback functions never attempt to block. For example, a timer callback function must not call vTaskDelay(), vTaskDelayUntil(), or specify a non zero block time when accessing a queue or a semaphore.

Software timers (2/3)

- It is not precise, intended to handle periodic actions and delay generation
  - Can be conditionally used to extend number of hardware timers in STM32
- Two types od software timers are available:
  - Periodic (execute its callback periodically with autoreload functionality)



  - One Pulse (execute its callback only once with an option of manual re-trigger)



life.augmented

Software timers (3/3)

- When Timers are enabled (`configUSE_TIMERS` enabled) , the scheduler creates automatically the timers service task (daemon) when started (calling `xTimerCreateTimerTask()` function).
- The timers service task is used to control and monitor (internally) all timers that the user will create.
- The timers task parameters are set through the following defines (in FreeRTOSConfig.h):
  - `configTIMER_TASK_PRIORITY` : priority of the timers task
  - `configTIMER_TASK_STACK_DEPTH` : timers task stack size (in words)
- The scheduler also creates automatically a message queue used to send commands to the timers task (timer start, timer stop ...) .
- The number of elements of this queue (number of messages that can be hold) are configurable through the define:

  `configTIMER_QUEUE_LENGTH`.

life.augmented

configUSE_TIMERS

Software timers configuration

Config field (default value)	description
configUSE_TIMERS (0 – disabled)	1 – includes software timers functionality and automatically creates timer service task on scheduler start 0 – disabled, no timer service task
configTIMER_TASK_PRIORITY ()	Priority for timer service task from the range between IDLE task priority and configMAX_PRIORITIES-1
configTIMER_QUEUE_LENGTH ()	This sets the maximum number of unprocessed commands that the timer command queue can hold at any one time.
configTIMER_TASK_STACK_DEPTH ()	Sets the size of the stack (in words, not bytes) allocated to the timer service task.

More information:
https://www.freertos.org/Configuring-a-real-time-RTOS-application-to-use-software-timers.html

life.augmented

Software Timers parameters (CMSIS_OS v2.x) cmsis_os2.h

Software timers parameters are grouped within lists at `osTimerAttr_t` objects (**cmsis_os2.h** file)

Field name	(CubeMX configurable), default	comment
const char *name	(CubeMX)	Name of the timer
uint32_t attr_bits		Attribute bits
void *cb_mem		Pointer to timer control block memory area
uint32_t cb_size		Timer control block (TCB) size

Software timers CMSIS_OS v2.x

- Software timer creation

```
const osTimerAttr_t attr = {.name = “timer_name“ };
osTimerId_t osTimerNew (osTimerFunc_t func, osTimerType_t type, void *argument, const osTimerAttr_t *attr)
```

Timer handle

Pointer to callback function

Timer type: one-shot or periodic

Argument to timer callback function

Timer attributes (NULL – defaults)

- Software timer start

```
osStatus_t osTimerStart (osTimerId_t timer_id, uint32_t ticks)
```

Return status

Timer handle

Timer period

- Software timer stop

```
osStatus_t osTimerStop (osTimerId_t timer_id)
```

Return status

Timer handle

CMSIS_OS v1.x
CMSIS_OS v2.x

Software timers APIs

CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osTimerCreate()	osTimerNew()	xTimerCreateStatic() xTimerCreate()
osTimerStart()	osTimerStart()	xTimerChangePeriod() xTimerChangePeriodFromISR()
osTimerStop()	osTimerStop()	xTimerStop() xTimerStopFromISR()
osTimerDelete()	osTimerDelete()	xTimerDelete()
	osTimerIsRunning()	xTimerIsTimerActive()
-	-	xTimerGetTimerDaemonTaskHandle()
-	-	xTimerGetPeriod()
-	-	xTimerGetExpiryTime()
-	osTimerGetName()	pcTimerGetName()
-	-	xTimerGenericCommand()

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

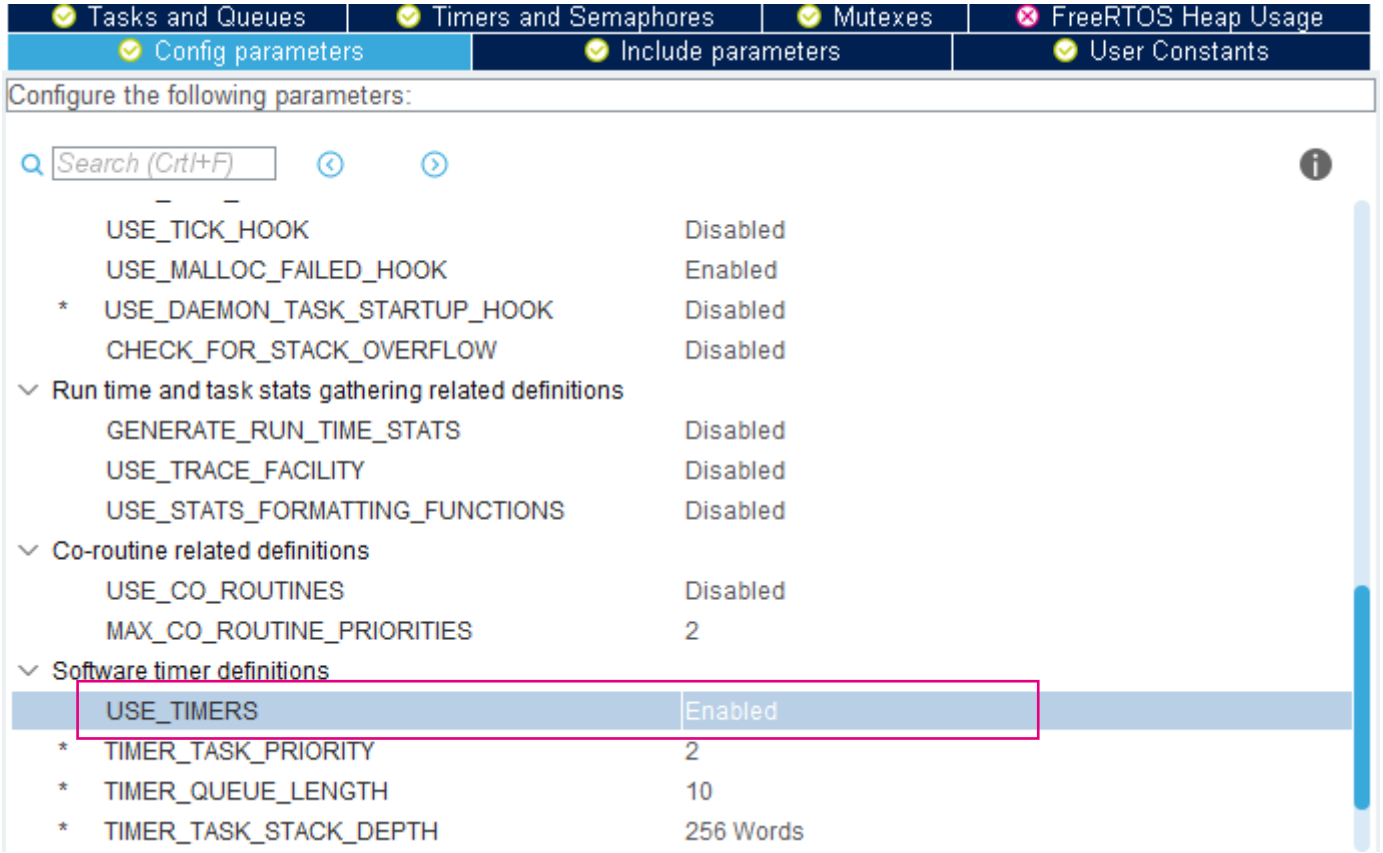
FreeRTOS – CMSIS_OS v2
Software timers lab



life.augmented

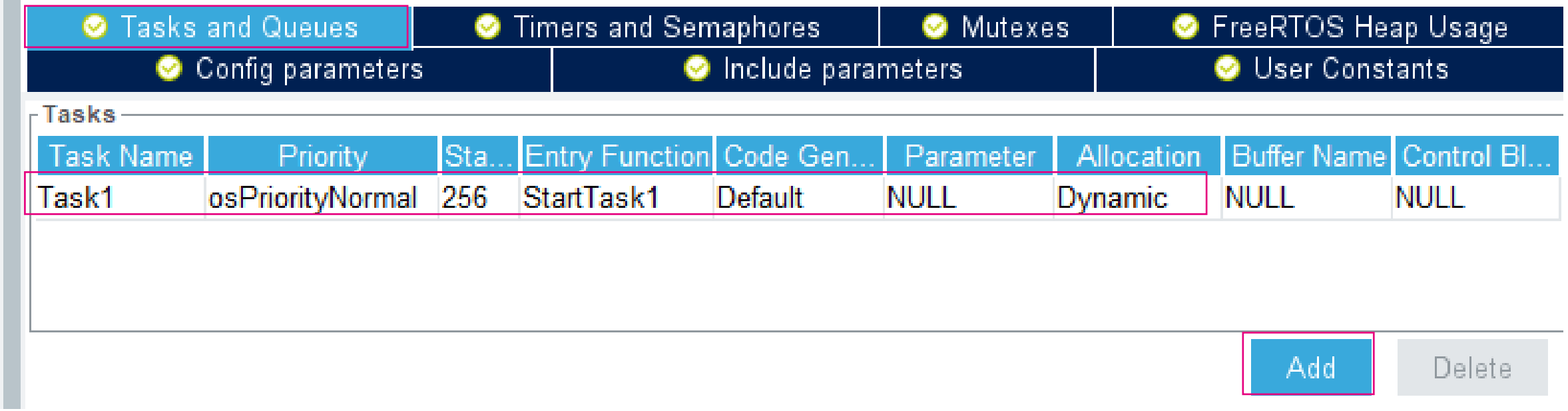
Software timers lab

- Software timers are disabled by default in STM32CubeMX (in CMSIS_OS v1)
- To enable them:
  - Select Config parameters tab
  - Set `USE_TIMERS` value to Enabled
  - Other software timers parameters we will keep in default configuration



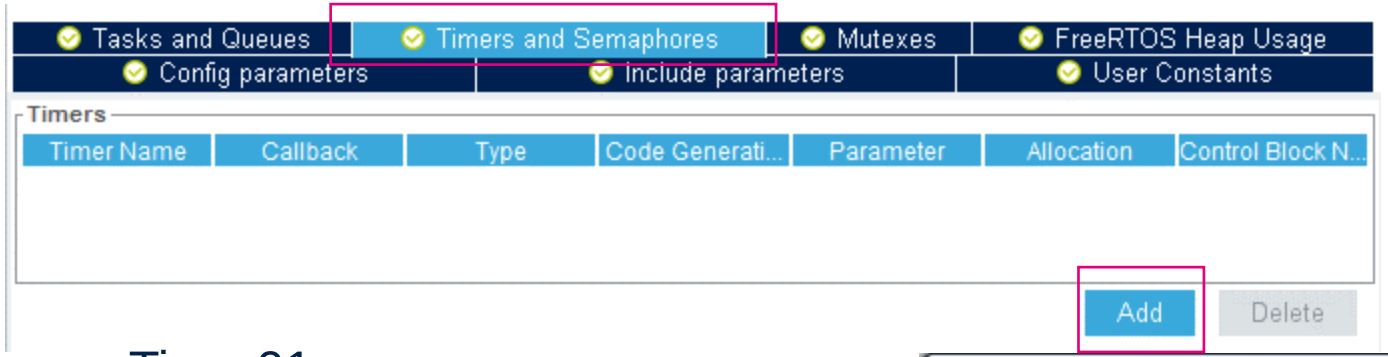
Software timers lab

- Using one of previous exercises
- Create one task, Task1 with entry function StartTask1 and normal priority

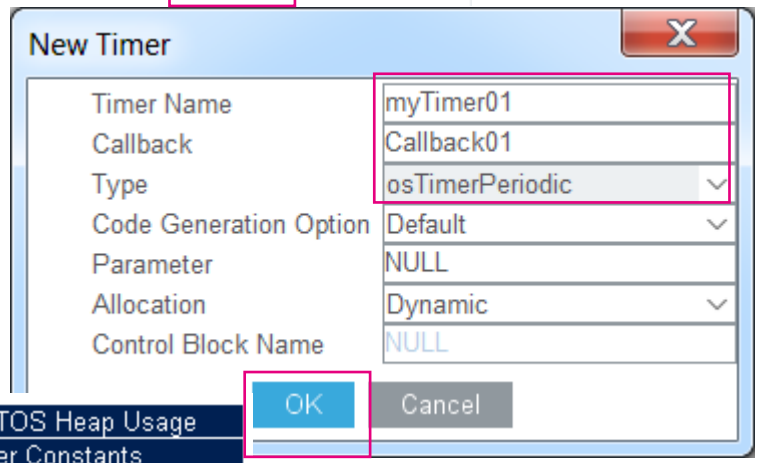


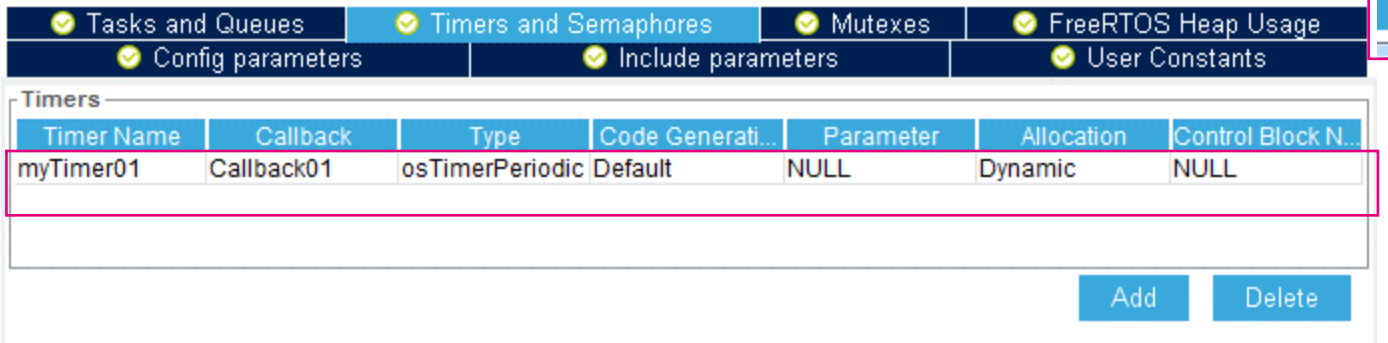
Software timers lab

- Create a new timer
- Select Timers and Semaphores tab
- Click Add button in Timers section



- Set timer name: i.e. myTimer01
- Timer callback name: i.e. Callback01
- Type: Periodic
- Click OK button
- Generate the code (Ctrl+s) and open main.c





life.augmented

Software timers lab code processing – main.c file

- Software timer handle definition

```
/* Private variables ---------------------------------------------------------*/
osThreadId_t Task1Handle;
osTimerId_t myTimer01Handle;
```

- Software timer creation

```
/* Create the timer(s) */
 /* definition and creation of myTimer01 */
 const osTimerAttr_t myTimer01_attributes = { .name = "myTimer01" };
 myTimer01Handle = osTimerNew(Callback01, osTimerPeriodic, NULL, &myTimer01_attributes);
```

- Software timer start

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  osTimerStart(myTimer01Handle,1000);
  /* Infinite loop */
  for(;;)
  {
    osDelay(2000);
    Task_action('1');
  }
  /* USER CODE END 5 */
}
```

Software timers lab code processing – main.c file

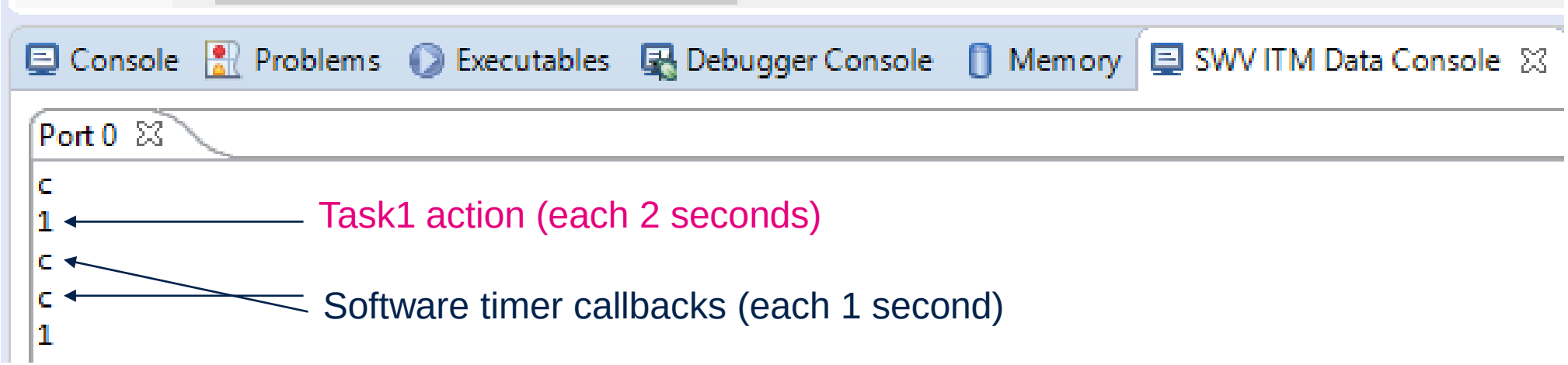
- Timer callback functions execute in the context of the timer service task.
- Timer callbacks are not called from interrupt context.
- There should be no blocking functions inside (like in hooks)

```
/* Callback01 function */
void Callback01(void const * argument)
{
  /* USER CODE BEGIN Callback01 */
   Task_action('c');
  /* USER CODE END Callback01 */
}
```

```
/* Callback01 function */
void Callback01(void const * argument)
{
  /* USER CODE BEGIN Callback01 */
  Task_action('c');
  osDelay(100);
  /* USER CODE END Callback01 */
}
```

Software timers lab

- Example of the result (using Task_action() to print simple messages over SWO)



Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2 advanced topics



Hooks

- Hooks are the callbacks supported by FreeRTOS core
- Those can help with FreeRTOS fault handling
- Type of hooks:
  - Idle Hook
  - Tick Hook
  - Malloc Failed Hook
  - Stack Overflow Hook

- STM32CubeMX, STM32CubeIDE create hook functions in freertos.c file

configUSE_IDLE_HOOK

Idle task and „idle task hook”

- Idle task is automatically created by scheduler within **`osKernelStart()`** function
- It has the lowest possible priority
- It runs only if there are no tasks in ready state
- It can share same priority with other tasks
- Specific function (called idle task hook function) can be called automatically from idle task.
  Its prototype is strictly defined:
  - **`void vApplicationIdleHook(void);`** **// [weak] version in freertos.c file**
  - **`configUSE_IDLE_HOOK`** must be set to **1** in FreeRTOSConfig.h to get it called
  - it must never attempt to block or suspend
  - it is responsible to cleanup resources after deletion of other task
  - it is executed every iteration of the idle task loop, do not put any endless loop inside

```
void vApplicationIdleHook(void)
{
        tick_IDLE++;
}
```

Correct

```
void vApplicationIdleHook(void)
{
        while(1)
        {
                tick_IDLE++;
        }
}
```

Wrong

life.augmented

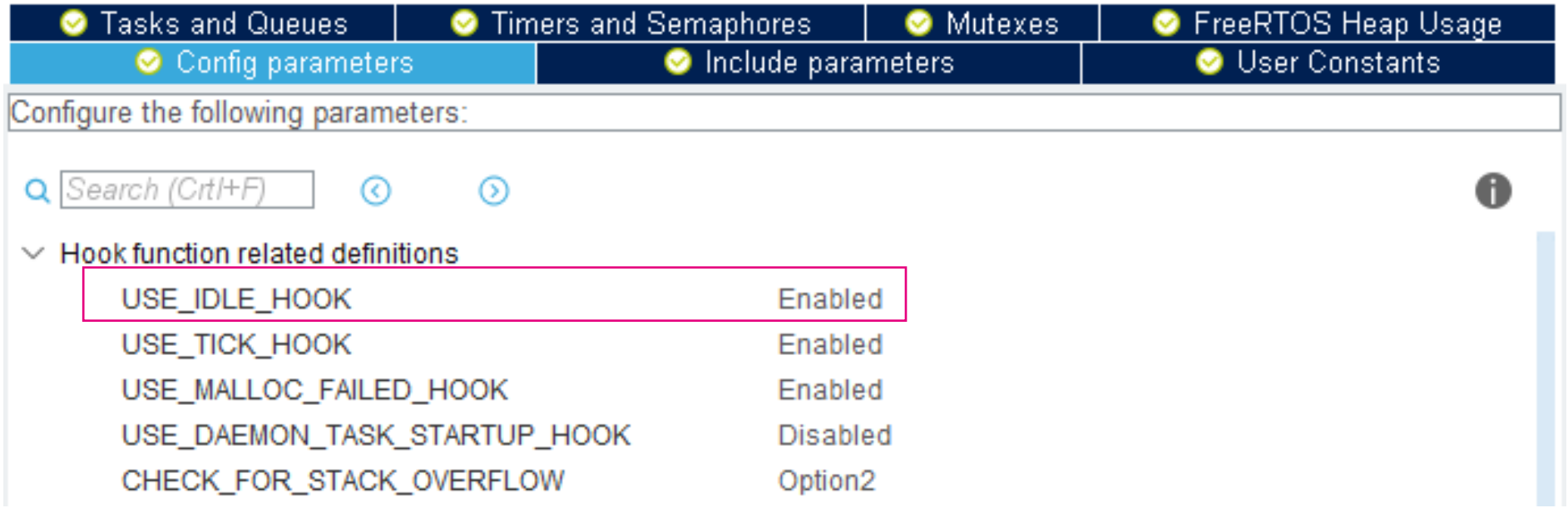
Idle task code

- Idle task code is generated automatically when the scheduler is started
- It is `portTASK_FUNCTION()` function within task.c file
- It is performing the following operations (in endless loop):
  - Check for deleted tasks to clean the memory
  - `taskYIELD()` if we are not using preemption (`configUSE_PREEMPTION`=0)
  - Get yield if there is another task waiting and we set `configIDLE_SHOULD_YIELD`=1
  - Executes `vApplicationIdleHook()` if `configUSE_IDLE_HOOK`=1
  - Perform low power entrance if `configUSE_TICKLESS_IDLE`!=0)

life.augmented

Idle hook

- It is necessary to enable it within Config parameters (part of FreeRTOSConfig.h configuration file)



configUSE_IDLE_HOOK

Idle hook

- Idle hook callback in freertos.c created by STM32CubeMX

```
/* USER CODE END FunctionPrototypes */
/* Hook prototypes */
void vApplicationIdleHook(void);

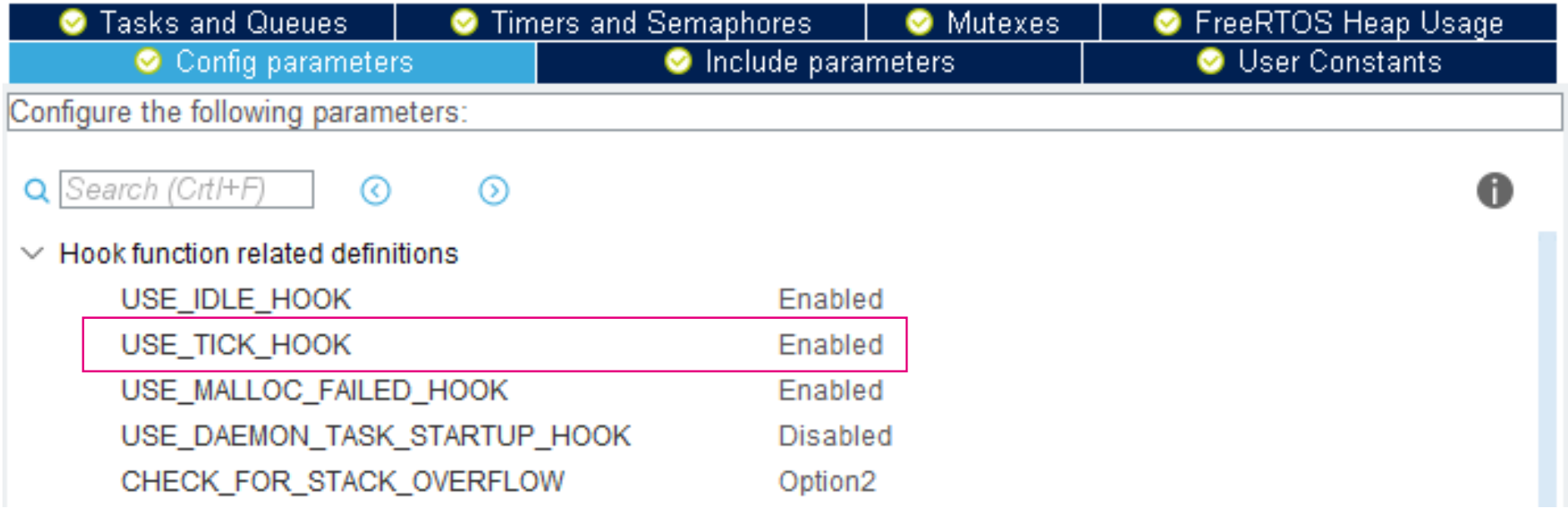
/* USER CODE BEGIN 2 */
__weak void vApplicationIdleHook( void )
{
   /* vApplicationIdleHook() will only be called if configUSE_IDLE_HOOK is set
   to 1 in FreeRTOSConfig.h. It will be called on each iteration of the idle
   task. It is essential that code added to this hook function never attempts
   to block in any way (for example, call xQueueReceive() with a block time
   specified, or call vTaskDelay()). If the application makes use of the
   vTaskDelete() API function (as this demo application does) then it is also
   important that vApplicationIdleHook() is permitted to return to its calling
   function, because it is the responsibility of the idle task to clean up
   memory allocated by the kernel to any task that has since been deleted. */
}
/* USER CODE END 2 */
```

- Do not use blocking functions (osDelay(), …) in this function or while(1)

life.augmented

Tick hook

- Every time the SysTick interrupt is trigger the Tick Hook is called
- Is possible use Tick Hook for periodic events like watchdog refresh
- It is necessary to enable it within Config parameters (part of FreeRTOSConfig.h configuration file)



Tick hook

- Tick hook callback in freertos.c created by STM32CubeMX / STM32CubeIDE

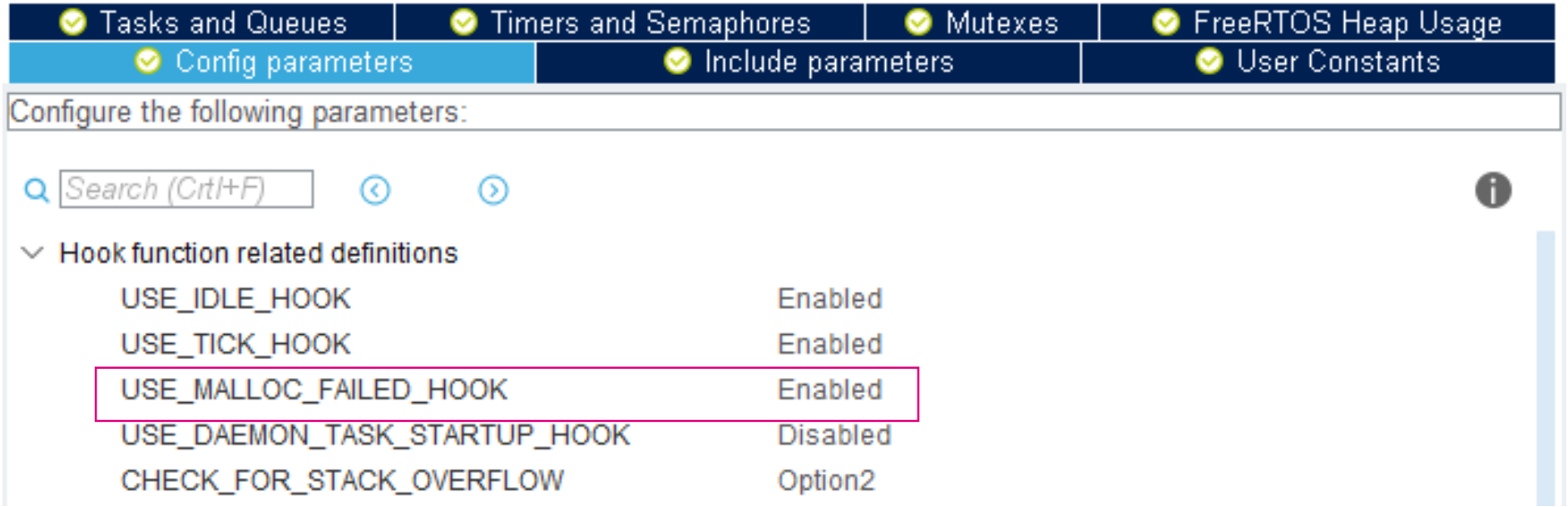
```
/* Hook prototypes */
void vApplicationTickHook(void);

/* USER CODE BEGIN 3 */
__weak void vApplicationTickHook( void )
{
   /* This function will be called by each tick interrupt if
   configUSE_TICK_HOOK is set to 1 in FreeRTOSConfig.h. User code can be
   added here, but the tick hook is called from an interrupt context, so
   code must not attempt to block, and only the interrupt safe FreeRTOS API
   functions can be used (those that end in FromISR()). */
}
/* USER CODE END 3 */
```

- Do not use blocking functions (osDelay, …) in this function or while(1)
- Use only the interrupt safe FreeRTOS functions (with suffix FromISR()).

Malloc failed hook

- This callback is called if the memory allocation process fails (pvPortMalloc() returns NULL)
- Helps to react on malloc problems, when function return is not handled
- It is necessary to enable it within Config parameters (part of FreeRTOSConfig.h configuration file)



Malloc failed hook

- Malloc Failed hook callback skeleton is present in freertos.c created by STM32CubeMX

```
/* Hook prototypes */
void vApplicationMallocFailedHook(void);

/* USER CODE BEGIN 5 */
__weak void vApplicationMallocFailedHook(void)
{
   /* vApplicationMallocFailedHook() will only be called if
   configUSE_MALLOC_FAILED_HOOK is set to 1 in FreeRTOSConfig.h. It is a hook
   function that will get called if a call to pvPortMalloc() fails.
   pvPortMalloc() is called internally by the kernel whenever a task, queue,
   timer or semaphore is created. It is also called by various parts of the
   demo application. If heap_1.c or heap_2.c are used, then the size of the
   heap available to pvPortMalloc() is defined by configTOTAL_HEAP_SIZE in
   FreeRTOSConfig.h, and the xPortGetFreeHeapSize() API function can be used
   to query the size of free heap space that remains (although it does not
   provide information on how the remaining heap might be fragmented). */
}
/* USER CODE END 5 */
```

- Do not use blocking functions (osDelay() , …) in this function or while(1)

Malloc failed hook

- Let's try to implement and test Malloc Failed hook mechanism
- Modify line 143 in **cmsis_os.h** to enable Memory Pools

```
/*cmsis_os.h file, line 143 change 0 to 1*/
#define osFeature_Pool          1
```

- Simple example of Malloc Failed hook (**main.c**):

```
/* USER CODE BEGIN 4 */
void vApplicationMallocFailedHook(void)
{
  Task_action('0');
}
```

- Do impossible memory allocation within one of our tasks (**main.c**)

```
/* Private variables ------------*/
osPoolId PoolHandle;
```

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  osPoolDef(Memory,0x10000000,uint8_t);
  /* Infinite loop */
  for(;;)
  {
    PoolHandle = osPoolCreate(osPool(Memory));
    osDelay(5000);
  }
  /* USER CODE END 5 */
}
```

Impossible memory allocation

Stack overflow protection check of stack ‘high watermark’

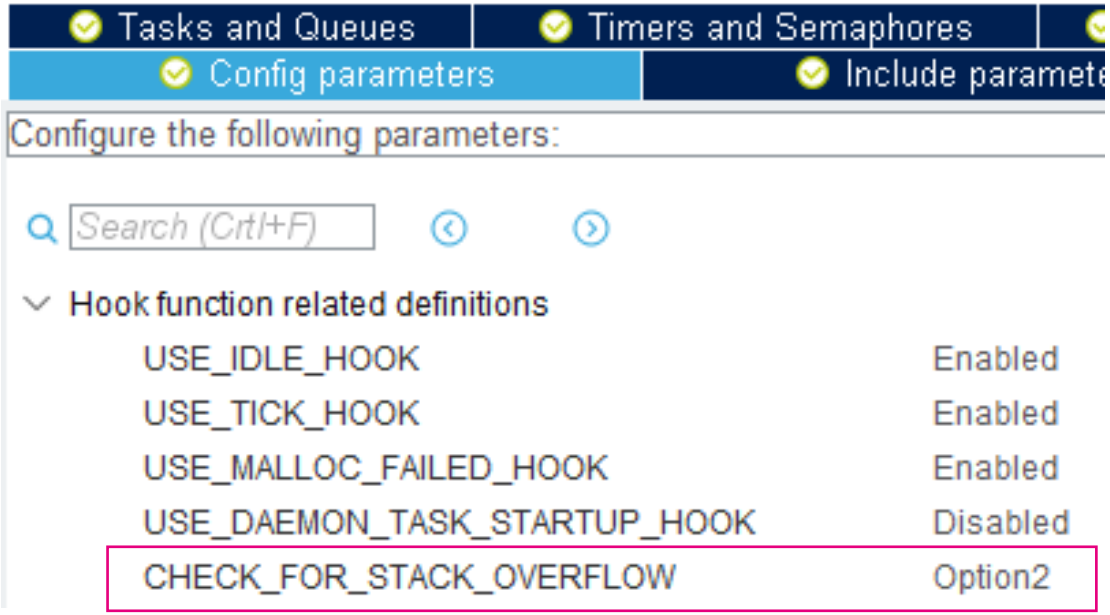
- During task creation, its stack memory space is filled with 0xA5 data
- During run time we can check how much stack is used by task – stack ‘high water mark’
- To turn on this mechanism, some additional configuration of FreeRTOS is required (FreeRTOSConfig.h file or STM32CubeMX FreeRTOS configuration window):
  - `configUSE_TRACE_FACILITY` should be defined to 1
  - `INCLUDE_uxTaskGetStackHighWaterMark` should be defined to 1
- There is a dedicated function to perform this operation:

```
uxTaskGetStackHighWaterMark(xTaskHandle xTask);
```

- After call it with task handle as an argument returns the minimum amount of remaining stack for xTask is presented (NULL means task which is currently in RUN mode).

Stack overflow protection runtime stack check mechanism in STM32CubeMX

- FreeRTOS is able to check stack against overflow
- Two options are available (to be configured within Config parameters (FreeRTOSConfig.h file)):
  - Option 1
  - Option 2



life.augmented

Stack overflow protection runtime stack check mechanism

Stack Overflow Detection - Option 1

- Stack can reach its deepest value after the OS kernel is moving the task from RUN to READY or BLOCKED state. At this point kernel can check whether stack pointer remains within valid stack space. In case it is outside of the valid stack range a stack overflow hook function is called.
- This method is quick, but it can't guarantee catching all stack overflows.
  To use this option only we need to set **configCHECK_FOR_STACK_OVERFLOW** to 1 within FreeRTOSConfig.h file or select Option1 within CHECK_FOR_STACK_OVERFLOW option when using STM32CubeMX or STM32CubeIDE configurators.

Stack Overflow Detection - Option 2

- When task is first created, its stack is filled with a known value (**0xA5**). When moving the task from RUN mode, kernel can check **last 20 bytes** within valid stack range to ensure that these known values have not been overwritten. Stack overflow hook function is called if any of these 20 bytes not remain at their initial value.
- This method is less efficient than method one, but still fast. It is very likely to catch stack overflows but is still not guaranteed to catch all overflows.
- To use this method in combination with option 1 set **configCHECK_FOR_STACK_OVERFLOW** to 2 within FreeRTOSConfig.h file or select Option2 within CHECK_FOR_STACK_OVERFLOW option when using STM32CubeMX or STM32CubeIDE configurators.

Stack overflow protection
stack overflow hook implementation

- Stack overflow hook function is a function called by the kernel at detected stack overflow
- It should be implemented by the user. Its declaration should look like:

```
vApplicationStackOverflowHook(xTaskHandle *pxTask, signed char *pcName);
```

- Its skeleton is generated by STM32CubeMX in freertos.c file

```
/* Hook prototypes */
void vApplicationStackOverflowHook(xTaskHandle xTask, signed char *pcTaskName);

/* USER CODE BEGIN 4 */
__weak void vApplicationStackOverflowHook(xTaskHandle xTask, signed char *pcTaskName)
{
   /* Run time stack overflow checking is performed if
   configCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2. This hook function is
   called if a stack overflow is detected. */
}
/* USER CODE END 4 */
```

- Do not use blocking functions (osDelay() , …) or while(1) in this function

Statistics

- To collect runtime statistics of OS components, there is dedicated function:

  `osThreadList()`

- This function is calling **`vTaskList()`** within FreeRTOS API and is collecting information about all tasks and put them to the table
- Function triggering and data formatting should be implemented by the user
- To run this function you need to set two definitions (define its values to 1):
  - **configUSE_TRACE_FACILITY**
  - **configUSE_STATS_FORMATTING_FUNCTIONS** → it should be added manually to FreeRTOSConfig.h or within STM32CubeMX configuration window for FreeRTOS

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Low power modes



life.augmented

FreeRTOS and low power modes
tickless idle mode operation

- Kernel can stop tick interrupt and place MCU in low power mode, on exit from this mode tick counter is updated
- Enabled when setting `configUSE_TICKLESS_IDLE` as 1
- The kernel will call a macro (tasks.c) `portSUPPRESS_TICKS_AND_SLEEP()` when the Idle task is the only task able to run (and no other task is scheduled to exit from blocked state after n* ticks)
- FreeRTOS implementation of `portSUPRESS_TICKS_AND_SLEEP` for cortexM3/M4 enters MCU in sleep low power mode
- Wakeup from sleep mode can be from a system interrupt/event
- User implementation can be done by setting `configUSE_TICKLESS_IDLE` above 1 (to avoid usage of kernel macros)
- Lowest power consumption can be achieved by replacing default SysTick by LowPower timers (LPTIM or RTC) as tick timer

*) n value is defined in FreeRTOS.h file

life.augmented

Idle task code

- Idle task code is generated automatically when the scheduler is started
- It is `portTASK_FUNCTION()` function within task.c file
- It is performing the following operations (in endless loop):
  - Check for deleted tasks to clean the memory
  - `taskYIELD()` if we are not using preemption (`configUSE_PREEMPTION`=0)
  - Get yield if there is another task waiting and we set `configIDLE_SHOULD_YIELD`=1
  - Executes `vApplicationIdleHook()` if `configUSE_IDLE_HOOK`=1
  - Perform low power entrance if `configUSE_TICKLESS_IDLE`!=0) -> let's look closer on this

Perform low power entrance

- Check expected idle time and if it is bigger than `configEXPECTED_IDLE_TIME_BEFORE_SLEEP` (set to 2 in FreeRTOS.h) then continue
- Suspend all tasks (stop scheduler)
- Check again expected idle time by `prvGetExpectedIdleTime()`
- execute `configPRE_SUPPRESS_TICKS_AND_SLEEP_PROCESSING` with expected idle time and if is bigger than `configEXPECTED_IDLE_TIME_BEFORE_SLEEP` (set to 2 in FreeRTOS.h) then continue
- Execute `portSUPPRESS_TICKS_AND_SLEEP()` with expected idle time and enter into low power mode

Low power mode

- Wakeup from low power mode and resume all tasks (start scheduler)

life.augmented

Perform low power entrance configPRE_SUPRESS_TICKS_AND_SLEEP_PROCESSING

- It is an empty macro defined in FreeRTOS.h file, needs to be defined by the user
- We should define this macro to set `xExpectedIdleTime` to 0 if the application does not want `portSUPRESS_TICKS_AND_SLEEP()` to be called

life.augmented

Perform low power entrance portSUPPRESS_TICKS_AND_SLEEP

- It is an empty macro defined in FreeRTOS.h file, needs to be defined by the user
- It is usually done in port functions (i.e. portmacro.h for gcc)
- There is an assignment to function i.e. `vPortSuppressTicksAndSleep()` which is defined as “weak” within port.c
- This function is called with the scheduler suspended

Perform low power entrance vPortSuppressTicksAndSleep()

- Calculate reload value (max sleep time)
- Read current value of the tick timer
- Stop tick timer
- Enter into critical section by disable interrupts (`cpsid I`)
- Check if there was no context switch in between
  - if yes -> abort low power mode entry
  - if not:
    - Start selected low power timer to expected delay value
    - Perform any other preparation to put application into low power mode *)
    - Enter into desired low power mode

life.augmented

*) by calling configPRE_STOP_PROCESSING() or configPRE_SLEEP_PROCESSING() macros defined in FreeRTOSConfig.h file (bodies with weak attribute in freertos.c file)

Perform low power entrance why we wakeup while interrupts are blocked by PRIMASK?

- WFI is a hint instruction that suspends execution until one of the following events occurs:
  - a non-masked interrupt occurs and is taken
  - an interrupt masked by PRIMASK becomes pending
  - a Debug Entry request.
- Source:

  http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dui0552a/BABFEFIG.html

Perform low power exit vPortSuppressTicksAndSleep()

- On wakeup call function*) to restore non-low power configuration
- Read low power counter values to update tick timer
- Stop low power counter and its interrupt
- Restart tick timer
- Update tick number using function `vTaskStepTick()`

life.augmented

*) configPOST_STOP_PROCESSING() or configPOST_SLEEP_PROCESSING() macros defined in FreeRTOSConfig.h file (bodies with weak attribute in freertos.c file)

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

FreeRTOS – CMSIS_OS v2
Low power modes – tickless mode lab



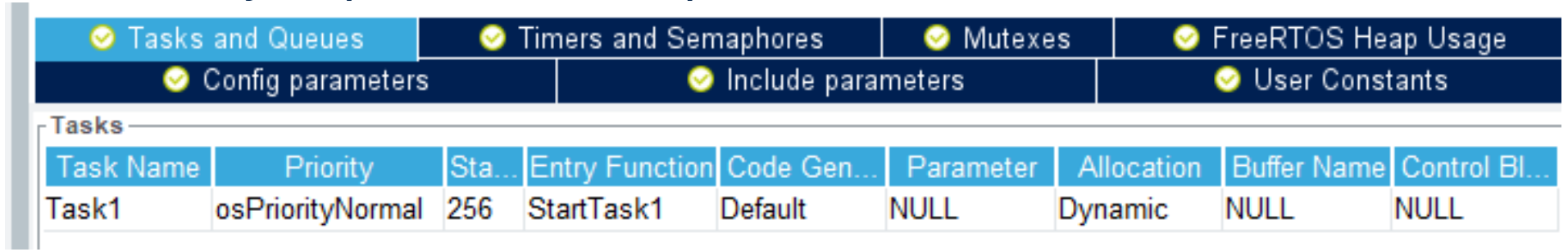
life.augmented

Tickless mode lab

- On next slides there is step-by-step guide how to make tickles mode example
- We would need for this exercise multimeter with uA range to measure current consumption or X-NUCLEO_LPM01 with CubeMonitorPower application
- As a target board NUCLEO-L476RG will be used
- STM32CubeIDE will be used as toolchain

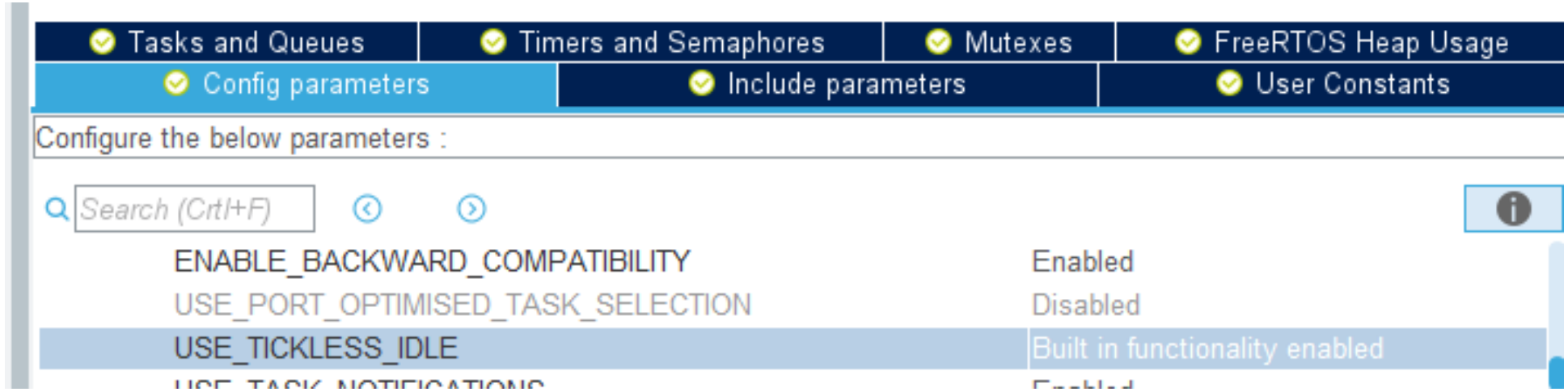
Tickless mode lab
FreeRTOS additional configuration

- We can reuse any of previous examples with one Task, no SW timers



Task Name	Priority	Sta...	Entry Function	Code Gen...	Parameter	Allocation	Buffer Name	Control Bl...
Task1	osPriorityNormal	256	StartTask1	Default	NULL	Dynamic	NULL	NULL

- Within FreeRTOS Config parameters please enable tickless mode (USE_TICKLESS_IDLE) by selecting „**Build in functionality enabled**”

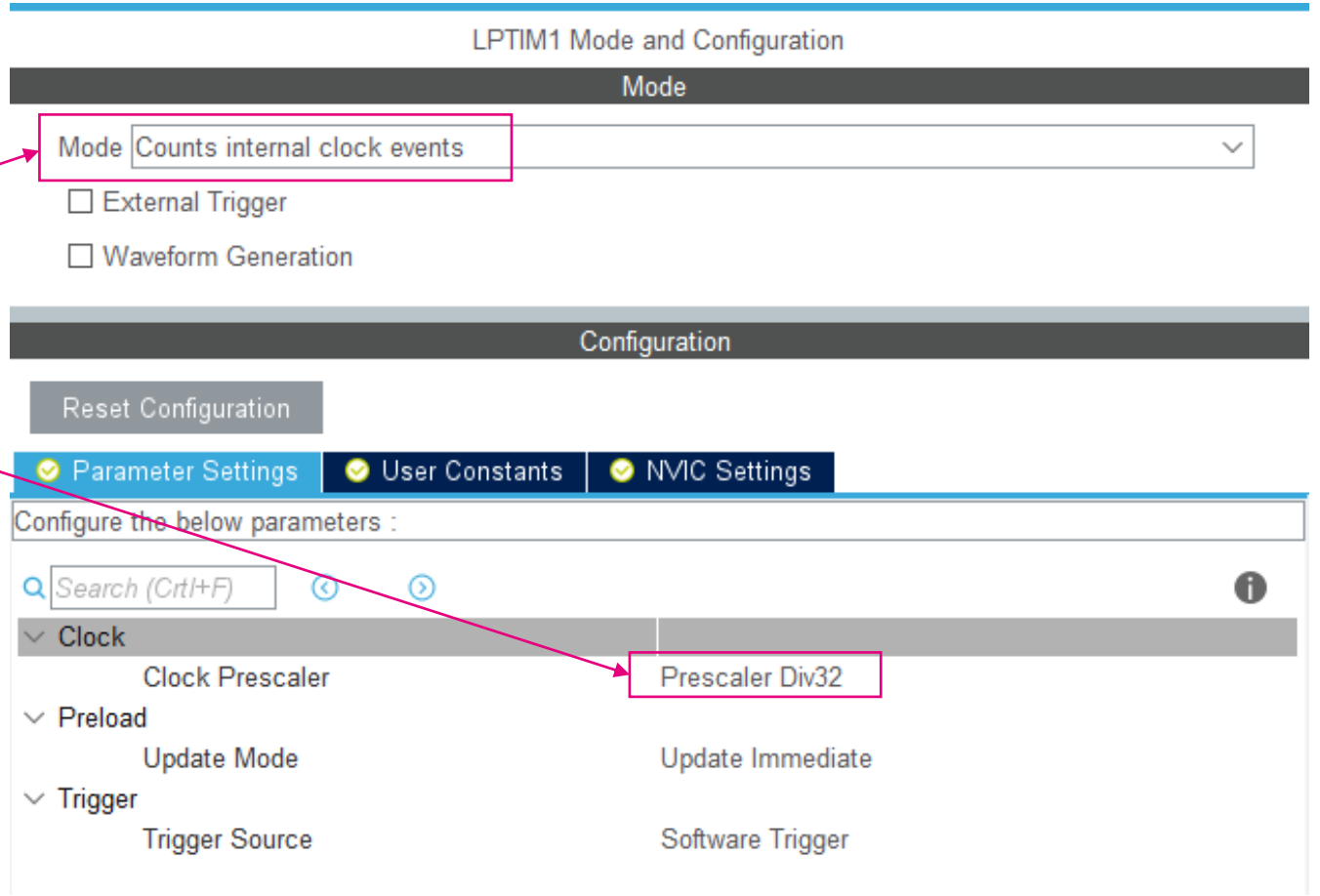


life.augmented

Tickless mode lab
LPTIM configuration (timer to be active in LP mode)

- We need to configure a timer which will work within selected low power mode. To reach STOP modes we can use LPTIM or RTC clocked by LSE.
- We will continue with LPTIM1

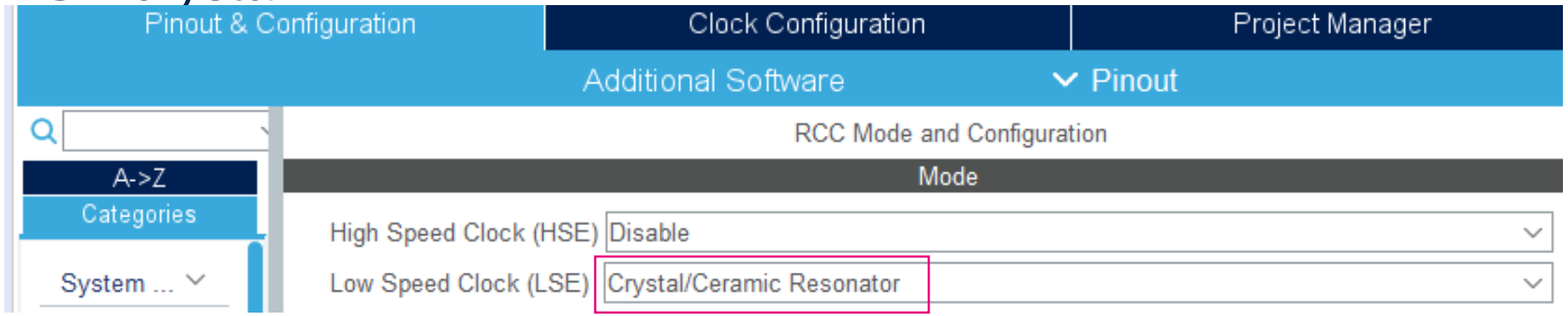
- To have 1ms quantum we need to set its frequency to **1kHz**, thus:
  - Count internal clock events
  - 32kHz LSE clock as source (see next slide) divided by 32
- Additionally, within NVIC Settings, please enable interrupt on LPTIM1



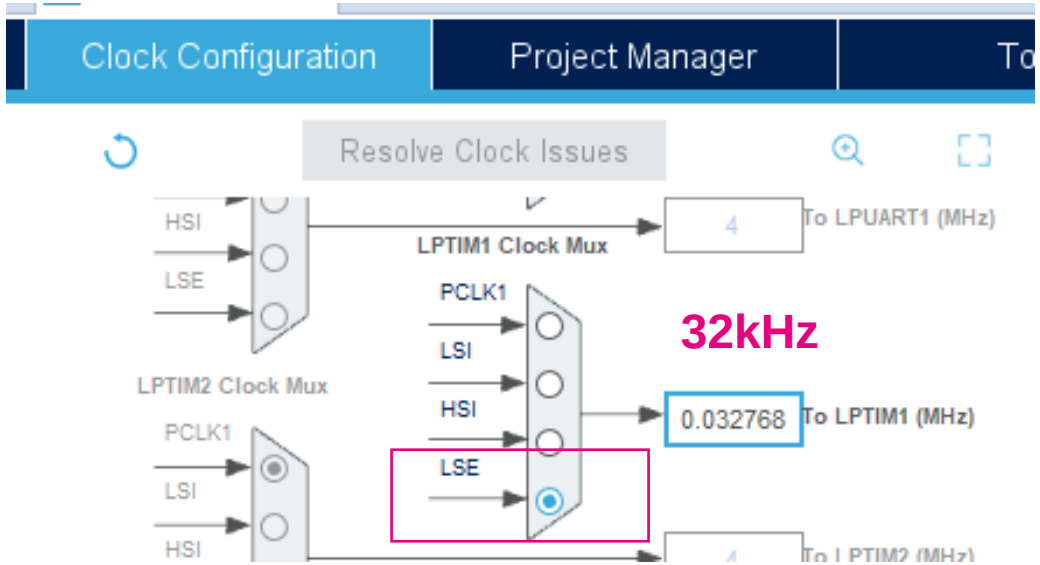
life.augmented

Tickless mode lab
clock configuration for LPTIM1 (LSE)

- Enable LSE crystal



- Select LSE as a clock for LPTIM1 (keeping MSI 4MHz as default system clock)
- Generate the code (Ctrl+s)
- Open main.c file



life.augmented

Tickless mode lab code processing – FreeRTOSConfig.h file

- Depending on the verstion of STM32CubeIDE / STM32CubeMX and libraries you are using you may face different implementations of `configPRE_SLEEP_PROCESSING` and `configPOST_SLEEP_PROCESSING` macros. We would need its simplest definition like below:

```
#if configUSE_TICKLESS_IDLE == 1
#define configPRE_SLEEP_PROCESSING                  PreSleepProcessing
#define configPOST_SLEEP_PROCESSING                 PostSleepProcessing
```

- In case of more complex definition please change it to above form

Tickless mode lab code processing – main.c file

- Define StartTask1() function with some space for tickless mode

```
void StartTask1(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
    osDelay(5000);    //time dedicated to be in tickless mode
    Task_action('1'); //sign of life by LED toggle or sending ,1’ to SWO
    HAL_Delay(500);  //remain active for 0.5 second
  }
  /* USER CODE END 5 */
}
```

Task_action() code – our choice for the session

- Function declaration

```
/* USER CODE BEGIN PFP */
void Task_action(char message);
```

- Using SWO and Serial Wire Viewer (SWV) to send simple text

```
/* USER CODE BEGIN 4 */
void Task_action(char message)
{
  ITM_SendChar(message);
  ITM_SendChar('\n');
}
```

Tickless mode lab code processing – freertos.c file

- Make LPTIM1 handler visible within freertos.c

```
/* USER CODE BEGIN PD */
  extern LPTIM_HandleTypeDef hlptim1;
/* USER CODE END PD */
```

- Prepare application to enter into low power mode:
  - Disable tick interrupt (used by HAL and redirected to TIM6)
  - Program LPTIM with an expected IDLE time to wake up the system

```
_weak void PreSleepProcessing(uint32_t *ulExpectedIdleTime)
{
  /* place for user code */
  HAL_SuspendTick(); //1
  HAL_LPTIM_TimeOut_Start_IT(&hlptim1, 0xFFFF, *ulExpectedIdleTime); //2
}
```

Tickless mode lab code processing – freertos.c file

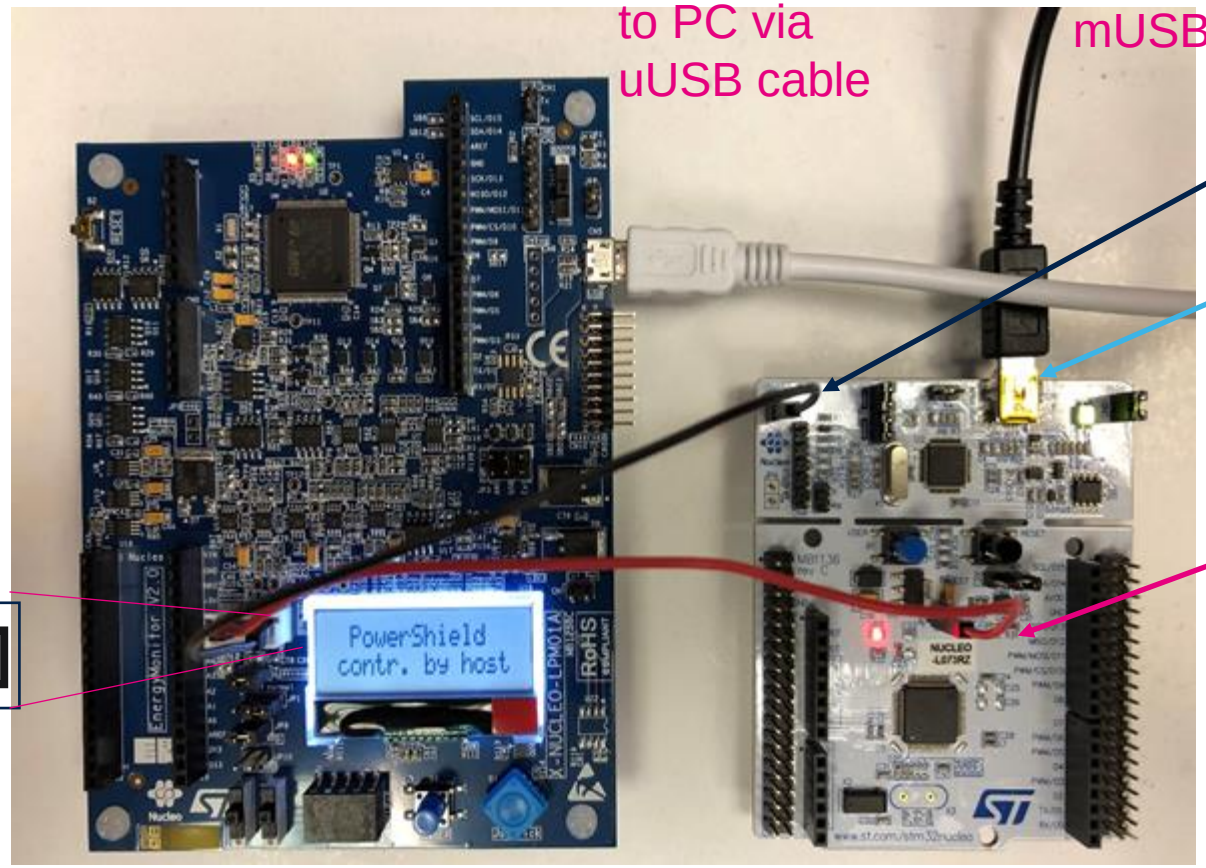
- Prepare application to exit from low power mode:
  - Stop LPTIM
  - Enable tick interrupt (used by HAL and redirected to TIM6)

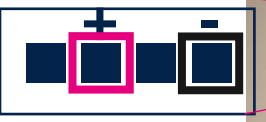
```
_weak void PostSleepProcessing(uint32_t *ulExpectedIdleTime)
{
  /* place for user code */
  HAL_LPTIM_TimeOut_Stop_IT(&hlptim1); //1
  HAL_ResumeTick(); //2
}
```

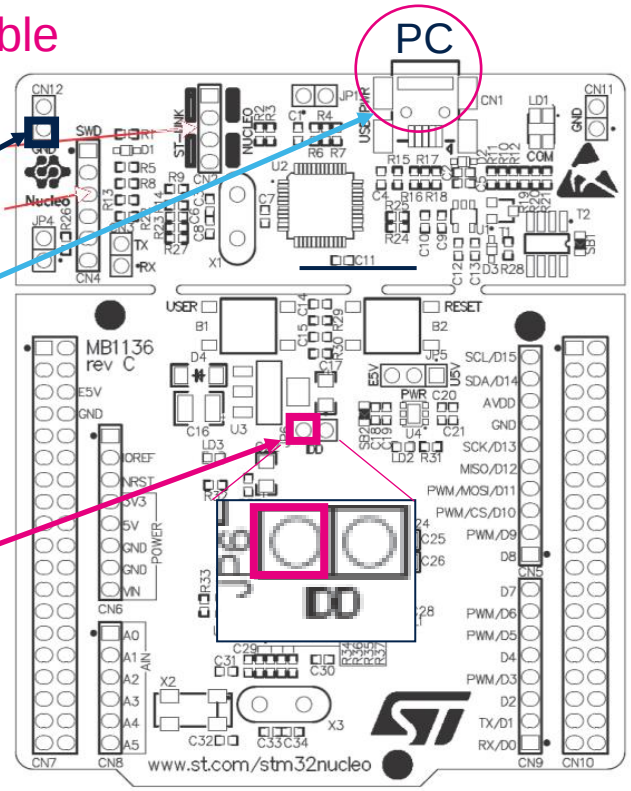
- After those operation we will enter into SLEEP mode for the time where there will be no active task (in our example max 5seconds)
- Compile the code and start debug session, then connect meter to JP6 on NUCLEO board and observe the current values

X-NUCLEO-LPM01 connection

- NUCLEO-L476RG board used as test platform
- X-NUCLEO-LPM01A* board used as programmable PSU





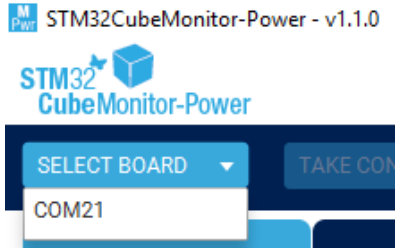


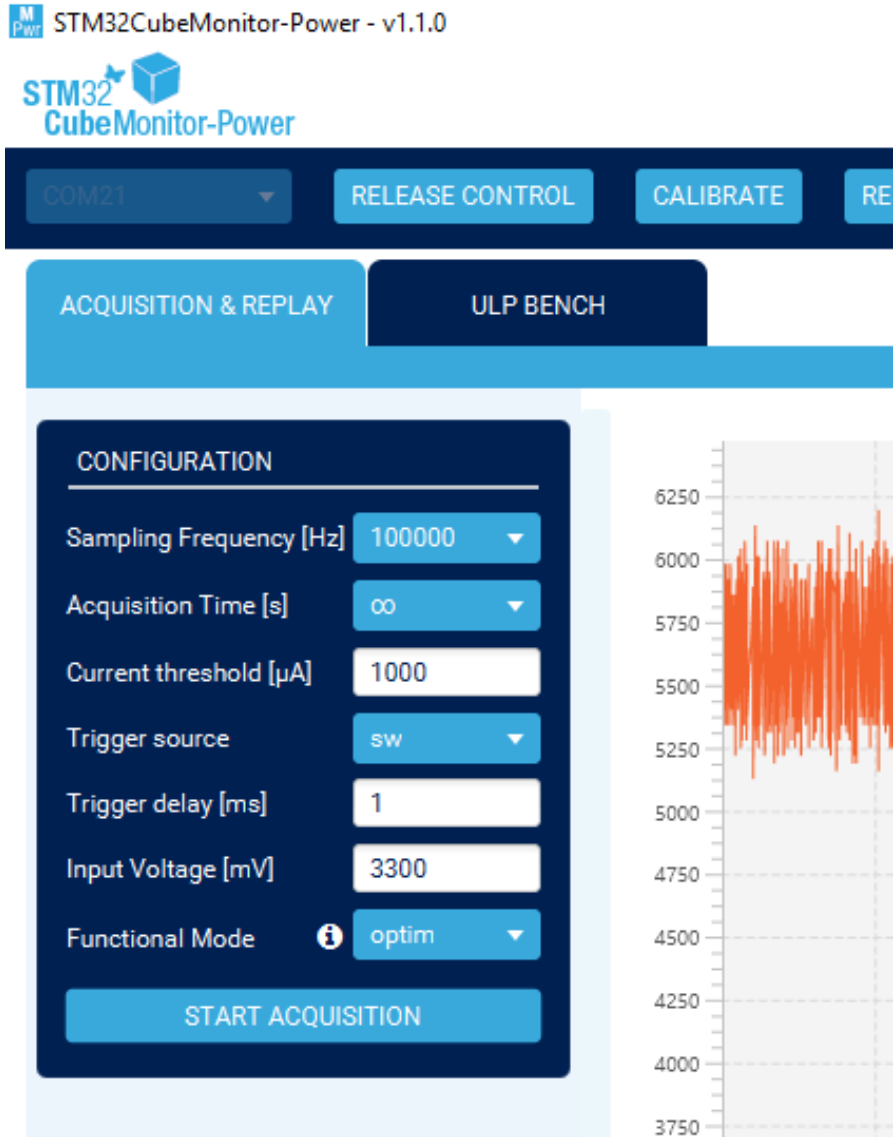
* https://www.st.com/content/st_com/en/products/evaluation-tools/product-evaluation-tools/stm32-nucleo-expansion-boards/x-nucleo-lpm01a.html

life.augmented

STM32CubeMonitor-Power

- After connection of the boards run STM32CubeMonitor-Power
- From „Select board” choose COM port assigned to X-NUCLEO-LPM01a board
- Press „Take control” button
- Within Configuration, please select 100kHz as sampling frequency, infinite acqusition time
- To start measurements press „START ACQUISITION” button
- To stop measurements press „STOP ACQUISITION” button



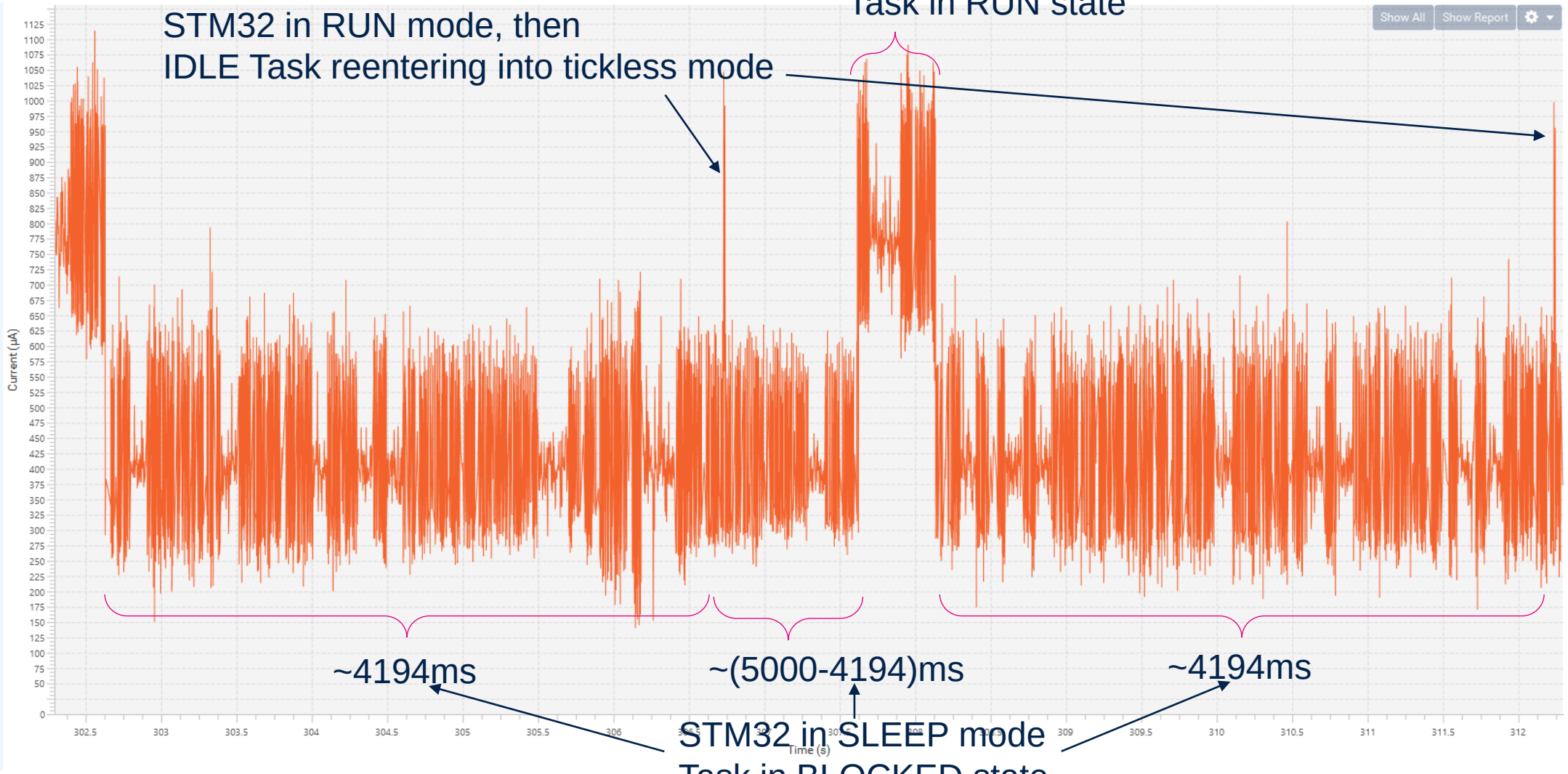


life.augmented

Tickless mode lab – SLEEP

Few ms
STM32 in RUN mode, then
IDLE Task reentering into tickless mode

0.5s,
STM32 in RUN mode
Task in RUN state



~4194ms

~(5000-4194)ms

~4194ms

STM32 in SLEEP mode
Task in BLOCKED state

life.augmented

Tickless mode lab code processing – port.c file

- To reach lower current consumption we need to replace default SLEEP mode with one of STOP ones
- It can be done within `vPortSuppressTicksAndSleep()` function stored in port.c file between execution of `configPRE_SLEEP_PROCESSING()` and `configPOST_SLEEP_PROCESSING()` functions, example of change is presented below

```
if ( xModifiavleIdleTime > 0 )
{
  __asm volatile( "dsb" ::: "memory" );
  __asm volatile( "wfi" );
  __asm volatile( "isb" );
}
```

SLEEP (default)

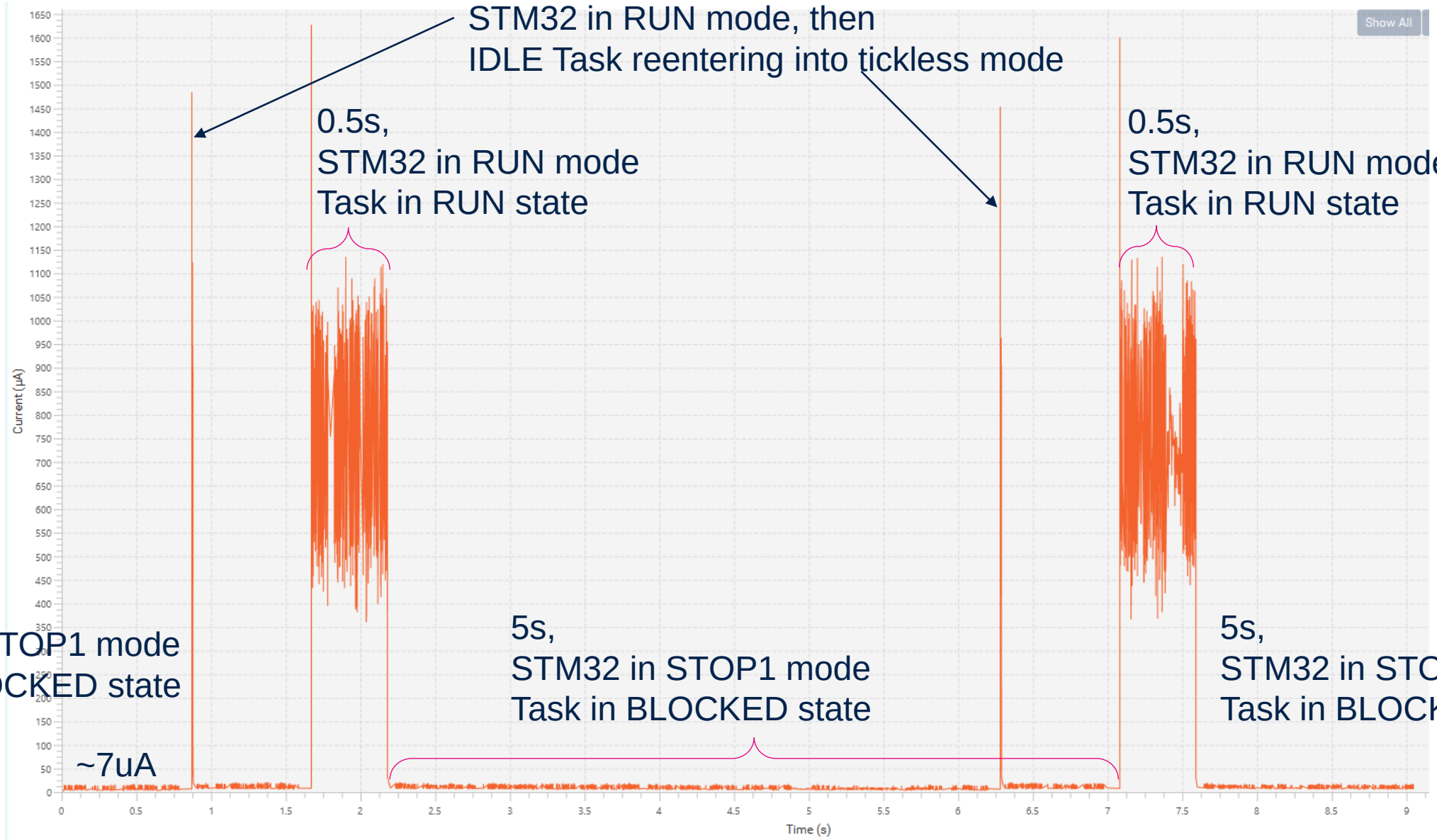
```
#include "main.h"
...
/* Restart SysTick. */
//portNVIC_SYSTICK_CTRL_REG |= portNVIC_SYSTICK_ENABLE_BIT;
…
if ( xModifiavleIdleTime > 0 )
{
  __asm volatile( "dsb" ::: "memory" );
  HAL_PWREx_EnterSTOP1Mode(PWR_STOPENTRY_WFI);
  __asm volatile( "isb" );
}
```

STOP1

Tickless mode lab final effect

- After those operation we will enter into LP mode for the time where there will be no active task (in our example max 5seconds)
- Compile the code and start debug session, then connect meter to JP6 on NUCLEO board and observe the current values
- Try different STOP modes (STOP0, STOP1, STOP2)

Tickless mode lab – STOP1



5s,
STM32 in STOP1 mode
Task in BLOCKED state

life.augmented

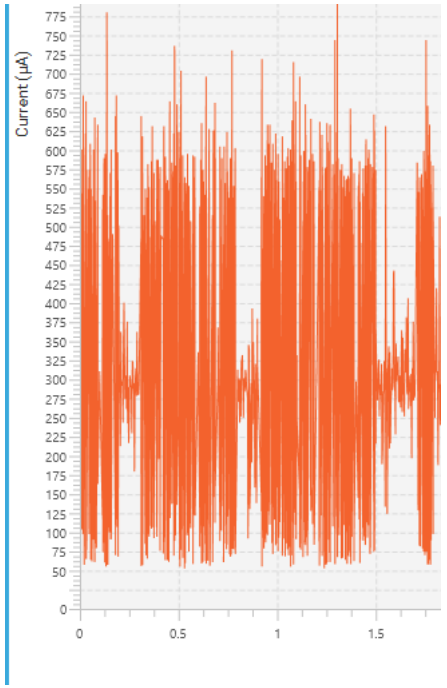
Tickless mode lab – STOP1



life.augmented

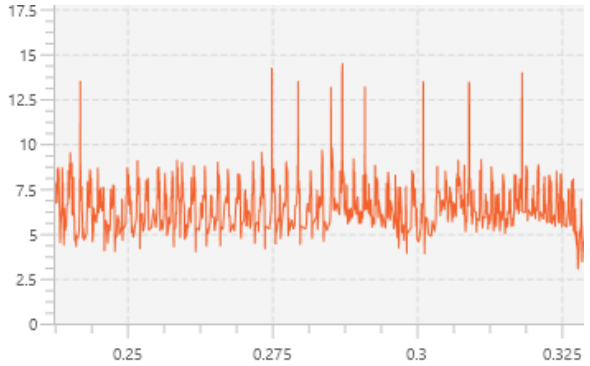
Tickless mode lab – STOP0, STOP1, STOP2 phases

~600uA



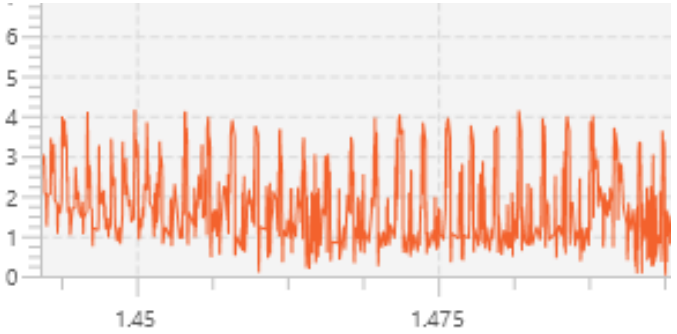
STM32 in STOP0 mode
Task in BLOCKED state

~7uA



STM32 in STOP1 mode
Task in BLOCKED state

~3uA



STM32 in STOP2 mode
Task in BLOCKED state

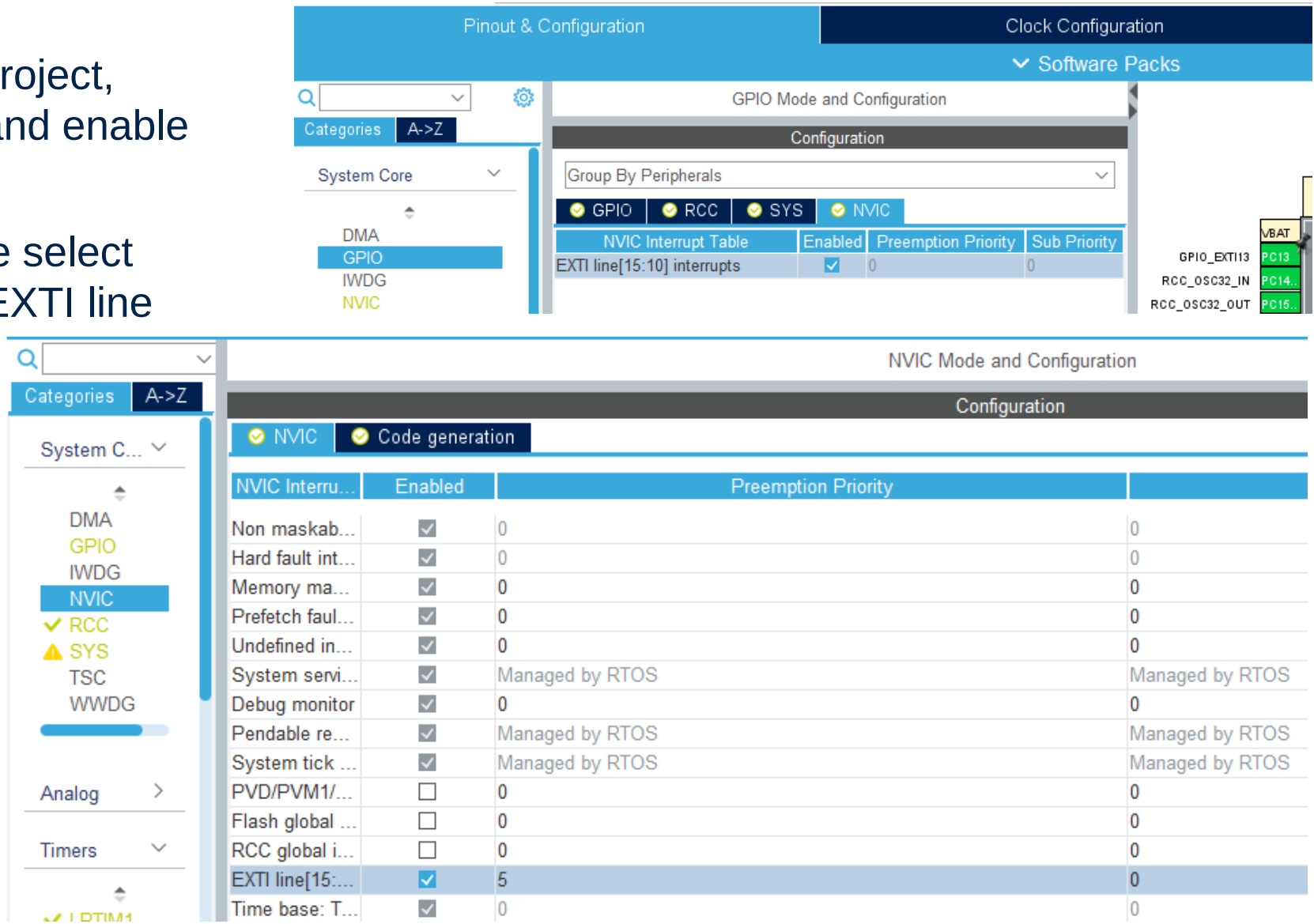
life.augmented

Tickless mode lab final effect

- After those operation we will enter into LP mode for the time where there will be no active task (in our example max 5seconds)
- Compile the code and start debug session, then connect meter to JP6 on NUCLEO board and observe the current values
- Try different STOP modes (STOP0, STOP1, STOP2)
- Advanced task: please replace osDelay() within the task function body with wait for binary semaphore with the same timeout. Semaphore can be given from an interrupt, i.e. connected to BLUE_BUTTON. What is the difference in application behaviour?

Tickless mode lab
adding binary semaphore and button support

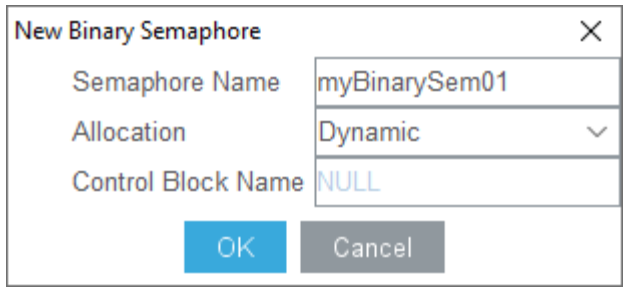
- Within existing STM32CubeMX project, please configure PC13 as EXTI and enable its interrupt
- Within NVIC configuration, please select preemption priority level to 5 for EXTI line [15:10] interrupt

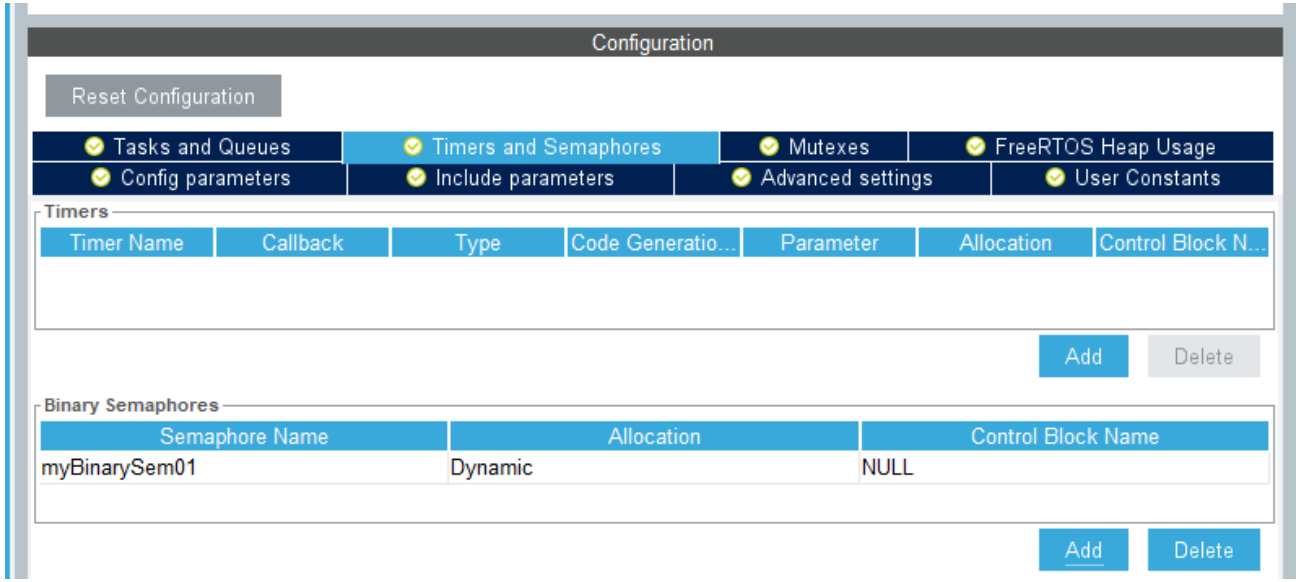


life.augmented

Tickless mode lab
adding binary semaphore and button support

- Within FreeRTOS configuration, please add binary semaphore





- Regenerate the code, open main.c file

Adding binary semaphore with IRQ code processing – main.c file

- Semaphore can be released from interrupt if its interrupt priority is below (higher number) configured `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY` (5)
- Using HAL libraries we can release semaphore in the GPIO_EXTI13 callback

```
/* USER CODE BEGIN 4 */

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  osSemaphoreRelease(myBinarySem01Handle);
}
```

- Within StartTask1() we need to replace osDelay(5000); with wait for semaphore code:

```
for(;;)
  {
        osSemaphoreAcquire(myBinarySem01Handle, 5000);
        Task_action('1'); //sign of life by LED toggle or sending ,1’ to SWO
        HAL_Delay(500);
 }
```

life.augmented

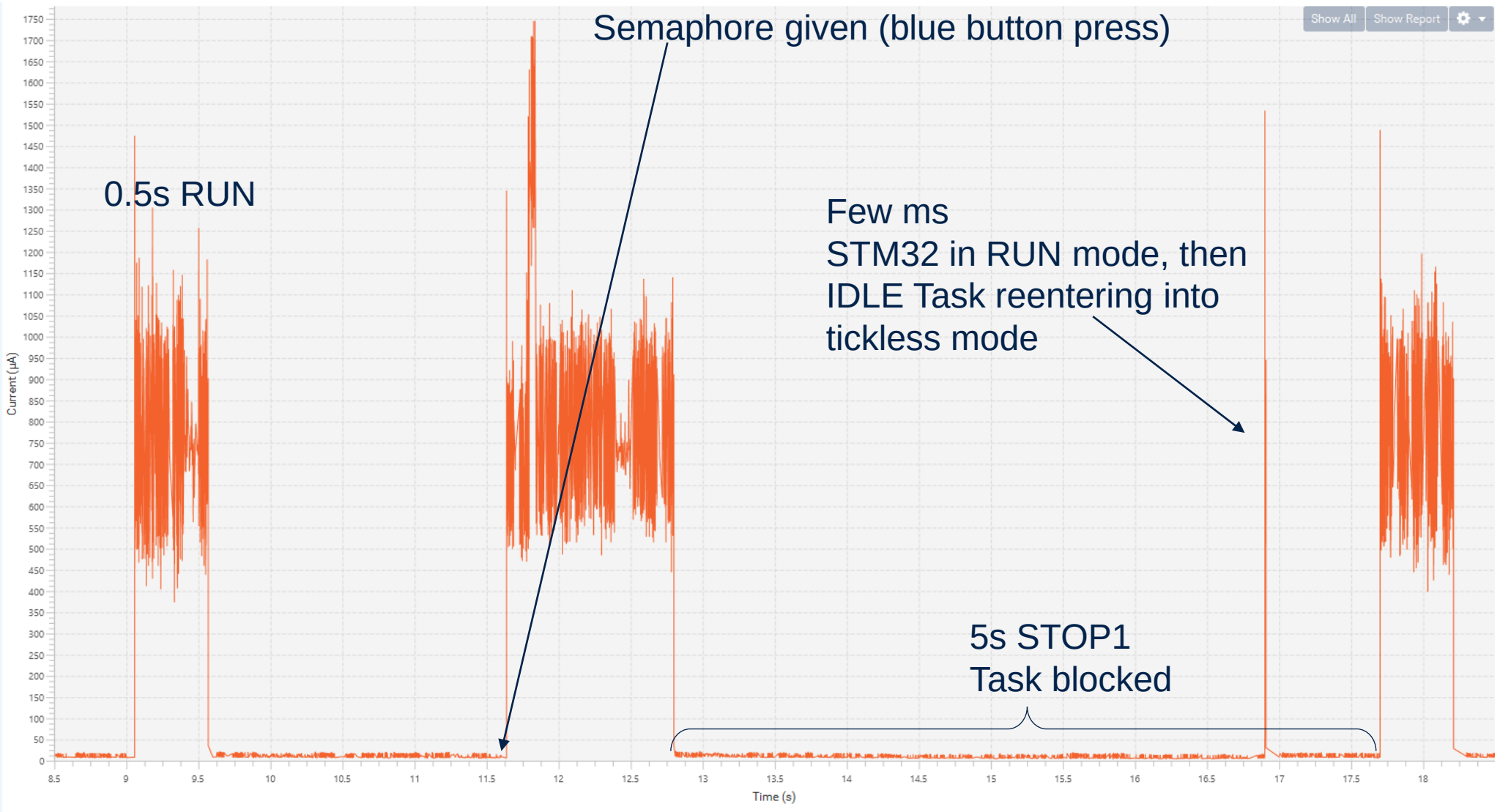
Tickless mode lab code processing – port.c file

- As code re-generation removes all the code modifications done outside USER CODE section, we need to repeat port.c three modifications
- Within `vPortSuppressTicksAndSleep()` function stored in port.c file between execution of `configPRE_SLEEP_PROCESSING()` and `configPOST_SLEEP_PROCESSING()` functions, example of change is presented below

```
#include "main.h„ //to see HAL functions
...
/* Restart SysTick. */
//portNVIC_SYSTICK_CTRL_REG |= portNVIC_SYSTICK_ENABLE_BIT;
…
if ( xModifiavleIdleTime > 0 )
{
  __asm volatile( "dsb" ::: "memory" );
  HAL_PWREx_EnterSTOP1Mode(PWR_STOPENTRY_WFI);
  __asm volatile( "isb" );
}
```

STOP1

Tickless mode lab – STOP1, semaphore with EXTI IRQ



life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Troubleshooting and typical mistakes



life.augmented

Using the same timer for HAL timebase and OS tick timer

Possible effect

- At code generation stage: STM32CubeMX and STM32CubeIDE are generating a warning that we need to change HAL time base timer (within SYS peripherals group).
- During runtime: All delays/timeouts within HAL /LL libraries will be affected by OS functions (by its critical sections where SysTick is frozen)

Solution

- Select different timer within SYS peripheral group (the best ones are timers without any input/output channels like TIM6 or TIM7).

More information

OS functions dedicated to task-only execution

Possible effect

- OS function has no effect when called from interrupts

Reason

- Selected function cannot be called from interrupt.

Solution

- Check the return value. If it is osErrorISR (-6) it means that function musn’t be called from interrupt, but only from the task code. As an action it is necessary to call the function from any task code or change the function which should be called from interrupt. Another option is to use IRQ to trigger any other task to perform this action.

Improper configuration of interrupts

Possible effect

- Sometimes OS function has no effect or not expected effect when called from interrupts
- Landing in vPortValidateInterruptPiority() in case configASSERT is set (default setting in case of STM32CubeMX/STM32CubeIDE usage). It can be found in FreeRTOSConfig.h file

Solution

- Set configASSERT within FreeRTOSConfig.h file
- Check it IRQ has priority lower or equal (numerically higher) to **configMAX_SYSCALL_INTERRUPT_PRIORITY** (stored within FreeRTOSConfig.h file and visible within FreeRTOS->Config parameters, LIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY within STM32CubeMX and STM32CubeIDE

More information

Some tasks are not executed

Possible effect

- Task is never selected by scheduler

Solution

- Check whether the task is not waiting for any of OS components like semaphores, queues, mutexes
- Check whether there is no higher priority task which was intended to be put into blocked state (by osDelay()) but instead osThreadYield() has been used (which sends task to ready list)
- Check whether we are not running OS in cooperative mode
- Verify the priority level of the task versus other tasks in the system

Wrong usage of semaphores and queues

Possible effect

- Unblocking the task (waiting for queue/semaphore) even queue/semaphore has not been received.

Solution

- Check return value (osStatus_t) type of Acquire/Take functions. In case of negative value analyze whether it was timeout or any other effect.
- Increase timeout used within Acquire/Take functions. In case semaphore/queue take should be the only unblocking condition - use osWaitForever timeout parameter
- Check semaphore initial content (creation phase). In case semaphore is already given upon creation (initial number of tokens is non zero) – change it to 0.

Wrong parameters of software timers task

Possible effect

- Software timers callback is not executed or is executed not expected way (in terms of timing)

Root cause

- Software timers task priority is either too low (then all other tasks) or has invalid value (please notice that within CMSIS_OS v2 there is a gap between 1 and 8 of defined priority value and the detault value set to TIMER_TASK_PRIORITY is set to 2)

Solution

- Check TIMER_TASK_PRIORITY and align it to other tasks used in the application.

Too small stack for the task

Possible effect

- Before code generation: STM32CubeMX and STM32CubeIDE are generating a warning that we need to change the size of OS heap (TOTAL_HEAP_SIZE) within FreeRTOSConfig.h (or FreeRTOS Config parameters within STM32CubeMX or STM32CubeIDE)
- Within code run (i.e. after local variables declaration within task entry functin): Trigger of HardFault interrupt.

Solution

- Increase the stack size dedicated to the task within its osThreadAttr_t field.

OS heap is too small

Possible effect

- At project creation: STM32CubeMX and STM32CubeIDE are generating a warning (FreeRTOS Heap Usage tab in FreeRTOS configuration) that there is not enough heap space to allocate selected OS components
- During run time (once we create new OS components from the code): OS handlers of newly created components are equal to 0 what means that are not created due to memory allocation issues.

Solution

- Increase TOTAL_HEAP_SPACE within FreeRTOSConfig.h file or within FreeRTOS Config paratmers once using STM32CubeMX or STM32CubeIDE.

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented

Summary



life.augmented

Summary of this session

The main goals of this sessions were answers to the following questions:

- What it is an operating system ?
- What are its main components ?
- What are the main advantages and disadvantages of operating system ?
- How to start application development based on OS ?
- When we would need it within embedded MCU based application ?
- What are typical mistakes done during OS based application development ?

Let’s have a look whether it was possible to answer to them.

life.augmented

What is an operating system ?

- We can define it as a program which is managing execution of series of smaller programs (tasks) which are doing the actions within embeddded application
- Real time operating system which we are considering here is more deterministic version, where all operations are tightly connected to hardware components of the system by interrupts.

What are the main components of the operating system ?

- The most important component of the operating system is scheduler which is managing the tasks and other components and made an interraction with hardware components of the system
- Next components group are tasks (threads in CMSIS_OS v2). Those are mini programs performing scheduled actions within the system
- Another group of components are communication mechanisms for interaction among tasks or between task and interrupts. Within this group we can find semaphores, queues, mutexes, Event Flags and Thread Flags
- We have as well software timers which are responsible for action triggering witin specified time.

life.augmented

What are main + / - of the operating system ?

Among main advantages of OS I would list:

- More determinism within application done by split the code into smaller actions which are executed periodically
- Additional resources (semaphores, queues, mutexes) which support resources management (shared memory, display interface).
- Built-in stack protection mechanisms gives more control on the application and recovery point in case of issues related to tasks.

Main disadvantages os operating systems could be:

- Different way of application creation and its execution
- RAM and FLASH memory bigger usage
- Slower execution speed due to time consuming context switch

life.augmented

How to start application development on STM32 and OS ?

- The fastest way to start with OS on STM32 is to use STM32CubeMX or STM32CubeIDE and add FreeRTOS there. Within those programs a ready to use project template will be generated and the role of the user would be to fill tasks functions bodies.
- Another method is to reuse existing examples, which are present within /Application folder for selected STM32 based board within its Cube library pack
- In all above cases user will work using CMSIS_OS v1 or v2 API.

When we need to use OS in embedded system ?

Usually we are using operating systems within embedded applications which:

- Should perform some actions on regular timer intervals (like USB or Ethernet data packets).
- Is using shared resources which needs to be properly managed and should have an exclusive access (like shared memory, display interface)
- Are built from processes / functions which are dependent from each other and some synchronization mechanism needs to be used

What are the typical mistakes during work with OS ?

I would list here the following points

- Using the same timer which is used for timeslice to other purposes
- Improper interrupts priority configuration
- Not enough RAM memory allocated for OS heap
- Misunderstanding of the timeout role in some of the functions

- We have covered this within „Troubleshooting and typical mistakes” part in this training

life.augmented

More information

- More information about **FreeRTOS and its native API** you can find on below web page:

  https://freertos.org/

- More information about **CMSIS_OS v2** you can find on below web page:

  https://www.keil.com/pack/doc/CMSIS_Dev/RTOS2/html/group__CMSIS__RTOS.html

- You can refer as well to dedicated user manual (**UM1722**)

  https://www.st.com/content/ccc/resource/technical/document/user_manual/2d/60/ff/15/8c/c9/43/77/DM00105262.pdf/files/DM00105262.pdf/jcr:content/translations/en.DM00105262.pdf

- Ready to use FreeRTOS examples with CMSIS_OS layer are available within Cube library for each STM32 line, usually in Applications folder, like:

  .\STM32Cube_FW_L4_V1.15.1\Projects\NUCLEO-L476RG\Applications\FreeRTOS\

Thank you for your particiapation

- More information about the session (prerequisites, link to materials) are present within dedicated web page at:

  https://www.st.com/content/st_com/en/support/learning/stm32-education/stm32-moocs/FreeRTOS-CMSIS_OSv2.html

- In case of any questions please post them to our support forum at:

  Community.st.com

- If you would like to share your opinion about this session, please use the following anonymous survey

  xxx

life.augmented

Thank you

© STMicroelectronics - All rights reserved.
The STMicroelectronics corporate logo is a registered trademark of the STMicroelectronics group of companies. All other names are the property of their respective owners.

life.augmented